ETL for Business Intelligence

A practical guide to data provisioning, dimensional modelling, and pipeline design

---

Chapter 1: Foundations — What ETL Is and Why It Exists

> ***ETL for Business Intelligence***
>
> *A practical guide to data provisioning, dimensional modelling, and pipeline design*
>
> *© Patrick Dolinger, NSCC Institute of Technology*
>
> *Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0)*
>
> *You are free to share and adapt this material for any purpose, provided appropriate credit is given.*

Chapter Overview

This chapter establishes the conceptual foundation for everything that follows. Before writing a single line of SQL or placing a single component on an SSIS canvas, you need to understand *why* ETL exists, *what problem it solves*, and *where it sits* in the broader landscape of data management and business intelligence.

By the end of this chapter you will be able to:

- Explain what ETL is and describe each of its three stages
- Distinguish between OLTP and OLAP systems and explain why both are necessary
- Describe the layered architecture of a modern BI pipeline
- Explain the role of a data warehouse in an analytical ecosystem
- Identify where SSIS fits as a professional ETL tool
- Describe the CabotTrail Outdoor data environment used as the worked example throughout this book

Table of Contents

1. The Problem ETL Solves

Every organization collects data. A retailer records sales transactions. A manufacturer tracks inventory movements. A hospital logs patient visits. In each case, that data is captured in an **operational system** — a database designed to support the day-to-day running of the business.

Operational systems are excellent at what they do. They are fast, reliable, and precise. But they are designed for a fundamentally different purpose than answering questions like:

- *Which product categories generated the most revenue last quarter?*
- *How does this month's return rate compare to the same month last year?*
- *Which sales territories are growing and which are declining?*

These are **analytical questions** — they require aggregating, comparing, and trending data across time and across multiple business areas. Answering them from an operational system is possible, but it is slow, complex, and risky. Complex analytical queries compete with operational transactions for database resources. A poorly written analytical query can bring an operational system to its knees at exactly the wrong moment — during a busy sales period, or at month-end close.

The solution is to **separate the two workloads**. Build a second system — an analytical system — that is specifically designed to answer business questions efficiently. Then build a process to move data from the operational system into the analytical system, transforming it along the way to make it more useful for analysis.

That process is **ETL: Extract, Transform, Load**.

ETL is the bridge between the world of operational data and the world of analytical insight.

---

2. OLTP vs OLAP: Two Different Jobs

The two types of systems described above have formal names:

- **OLTP** — Online Transaction Processing
- **OLAP** — Online Analytical Processing

Understanding the fundamental differences between them is not just academic — it directly shapes every design decision you will make as an ETL developer.

2.1 OLTP: Designed for Operations

An OLTP system is optimized for **recording and retrieving individual transactions quickly and accurately**. Think of it as the database that powers a point-of-sale terminal, an order management system, or an inventory application.

Key characteristics of OLTP systems:

Characteristic	Description
Workload	Many small, fast read/write transactions happening simultaneously
Data model	Highly normalized (3NF or above) — data is stored without redundancy
Query pattern	Precise lookups: *"What is the current stock level for ProductID 42?"*
Data volume per query	Small — typically one or a few rows
Update frequency	Continuous — every transaction modifies the database
Data history	Current state only — historical changes are often overwritten
Users	Operational staff: clerks, warehouse workers, customer service agents

The normalization used in OLTP systems serves a specific purpose: it prevents **data anomalies**. When customer information is stored in one place and referenced everywhere else via a foreign key, updating a customer's address requires changing exactly one row. There is no risk of inconsistency. This is the core value of normalization — correctness over convenience.

2.2 OLAP: Designed for Analysis

An OLAP system is optimized for **answering complex questions across large volumes of historical data**. Think of it as the database that powers a BI dashboard, an executive report, or a data analyst's query workbench.

Key characteristics of OLAP systems:

Characteristic	Description
Workload	Fewer, larger queries that aggregate millions of rows
Data model	Denormalized — data is structured for query convenience, not storage efficiency
Query pattern	Aggregations: *"What was total revenue by product category and territory for 2024?"*
Data volume per query	Large — often scanning millions of rows to produce a summary
Update frequency	Periodic — typically batch-loaded nightly or weekly
Data history	Deep history preserved — years of data retained for trend analysis
Users	Analysts, managers, executives, BI tools

The denormalization used in OLAP systems serves an equally specific purpose: it reduces **join complexity** at query time. When a sales analyst wants to see revenue by product category, having `CategoryName` stored directly on the product row in a dimension table means one table read instead of three joins. At millions of rows, that difference is significant.

2.3 Why You Cannot Use One System for Both

The differences between OLTP and OLAP are not merely philosophical — they represent genuine engineering trade-offs that make one system fundamentally unsuitable for the other's workload.

Running analytics on an OLTP system causes problems:

1. **Resource contention** — a full-table scan for an analytical query competes with transaction processing for I/O, CPU, and locks. Operational users experience slowdowns or timeouts.
2. **Locking conflicts** — analytical queries may hold read locks on data that operational transactions need to update, causing deadlocks.
3. **Incomplete history** — OLTP systems often overwrite historical data (SCD Type 1 by default). An analytical query asking "what province was this customer in last year?" may find the answer has been overwritten.

4. **Structural mismatch** — the normalized structure that makes OLTP efficient requires many joins for even simple analytical questions. These queries are difficult to write and slow to execute.

Maintaining an OLTP on an OLAP system causes different problems:

1. **Write performance** — denormalized structures require updating many rows when a single fact changes (the update anomaly). Real-time transaction processing becomes unreliable.
2. **Data integrity risks** — without the constraints and normalization of OLTP, data inconsistencies can creep in silently.
3. **Storage waste** — redundant storage of repeated values is acceptable in a batch-loaded analytical system but wasteful in a continuously updated operational one.

The separation of OLTP and OLAP workloads onto separate systems is therefore not a luxury — it is an architectural necessity. ETL is the mechanism that makes this separation practical.

2.4 CabotTrail Outdoor: A Concrete Example

Throughout this book, a fictional outdoor gear retailer called **CabotTrail Outdoor** serves as the worked example. Its OLTP database, `CabotTrailOutdoor`, stores operational data across four schemas:

```
Sales        → Customers, Orders, OrderLines, Invoices, InvoiceLines,
               CustomerTransactions, Returns
Purchasing   → PurchaseOrders, PurchaseOrderLines, SupplierTransactions
Inventory    → Products, ProductCategories, ProductCategoryAssignments,
               ProductHoldings
Application  → People, Employees, Cities, StateProvinces, Countries,
               DeliveryMethods, TransactionTypes, PaymentMethods
```

To answer a simple business question — *"Which sales rep generated the most revenue last quarter?"* — requires joining at minimum six tables across two schemas. The OLTP can answer the question, but it is not designed for it.

The analytical target, `CabotTrailOutdoorDW`, restructures this same data into a dimensional model where the same question requires a single three-table join.

This contrast — the same data, two different structures, for two different purposes — is the lived experience of ETL work.

---

3. The BI Pipeline: A Layered Architecture

Modern business intelligence does not move data directly from an operational system to a report. It passes through several distinct layers, each with a specific purpose.

3.1 Source Systems

Source systems are the origin of all data. In enterprise environments there are typically multiple source systems — each optimized for its operational purpose, often running different database platforms, and often with overlapping or conflicting data.

Common source system types:

Type	Examples	Challenges for ETL
Relational OLTP	SQL Server, Oracle, PostgreSQL	Multiple schemas, complex joins required
ERP systems	SAP, Oracle ERP, Microsoft Dynamics	Proprietary schemas, complex licensing for data access
CRM systems	Salesforce, HubSpot	REST APIs, rate limits, JSON data formats
Flat files	CSV, Excel, fixed-width text	No data types enforced, encoding issues, irregular formats
Web APIs	REST, GraphQL	Pagination, authentication, schema changes without notice
Cloud data stores	Azure Blob, S3, BigQuery	Network latency, cost per query, authentication complexity

In the CabotTrail environment, the source system is a single SQL Server OLTP database — the simplest possible scenario. Real-world ETL projects almost always involve multiple heterogeneous sources.

3.2 The Staging Area

The staging area is a temporary holding zone where source data lands after extraction but before transformation. It is often underappreciated by newcomers to ETL but plays a critical role.

Why a staging area matters:

1. **Source system protection** — extracting once to staging means you only touch the operational system once per load cycle. If transformation fails and needs to be re-run, you re-process from staging — not from the source.
2. **Auditability** — staging preserves a snapshot of exactly what arrived from the source. If a transformation produced unexpected results, you can compare the staging data to the transformed output to find the discrepancy.
3. **Performance isolation** — complex transformations run against staging tables, not against the live operational system.
4. **Multiple target support** — the same staging data can feed multiple transformation streams for different targets without re-extracting from source.

Staging tables typically have minimal constraints — no foreign keys, minimal indexes, nullable columns. The goal is to land the data fast and faithfully, not to enforce business rules. Business rules are applied in the Transform stage.

> ***CabotTrail note:*** *The `CabotTrailOutdoorDW` database includes a `Staging` schema for this purpose. In this book's worked examples, some transformations use staging tables explicitly; others perform the extraction and transformation in a single SSIS Data Flow for simplicity. Production ETL systems almost always use explicit staging.*

3.3 The Data Warehouse

The data warehouse is the integrated, subject-oriented, non-volatile, time-variant store of data that feeds analytical systems. This four-part definition, from Bill Inmon's foundational work on data warehousing, is worth unpacking:

- **Integrated** — data from multiple sources is consolidated using common definitions, formats, and keys
- **Subject-oriented** — organized around business subjects (Sales, Purchasing, Inventory) rather than operational applications
- **Non-volatile** — once loaded, data is not changed or deleted (corrections are applied as new records)
- **Time-variant** — history is preserved; every record carries a timestamp or date key

The data warehouse is where ETL delivers its results. Its structure — the dimensional model — is the subject of Chapter 2.

3.4 Data Marts

A **data mart** is a subject-specific subset of the data warehouse, optimized for a particular business area or audience. While a data warehouse might contain all enterprise data, a Sales data mart contains only what the sales team needs — in a structure simplified for their specific analytical tools and questions.

In the CabotTrail environment, five data marts are derived from the central DW:

Data Mart	Business area	Key fact table
`CabotTrailOutdoorsSales`	Revenue and order analysis	`fact.Sales`
`CabotTrailOutdoorsReturns`	Product returns and refunds	`fact.Returns`
`CabotTrailOutdoorsPurchasing`	Supplier and procurement	`fact.Purchasing`
`CabotTrailOutdoorsInventory`	Stock levels and valuation	`fact.Inventory`
`CabotTrailOutdoorsTransactions`	Financial transactions	`fact.CustomerTransaction`, `fact.SupplierTransaction`

The relationship between the DW and data marts reflects a key architectural principle: **transform once, serve many**. The DW performs the expensive integration work; data marts serve specific audiences efficiently from the integrated result.

3.5 Presentation Layer

The presentation layer is where business users interact with the data. BI tools like Microsoft Power BI, Tableau, or SSRS connect to data marts and data warehouses to produce reports, dashboards, and ad-hoc analyses.

ETL developers rarely build the presentation layer — that is typically the domain of BI developers and data analysts. But ETL developers profoundly influence what is possible in the presentation layer. A well-designed dimensional model makes complex analytical questions simple to express in a BI tool. A poorly designed model forces BI developers into workarounds that produce incorrect results.

---

4. What Is a Data Warehouse?

The term "data warehouse" is used loosely in industry conversations. For the purposes of this book, we use the precise definition established by Ralph Kimball, whose dimensional modelling approach underpins the vast majority of commercial data warehouse implementations:

> *"A data warehouse is a system that extracts, cleans, conforms, and delivers source data into a dimensional data store and then supports and implements querying and analysis for the purpose of*

> *decision making."*
> *— Ralph Kimball, The Data Warehouse Toolkit, 3rd Edition*

4.1 Key Properties

Historical depth. Unlike an OLTP system that reflects the current state of the world, a data warehouse accumulates history. The CabotTrail DW retains four years of sales data (2022–2025). Analysis of trends, seasonality, and year-over-year growth is only possible because history is preserved.

Integrated definitions. When multiple source systems use different terms for the same concept — one system calls it "CustomerID", another calls it "ClientCode" — the data warehouse resolves these differences. All source identifiers are mapped to a single, consistent definition.

Subject orientation. The DW is organized around what the business cares about — Sales, Purchasing, Inventory — not around the applications that captured the data. A sale might touch the order management system, the inventory system, and the finance system in the OLTP world. In the DW, all of that is consolidated into a single Sales subject area.

Surrogate keys. OLTP systems use **natural keys** — identifiers that exist in the real world (CustomerID from the CRM, ProductCode from the ERP). Data warehouses replace these with **surrogate keys** — system-generated integers with no business meaning. This provides independence from source system changes and enables Slowly Changing Dimension tracking (covered in Chapter 4).

4.2 The Dimensional Model

The dimensional model is the dominant design pattern for data warehouses. It organizes data into two types of tables:

Fact tables record measurements of business events — a sale, a purchase, a return. They are typically very wide (many measure columns) and very long (millions of rows). They contain foreign keys to all related dimension tables.

Dimension tables provide the context for those measurements — who, what, where, when, and how. They are typically wide (many descriptive columns) and relatively short (thousands or tens of thousands of rows). They contain the surrogate primary key, the natural key from the source, and all descriptive attributes.

The resulting structure — a central fact table surrounded by dimension tables — is called a **star schema**, named for its visual resemblance to a star when drawn as an entity-relationship diagram.

```
                    dim.Calendar
                         |
                    (DateKey)
                         |
dim.Customer ──────  fact.Sales ──────  dim.Product
(CustomerKey)    (CustomerKey,    (ProductKey)
                  ProductKey,
                  EmployeeKey,
dim.Employee ──   DeliveryMethodKey) ── dim.DeliveryMethod
(EmployeeKey)
```

The star schema is simple to query, simple to explain to business users, and simple for BI tools to navigate. These properties — not theoretical elegance — explain its dominance in practice.

---

5. What Is ETL?

With the pipeline architecture understood, ETL can be defined precisely.

ETL (Extract, Transform, Load) is the process of moving data from one or more source systems to a target system, applying business rules and transformations along the way.

Each stage has a distinct purpose and distinct technical challenges.

5.1 Extract

The Extract stage reads data from source systems. It sounds simple, but it involves a set of non-trivial decisions:

What to extract. Not all source data is needed in the target. Operational fields like `LastModifiedBy` or `LockVersion` that serve the OLTP application have no analytical value and are excluded.

How to extract. The simplest approach is a full extract — read every row in every source table on every run. This is reliable and simple but expensive for large tables. Incremental extraction reads only rows that have changed since the last load, using a timestamp, a change sequence number, or Change Data Capture (CDC). Full vs incremental extraction is one of the most consequential ETL design decisions.

When to extract. Extraction must not interfere with operational workloads. Nightly batch windows exist for exactly this reason — extract during off-peak hours.

Connection and authentication. Source systems may require specific database drivers, network connectivity, firewall rules, and service account permissions. These are operational concerns that must be designed and documented.

In the CabotTrail environment, all extraction is from a single SQL Server instance using Windows Authentication. Real-world projects involve considerably more complexity.

5.2 Transform

The Transform stage is where the analytical value is created. Raw source data rarely arrives in a form that is immediately useful for analysis. Transformation applies business rules, resolves inconsistencies, derives new measures, and restructures the data for the target schema.

Transformation tasks fall into several categories:

Data cleansing. Correcting or handling dirty data — NULL values, out-of-range values, inconsistent formatting, duplicate records. Every ETL system needs a strategy for what to do when data quality rules are violated: reject the row, substitute a default value, log and continue, or stop the load entirely.

Data type conversion. Source systems use a variety of data types that may not map directly to target types. A date stored as a string in a source file must be parsed and converted to a proper DATE type in the target. A numeric code stored as NVARCHAR in the source must be cast to INT in the dimension.

Structural transformation. The source schema is normalized; the target schema is denormalized. Transformation flattens multiple normalized source tables into a single wide dimension table. This is the most complex and most valuable transformation category.

Derivation. New columns are calculated from source data. Gross profit margin is calculated from line total and unit cost. Sales territory is derived from province code. Customer tier is derived from credit limit. These derived values add analytical power that did not exist in the source.

Surrogate key lookup. Fact tables contain foreign keys to dimension tables. These keys are the DW's surrogate keys — not the source system's natural keys. Every natural key in a fact row must be looked up against its dimension to find the corresponding surrogate key. A customer identified as `CustomerID = 42` in the source becomes `CustomerKey = 7` in the DW after the lookup.

Conforming. When data comes from multiple sources, the same concept may be expressed differently. A "cancelled" status in one system might be "CNCL" in another. Transformation maps both to a single agreed-upon value in the target — this is called **conforming**, and it is one of ETL's most important functions.

5.3 Load

The Load stage writes the transformed data into the target. It is the stage that is most visible — after it completes, analysts can query the new data. But load strategy has important implications for performance, reliability, and data integrity.

Full load. Truncate the target table and reload from scratch. Simple, reliable, and produces a clean state on every run. The appropriate strategy when source data volumes are manageable and the load window is sufficient.

Incremental load. Apply only the changes since the last load — new rows are inserted, changed rows are updated, deleted rows are handled according to business rules. Faster than full load for large datasets but significantly more complex to implement correctly.

Upsert (MERGE). A hybrid approach using the SQL `MERGE` statement — rows that match are updated, rows that do not match are inserted. Covered in depth in Chapter 5.

Load order. In a dimensional model, dimension tables must be loaded before fact tables. A fact row contains foreign keys to dimension rows — those dimension rows must exist before the fact can be inserted. The dependency chain must be documented and respected.

Error handling. What happens when a row fails to load? A missing lookup key, a constraint violation, a NULL in a NOT NULL column — these failures must be anticipated and handled. Unhandled load errors cause packages to fail silently or noisily, neither of which is acceptable in production.

---

6. SSIS: ETL as a Professional Tool

The ETL concepts described above can be implemented in many ways — T-SQL scripts, Python, Azure Data Factory, Informatica, Talend, or dozens of other tools. This book uses **SQL Server Integration Services (SSIS)** as its primary implementation tool.

6.1 What SSIS Is

SSIS is Microsoft's enterprise ETL platform, included with SQL Server. It provides:

- A **visual development environment** (in SQL Server Data Tools / Visual Studio) where ETL pipelines are built as graphical workflows rather than code
- A **high-performance data movement engine** optimized for bulk operations against SQL Server and other data sources
- **Built-in transformations** for the most common ETL operations — lookups, derived columns, data type conversions, aggregations, conditional splits
- **Logging and monitoring** through the SSIS Catalog (`SSISDB`) in SQL Server
- **Scheduling integration** with SQL Server Agent for automated execution
- **Error handling** at the row level — individual rows that fail transformation can be redirected to error outputs rather than causing the entire load to fail

6.2 Why SSIS for This Book

SSIS is the dominant ETL tool in SQL Server environments, which represent a large portion of the enterprise BI market. It is well-documented, widely deployed, and deeply integrated with the SQL Server tools that students already know from previous courses.

More importantly, SSIS makes the structure of ETL **visible**. When you drag a Lookup transformation onto the Data Flow canvas and connect it between a Source and a Destination, the architecture of key resolution is physically represented. The tool teaches the concept by making it tangible.

That said, the concepts in this book — extraction strategies, transformation patterns, surrogate key management, SCD handling, testing frameworks, load order dependencies — are not SSIS-specific. They apply equally to Azure Data Factory, AWS Glue, Apache NiFi, or any other ETL platform. SSIS is the vehicle; the concepts are the destination.

6.3 SSIS Architecture

An SSIS solution is organized as follows:

```
SSIS Project (.dtproj)
    └── SSIS Packages (.dtsx)
            ├── Control Flow
            │       ├── Execute SQL Task
            │       ├── Data Flow Task ──────┐
            │       ├── Sequence Container   │
            │       └── Execute Package Task │
            │                                │
            └── Data Flow (inside Data Flow Task)
                    ├── OLE DB Source
                    ├── Transformations
                    │       ├── Lookup
                    │       ├── Derived Column
                    │       ├── Conditional Split
                    │       └── Data Conversion
                    └── OLE DB Destination
```

The Control Flow is the orchestration layer. It determines what runs, in what order, and what to do when something fails. It contains tasks and containers connected by precedence constraints (success, failure, or completion arrows).

The Data Flow is the transformation engine. It operates on a streaming pipeline — rows flow from a source through a series of transformations to a destination. Unlike the Control Flow, which runs tasks sequentially, the Data Flow processes rows in parallel through the pipeline, buffering data in memory for performance.

This separation — orchestration in the Control Flow, transformation in the Data Flow — is one of SSIS's most important architectural characteristics. It means complex orchestration logic (retries, branching, looping) and complex transformation logic (multi-step row processing) are kept cleanly separate.

---

7. The CabotTrail Outdoor Environment

Throughout this book, every concept is illustrated using the CabotTrail Outdoor database environment. This section describes that environment in full so you can refer back to it as needed.

7.1 The Business

CabotTrail Outdoor is a fictional outdoor gear retailer based in Nova Scotia, Canada. It sells products across thirteen categories — from Shelter and Sleeping to Accessories and Navigation — through a network of wholesale and retail customers across Canada.

The business has:
- **100 customers** across Canadian provinces
- **142 products** supplied by multiple vendors
- **50 employees** across sales and operations roles
- **4+ years of transaction history** (2022–2025) representing over 16,000 invoice lines

7.2 The Database Pipeline

```
CabotTrailOutdoor          OLTP source — normalized, operational
        ↓ ETL
CabotTrailOutdoorDW        Data warehouse — dimensional model
        ↓ ETL
CabotTrailOutdoorsSales    Sales data mart — star schema
CabotTrailOutdoorsReturns  Returns data mart — star schema
CabotTrailOutdoorsPurchasing  Purchasing data mart — star schema
CabotTrailOutdoorsInventory   Inventory data mart — star schema
CabotTrailOutdoorsTransactions  Transactions data mart — two facts
```

7.3 The OLTP Schema

`CabotTrailOutdoor` is organized into four schemas:

Sales schema — customer-facing transactions:
- `Customers` — 100 customers with credit limits and account dates
- `Orders` / `OrderLines` — purchase orders from customers
- `Invoices` / `InvoiceLines` — fulfilled orders with pricing and tax

- `CustomerTransactions` — financial transactions (invoices, payments, credits)
- `Return` — product returns with reasons and refund amounts

Purchasing schema — supplier-facing transactions:

- `Suppliers` — product suppliers with payment terms
- `PurchaseOrders` / `PurchaseOrderLines` — orders placed with suppliers
- `SupplierTransactions` — financial transactions with suppliers

Inventory schema — product management:

- `Products` — 142 products with pricing and weight
- `ProductCategories` — 13 product categories with `StandardCostPct` margin rates
- `ProductCategoryAssignments` — M:M junction (products can belong to multiple categories)
- `ProductHoldings` — current stock levels, reorder points, and target stock levels

Application schema — reference data:

- `People` / `Employees` — staff records
- `Cities` / `StateProvinces` / `Countries` — geography hierarchy
- `DeliveryMethods` — 12 delivery options
- `TransactionTypes` — invoice, payment, credit note classifications
- `PaymentMethods` — payment method lookup

7.4 The Data Warehouse Schema

`CabotTrailOutdoorDW` organizes the same data into a dimensional model:

Dimension schema:

Table	Source	Rows
`DimDate`	Generated	4,017
`DimCustomer`	Sales.Customers + Application.Cities/Provinces	100
`DimProduct`	Inventory.Products + Categories + Suppliers	142
`DimSupplier`	Purchasing.Suppliers + Application.Cities	varies
`DimEmployee`	Application.People + Employees	50
`DimGeography`	Application.Cities + Provinces + Countries	varies
`DimDeliveryMethod`	Application.DeliveryMethods	12
`DimTransactionType`	Application.TransactionTypes	varies
`DimPaymentMethod`	Application.PaymentMethods	varies
`DimProductCategory`	Inventory.ProductCategories	13

Fact schema:

Table	Grain	Rows
`FactSales`	One row per invoice line	16,359
`FactPurchasing`	One row per PO line	905
`FactReturns`	One row per return	500
`FactInventory`	One row per product (snapshot)	142
`FactCustomerTransactions`	One row per customer transaction	9,346
`FactSupplierTransactions`	One row per supplier transaction	240

7.5 A Note on the Data

The CabotTrail data was deliberately designed to support learning:

- **Varied margins** — product categories have different `StandardCostPct` values (45% to 70%), producing margins ranging from 30% (Shelter) to 55% (Accessories). This makes margin analysis meaningful rather than uniformly flat.
- **Growing sales trend** — order volumes increase year over year (743 orders in 2022 → 1,559 in 2025), reflecting business growth.

- **Realistic return reasons** — seven distinct return reasons with corresponding refund rates (100% for defective items, 50% for change-of-mind) make return analysis meaningful.
- **Geographic distribution** — customers are spread across Canadian provinces, enabling territory analysis.

---

8. Chapter Summary

This chapter established the conceptual foundation for ETL and business intelligence:

- **OLTP systems** are designed for operational transactions — normalized, fast, current-state. **OLAP systems** are designed for analytical queries — denormalized, historical, aggregated. They serve different purposes and must be kept separate.
- The **BI pipeline** moves data through several layers: source systems → staging area → data warehouse → data marts → presentation layer. Each layer has a specific purpose.
- A **data warehouse** is an integrated, subject-oriented, non-volatile, time-variant store of historical data organized in a **dimensional model** of fact and dimension tables.
- **ETL** bridges the OLTP and OLAP worlds in three stages: **Extract** (read from source), **Transform** (apply business rules, cleanse, derive, restructure), and **Load** (write to target).
- **SSIS** is Microsoft's professional ETL platform. It provides a visual development environment, a high-performance data movement engine, built-in transformations, logging, and scheduling integration.
- The **CabotTrail Outdoor** environment — with its OLTP source, data warehouse, and five data marts — serves as the worked example throughout this book.

---

9. Review Questions

1. Explain in your own words why a business cannot simply run analytical queries against its OLTP database. Give two specific reasons.
2. A retail company stores customer records in an OLTP database. When a customer moves to a new city, the customer record is updated. Explain what information is lost and why this matters for analytical systems.
3. What is the purpose of a staging area in the ETL pipeline? Why not load directly from the source to the data warehouse?

4. Describe the difference between a data warehouse and a data mart. In what circumstances would a business choose to maintain both?
5. A colleague suggests that ETL could be replaced simply by having analysts connect their BI tools directly to the OLTP database. What arguments would you make for and against this approach?
6. In a dimensional model, what is the difference between a fact table and a dimension table? What type of information goes in each?
7. The CabotTrail `Inventory.ProductCategoryAssignments` table is a junction table connecting products to categories in a many-to-many relationship. When this data is loaded into the data warehouse dimension `DimProduct`, how is the many-to-many relationship resolved? What design decision is made and why?
8. Why do data warehouses use surrogate keys rather than the natural keys from source systems? Give at least two reasons.

Deeper Dive

Going Further with ETL Foundations

The Inmon vs Kimball Debate

The data warehousing field has a long-running architectural debate between two foundational approaches.

Bill Inmon (often called the "father of the data warehouse") proposed a **top-down** approach: build a fully normalized, enterprise-wide data warehouse first, then derive data marts from it. Inmon's data warehouse is in 3NF — it looks like a very large, well-organized OLTP database. Data marts are then built for specific subject areas. This approach prioritizes data integrity and a single source of truth across the enterprise.

Ralph Kimball proposed a **bottom-up** approach: build dimensional data marts for individual subject areas first. Each mart is a star schema optimized for its subject. The enterprise data warehouse emerges as the union of conforming dimensions and facts across all the marts. This approach prioritizes speed to delivery and analytical usability.

In practice, most modern implementations are hybrids. The **CabotTrail architecture** most closely resembles the Kimball approach — a central DW in dimensional form, with subject-specific data marts derived from it. The DW serves as an integration layer; the marts serve as the analytical layer.

For a thorough treatment of both approaches and their trade-offs, see:
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional*

Modeling (3rd ed.). Wiley.
- Inmon, W. H. (2005). *Building the Data Warehouse* (4th ed.). Wiley.

ELT: The Cloud-Era Inversion

A note on terminology that causes confusion: modern cloud data platforms have popularized **ELT** (Extract, Load, Transform) as an alternative pattern. In ELT:

1. Raw data is extracted from sources and loaded directly into the target platform (e.g., Snowflake, BigQuery, Azure Synapse) with minimal or no transformation
2. Transformation occurs *inside* the target platform using SQL, dbt, or similar tools

ELT is made practical by the massive parallel processing power of modern cloud data warehouses, which can transform data at scale in SQL far faster than ETL tools could transform it in transit. Tools like **dbt (Data Build Tool)** have formalized this approach.

The conceptual stages — extract, transform, load — remain the same. What changes is *where* transformation happens. The principles of dimensional modelling, data quality, and governance discussed in this book apply equally to ELT architectures.

Change Data Capture (CDC)

The extraction strategies described in this chapter (full extract vs incremental by timestamp) have a third, more sophisticated option: **Change Data Capture (CDC)**.

CDC works by reading the database transaction log — the record of every INSERT, UPDATE, and DELETE that has occurred. Rather than querying the source tables and comparing values to detect changes, CDC reads the log directly and extracts exactly what changed, with the type of change (insert/update/delete) recorded.

SQL Server has native CDC support:

```
-- Enable CDC on a database
EXEC sys.sp_cdc_enable_db;

-- Enable CDC on a specific table
EXEC sys.sp_cdc_enable_table
    @source_schema = 'Sales',
    @source_name   = 'Orders',
    @role_name     = NULL;
```

CDC provides several advantages over timestamp-based incremental extraction:
- Captures deletes (timestamps cannot detect deleted rows)
- Captures multiple changes to the same row within a single extraction window

- Does not require a `LastModifiedDate` column on source tables
- Extremely low impact on source system performance

CDC is an advanced topic beyond the scope of this book's main chapters, but it is worth knowing it exists. Microsoft's documentation provides a thorough introduction: About Change Data Capture — SQL Server

---

Industry Perspectives

Kimball on the Purpose of the Data Warehouse

Ralph Kimball's most concise statement of purpose for the data warehouse is worth committing to memory:

> *"The data warehouse is the foundation for business intelligence. It is the single authoritative source of truth for the enterprise. It must be designed with the end in mind — the business questions that need to be answered."*

This "end in mind" philosophy — starting with the business question and working backward to the data structure — is a thread that runs through all of Kimball's work. It is the reason dimensional modelling produces schemas that business analysts find intuitive: they were designed for the questions analysts ask, not for the convenience of the database administrator.

The Kimball Group's online resource library is freely available and contains decades of articles, techniques, and worked examples: Kimball Techniques — Kimball Group

Microsoft on SSIS Architecture

Microsoft's official documentation for SSIS provides deep technical coverage of the execution engine, buffer management, and performance tuning:
- SQL Server Integration Services — Overview
- Integration Services Data Flow
- SSIS Catalog (SSISDB)

On the Four Properties of a Data Warehouse

Bill Inmon's original four properties — integrated, subject-oriented, non-volatile, time-variant — were first published in:
- Inmon, W. H. (1992). *Building the Data Warehouse*. QED Technical Publishing Group.

These properties have stood for over thirty years as the clearest statement of what distinguishes a data warehouse from other types of databases. They are worth revisiting periodically as the field evolves.

References and Further Reading

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley. — The foundational text for dimensional modelling. Chapters 1–3 cover the concepts introduced in this chapter.
2. Inmon, W. H. (2005). *Building the Data Warehouse* (4th ed.). Wiley. — The original text on data warehouse architecture from the enterprise perspective.
3. Microsoft. (2024). *SQL Server Integration Services documentation*. https://learn.microsoft.com/en-us/sql/integration-services/
4. Microsoft. (2024). *What is a data warehouse?* Azure documentation. https://learn.microsoft.com/en-us/azure/synapse-analytics/sql-data-warehouse/sql-data-warehouse-overview-what-is
5. Kimball Group. (n.d.). *Kimball Techniques*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/
6. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media. — Chapter 3 covers storage engines and the distinction between OLTP and OLAP workloads from a systems perspective.
7. Redmond, E., & Wilson, J. R. (2012). *Seven Databases in Seven Weeks*. Pragmatic Bookshelf. — Provides comparative context for understanding why different database architectures exist.
8. Microsoft. (2024). *About Change Data Capture (SQL Server)*. https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/about-change-data-capture-sql-server

Chapter 2: Dimensional Modelling — Stars, Schemas, and the Language of Analytics

> ***ETL for Business Intelligence***
> *A practical guide to data provisioning, dimensional modelling, and pipeline design*
>
>

Chapter Overview

Chapter 1 established *why* data warehouses exist and *what* ETL does. This chapter goes deeper into the structure of the data warehouse itself — the dimensional model.

Dimensional modelling is the design language of business intelligence. It is how ETL developers, BI developers, and business analysts communicate about data. A well-designed dimensional model makes complex analytical questions simple to express, fast to execute, and easy for non-technical users to understand. A poorly designed one forces workarounds that produce incorrect results and erode trust in the data.

This chapter covers dimensional modelling from its theoretical foundations through its practical application in the CabotTrail data warehouse. It is one of the most important chapters in this book — the concepts here underpin every ETL design decision that follows.

By the end of this chapter you will be able to:

- Define and apply the core components of a dimensional model: facts, dimensions, and keys
- Describe the star schema and explain why it is the dominant pattern for analytical databases
- Explain the grain of a fact table and describe why grain definition is the most important design decision in dimensional modelling
- Distinguish between additive, semi-additive, and non-additive measures and explain the implications for aggregation
- Identify and describe the major dimension types: conformed, role-playing, degenerate, junk, and slowly changing

- Explain the difference between a star schema and a snowflake schema and describe when each is appropriate
- Apply dimensional modelling concepts to the CabotTrail data warehouse

---

Table of Contents

---

1. The Philosophy of Dimensional Modelling

Dimensional modelling was developed by Ralph Kimball in the 1990s as a direct response to a practical problem: data warehouses built using normalized (3NF) structures were correct but unusable. Analysts struggled with complex joins. Queries were slow. Business users could not understand the schema. Reports took months to build.

Kimball's insight was that **analytical queries have a consistent structure** that can be anticipated and designed for. Every business question has two parts:

1. **A measurement** — what happened? (revenue, quantity, cost, count)
2. **A context** — when, where, by whom, and for what? (date, geography, customer, product)

This structure maps directly to the two components of a dimensional model:

- **Fact tables** — the measurements
- **Dimension tables** — the context

The resulting design is optimized not for storage efficiency or update performance (those are OLTP priorities) but for **query simplicity, query speed, and business user comprehension**. These three goals are explicitly stated by Kimball as the success criteria for dimensional modelling.

> *"The goal of the data warehouse is to provide the best possible query and analysis performance on the most important business data, presented in the most understandable format."*
> *— Ralph Kimball, The Data Warehouse Toolkit, 3rd Edition*

This philosophy has a practical implication: **if business users cannot understand the schema, it is a design failure, regardless of how technically correct it is.** Dimensional modelling sacrifices normalization in service of comprehension. That is not a bug — it is the point.

---

2. Facts: Measuring the Business

A **fact table** records measurements of business events. Each row in a fact table represents one occurrence of a measurable business event at a specific grain (defined in section 2.2).

2.1 Fact Table Structure

A fact table contains two types of columns:

1. **Foreign keys** to dimension tables — these provide the context for each measurement
2. **Measure columns** — the numeric values being measured

```
-- CabotTrail: Fact.FactSales structure
SELECT  TOP 3
        -- Foreign keys (context)
        OrderDateKey,       -- → DimDate
        CustomerKey,        -- → DimCustomer
        ProductKey,         -- → DimProduct
        EmployeeKey,        -- → DimEmployee
        DeliveryMethodKey,  -- → DimDeliveryMethod

        -- Degenerate dimensions (no lookup table)
        InvoiceID,
        OrderID,
        OrderLineID,

        -- Measures
        OrderedQuantity,
        PickedQuantity,
        UnitPrice,
        TaxRate,
        LineTotal,
        TaxAmount,
        LineTotalIncludingTax,
        UnitCost,
        GrossProfit,
        GrossProfitMarginPct
FROM    CabotTrailOutdoorDW.Fact.FactSales;
```

Fact tables tend to be **wide** (many columns) and **deep** (millions of rows). They are the largest tables in the data warehouse by row count and are the primary target of analytical queries.

2.2 The Grain: The Most Important Design Decision

The **grain** of a fact table defines exactly what one row represents. It is the single most important design decision in dimensional modelling. Getting the grain wrong produces fact tables that either double-count measures or fail to answer important questions.

Kimball states this directly:

> *"Declaring the grain is the pivotal step in the design. The grain must be stated before choosing dimensions or facts. The grain establishes what one row of the fact table represents. You cannot go further until you have clearly defined the grain."*
> *— Ralph Kimball, The Data Warehouse Toolkit, 3rd Edition*

The grain must be stated in business terms, not technical terms:

Bad grain statement	Good grain statement
"One row per invoice line"	"One row per product on each customer invoice"
"One row per transaction"	"One row per financial transaction between CabotTrail and a customer"
"One row per snapshot"	"One row per product as of the most recent stocktake date"

For `Fact.FactSales` in the CabotTrail DW:

> ***Grain:*** *One row per product on each customer invoice — representing the quantity ordered, quantity picked, unit price, line total, tax, and gross profit for a single product on a single invoice.*

This grain definition determines:
- Which dimensions are present (every dimension that can vary at the invoice-line level)
- Which measures are included (only measures that are meaningful at the invoice-line level)
- What cannot be answered from this table alone (anything requiring cross-invoice aggregation that is better answered from a separate aggregate fact)

2.3 Types of Measures

Not all measures behave the same way when aggregated. Understanding measure types is essential for designing fact tables that produce correct results.

Additive Measures

An **additive measure** can be summed meaningfully across all dimensions. It is the simplest and most common type.

`LineTotal` is additive — you can sum it across time (total revenue for the year), across customers (total revenue per customer), across products (total revenue per product), or across all dimensions at once (total revenue company-wide). Every aggregation produces a correct and meaningful result.

```
-- Additive: summing LineTotal across any dimension is valid
SELECT  SUM(LineTotal)  AS TotalRevenue  FROM Fact.FactSales;                          -
- Company total
SELECT  CustomerKey,    SUM(LineTotal)   FROM Fact.FactSales GROUP BY CustomerKey;     -
- Per customer
SELECT  OrderDateKey,   SUM(LineTotal)   FROM Fact.FactSales GROUP BY OrderDateKey;    -
- Per day
```

Other additive measures in `Fact.FactSales`: `TaxAmount`, `GrossProfit`, `OrderedQuantity`, `PickedQuantity`, `LineTotalIncludingTax`, `UnitCost`.

Semi-Additive Measures

A **semi-additive measure** can be summed across some dimensions but not others. The classic example is any balance or stock level — it makes sense to sum across locations or products, but summing across time produces a meaningless result.

`QuantityOnHand` in `Fact.FactInventory` is semi-additive. The total stock across all products at a point in time is meaningful. But summing stock levels across different snapshot dates would double-count inventory that existed at multiple points — a product with 50 units on January 1 and 50 units on February 1 does not mean 100 units were available.

```
-- Semi-additive: summing across products is valid
SELECT  SUM(QuantityOnHand) AS TotalUnitsOnHand FROM Fact.FactInventory;  -- Valid

-- Semi-additive: summing across snapshots is WRONG if multiple snapshots exist
-- (CabotTrail has a single snapshot, so this is currently safe — but design must
anticipate multiple)
SELECT  SnapshotDateKey, SUM(QuantityOnHand) FROM Fact.FactInventory
GROUP BY SnapshotDateKey;
-- Correct for a single date; misleading if rows for multiple dates are present
```

The correct aggregation for semi-additive measures across time is typically `MAX`, `MIN`, `AVG`, or the last value — not `SUM`.

Non-Additive Measures

A **non-additive measure** cannot be summed meaningfully across any dimension. Ratios, percentages, and rates fall into this category.

`GrossProfitMarginPct` in `Fact.FactSales` is non-additive. You cannot sum margin percentages across products or customers to get a meaningful total — adding 45% and 33% does not produce a company-wide margin of 78%. The correct aggregation is a weighted average, computed from the underlying additive measures:

```
-- NON-ADDITIVE: do NOT do this
SELECT SUM(GrossProfitMarginPct) AS WrongMargin FROM Fact.FactSales;

-- CORRECT: compute from additive measures
SELECT
    ROUND(SUM(GrossProfit) / NULLIF(SUM(LineTotal), 0) * 100, 2) AS CorrectMarginPct
FROM Fact.FactSales;
```

> ***Design principle:*** *Store non-additive measures in fact tables for convenience, but always document them as non-additive and provide guidance on correct aggregation. Many BI tools will blindly sum a percentage column if told it is a measure — this produces dramatically wrong results.*

Factless Facts

A special case: a **factless fact table** records the occurrence of an event with no numeric measures at all. The fact is the occurrence itself.

Examples:
- Student attendance (StudentKey, CourseKey, DateKey — no numeric measure)
- Product promotions (ProductKey, PromotionKey, DateKey — records that a product was on promotion, with no sales amount)
- Website page views (UserKey, PageKey, DateKey, SessionKey)

Factless facts are less common than measure facts but important to recognize. They are used to answer questions like "which products were promoted but had no sales during the promotion period?" — which requires a join between a factless fact and a sales fact.

---

3. Dimensions: Providing Context

A **dimension table** provides the context for fact table measurements. Every dimension answers one of the classic analytical questions: *who, what, when, where, why, or how.*

3.1 Dimension Table Structure

A dimension table contains:

1. **A surrogate key** — the primary key, system-generated, with no business meaning (covered in section 4)
2. **A natural key** — the source system's identifier for the entity (e.g., `CustomerID` from the OLTP)
3. **Descriptive attributes** — all the textual and categorical information that gives the dimension its analytical value

```
-- CabotTrail: Dimension.DimProduct structure
SELECT  TOP 3
        ProductKey,             -- Surrogate key (PK)
        ProductID,              -- Natural key (from OLTP)
        ProductCode,            -- Natural identifier
        ProductName,            -- Descriptive attribute
        ProductCategoryKey,     -- FK to DimProductCategory (snowflake)
        ColorName,              -- Descriptive attribute
        PackageTypeName,        -- Descriptive attribute
        UnitPrice,              -- Reference measure (not in fact table)
        RecommendedRetailPrice, -- Reference measure
        TypicalWeightPerUnit,   -- Descriptive attribute
        IsDiscontinued,         -- Status flag
        SupplierKey             -- FK to DimSupplier (snowflake)
FROM    CabotTrailOutdoorDW.Dimension.DimProduct;
```

3.2 The Wide Dimension

Dimension tables should be **wide** — they should contain as many descriptive attributes as possible. The goal is to allow analysts to slice and filter data without requiring joins to other tables.

A narrow dimension forces analysts to join to lookup tables to get the attributes they need. This is precisely the complexity that dimensional modelling is designed to eliminate.

Consider the difference:

Narrow DimProduct (poor design):

```
ProductKey | ProductID | ProductName | ProductCategoryID | SupplierID
```

To filter by category name or supplier name, an analyst must join to `DimProductCategory` and `DimSupplier`.

Wide DimProduct (good design — CabotTrail datamart):

```
ProductKey | ProductID | ProductName | CategoryName | SupplierName | SupplierCountry
| ...
```

All filtering can be done from a single table.

The CabotTrail data marts demonstrate this principle. In the `CabotTrailOutdoorsSales` datamart, `dim.Product` flattens the DW's `DimProduct`, `DimProductCategory`, and `DimSupplier` into a single wide dimension. An analyst can filter by `CategoryName`, `SupplierName`, or `SupplierCountry` without any joins:

```
-- Wide dimension: category and supplier in one table
USE CabotTrailOutdoorsSales;

SELECT  ProductName,
        CategoryName,       -- From DimProductCategory in DW
        SupplierName,       -- From DimSupplier in DW
        SupplierCountry,    -- From DimGeography via DimSupplier in DW
        UnitPrice
FROM    dim.Product
WHERE   CategoryName = 'Sleeping'
ORDER BY UnitPrice DESC;
```

3.3 Hierarchies Within Dimensions

Dimensions often contain **hierarchies** — structured levels of granularity that allow drill-down analysis.

A **balanced hierarchy** has the same number of levels in every branch:

```
Calendar hierarchy (balanced):
Year → Quarter → Month → Day

Geography hierarchy (balanced):
Country → Province → City
```

A **ragged (unbalanced) hierarchy** has different depths in different branches:

```
Organization hierarchy (ragged):
CEO → VP → Director → Manager → Employee
(some managers report directly to VPs, skipping Director)
```

Hierarchies in dimension tables are implemented as multiple columns, one per level:

```
-- Calendar hierarchy in DimDate
SELECT  YearNumber,     -- Year level
        QuarterNumber,  -- Quarter level
        MonthNumber,    -- Month level
        FullDate        -- Day level
FROM    CabotTrailOutdoorDW.Dimension.DimDate
WHERE   YearNumber = 2024
ORDER BY FullDate;
```

BI tools like Power BI and Tableau can automatically detect and navigate hierarchies when the column relationships are defined correctly in the data model.

---

4. Keys: Natural, Surrogate, and Beyond

Key management is one of the most technically important aspects of dimensional modelling, and one of the most commonly misunderstood by developers coming from an OLTP background.

4.1 Natural Keys

A **natural key** is an identifier that exists in the real world or in a source system — `CustomerID`, `ProductCode`, `InvoiceNumber`. Natural keys have business meaning. They are the identifiers that operational users know and recognize.

Natural keys have important limitations in data warehouses:

They can change. A supplier acquires another company and merges product lines — `ProductCode` values are reassigned. A customer management system is replaced and `CustomerID` sequences are restarted. When natural keys change, all historical fact rows that reference the old key are invalidated.

They can be reused. A product is discontinued and its code is reassigned to a new product. Historical sales data now appears to be for the new product — a silent, catastrophic data quality failure.

They are not unique across sources. When data comes from multiple source systems, `CustomerID = 42` in System A and `CustomerID = 42` in System B are different customers. Natural keys from different sources cannot coexist in a single dimension without collision.

They encode business logic. Some natural keys carry meaning: `PROD-NS-042` might encode region, category, and sequence. If the encoding scheme changes, historical data becomes inconsistent.

4.2 Surrogate Keys

A **surrogate key** is a system-generated integer that has no business meaning. It is created by the data warehouse ETL process and exists only within the DW. It solves every limitation of natural keys:

- It never changes — once assigned to a dimension row, it is permanent
- It is never reused — IDENTITY columns in SQL Server guarantee this
- It is unique across all sources — generated by the DW, not the source
- It encodes no business logic — it is just a number

```
-- Surrogate keys in DimCustomer
SELECT
    CustomerKey,    -- Surrogate key (DW-generated, no business meaning)
    CustomerID,     -- Natural key (from CabotTrailOutdoor.Sales.Customers)
    CustomerName
FROM    CabotTrailOutdoorDW.Dimension.DimCustomer
WHERE   CustomerID <> 0   -- Exclude unknown member
ORDER BY CustomerKey;
```

Both keys are preserved in the dimension table. `CustomerKey` is used for all joins within the DW (foreign keys in fact tables always reference the surrogate key). `CustomerID` is preserved so that analysts can trace a DW record back to the source system record.

4.3 The Unknown Member

Every well-designed dimension table contains a special row called the **unknown member** (also called the default member or error member). This row has a surrogate key of 0 (or -1) and represents "unknown" or "not applicable."

```
-- The unknown member in DimCustomer
SELECT  CustomerKey, CustomerID, CustomerName
FROM    CabotTrailOutdoorDW.Dimension.DimCustomer
WHERE   CustomerID = 0;
-- Returns: CustomerKey=0, CustomerID=0, CustomerName='Unknown'
```

The unknown member serves as the default FK target for fact rows where the dimension value is genuinely unknown or not applicable. Without it, a fact row with a NULL foreign key would violate referential integrity. With it, any fact row that cannot be resolved to a real dimension member can still be loaded — pointing to the unknown member — without breaking the data model.

> ***Common mistake:*** *Many developers skip the unknown member and use NULL foreign keys instead. This creates problems for BI tools, which often cannot handle NULL FK values correctly, and for SQL queries, where* `WHERE CustomerKey = NULL` *never returns rows (NULL comparisons require* `IS NULL` *).*

4.4 Composite Keys in Dimensions

Dimension tables always use a single surrogate key as their primary key. This is non-negotiable in Kimball's approach.

In OLTP systems, some entities are identified by composite keys — `(OrderID, OrderLineID)`, for example. In dimensional modelling, these composite identifiers are handled differently:

- If the entity becomes a dimension, assign a surrogate key and store the composite natural key as two separate columns
- If the identifier belongs to a fact (an order line), it becomes a **degenerate dimension** — stored on the fact table as a regular column, with no corresponding dimension table (covered in section 7.4)

---

5. The Star Schema

The **star schema** is the foundational pattern of dimensional modelling. It consists of one or more fact tables at the centre, each surrounded by its dimension tables. When drawn as an entity-relationship diagram, the pattern resembles a star.

5.1 Structure

5.2 Why the Star Schema Works

The star schema's performance and usability advantages stem from two structural properties:

Single-level joins. Every fact row connects directly to its dimension rows via a single join. To answer "what was total revenue by product category and customer territory?", a BI tool or SQL query needs only two joins — from the fact to `dim.Product` (for `CategoryName`) and from the fact to `dim.Customer` (for `SalesTerritory`). There are no intermediate tables, no chained joins.

```
-- Star schema: two joins answer a multi-dimensional question
USE CabotTrailOutdoorsSales;

SELECT
    prod.CategoryName,
    cust.SalesTerritory,
    SUM(fs.LineTotal)   AS Revenue
FROM    fact.Sales fs
INNER JOIN dim.Product prod  ON prod.ProductKey  = fs.ProductKey
INNER JOIN dim.Customer cust ON cust.CustomerKey = fs.CustomerKey
GROUP BY prod.CategoryName, cust.SalesTerritory
ORDER BY prod.CategoryName, Revenue DESC;
```

Predictable query patterns. BI tools like Power BI and Tableau are optimized for star schemas. They can automatically generate correct aggregation queries because the schema structure is predictable — facts join to dimensions; dimensions provide attributes for grouping and filtering. A normalized schema requires the BI tool to understand complex join chains, which most cannot do reliably.

5.3 Star Schema Design Rules

Kimball defines a set of design rules for star schemas that are worth making explicit:

1. **Every dimension table has a single surrogate primary key.** Never composite PKs in dimensions.
2. **Every fact table has a foreign key to every relevant dimension.** If the fact event involves a customer, a product, a date, and an employee, all four foreign keys must be present.
3. **Dimension tables are denormalized.** All attributes of an entity are stored on the dimension row, not in separate lookup tables (with some exceptions — see snowflake schema).
4. **Fact tables contain only foreign keys and measures.** Textual descriptions belong in dimensions.
5. **Every dimension is at the same grain as the fact.** A dimension that varies at a finer grain than the fact cannot be correctly joined.

5.4 The CabotTrail Sales Star Schema

The `CabotTrailOutdoorsSales` data mart is a textbook star schema:

```
-- Verify the star schema structure
USE CabotTrailOutdoorsSales;

SELECT
    fk.name                 AS ForeignKey,
    tp.name                 AS FactTable,
    tr.name                 AS DimensionTable,
    cp.name                 AS FactColumn,
    cr.name                 AS DimensionColumn
FROM    sys.foreign_keys fk
INNER JOIN sys.foreign_key_columns fkc ON fkc.constraint_object_id = fk.object_id
INNER JOIN sys.tables tp  ON tp.object_id  = fkc.parent_object_id
INNER JOIN sys.tables tr  ON tr.object_id  = fkc.referenced_object_id
INNER JOIN sys.columns cp ON cp.object_id  = fkc.parent_object_id
                         AND cp.column_id  = fkc.parent_column_id
INNER JOIN sys.columns cr ON cr.object_id  = fkc.referenced_object_id
                         AND cr.column_id  = fkc.referenced_column_id
ORDER BY tp.name, fk.name;
```

The result confirms five FK relationships from `fact.Sales` to five dimension tables — a classic five-pointed star.

---

6. The Snowflake Schema

A **snowflake schema** extends the star schema by normalizing one or more dimension tables — breaking out repeated values into separate lookup tables connected by foreign keys. The result, when drawn, resembles a snowflake rather than a star.

6.1 Structure

```
                DimProductCategory (CategoryKey PK)
                        │
                   CategoryKey (FK)
                        │
DimSupplier ── SupplierKey (FK) ── DimProduct ── ProductKey (FK) ── Fact.FactSales
```

In this snowflake arrangement, `DimProduct` does not store `CategoryName` directly. Instead it stores `ProductCategoryKey` — a FK to `DimProductCategory`. To get the category name, a query must join `DimProduct` to `DimProductCategory`.

6.2 Snowflake in the CabotTrail DW

The `CabotTrailOutdoorDW` uses a snowflake schema. `DimProduct` references both `DimProductCategory` and `DimSupplier`:

```
-- Snowflake: DimProduct references two other dimension tables
USE CabotTrailOutdoorDW;

SELECT
    p.ProductName,
    pc.CategoryName,        -- Requires join to DimProductCategory
    s.SupplierName,         -- Requires join to DimSupplier
    p.UnitPrice
FROM    Dimension.DimProduct p
INNER JOIN Dimension.DimProductCategory pc
    ON pc.ProductCategoryKey = p.ProductCategoryKey
INNER JOIN Dimension.DimSupplier s
    ON s.SupplierKey = p.SupplierKey
WHERE   p.IsDiscontinued = 0
ORDER BY pc.CategoryName, p.ProductName;
```

Compare this to the equivalent query against the star schema data mart:

```
-- Star schema: no joins needed for category and supplier
USE CabotTrailOutdoorsSales;

SELECT  ProductName, CategoryName, SupplierName, UnitPrice
FROM    dim.Product
WHERE   IsDiscontinued = 0
ORDER BY CategoryName, ProductName;
```

6.3 Star vs Snowflake: When to Use Each

The choice between star and snowflake is one of the most debated topics in dimensional modelling. The practical considerations are:

Factor	Star Schema	Snowflake Schema
Query simplicity	Fewer joins — simpler queries	More joins — more complex queries
BI tool compatibility	Optimal — designed for star	Requires additional relationship definitions
Storage	Redundant text stored per dimension row	Text stored once in lookup tables
ETL complexity	Simpler — load one flat table	More complex — load in dependency order
Maintainability	Changing a category name requires updating many rows	Change category name in one lookup row
Query performance	Fewer joins = faster at query time	More joins = slower at query time

Kimball's recommendation is unambiguous: use the star schema for the presentation layer (data marts). The redundancy is intentional and acceptable. The query simplicity and BI tool compatibility outweigh the storage cost.

The snowflake is appropriate at the **data warehouse integration layer** — where data quality and referential integrity are priorities, and where analysts are not querying directly. This is exactly how CabotTrail uses it: the DW uses a snowflake for integrity; the data marts denormalize it into stars for consumption.

> ***Key principle:*** *Snowflake at the integration layer; star at the presentation layer. ETL flattens the snowflake into a star as part of the data mart load process.*

---

7. Dimension Types

Not all dimensions are alike. Kimball describes a taxonomy of dimension types, each with specific design patterns. Understanding these types is essential for building ETL processes that handle them correctly.

7.1 Conformed Dimensions

A **conformed dimension** is a dimension that is shared across multiple fact tables or data marts, using identical surrogate keys, attribute names, and attribute values. It enables meaningful cross-fact analysis.

The `DimDate` dimension in `CabotTrailOutdoorDW` is a conformed dimension. `Fact.FactSales`, `Fact.FactPurchasing`, `Fact.FactReturns`, and `Fact.FactCustomerTransactions` all use `DimDate`

with the same `DateKey`. This means an analyst can write a query that joins sales and returns through their shared date dimension to compare revenue and refunds by month — a query that would be impossible if each fact used its own date structure.

```
-- Conformed dimension enables cross-fact analysis
USE CabotTrailOutdoorDW;

SELECT
    d.YearNumber,
    d.MonthNumber,
    d.MonthName,
    SUM(fs.LineTotal)       AS Revenue,
    SUM(fr.RefundAmount)    AS Refunds,
    ROUND(SUM(fr.RefundAmount) / NULLIF(SUM(fs.LineTotal), 0) * 100, 2) AS RefundRatePct
FROM    Dimension.DimDate d
LEFT JOIN Fact.FactSales fs    ON fs.OrderDateKey  = d.DateKey
LEFT JOIN Fact.FactReturns fr  ON fr.ReturnDateKey = d.DateKey
WHERE   d.YearNumber BETWEEN 2022 AND 2025
GROUP BY d.YearNumber, d.MonthNumber, d.MonthName
ORDER BY d.YearNumber, d.MonthNumber;
```

> *"Conforming dimensions are the glue that holds the enterprise data warehouse together."*
> *— Ralph Kimball, The Data Warehouse Toolkit, 3rd Edition*

The ETL implication is important: conformed dimensions must be loaded once, centrally, before any fact tables that depend on them. They cannot be loaded independently by each fact's ETL process, or they will diverge.

7.2 Role-Playing Dimensions

A **role-playing dimension** is a single physical dimension table that is referenced multiple times by the same fact table, each time in a different role.

The date dimension is the most common example. `Fact.FactSales` has three date keys:

- `OrderDateKey` — the date the order was placed
- `InvoiceDateKey` — the date the invoice was issued
- `DueDateKey` — the payment due date

All three reference `DimDate`, but each plays a different business role. In SQL queries, this is handled with multiple joins using different aliases:

```
-- Role-playing dimension: DimDate used three times
USE CabotTrailOutdoorDW;

SELECT
    order_date.FullDate     AS OrderDate,
    invoice_date.FullDate   AS InvoiceDate,
    due_date.FullDate       AS DueDate,
    fs.LineTotal
FROM    Fact.FactSales fs
INNER JOIN Dimension.DimDate order_date
    ON order_date.DateKey   = fs.OrderDateKey
INNER JOIN Dimension.DimDate invoice_date
    ON invoice_date.DateKey = fs.InvoiceDateKey
INNER JOIN Dimension.DimDate due_date
    ON due_date.DateKey     = fs.DueDateKey
WHERE   order_date.YearNumber = 2024
ORDER BY order_date.FullDate;
```

In BI tools, role-playing dimensions often require creating **views** — one view per role — so that the tool can present each role as a separate dimension with an appropriate name:

```
-- Views for role-playing dimension roles
CREATE VIEW Dimension.DimOrderDate    AS SELECT * FROM Dimension.DimDate;
CREATE VIEW Dimension.DimInvoiceDate  AS SELECT * FROM Dimension.DimDate;
CREATE VIEW Dimension.DimDueDate      AS SELECT * FROM Dimension.DimDate;
```

7.3 Slowly Changing Dimensions (SCD)

A **slowly changing dimension (SCD)** is a dimension whose attribute values change over time — not every transaction, but occasionally. How to handle these changes is one of the most important design decisions in dimensional modelling.

The change handling strategy determines whether historical fact rows continue to reflect the attribute values that were true *at the time of the event*, or whether they reflect the current values regardless of history.

SCD Type 1: Overwrite

The simplest approach — just update the attribute value in the existing dimension row. No history is preserved.

```
Before change:
CustomerKey=15 | CustomerID=42 | CustomerName='Fundy Bay Outfitters' | ProvinceCode='NS'

After customer moves to Ontario, Type 1 update:
CustomerKey=15 | CustomerID=42 | CustomerName='Fundy Bay Outfitters' | ProvinceCode='ON'
```

Historical fact rows for this customer now reflect `ProvinceCode='ON'` — even for sales that occurred when the customer was in Nova Scotia. The history is gone.

Use Type 1 when: the old value was wrong (data correction), or history genuinely does not matter for this attribute.

SCD Type 2: Add a New Row

The most powerful and most common approach — insert a new dimension row for each change, marking the old row as expired and the new row as current.

```
Before change:
CustomerKey=15 | CustomerID=42 | ProvinceCode='NS' | ValidFrom='2022-01-01' |
ValidTo='9999-12-31' | IsCurrent=1

After change (Type 2):
CustomerKey=15 | CustomerID=42 | ProvinceCode='NS' | ValidFrom='2022-01-01' |
ValidTo='2024-06-30' | IsCurrent=0
CustomerKey=87 | CustomerID=42 | ProvinceCode='ON' | ValidFrom='2024-07-01' |
ValidTo='9999-12-31' | IsCurrent=1
```

The old surrogate key (15) is preserved. Historical fact rows continue to join to `CustomerKey=15`, which correctly shows `ProvinceCode='NS'`. New fact rows join to `CustomerKey=87`, correctly showing `ProvinceCode='ON'`.

Use Type 2 when: analysts need to see attribute values as they were at the time of the event. Customer location, product category, and employee department are common Type 2 candidates.

```
-- Query that correctly respects SCD Type 2 history
-- All-time revenue by the province the customer was in AT THE TIME OF SALE
SELECT
    dc.ProvinceCode,
    SUM(fs.LineTotal) AS Revenue
FROM    Fact.FactSales fs
INNER JOIN Dimension.DimCustomer dc ON dc.CustomerKey = fs.CustomerKey
GROUP BY dc.ProvinceCode
ORDER BY Revenue DESC;
-- Each fact row joins to the customer row that was current at load time
-- Historical province values are preserved correctly
```

SCD Type 3: Add a Column

A limited approach — add a `PreviousValue` column alongside the current value. Only the most recent prior value is retained.

```
CustomerKey=15 | CustomerID=42 | ProvinceCode='ON' | PreviousProvinceCode='NS'
```

Use Type 3 when: only the immediate before-and-after of the most recent change matters, and multiple historical changes are not needed. This is uncommon in practice.

SCD Type 6: Hybrid (1+2+3)

An advanced pattern that combines Types 1, 2, and 3 into a single implementation — named "Type 6" because 1 + 2 + 3 = 6. It maintains full row history (Type 2) while also storing the current value on all rows (Type 1) and the immediately prior value (Type 3). This allows queries to filter by current value without needing to know which rows are current.

SCD Type 6 is covered in the Deeper Dive section of this chapter.

7.4 Degenerate Dimensions

A **degenerate dimension** is a dimension that has no corresponding dimension table. The dimensional value is stored directly on the fact row as a regular column.

The classic example is an invoice number or order number. `InvoiceID`, `OrderID`, and `OrderLineID` in `Fact.FactSales` are degenerate dimensions. They are identifiers — they provide context and allow drill-through to source system details — but they are not analyzed in their own right. There is no `DimInvoice` table with attributes about the invoice, because the invoice's attributes are already fully represented by the other dimensions (customer, date, employee, delivery method) and measures.

```
-- Degenerate dimensions used for drill-through (finding the original transaction)
SELECT
    fs.InvoiceID,       -- Degenerate dimension: drill back to source
    fs.OrderID,         -- Degenerate dimension
    cust.CustomerName,
    fs.OrderDate,
    fs.LineTotal
FROM    CabotTrailOutdoorsSales.fact.Sales fs
INNER JOIN CabotTrailOutdoorsSales.dim.Customer cust
    ON cust.CustomerKey = fs.CustomerKey
WHERE   fs.InvoiceID = 10042;
```

Degenerate dimensions are very common in transaction-level fact tables. The rule of thumb: if an identifier has no descriptive attributes of its own that analysts would want to filter or group by, make it a degenerate dimension.

7.5 Junk Dimensions

A **junk dimension** is a dimension that consolidates a collection of low-cardinality flags, indicators, and codes that do not belong to any other dimension.

Consider a transaction that has the following attributes:

- `IsOnline` (Y/N)
- `IsGift` (Y/N)
- `IsRush` (Y/N)
- `PromotionCode` (5 values)

Each of these has too few values to justify its own dimension table and too little analytical importance to put in `DimCustomer` or `DimProduct`. Adding them directly to the fact table as columns works but creates a cluttered fact table with many low-value columns.

A junk dimension collects them:

```
-- Junk dimension example (not in CabotTrail, but illustrative)
CREATE TABLE Dimension.DimTransactionFlags
(
    TransactionFlagsKey INT NOT NULL IDENTITY(1,1),
    IsOnline            BIT NOT NULL,
    IsGift              BIT NOT NULL,
    IsRush              BIT NOT NULL,
    PromotionCode       NVARCHAR(10) NULL,
    CONSTRAINT PK_DimTransactionFlags PRIMARY KEY (TransactionFlagsKey)
);
```

The junk dimension contains one row for every combination of flag values. The fact table then has a single `TransactionFlagsKey` FK instead of four separate columns.

---

8. The Date Dimension: A Special Case

The date dimension deserves special treatment because it is the most universally important dimension in any data warehouse. Nearly every fact table includes at least one date key. The date dimension enables time intelligence — the ability to analyze data by day, week, month, quarter, year, fiscal period, and any other time-based grouping.

8.1 Why Pre-Populate a Date Dimension?

An alternative to a pre-populated `DimDate` table would be to derive date attributes dynamically using SQL date functions (`YEAR()`, `MONTH()`, `DATENAME()`). The date dimension is chosen instead for several reasons:

Performance. Joining to a pre-populated integer key (`DateKey = 20240315`) is dramatically faster than computing `YEAR(OrderDate)`, `MONTH(OrderDate)`, etc. on millions of fact rows at query time.

Business calendar support. The date dimension can encode business-specific attributes that cannot be derived from the date itself: fiscal year (which may start in any month), public holidays (which vary by jurisdiction and year), and custom period definitions (promotional weeks, trading weeks).

Consistent definitions. "Q1" means January–March in one system and October–December in a fiscal year starting in October. The date dimension encodes the agreed business definition once, and all reports use it consistently.

Simplified BI tool configuration. Power BI and other BI tools can automatically configure time intelligence (year-to-date, same period last year) when the date dimension is properly structured with a continuous range of dates.

8.2 The CabotTrail Date Dimension

`Dimension.DimDate` in the CabotTrail DW contains 4,017 rows — one per calendar day across the date range of the data. Key columns include:

```
SELECT TOP 5
    DateKey,                -- YYYYMMDD integer PK
    FullDate,               -- DATE value
    DayNumber,              -- Day of month (1-31)
    DayName,                -- 'Monday', 'Tuesday', etc.
    DayOfWeekNumber,        -- 1=Sunday ... 7=Saturday
    WeekNumber,             -- ISO week number
    MonthNumber,            -- 1-12
    MonthName,              -- 'January', etc.
    MonthShortName,         -- 'Jan', etc.
    QuarterNumber,          -- 1-4
    QuarterName,            -- 'Q1', 'Q2', etc.
    YearNumber,             -- Calendar year
    FiscalYearNumber,       -- Fiscal year (April 1 - March 31)
    FiscalQuarterNumber,    -- Fiscal quarter (1-4)
    FiscalPeriodNumber,     -- Fiscal month (1-12)
    IsWeekday,              -- 1 if Mon-Fri
    IsWeekend,              -- 1 if Sat-Sun
    IsPublicHoliday,        -- 1 if public holiday
    PublicHolidayName       -- Holiday name if applicable
FROM    CabotTrailOutdoorDW.Dimension.DimDate
ORDER BY DateKey;
```

8.3 The Integer Date Key

The date key is stored as an 8-digit integer in the YYYYMMDD format (e.g., `20240315` for March 15, 2024). This is a widespread convention in dimensional modelling with two practical advantages:

Human readability. A data analyst can read `OrderDateKey = 20240315` and immediately understand it means March 15, 2024. A surrogate key of `OrderDateKey = 8847` tells them nothing.

Range filtering without joins. Date range queries can be expressed as integer comparisons without joining to `DimDate`:

```
-- Fast integer range filter — no join to DimDate needed
SELECT  SUM(LineTotal)  AS Revenue
FROM    Fact.FactSales
WHERE   OrderDateKey BETWEEN 20240101 AND 20241231;
```

The trade-off is that the integer key must be computed during ETL from a `DATE` or `DATETIME` source column. This is a simple computation in SSIS or T-SQL:

```
-- Convert a date to an integer DateKey
CAST(FORMAT(OrderDate, 'yyyyMMdd') AS INT) AS OrderDateKey
-- or equivalently:
YEAR(OrderDate) * 10000 + MONTH(OrderDate) * 100 + DAY(OrderDate) AS OrderDateKey
```

---

9. The CabotTrail Dimensional Model

With the concepts established, we can now examine the full CabotTrail dimensional model as a design case study.

9.1 Dimension Inventory

Dimension	Key	Natural Key	Type	Special features
`DimDate`	`DateKey`	`FullDate`	Conformed	Integer YYYYMMDD key; fiscal calendar; holiday flags
`DimCustomer`	`CustomerKey`	`CustomerID`	SCD Type 1 (DW)	Geography flattened from 3 OLTP tables
`DimProduct`	`ProductKey`	`ProductID`	SCD Type 1 (DW)	Snowflakes to DimProductCategory + DimSupplier
`DimSupplier`	`SupplierKey`	`SupplierID`	SCD Type 1	Geography flattened from DimGeography
`DimEmployee`	`EmployeeKey`	`EmployeeID`	SCD Type 1	People + Employees joined at ETL
`DimGeography`	`GeographyKey`	`CityID`	SCD Type 1	City + Province + Country hierarchy
`DimDeliveryMethod`	`DeliveryMethodKey`	`DeliveryMethodID`	Static	12 delivery options; rarely changes
`DimTransactionType`	`TransactionTypeKey`	`TransactionTypeID`	Static	Invoice, Payment, Credit Note classifications
`DimPaymentMethod`	`PaymentMethodKey`	`PaymentMethodID`	Static	Payment method lookup
`DimProductCategory`	`ProductCategoryKey`	`ProductCategoryID`	Static	13 categories; source of StandardCostPct

9.2 Fact Inventory

Fact Table	Grain	Dimensions	Key measures
`FactSales`	One row per product per invoice	Date (×3), Customer, Product, Employee, Geography, DeliveryMethod	LineTotal, GrossProfit, GrossProfitMarginPct
`FactPurchasing`	One row per product per PO	Date (×3), Product, Supplier, Employee	LineTotal, ReceivedTotal, DaysToDelivery
`FactReturns`	One row per return	Date (×3), Customer, Product, Employee, Geography	RefundAmount, RefundAsPctOfSale, DaysToReturn
`FactInventory`	One row per product (snapshot)	Date, Product, Supplier, ProductCategory	QuantityOnHand, StockValueAtCost (semi-additive)
`FactCustomerTransactions`	One row per financial transaction	Date, Customer, TransactionType, Employee	TransactionAmount, AmountExcludingTax, TaxAmount
`FactSupplierTransactions`	One row per financial transaction	Date, Supplier, TransactionType, Employee	TransactionAmount, AmountExcludingTax, TaxAmount

9.3 Design Decisions Worth Noting

Role-playing dates. `FactSales`, `FactPurchasing`, and `FactReturns` all use `DimDate` in multiple roles (OrderDate, InvoiceDate, DueDate; OrderDate, ExpectedDeliveryDate, ActualDeliveryDate; ReturnDate, OrderDate, InvoiceDate). The date dimension is a role-playing dimension in all three facts.

Semi-additive fact. `FactInventory` is a periodic snapshot fact. `QuantityOnHand` and all stock value measures are semi-additive — they can be summed across products but not across snapshot dates. If multiple snapshots are loaded in the future, this constraint must be respected in all reporting queries.

Computed measures stored in fact. `GrossProfitMarginPct` and `DaysToReturn` are computed values stored on the fact table rather than calculated at query time. This is a deliberate performance and consistency choice — the calculation is performed once at ETL time and stored for repeated fast retrieval. The ETL developer is responsible for ensuring the stored calculation matches the agreed business definition.

StandardCostPct in DimProductCategory. Product cost rates are stored as `StandardCostPct` on `Inventory.ProductCategories` in the OLTP and carried into `DimProductCategory` in the DW. This allows `UnitCost` and `GrossProfit` to be computed during ETL using per-category rates rather than a flat rate — producing meaningful margin variation across product lines.

---

10. Chapter Summary

- **Dimensional modelling** is a design methodology optimized for analytical queries, business user comprehension, and BI tool compatibility. It organizes data into fact tables (measurements) and dimension tables (context).
- The **grain** of a fact table defines what one row represents. It is the most important design decision and must be defined before choosing dimensions or measures.
- **Measures** are additive (sum freely), semi-additive (sum across some dimensions only), or non-additive (never sum — compute from additive measures). Treating a non-additive measure as additive produces incorrect results.
- **Dimension tables** are wide, denormalized, and surrounded by descriptive attributes. They contain a surrogate primary key and the natural key from the source.
- **Surrogate keys** are system-generated integers used throughout the DW in place of natural keys. They protect against natural key changes, reuse, and cross-source collision.
- The **star schema** — one fact surrounded by directly-joined dimensions — is the optimal structure for analytical queries and BI tools.
- The **snowflake schema** normalizes dimension tables. It is more appropriate at the DW integration layer than at the data mart presentation layer.
- **Dimension types** include conformed (shared across facts), role-playing (referenced multiple times by one fact), slowly changing (tracked with Type 1, 2, or 3 strategies), degenerate (no corresponding table), and junk (combined low-cardinality flags).
- The **date dimension** is the most universal dimension in any DW. It is pre-populated with business calendar attributes and uses an integer YYYYMMDD key for performance and readability.

---

11. Review Questions

1. Define the grain of `Fact.FactSales` in the CabotTrail DW in business terms. What would change about the schema if the grain were changed to "one row per invoice" instead of "one row per invoice line"?
2. `GrossProfitMarginPct` is stored in `Fact.FactSales` as a pre-computed value. Explain why it is non-additive and describe the correct way to compute a company-wide margin percentage from the fact table.
3. `Fact.FactInventory` has `QuantityOnHand` as a semi-additive measure. A developer writes the query `SELECT SUM(QuantityOnHand) FROM Fact.FactInventory GROUP BY SnapshotDateKey` and presents the result as "total units available over time." Explain what is wrong with this query and what the correct approach should be.
4. The CabotTrail DW uses a snowflake schema at the DW layer but star schemas at the data mart layer. Explain the reasoning behind this two-layer approach. What happens in the ETL process between the DW and the data marts?
5. A new source system provides customer data where `CustomerID` values overlap with existing customer IDs from the original source system (`CustomerID = 42` exists in both). How do surrogate keys in the data warehouse solve this problem?
6. A product moves from the `Apparel - Tops` category to `Outerwear`. Which SCD type is appropriate for this change if the business requires that historical sales reports continue to show the product in its original category? Describe the process for implementing this change.
7. `DimDate` is used three times in `Fact.FactSales` as a role-playing dimension. Write the SQL to query average days between order date and invoice date for each calendar month in 2024, using the role-playing dimension pattern correctly.
8. `InvoiceID` and `OrderID` are degenerate dimensions in `Fact.FactSales`. Explain why they are not candidates for their own dimension tables. What purpose do they serve on the fact row?

Deeper Dive

Going Further with Dimensional Modelling

The Four-Step Design Process

Kimball defines a systematic four-step process for designing a dimensional model. Following it in sequence prevents the most common design errors:

Step 1 — Select the business process.
Choose the specific business process to model (e.g., sales order processing, purchasing, returns). Each process typically maps to one or more fact tables. Do not try to model everything in one fact — separate business processes should be in separate facts.

Step 2 — Declare the grain.
State exactly what one row in the fact table represents. This must be done before anything else. The grain drives all subsequent decisions.

Step 3 — Identify the dimensions.
Given the grain, identify every dimension that can be known at the time of the event. If the grain is "one row per invoice line," then date, customer, product, sales rep, and delivery method can all be known at invoice line time. They are all legitimate dimensions.

Step 4 — Identify the facts.
Given the grain, identify every numeric measure that is true at the grain level. Unit price, quantity, line total, tax amount — all are known per invoice line. *Total customer lifetime value* is not known per invoice line — it is an aggregate computed from many invoice lines. It does not belong as a measure in this fact table.

This four-step process is described in detail in Kimball's *The Data Warehouse Toolkit*, Chapter 1, and in the online Kimball Techniques library: Kimball Group — Dimensional Modeling Techniques

SCD Type 6: The Hybrid Approach

SCD Type 6 combines Types 1, 2, and 3 into a single dimension design. The name comes from the arithmetic: 1 + 2 + 3 = 6. It is the most sophisticated SCD approach and addresses a practical limitation of pure Type 2.

With pure Type 2, a query that wants all current customers in Ontario must specify `WHERE IsCurrent = 1 AND ProvinceCode = 'ON'`. A query that wants historical sales correctly attributed to the province *at the time of sale* simply joins without the IsCurrent filter. Both work correctly.

But a common business question breaks both approaches: *"Show me total lifetime sales for current customers in Ontario, using their current province, regardless of what province they were in when they placed each order."*

Type 2 cannot answer this cleanly because the fact rows point to historical dimension rows with the old province. Type 6 solves this by adding a `CurrentProvinceCode` column to every dimension row (updated Type 1-style on every run) alongside the historical `ProvinceCode` (preserved Type 2-style):

```
-- Type 6 dimension: both historical and current attribute values on every row
SELECT
    CustomerKey,
    CustomerID,
    CustomerName,
    ProvinceCode            AS ProvinceAtTimeOfSale,  -- Historical (Type 2)
    CurrentProvinceCode,                               -- Current (Type 1, updated on
each load)
    ValidFrom,
    ValidTo,
    IsCurrent
FROM    Dimension.DimCustomer_Type6;
```

This allows:
- Historical accuracy: `WHERE ProvinceCode = 'NS'` — who was in NS at time of sale
- Current state: `WHERE CurrentProvinceCode = 'ON'` — who is currently in ON
- Both at once: full flexibility

Type 6 is more complex to implement and maintain than pure Type 2, but it is the right choice when both historical and current perspectives are needed simultaneously.

Kimball's full description of SCD types, including Type 6: Kimball Group — Slowly Changing Dimension Techniques

Factless Fact Tables in Practice

Factless facts are underused and underappreciated. They enable a class of analytical question that cannot be answered by measure facts alone: questions about absence.

"Which products were never sold?"

```
-- Factless fact approach: all product-date combinations
-- that SHOULD have sales, left joined to actual sales
SELECT  p.ProductName,
        p.CategoryName,
        COUNT(fs.SalesKey) AS TimesSold
FROM    Dimension.DimProduct p
LEFT JOIN Fact.FactSales fs ON fs.ProductKey = p.ProductKey
WHERE   p.ProductID <> 0
GROUP BY p.ProductName, p.CategoryName
HAVING  COUNT(fs.SalesKey) = 0;
```

"Which customers placed no orders in Q3 2024?"

```
SELECT  dc.CustomerName,
        dc.SalesTerritory
FROM    Dimension.DimCustomer dc
WHERE   dc.IsCurrent = 1
AND     dc.CustomerID <> 0
AND     NOT EXISTS (
    SELECT 1
    FROM   Fact.FactSales fs
    INNER JOIN Dimension.DimDate dd ON dd.DateKey = fs.OrderDateKey
    WHERE  fs.CustomerKey = dc.CustomerKey
    AND    dd.YearNumber   = 2024
    AND    dd.QuarterNumber = 3
);
```

A formal factless fact table — pre-populated with all expected combinations (every customer × every quarter, or every product × every promotion period) — makes these absence queries simpler and more performant than the correlated subquery approach above.

Aggregate Navigation

In large data warehouses with hundreds of millions of fact rows, query performance on the line-item fact table becomes a concern. **Aggregate navigation** is the practice of maintaining pre-computed aggregate fact tables at higher grains (daily totals, monthly totals) and automatically routing queries to the appropriate aggregate.

The CabotTrail aggregate table example from Chapter 8 (`Fact.FactSalesMonthly`) is a simple form of aggregate navigation. Production implementations use tools like Microsoft Analysis Services (SSAS) or the Power BI aggregations feature to manage this automatically — queries directed at the line-item fact are transparently redirected to the appropriate aggregate when the aggregate satisfies the query.

Kimball's treatment of aggregates: Kimball Group — Aggregate Fact Tables

The Bus Architecture: Conforming Across the Enterprise

When multiple dimensional models are built for different business areas (Sales, Purchasing, HR, Finance), they need to share common dimensions to enable cross-functional analysis. Kimball's answer is the **enterprise data warehouse bus architecture** — a matrix that maps every fact to every conformed dimension it uses.

The bus matrix for CabotTrail would look like:

	DimDate	DimCustomer	DimProduct	DimSupplier	DimEmployee	DimGeography
FactSales	▪		▪	▪		
FactPurchasing	▪	▪			▪	▪
FactReturns	▪			▪	▪	
FactInventory						
FactCustomerTransactions						
FactSupplierTransactions						

Where two facts share a conformed dimension, cross-fact analysis is possible. `DimProduct` is shared by FactSales, FactPurchasing, FactReturns, and FactInventory — enabling analysis that spans sales, procurement, returns, and stock levels for the same product.

The bus matrix is a planning tool for ETL development: it determines which conformed dimensions must be loaded first, and which fact loads depend on them.

Full description of the bus architecture: Kimball Group — Enterprise Data Warehouse Bus Architecture

---

Industry Perspectives

Kimball on Grain

Kimball's writing on grain definition is some of the most practically useful guidance in the dimensional modelling literature. The key insight is that declaring the grain is a *commitment* — once made, every dimension and measure decision is constrained by it. Changing the grain after the fact table is built and loaded is enormously expensive.

A common grain mistake is declaring the grain too coarsely: "one row per order" when the business actually needs "one row per product per order." This prevents analysis by product within an order — a fundamental omission for any retail or distribution business. When in doubt, choose the finest grain supported by the

source system. Aggregates can always be computed upward from a fine grain; detail can never be recovered from a coarse grain after the fact.

Microsoft on SSAS and Tabular Models

For readers interested in how dimensional models are consumed by analytical engines, Microsoft's Analysis Services (SSAS) has deep support for both multidimensional (cube-based) and tabular (in-memory column-store) models built on top of dimensional data warehouses:

- Analysis Services documentation
- Tabular modeling in SSAS

Power BI's data model is effectively a tabular SSAS model hosted in the Power BI service — dimensional modelling concepts apply directly.

References and Further Reading

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley. — Chapters 1–5 cover all concepts in this chapter in authoritative detail.
2. Kimball Group. (n.d.). *Dimensional Modeling Techniques*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/
3. Kimball Group. (n.d.). *Slowly Changing Dimension Techniques*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/050-slowly-changing-dimension-types-1-2-3-4-5-6/
4. Kimball Group. (n.d.). *Enterprise Data Warehouse Bus Architecture*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/009-enterprise-data-warehouse-bus-architecture/
5. Kimball Group. (n.d.). *Aggregate Fact Tables*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/059-aggregate-fact-table-cube/
6. Microsoft. (2024). *Analysis Services Overview*. https://learn.microsoft.com/en-us/analysis-services/analysis-services-overview
7. Microsoft. (2024). *Design tables in Azure Synapse Analytics*. https://learn.microsoft.com/en-us/azure/synapse-analytics/sql-data-warehouse/sql-data-warehouse-tables-overview — Applies dimensional modelling concepts in a cloud DW context.

8. Adamson, C. (2010). *Star Schema: The Complete Reference*. McGraw-Hill. — A thorough practical reference for star schema design that complements Kimball's toolkit.

9. Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann. — An alternative to Kimball for enterprise DW design at extreme scale; useful context for understanding where dimensional modelling fits in the broader landscape.

Chapter 3: Gap Analysis and Source-to-Target Mapping

Chapter Overview

Chapters 1 and 2 established *what* ETL does and *what structure* the data warehouse uses. This chapter addresses the critical design work that must happen before a single ETL component is built: understanding the gap between what the source has and what the target needs, and documenting precisely how to bridge it.

Gap analysis and **source-to-target (S2T) mapping** are the ETL developer's design documents. They are the equivalent of an architect's blueprints — the detailed specifications that govern every technical decision that follows. An ETL project built without them will either fail or produce a system that cannot be maintained, tested, or modified with confidence.

This chapter is deliberately practical. It covers the formal methodology and the SQL queries that make that methodology executable in the CabotTrail environment.

By the end of this chapter you will be able to:

- Define the three types of ETL gaps and identify examples of each
- Conduct a systematic source data analysis and profile a source table
- Build a complete source-to-target mapping document for a dimension or fact table
- Identify and document data quality issues using a formal framework
- Apply referential integrity checks to validate source data before ETL begins
- Understand the relationship between the S2T mapping and the SSIS packages that implement it

Table of Contents

---

1. Why Design Before You Build

A common instinct among developers encountering a new ETL project is to start building immediately — connect to the source, pull data, write transformations, load the target. This approach produces working code quickly and feels productive. It is also one of the most reliable paths to an ETL system that cannot be trusted.

The reason is straightforward: ETL transforms data according to business rules. Those rules must be agreed upon, documented, and validated before they are encoded in a transformation pipeline. If the rules are encoded first and documented afterward (or never), several things happen:

The rules become invisible. Three months after a package is built, no one can answer the question "why does `SalesTerritory` have the value 'Atlantic — Nova Scotia' for this customer?" The answer is buried in a SSIS Derived Column expression that took twelve minutes to find.

Testing becomes impossible. A test suite verifies that the system produces correct output. Correct output requires a specification. A specification requires documentation. Without the S2T mapping, there is no specification to test against.

Changes become dangerous. When a business rule changes — and it will — the developer needs to find every place that rule is implemented and update all of them consistently. Without documentation, finding those places requires archaeology.

Onboarding becomes impossible. A new developer joining the project has no way to understand what the system is supposed to do without reading every package, every SQL statement, and every expression — and hoping they are all consistent.

The S2T mapping is the solution to all of these problems. It is a living document that precedes implementation, guides implementation, and is maintained alongside implementation throughout the life of the ETL system.

---

2. The Three Types of ETL Gaps

A **gap** is any difference between what the source provides and what the target requires. Every ETL project has gaps. The work of gap analysis is to find them all, categorize them, and document how each will be resolved.

There are three fundamental gap types.

2.1 Missing Data Gaps

A **missing data gap** occurs when the target requires a column or attribute that does not exist in any source system.

Missing data gaps are the most significant because they require either sourcing the data from a new place, deriving it from existing data, or accepting that the target column will always be NULL or a default value.

CabotTrail examples:

Target Column	Target Table	Gap Description	Resolution
`SalesTerritory`	`dim.Customer` (datamart)	No `SalesTerritory` column exists in any OLTP table	Derive from `ProvinceCode` using a CASE expression during ETL
`PostalCode`	`dim.Customer` (datamart)	Postal code not captured in `Application.Cities`	NULL default — not available in source; document as gap
`ProductDescription`	`dim.Product` (datamart)	`Inventory.Products` has no description column	NULL default
`IsOnCreditHold`	`dim.Customer` (datamart)	`Sales.Customers.IsOnCreditHold` was not loaded into DW	Default to 0 — not available via DW; source directly from OLTP if needed

Missing data gaps require a design decision. The options are:

1. **Derive it** — compute the value from existing data (e.g., `SalesTerritory` from `ProvinceCode`)
2. **Source it elsewhere** — find the value in another system or table
3. **Default it** — use a fixed default value and document why
4. **Accept NULL** — document that the column is intentionally unpopulated

The decision must be made by a business stakeholder, not unilaterally by the ETL developer. "The source doesn't have it so I left it NULL" is not an acceptable resolution unless the business has agreed that NULL is correct.

2.2 Data Quality Gaps

A **data quality gap** occurs when source data exists but is dirty, inconsistent, incomplete, or violates business rules that the target enforces.

Data quality gaps are often the most numerous and the most time-consuming to resolve. They represent the real-world messiness of operational data — data that was entered by humans under time pressure, migrated from legacy systems, or generated by applications that did not enforce consistent rules.

Common data quality gap types:

Type	Description	Example
NULL violations	A NOT NULL target column receives NULL from source	`CustomerName` is NULL for 3 records
Type mismatches	Source stores a value as the wrong type	`OrderDate` stored as NVARCHAR('2024-03-15') instead of DATE
Range violations	A numeric value falls outside the valid range	`UnitPrice = -5.00` in a source with no negative price validation
Referential integrity	A FK value in the source references a PK that does not exist	`Orders.CustomerID = 999` but `CustomerID = 999` does not exist in `Customers`
Encoding inconsistency	The same concept expressed differently	Province stored as 'NS', 'N.S.', 'Nova Scotia', and 'nova scotia'
Duplicates	The same entity appears multiple times with different keys	Two `Customers` rows for 'Fundy Bay Outfitters' with different CustomerIDs
Orphaned records	Records with no valid parent	Returns with no matching order line
Future dates	Transaction dates in the future	`InvoiceDate = '2099-01-01'` (a data entry error)
Truncated data	Values cut off because a source column was too narrow	`CustomerName = 'The Great Outdoors Supply Comp'` (truncated at 30 chars)

2.3 Structural Gaps

A **structural gap** occurs when the source organizes data differently from how the target needs it — requiring joins, splits, pivots, or other structural transformations.

Structural gaps are the most technically interesting because they require the most sophisticated ETL logic.

CabotTrail examples:

Structural gap	Description	Resolution
Geography normalization	Customer geography is spread across `Customers`, `Cities`, `StateProvinces`, `Countries` — 4 tables	JOIN all four at extract time into a single flat row
M:M category relationship	A product can belong to multiple categories (junction table)	Apply business rule: use MIN(ProductCategoryID) as primary category
Snowflake to star	DW `DimProduct` snowflakes to `DimProductCategory` and `DimSupplier`; datamart needs a flat wide dimension	JOIN all three in extract query; flatten into single dim.Product row
Integer date key	Source stores dates as DATE; DW requires YYYYMMDD integer	Convert using `CAST(FORMAT(OrderDate, 'yyyyMMdd') AS INT)`
Surrogate key lookup	Source stores `CustomerID` (natural key); fact requires `CustomerKey` (surrogate)	Lookup transformation in SSIS; JOIN in T-SQL
Fiscal year derivation	Source has no fiscal year attributes; DW `DimDate` requires fiscal year, quarter, period	Compute from calendar month using business fiscal calendar rules

3. Source Data Analysis

Before gaps can be identified, the source data must be thoroughly understood. **Source data analysis** is the systematic profiling of every source table to establish its structure, content, ranges, completeness, and quality.

Source data analysis answers the questions:
- What is in this table?
- How many rows?
- What is the date range?
- Are there NULLs? How many, in which columns?
- Are there outliers or unexpected values?
- Are referential integrity relationships intact?

3.1 Table-Level Profiling

The first step is to understand the shape and size of every source table.

```
-- Fast row counts for all OLTP tables (uses metadata — no full scan)
USE CabotTrailOutdoor;

SELECT
    SCHEMA_NAME(t.schema_id)    AS [Schema],
    t.name                      AS [Table],
    p.rows                      AS [RowCount],
    -- Table creation date
    t.create_date               AS CreatedDate
FROM    sys.tables t
INNER JOIN sys.partitions p
    ON p.object_id = t.object_id
    AND p.index_id IN (0, 1)   -- Heap (0) or clustered index (1)
ORDER BY [Schema], t.name;
```

```
-- Column inventory: data types, nullability, defaults
SELECT
    c.TABLE_SCHEMA,
    c.TABLE_NAME,
    c.COLUMN_NAME,
    c.DATA_TYPE
        + CASE
            WHEN c.CHARACTER_MAXIMUM_LENGTH IS NOT NULL
            THEN '(' + CAST(c.CHARACTER_MAXIMUM_LENGTH AS VARCHAR) + ')'
            WHEN c.NUMERIC_PRECISION IS NOT NULL
             AND c.DATA_TYPE IN ('decimal', 'numeric')
            THEN '(' + CAST(c.NUMERIC_PRECISION AS VARCHAR)
                + ',' + CAST(c.NUMERIC_SCALE AS VARCHAR) + ')'
            ELSE ''
          END                   AS DataType,
    c.IS_NULLABLE,
    c.COLUMN_DEFAULT,
    c.ORDINAL_POSITION
FROM    INFORMATION_SCHEMA.COLUMNS c
WHERE   c.TABLE_SCHEMA IN ('Sales', 'Purchasing', 'Inventory', 'Application')
ORDER BY c.TABLE_SCHEMA, c.TABLE_NAME, c.ORDINAL_POSITION;
```

3.2 Column-Level Profiling

After understanding the structure, profile the content of key columns in each source table. Column profiling establishes value distributions, NULL rates, and data ranges.

```
-- Profile: Sales.Orders
USE CabotTrailOutdoor;

SELECT
    COUNT(*)                            AS TotalRows,
    COUNT(DISTINCT CustomerID)          AS UniqueCustomers,
    COUNT(DISTINCT SalesRepID)          AS UniqueSalesReps,
    COUNT(DISTINCT CityID)              AS UniqueCities,
    MIN(OrderDate)                      AS EarliestOrderDate,
    MAX(OrderDate)                      AS LatestOrderDate,
    -- NULL checks
    SUM(CASE WHEN OrderDate  IS NULL THEN 1 ELSE 0 END) AS NullOrderDates,
    SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) AS NullCustomerIDs,
    SUM(CASE WHEN SalesRepID IS NULL THEN 1 ELSE 0 END) AS NullSalesRepIDs,
    SUM(CASE WHEN CityID     IS NULL THEN 1 ELSE 0 END) AS NullCityIDs
FROM    Sales.Orders;
```

```
-- Profile: Sales.InvoiceLines — numeric value ranges and quality checks
SELECT
    COUNT(*)                            AS TotalRows,
    MIN(UnitPrice)                      AS MinUnitPrice,
    MAX(UnitPrice)                      AS MaxUnitPrice,
    ROUND(AVG(UnitPrice), 2)            AS AvgUnitPrice,
    MIN(Quantity)                       AS MinQuantity,
    MAX(Quantity)                       AS MaxQuantity,
    MIN(LineTotal)                      AS MinLineTotal,
    MAX(LineTotal)                      AS MaxLineTotal,
    -- Quality flags
    SUM(CASE WHEN UnitPrice <= 0  THEN 1 ELSE 0 END) AS ZeroOrNegativePrice,
    SUM(CASE WHEN Quantity  <= 0  THEN 1 ELSE 0 END) AS ZeroOrNegativeQty,
    SUM(CASE WHEN TaxRate   <  0  THEN 1 ELSE 0 END) AS NegativeTaxRate,
    SUM(CASE WHEN LineTotal <  0  THEN 1 ELSE 0 END) AS NegativeLineTotal
FROM    Sales.InvoiceLines;
```

```
-- Profile: Inventory.Products — NULL analysis and value distribution
SELECT
    COUNT(*)                            AS TotalProducts,
    SUM(CASE WHEN ProductCode IS NULL       THEN 1 ELSE 0 END) AS NullProductCodes,
    SUM(CASE WHEN ColorID     IS NULL       THEN 1 ELSE 0 END) AS NullColors,
    SUM(CASE WHEN IsDiscontinued = 1        THEN 1 ELSE 0 END) AS DiscontinuedCount,
    MIN(UnitPrice)                      AS MinUnitPrice,
    MAX(UnitPrice)                      AS MaxUnitPrice,
    ROUND(AVG(UnitPrice), 2)            AS AvgUnitPrice,
    MIN(TypicalWeightPerUnit)           AS MinWeight,
    MAX(TypicalWeightPerUnit)           AS MaxWeight
FROM    Inventory.Products
WHERE   ProductID <> 0;
```

3.3 Referential Integrity Checks

Referential integrity (RI) checks verify that FK values in child tables have corresponding PK values in parent tables. In a well-maintained OLTP, these should always pass. But legacy data migrations, application bugs, and data loads that bypass constraints can introduce orphaned records.

RI failures in the source are particularly dangerous for ETL because they cause **lookup failures** — a fact row that references a `CustomerID` that does not exist in the `Customers` table will fail the surrogate key lookup in SSIS, potentially causing the entire load to fail or silently dropping rows.

```
-- RI check: Orders with no matching Customer
SELECT  COUNT(*) AS OrphanOrders
FROM    Sales.Orders o
WHERE   NOT EXISTS (
    SELECT 1 FROM Sales.Customers c
    WHERE c.CustomerID = o.CustomerID
);

-- RI check: OrderLines with no matching Order
SELECT  COUNT(*) AS OrphanOrderLines
FROM    Sales.OrderLines ol
WHERE   NOT EXISTS (
    SELECT 1 FROM Sales.Orders o
    WHERE o.OrderID = ol.OrderID
);

-- RI check: InvoiceLines with no matching OrderLine
SELECT  COUNT(*) AS OrphanInvoiceLines
FROM    Sales.InvoiceLines il
WHERE   NOT EXISTS (
    SELECT 1 FROM Sales.OrderLines ol
    WHERE ol.OrderLineID = il.OrderLineID
);

-- RI check: Products with no matching Supplier
SELECT  COUNT(*) AS ProductsWithNoSupplier
FROM    Inventory.Products p
WHERE   NOT EXISTS (
    SELECT 1 FROM Purchasing.Suppliers s
    WHERE s.SupplierID = p.SupplierID
)
AND p.ProductID <> 0;

-- RI check: ProductCategoryAssignments with no matching Product
SELECT  COUNT(*) AS OrphanCategoryAssignments
FROM    Inventory.ProductCategoryAssignments pca
WHERE   NOT EXISTS (
    SELECT 1 FROM Inventory.Products p
    WHERE p.ProductID = pca.ProductID
);
```

> ***In the CabotTrail environment, all RI checks should return 0.*** *If they do not, the source database was not loaded correctly and must be corrected before ETL development begins. Document the result of every RI check in the source data analysis report.*

3.4 Cardinality Analysis

Understanding the cardinality (number of distinct values) of key columns helps predict ETL behaviour and identify potential design issues.

```
-- Cardinality of key columns in Products
SELECT
    COUNT(DISTINCT ProductID)       AS UniqueProducts,
    COUNT(DISTINCT SupplierID)      AS UniqueSuppliers,
    COUNT(DISTINCT ColorID)         AS UniqueColors,
    COUNT(DISTINCT PackageTypeID)   AS UniquePackageTypes
FROM    Inventory.Products
WHERE   ProductID <> 0;

-- How many products per category? (M:M junction analysis)
SELECT
    pc.ProductCategoryName,
    COUNT(pca.ProductID)            AS ProductCount
FROM    Inventory.ProductCategories pc
LEFT JOIN Inventory.ProductCategoryAssignments pca
    ON pca.ProductCategoryID = pc.ProductCategoryID
GROUP BY pc.ProductCategoryName
ORDER BY ProductCount DESC;

-- Products assigned to multiple categories (important for primary category logic)
SELECT
    p.ProductName,
    COUNT(pca.ProductCategoryID)    AS CategoryCount,
    STRING_AGG(pc.ProductCategoryName, ', ')
        WITHIN GROUP (ORDER BY pca.ProductCategoryID) AS Categories
FROM    Inventory.Products p
INNER JOIN Inventory.ProductCategoryAssignments pca
    ON pca.ProductID = p.ProductID
INNER JOIN Inventory.ProductCategories pc
    ON pc.ProductCategoryID = pca.ProductCategoryID
WHERE   p.ProductID <> 0
GROUP BY p.ProductName
HAVING  COUNT(pca.ProductCategoryID) > 1
ORDER BY CategoryCount DESC;
```

The multi-category query above reveals products that belong to more than one category. This is a **structural gap** — the target `DimProduct` needs a single category per product. The source data analysis reveals the

business decision that must be made: which category takes precedence? The CabotTrail ETL resolves this by always using the lowest `ProductCategoryID` (the primary assignment).

---

4. Data Quality Assessment

Source data analysis produces raw facts about the data. A **data quality assessment** turns those facts into a structured framework of rules, measurements, and severities that guides ETL error handling.

4.1 Data Quality Dimensions

Data quality is assessed across several dimensions — each measuring a different aspect of "good" data:

Dimension	Definition	Example
Completeness	Are all required values present?	`CustomerName IS NOT NULL`
Accuracy	Do values correctly represent the real world?	`UnitPrice > 0`
Consistency	Are values consistent within and across systems?	Province stored as 'NS' everywhere (not 'N.S.' or 'Nova Scotia')
Timeliness	Is the data current enough for its intended use?	Inventory snapshot taken within the last 24 hours
Uniqueness	Are entities represented exactly once?	No duplicate CustomerID values
Validity	Do values conform to defined formats or ranges?	`OrderDate` is a valid date, not '0000-00-00'
Referential integrity	Do FK values have corresponding PK records?	All `Orders.CustomerID` values exist in `Customers.CustomerID`

4.2 The Data Quality Rule Table

Document every data quality check as a formal rule with a defined pass condition, measurement query, and severity level.

Severity levels:

- **High** — a failure blocks the load; affected rows must be rejected or the ETL must stop
- **Medium** — a failure is logged and rows are flagged; the load continues
- **Low** — a failure is logged only; no operational impact

```
-- Execute and document these DQ rules for the CabotTrail source

-- DQ-001 [High] Sales.Orders: OrderDate must not be NULL
SELECT 'DQ-001' AS RuleID, 'High' AS Severity,
       'Sales.Orders' AS [Table], 'OrderDate' AS [Column],
       'No null order dates' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Sales.Orders WHERE OrderDate IS NULL;

-- DQ-002 [High] Sales.Orders: CustomerID must exist in Customers
SELECT 'DQ-002' AS RuleID, 'High' AS Severity,
       'Sales.Orders' AS [Table], 'CustomerID' AS [Column],
       'CustomerID must reference a valid customer' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Sales.Orders o
WHERE NOT EXISTS (SELECT 1 FROM Sales.Customers c WHERE c.CustomerID = o.CustomerID);

-- DQ-003 [High] Sales.InvoiceLines: UnitPrice must be > 0
SELECT 'DQ-003' AS RuleID, 'High' AS Severity,
       'Sales.InvoiceLines' AS [Table], 'UnitPrice' AS [Column],
       'Unit price must be positive' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Sales.InvoiceLines WHERE UnitPrice <= 0;

-- DQ-004 [High] Sales.InvoiceLines: Quantity must be > 0
SELECT 'DQ-004' AS RuleID, 'High' AS Severity,
       'Sales.InvoiceLines' AS [Table], 'Quantity' AS [Column],
       'Quantity must be positive' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Sales.InvoiceLines WHERE Quantity <= 0;

-- DQ-005 [Medium] Inventory.Products: ProductCode may be NULL
SELECT 'DQ-005' AS RuleID, 'Medium' AS Severity,
       'Inventory.Products' AS [Table], 'ProductCode' AS [Column],
       'ProductCode is nullable; document NULL count' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Inventory.Products WHERE ProductCode IS NULL AND ProductID <> 0;

-- DQ-006 [High] Inventory.Products: UnitPrice must be > 0
SELECT 'DQ-006' AS RuleID, 'High' AS Severity,
       'Inventory.Products' AS [Table], 'UnitPrice' AS [Column],
       'Unit price must be positive' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Inventory.Products WHERE UnitPrice <= 0 AND ProductID <> 0;

-- DQ-007 [High] Sales.Customers: CreditLimit must be >= 0
SELECT 'DQ-007' AS RuleID, 'High' AS Severity,
       'Sales.Customers' AS [Table], 'CreditLimit' AS [Column],
       'Credit limit cannot be negative' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Sales.Customers WHERE CreditLimit < 0;

-- DQ-008 [Medium] Application.StateProvinces: ProvinceCode must be 2 chars
```

```
SELECT 'DQ-008' AS RuleID, 'Medium' AS Severity,
       'Application.StateProvinces' AS [Table], 'StateProvinceCode' AS [Column],
       'Province code must be exactly 2 characters' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Application.StateProvinces WHERE LEN(StateProvinceCode) <> 2;

-- DQ-009 [High] Purchasing.PurchaseOrderLines: UnitCost must be > 0
SELECT 'DQ-009' AS RuleID, 'High' AS Severity,
       'Purchasing.PurchaseOrderLines' AS [Table], 'UnitCost' AS [Column],
       'Unit cost must be positive' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Purchasing.PurchaseOrderLines WHERE UnitCost <= 0;

-- DQ-010 [Medium] Sales.Return: ReturnDate must be >= InvoiceDate
SELECT 'DQ-010' AS RuleID, 'Medium' AS Severity,
       'Sales.Return' AS [Table], 'ReturnDate' AS [Column],
       'Return date must not precede invoice date' AS RuleDescription,
       COUNT(*) AS FailCount
FROM Sales.[Return] r
INNER JOIN Sales.OrderLines ol  ON ol.OrderLineID = r.OrderLineID
INNER JOIN Sales.Orders o       ON o.OrderID      = ol.OrderID
INNER JOIN Sales.Invoices i     ON i.OrderID      = o.OrderID
WHERE r.ReturnDate < i.InvoiceDate;
```

4.3 ETL Error Handling Strategy

Once data quality rules are defined, the ETL must implement a strategy for handling violations. There are three fundamental approaches:

Fail fast. The package detects a quality violation and stops immediately. The DW is not updated. Operations are alerted and the source data must be corrected before the next load attempt.

When to use: High-severity violations where partial loading would produce misleading results.

Reject and continue. Rows that violate quality rules are redirected to an error table and excluded from the main load. The load completes with clean rows; rejected rows are logged for investigation and remediation.

When to use: Medium-severity violations where most rows are clean and a partial load is preferable to no load.

Substitute and continue. The ETL substitutes a default or derived value for the missing or invalid one and logs the substitution. The load completes; substituted values are flagged for review.

When to use: Low-severity violations or cases where the business has agreed on a specific default (e.g., `ISNULL(ProductCode, 'No Code')`).

In SSIS, these strategies are implemented using the **Conditional Split** transformation and **Error Outputs**:

```
OLE DB Source
    ↓
Conditional Split
    ├── [Valid rows] UnitPrice > 0 AND Quantity > 0 → main flow → OLE DB Destination
    └── [Invalid rows] otherwise → Error OLE DB Destination (error log table)
```

---

5. Source-to-Target Mapping

The **source-to-target (S2T) mapping** is the core ETL design document. It is a complete specification — typically maintained as a spreadsheet — that describes every column in the target, where it comes from, and how it is transformed.

An S2T mapping serves three purposes simultaneously:

1. **Specification** — tells the ETL developer exactly what to build
2. **Test oracle** — defines what correct output looks like, enabling automated reconciliation
3. **Documentation** — answers the question "why does this column have this value?" for the life of the system

5.1 S2T Mapping Structure

A complete S2T mapping has the following columns for each target column:

Column	Description
Target Schema	Schema in the DW or data mart (e.g., `Dimension`, `Fact`, `fact`, `dim`)
Target Table	Table name
Target Column	Column name in the target
Target Data Type	Data type and precision (e.g., `NVARCHAR(100)`, `DECIMAL(18,2)`)
Nullable	Y or N
Source Database	Which database the source data comes from
Source Schema	Schema in the source database
Source Table	Primary source table
Source Column	Column name in the source
Transformation Rule	Any derivation, lookup, join, conversion, or cleansing applied
Default Value	Value to use when source is NULL or not available
DQ Rule Reference	Which DQ rule(s) apply to this column
Notes	Special handling, business rules, known issues

5.2 Transformation Rule Types

The transformation rule column is the most important part of the mapping. It must be precise enough that a developer can implement it without asking additional questions. Common rule types:

Rule type	Notation	Example
Direct copy	`Direct`	`CustomerName → CustomerName`
Rename	`Rename: [source name]`	`StateProvinceName → ProvinceName`
JOIN	`JOIN [table] ON [condition]`	`JOIN Application.Cities ON CityID = DeliveryCityID`
Lookup	`LOOKUP [DW table] ON [natural key] → [surrogate key]`	`LOOKUP DimCustomer ON CustomerID → CustomerKey`
Derivation	`DERIVE: [expression]`	`DERIVE: CASE ProvinceCode WHEN 'NS' THEN 'Atlantic – Nova Scotia' ...`
Conversion	`CONVERT: [source type] → [target type]`	`CONVERT: DATE → INT YYYYMMDD`
Aggregation	`AGG: [function]`	`AGG: MIN(ProductCategoryID) → PrimaryCategoryID`
Default	`DEFAULT: [value]`	`DEFAULT: 0`
NULL handling	`ISNULL([column], [default])`	`ISNULL(ProductCode, 'No Code')`
Computed	`COMPUTE: [expression]`	`COMPUTE: ROUND(UnitPrice * StandardCostPct * Quantity, 2)`
Identity	`IDENTITY`	Surrogate key generated by DW

5.3 Load Order and Dependencies

The S2T mapping must also document **load order dependencies** — which tables must be loaded before others.

In a dimensional model, the dependency chain is:

```
1. Static lookups (DimDeliveryMethod, DimTransactionType, DimPaymentMethod)
2. Date dimension (DimDate – no source dependencies)
3. Geography (DimGeography – no other DW dim dependencies)
4. Customers, Suppliers, Employees (depend on DimGeography)
5. Product Categories (no dependencies)
6. Products (depend on DimProductCategory, DimSupplier)
7. All Fact tables (depend on all their dimensions)
```

Documenting this in the S2T mapping prevents the most common ETL deployment failure: loading facts before their dimensions.

6. Worked Example: Mapping DimCustomer

`Dimension.DimCustomer` in `CabotTrailOutdoorDW` is built from three OLTP source tables. The complete S2T mapping follows.

6.1 Source Tables Involved

```
-- Identify all source tables for DimCustomer
USE CabotTrailOutdoor;

-- Primary source
SELECT TOP 3 * FROM Sales.Customers;

-- Geography level 1: City
SELECT TOP 3 * FROM Application.Cities;

-- Geography level 2: Province
SELECT TOP 3 * FROM Application.StateProvinces;

-- Geography level 3: Country
SELECT TOP 3 * FROM Application.Countries;
```

6.2 Gap Analysis: DimCustomer

Before mapping, identify all gaps:

Gap ID	Type	Description	Resolution
GAP-C01	Structural	Customer geography is split across 4 tables	JOIN all 4 at extract time
GAP-C02	Missing	No `SalesTerritory` attribute in any source table	Derive from `StateProvinceCode` using CASE
GAP-C03	Missing	No `PostalCode` in `Application.Cities`	NULL default; document as known gap
GAP-C04	Missing	`IsOnCreditHold` not loaded into DW from OLTP	Default to 0; note that OLTP has this column
GAP-C05	Structural	DW is Type 1 (no history); datamart requires `ValidFrom` / `ValidTo` / `IsCurrent`	Populate with static defaults: ValidFrom=first load date, ValidTo=9999-12-31, IsCurrent=1
GAP-C06	Structural	`CustomerGroupName` in OLTP maps to `CustomerCategoryName` in DW	Rename during ETL

6.3 Complete S2T Mapping: DimCustomer

Target Column	Type	Nullable	Source DB	Source Schema	Source Table	Source Colu
`CustomerKey`	INT	N	—	—	—	—
`CustomerID`	INT	N	CabotTrailOutdoor	Sales	Customers	CustomerID
`CustomerName`	NVARCHAR(100)	N	CabotTrailOutdoor	Sales	Customers	CustomerNan
`CustomerCategoryName`	NVARCHAR(50)	N	CabotTrailOutdoor	Sales	Customers	CustomerGro
`CreditLimit`	DECIMAL(18,2)	N	CabotTrailOutdoor	Sales	Customers	CreditLimit
`AccountOpenedDate`	DATE	Y	CabotTrailOutdoor	Sales	Customers	AccountOpen
`IsOnCreditHold`	BIT	N	—	—	—	—
`CityName`	NVARCHAR(60)	N	CabotTrailOutdoor	Application	Cities	CityName
`ProvinceName`	NVARCHAR(60)	N	CabotTrailOutdoor	Application	StateProvinces	StateProvince
`ProvinceCode`	NCHAR(2)	N	CabotTrailOutdoor	Application	StateProvinces	StateProvince
`CountryName`	NVARCHAR(60)	N	CabotTrailOutdoor	Application	Countries	CountryName
`PostalCode`	NVARCHAR(10)	Y	—	—	—	—
`SalesTerritory`	NVARCHAR(60)	Y	—	—	—	—
`ValidFrom`	DATE	N	—	—	—	—
`ValidTo`	DATE	N	—	—	—	—
`IsCurrent`	BIT	N	—	—	—	—

6.4 Extract Query for DimCustomer

The S2T mapping directly drives the extract query. Every JOIN, rename, and derivation in the mapping is implemented here:

```
-- Extract query for DimCustomer — driven by the S2T mapping above
USE CabotTrailOutdoor;

SELECT
    c.CustomerID,
    c.CustomerName,

    -- GAP-C06: Rename CustomerGroupName → CustomerCategoryName
    c.CustomerGroupName                         AS CustomerCategoryName,

    c.CreditLimit,
    c.AccountOpenedDate,

    -- GAP-C04: IsOnCreditHold not in DW source; default to 0
    0                                           AS IsOnCreditHold,

    -- GAP-C01: JOIN geography hierarchy (structural gap)
    ci.CityName,
    sp.StateProvinceName                        AS ProvinceName,
    sp.StateProvinceCode                        AS ProvinceCode,
    co.CountryName,

    -- GAP-C03: PostalCode not available; NULL default
    NULL                                        AS PostalCode,

    -- GAP-C02: SalesTerritory derived from ProvinceCode (missing data gap)
    CASE sp.StateProvinceCode
        WHEN 'NS' THEN 'Atlantic — Nova Scotia'
        WHEN 'NB' THEN 'Atlantic — New Brunswick'
        WHEN 'PE' THEN 'Atlantic — PEI'
        WHEN 'NL' THEN 'Atlantic — Newfoundland'
        WHEN 'QC' THEN 'Quebec'
        WHEN 'ON' THEN 'Ontario'
        WHEN 'MB' THEN 'Prairies'
        WHEN 'SK' THEN 'Prairies'
        WHEN 'AB' THEN 'Alberta'
        WHEN 'BC' THEN 'British Columbia'
        ELSE 'Other'
    END                                         AS SalesTerritory,

    -- GAP-C05: SCD static defaults
    CAST('2022-01-01' AS DATE)                  AS ValidFrom,
    CAST('9999-12-31' AS DATE)                  AS ValidTo,
    1                                           AS IsCurrent

FROM    Sales.Customers c

-- GAP-C01: JOIN to resolve geography
INNER JOIN Application.Cities ci
    ON ci.CityID = c.DeliveryCityID
INNER JOIN Application.StateProvinces sp
    ON sp.StateProvinceID = ci.StateProvinceID
INNER JOIN Application.Countries co
```

```
    ON co.CountryID = sp.CountryID

-- Exclude the unknown member
WHERE   c.CustomerID <> 0;
```

> *Notice how every line of this query can be traced back to a row in the S2T mapping table. The mapping is not just documentation — it is the specification from which the code is derived. If the mapping changes, the code changes in a predictable, documented way.*

---

7. Worked Example: Mapping FactSales

Fact table mapping is more complex than dimension mapping because it involves multiple lookups, derived measures, and a clearly defined grain.

7.1 Grain Declaration

Before mapping any columns, declare the grain:

> ***Grain:*** *One row per product on each customer invoice — representing the quantity ordered, quantity picked, unit price, tax, line total, unit cost (derived from `StandardCostPct`), gross profit, and gross profit margin for a single product on a single invoice.*

7.2 Source Tables Involved

`Fact.FactSales` is built from seven source tables:

Source table	Role
`Sales.InvoiceLines`	Primary source — line-level measures
`Sales.Invoices`	Invoice date, due date, delivery method, sales rep
`Sales.Orders`	Order date, city
`Sales.OrderLines`	Ordered quantity, picked quantity
`Sales.Customers`	Customer → used for CustomerKey lookup
`Inventory.ProductCategoryAssignments`	Primary category → for cost rate lookup
`Inventory.ProductCategories`	`StandardCostPct` → for UnitCost derivation

7.3 Gap Analysis: FactSales

Gap ID	Type	Description	Resolution
GAP-F01	Structural	`CustomerKey` required but source has `CustomerID`	LOOKUP DimCustomer on CustomerID → CustomerKey
GAP-F02	Structural	`ProductKey` required but source has `ProductID`	LOOKUP DimProduct on ProductID → ProductKey
GAP-F03	Structural	`EmployeeKey` required but source has `SalesRepID`	LOOKUP DimEmployee on EmployeeID → EmployeeKey
GAP-F04	Structural	`GeographyKey` required but source has `CityID`	LOOKUP DimGeography on CityID → GeographyKey
GAP-F05	Structural	`DeliveryMethodKey` required but source has `DeliveryMethodID`	LOOKUP DimDeliveryMethod on DeliveryMethodID → DeliveryMethodKey
GAP-F06	Structural	Date keys must be YYYYMMDD integers; source has DATE columns	CONVERT: `CAST(FORMAT(date, 'yyyyMMdd') AS INT)`
GAP-F07	Missing	`UnitCost` not on `InvoiceLines`; must be derived	COMPUTE: `UnitPrice × StandardCostPct × PickedQuantity`
GAP-F08	Missing	`GrossProfit` not on `InvoiceLines`	COMPUTE: `LineTotal - UnitCost`
GAP-F09	Missing	`GrossProfitMarginPct` not on `InvoiceLines`	COMPUTE: `GrossProfit / LineTotal × 100` (NULLIF guard)
GAP-F10	Structural	`LineTotalIncludingTax` is a computed column in OLTP; not selectable by name	COMPUTE: `LineTotal + TaxAmount`
GAP-F11	Structural	M:M category relationship; cost rate requires primary category	AGG: `MIN(ProductCategoryID)` per ProductID
GAP-F12	Missing	Exclude 2026 open orders (no invoices)	FILTER: `YEAR(OrderDate) < 2026`

7.4 Key Measures: Derivation Rules

The three most important derived measures in `FactSales` deserve explicit documentation of their derivation logic:

UnitCost:

```
Source:   il.UnitPrice (from Sales.InvoiceLines)
          × pcat.StandardCostPct (from Inventory.ProductCategories via primary category)
          × ol.PickedQuantity (from Sales.OrderLines)
Formula:  ROUND(il.UnitPrice * pcat.StandardCostPct * ol.PickedQuantity, 2)
Rationale: Represents the total cost of goods for the picked quantity of this line.
           StandardCostPct varies by product category (range: 0.45 to 0.70),
           producing meaningful margin variation across the product catalogue.
```

GrossProfit:

```
Source:   il.LineTotal - UnitCost (as derived above)
Formula:  ROUND(il.LineTotal - (il.UnitPrice * pcat.StandardCostPct * ol.PickedQuantity),
2)
Rationale: Revenue minus cost for this invoice line.
           Additive: can be summed across any dimension.
```

GrossProfitMarginPct:

```
Source:   GrossProfit / LineTotal × 100
Formula:  CASE WHEN il.LineTotal = 0 THEN 0
               ELSE ROUND((il.LineTotal - (il.UnitPrice * pcat.StandardCostPct *
ol.PickedQuantity))
                           / il.LineTotal * 100, 2)
          END
Rationale: Percentage margin for this line.
           NON-ADDITIVE: never sum this column. Always compute from GrossProfit /
LineTotal.
           Stored for convenience only.
```

7.5 Complete Extract Query for FactSales

```
-- FactSales extract query — implements all gaps from the mapping
USE CabotTrailOutdoor;

SELECT
    -- GAP-F06: Convert date columns to YYYYMMDD integer keys
    CAST(FORMAT(o.OrderDate,   'yyyyMMdd') AS INT)          AS OrderDateKey,
    CAST(FORMAT(i.InvoiceDate, 'yyyyMMdd') AS INT)          AS InvoiceDateKey,
    CAST(FORMAT(i.DueDate,     'yyyyMMdd') AS INT)          AS DueDateKey,

    -- GAP-F01 to F05: Natural keys for SSIS Lookup transformations
    -- These are replaced by surrogate keys during the SSIS Lookup stage
    c.CustomerID,
    il.ProductID,
    i.SalesRepID                                            AS EmployeeID,
    o.CityID,
    i.DeliveryMethodID,

    -- Degenerate dimensions
    i.InvoiceID,
    o.OrderID,
    il.OrderLineID,

    -- Direct measures
    ol.Quantity                                             AS OrderedQuantity,
    ol.PickedQuantity,
    il.UnitPrice,
    il.TaxRate,
    il.LineTotal,
    il.TaxAmount,

    -- GAP-F10: LineTotalIncludingTax computed inline
    ROUND(il.LineTotal + il.TaxAmount, 2)                   AS LineTotalIncludingTax,

    -- GAP-F07: UnitCost derived from StandardCostPct
    ROUND(il.UnitPrice * pcat.StandardCostPct
          * ol.PickedQuantity, 2)                           AS UnitCost,

    -- GAP-F08: GrossProfit
    ROUND(il.LineTotal
          - (il.UnitPrice * pcat.StandardCostPct
             * ol.PickedQuantity), 2)                       AS GrossProfit,

    -- GAP-F09: GrossProfitMarginPct (non-additive — stored for convenience)
    CASE
        WHEN il.LineTotal = 0 THEN 0
        ELSE ROUND(
            (il.LineTotal - (il.UnitPrice * pcat.StandardCostPct * ol.PickedQuantity))
            / il.LineTotal * 100, 2)
    END                                                     AS GrossProfitMarginPct

FROM    Sales.InvoiceLines il
INNER JOIN Sales.Invoices i     ON i.InvoiceID   = il.InvoiceID
INNER JOIN Sales.Orders o       ON o.OrderID     = i.OrderID
```

```
INNER JOIN Sales.OrderLines ol  ON ol.OrderLineID = il.OrderLineID
INNER JOIN Sales.Customers c    ON c.CustomerID  = i.CustomerID

-- GAP-F11: Primary category via MIN aggregation (resolve M:M)
INNER JOIN (
    SELECT  pca.ProductID,
            MIN(pca.ProductCategoryID)  AS PrimaryCategoryID
    FROM    Inventory.ProductCategoryAssignments pca
    GROUP BY pca.ProductID
) pri ON pri.ProductID = il.ProductID

-- GAP-F07: StandardCostPct from primary category
INNER JOIN Inventory.ProductCategories pcat
    ON pcat.ProductCategoryID = pri.PrimaryCategoryID

-- GAP-F12: Exclude open orders (2026) with no invoices
WHERE   YEAR(o.OrderDate) < 2026;
```

This extract query produces every row and column needed for `FactSales`, with surrogate key lookups remaining to be resolved in the SSIS Data Flow (covered in Chapter 4).

---

8. The Relationship Between Mapping and Implementation

The S2T mapping and the ETL implementation should maintain a one-to-one correspondence throughout the life of the system. This discipline is what makes ETL systems maintainable.

8.1 Traceability

Every column in the target should be traceable through the S2T mapping to its source. Every transformation in an SSIS package should have a corresponding row in the S2T mapping that describes what it does and why.

When a business analyst asks "why does this customer's territory show as 'Other' instead of 'Atlantic — Nova Scotia'?", the trace is:

1. `SalesTerritory` in `dim.Customer` → S2T mapping row → Transformation Rule: CASE on `ProvinceCode`
2. `ProvinceCode` in `dim.Customer` → S2T mapping row → Source: `Application.StateProvinces.StateProvinceCode`
3. Check: is `StateProvinceCode` populated for this customer? If not, CASE falls to `ELSE 'Other'`

Without the mapping, this trace requires reading SSIS package expressions — slow, error-prone, and inaccessible to non-developers.

8.2 Change Management

When a business rule changes — for example, Quebec is split into two territories ("Quebec East" and "Quebec West") — the change process is:

1. Update the S2T mapping (modify the CASE expression in the `SalesTerritory` derivation rule)
2. Update the SSIS Derived Column expression to match
3. Test that the new rule produces the expected output
4. Deploy and run the updated ETL

Without the mapping, step 1 is skipped — the change is made directly in the package. The next developer who looks at the package has no idea the territory logic was changed, when, or why.

8.3 The Mapping as a Test Specification

The S2T mapping can be directly translated into automated reconciliation tests:

```
-- Test derived from S2T mapping: DimCustomer SalesTerritory derivation
-- Expected: all NS customers have SalesTerritory = 'Atlantic — Nova Scotia'
SELECT  COUNT(*) AS FailCount
FROM    CabotTrailOutdoorDW.Dimension.DimCustomer
WHERE   ProvinceCode = 'NS'
AND     SalesTerritory <> 'Atlantic — Nova Scotia'
AND     CustomerID <> 0;
-- Should return 0

-- Test derived from S2T mapping: FactSales GrossProfit derivation
-- Expected: stored GrossProfit matches computed value from LineTotal and UnitCost
SELECT  COUNT(*) AS DerivationMismatches
FROM    CabotTrailOutdoorDW.Fact.FactSales
WHERE   ABS(GrossProfit - (LineTotal - UnitCost)) > 0.01  -- Allow $0.01 rounding
tolerance
AND     OrderDateKey > 0;
-- Should return 0
```

These tests are not ad hoc — they flow directly from the documented transformation rules. The S2T mapping is the specification; the tests verify conformance to the specification.

---

9. Chapter Summary

- **Gap analysis** identifies the differences between source and target before ETL development begins. There are three gap types: **missing data** (target needs something the source does not have), **data quality** (source data exists but is dirty), and **structural** (data exists but is organized differently).

- **Source data analysis** profiles source tables systematically — row counts, column distributions, NULL rates, value ranges, and referential integrity. It must be completed before ETL design begins.
- **Data quality assessment** documents quality rules as a formal table with rule ID, severity (High/Medium/Low), SQL measurement, and pass condition. Rules drive ETL error handling strategy: fail fast, reject and continue, or substitute and continue.
- The **source-to-target mapping** is the core ETL design document. It specifies every target column, its source, and its transformation rule. It drives implementation, enables testing, and supports change management.
- The **extract query** is derived directly from the S2T mapping. Every JOIN, rename, derivation, and conversion in the mapping appears as a corresponding SQL construct in the extract query.
- The S2T mapping must be **maintained alongside the implementation** throughout the life of the ETL system. It is the bridge between the business rules and the technical implementation.

---

10. Review Questions

1. A new dimension table `DimPromotion` is being added to the data warehouse. The source system stores promotions in a table `Marketing.Promotions` with columns `PromotionID`, `PromotionName`, `StartDate`, `EndDate`, and `DiscountPctText` (a NVARCHAR column that stores values like '15%' or '10.5%'). Identify the gap type(s) present and describe the transformation rule(s) needed.
2. During source data analysis of `Sales.InvoiceLines`, you find that `UnitPrice <= 0` for 3 rows out of 16,359. You have defined this as a High-severity data quality rule (DQ-003). Describe the two ETL error handling strategies available and give a specific reason to choose one over the other for this scenario.
3. Explain why referential integrity checks against the source data are particularly important for ETL that uses SSIS Lookup transformations. What happens in SSIS when a lookup key is not found, and what are the two ways to handle this?
4. The S2T mapping for `DimProduct` shows `ProductCategoryKey` with transformation rule `LOOKUP DimProductCategory ON ProductCategoryID → ProductCategoryKey`. What must be true about `DimProductCategory` before the `DimProduct` load can run? What happens if the sequence is reversed?
5. `GrossProfitMarginPct` is stored in `Fact.FactSales` with the S2T mapping noting it is non-additive. A developer writes a report that shows `SUM(GrossProfitMarginPct)` grouped by `CalendarYear`. What is wrong with this, and what should the query do instead?

6. The `SalesTerritory` derivation in `DimCustomer` uses a CASE expression on `ProvinceCode`. Six months after deployment, the business decides that Manitoba and Saskatchewan should be separated (currently both show as 'Prairies'). Walk through the complete change management process, starting with the S2T mapping.
7. A source data analysis reveals that `Purchasing.PurchaseOrders.ActualDeliveryDate` is NULL for 15% of rows — those are open orders not yet delivered. Is this a data quality gap? How should it be handled in the S2T mapping for `Fact.FactPurchasing`?
8. Why is it important to run referential integrity checks against the *source* data before loading the *target*? Give a specific example from the CabotTrail environment where an RI failure in the source would cause a specific failure in the ETL load.

Deeper Dive

Going Further with Gap Analysis and S2T Mapping

Data Profiling at Scale

The source data analysis queries shown in this chapter are effective for tables with thousands to hundreds of thousands of rows. At enterprise scale — billions of rows — running `COUNT(*)`, `MIN()`, `MAX()`, and `COUNT(DISTINCT)` across entire tables becomes impractical.

Enterprise ETL platforms and data observability tools address this with **statistical sampling** and **incremental profiling**:

- **Statistical sampling** — profile a random sample (1% or 10%) of the table rather than the full dataset. For normally distributed data, this produces accurate statistics at a fraction of the cost.
- **Incremental profiling** — rather than profiling the full table on every run, profile only the rows that changed since the last profile run (using a watermark timestamp). This keeps profiling overhead constant regardless of table size.

Microsoft SQL Server's **Data Quality Services (DQS)** provides a managed data quality platform with built-in profiling, rule definition, and cleansing workflows. While DQS is beyond the scope of this book, it represents the enterprise-scale evolution of the manual DQ approach described here:
SQL Server Data Quality Services

Data Lineage and Impact Analysis

The S2T mapping described in this chapter is a manual documentation artifact. Enterprise data management platforms automate lineage tracking — recording, at runtime, which source rows contributed to which target rows, through which transformation steps.

Data lineage answers "where did this data come from?" Impact analysis (the inverse) answers "if I change this source column, what downstream targets are affected?"

Tools that provide automated lineage in SQL Server environments include:
- **SQL Server Integration Services** — the SSIS Catalog (`SSISDB`) records package execution history but not column-level lineage
- **Microsoft Purview** — Microsoft's enterprise data governance platform with automated lineage across Azure data services
- **Apache Atlas** — open-source metadata management with lineage tracking
- **Informatica Data Catalog** — commercial metadata and lineage platform

Microsoft Purview documentation: Microsoft Purview data governance documentation

The Data Vault Approach to Source Analysis

The **Data Vault 2.0** methodology (Dan Linstedt) takes a different approach to the source-to-target problem. Rather than mapping sources to a pre-designed dimensional model, Data Vault loads raw source data into a highly normalized integration layer (Hubs, Links, and Satellites), preserving all source data exactly as received. Transformation to a dimensional model happens in a second layer (the "Information Mart").

This approach changes the nature of gap analysis: instead of asking "what does the target need?", Data Vault asks "what does the source have?" and loads everything. Dimensional model decisions are deferred to the reporting layer.

Data Vault is particularly effective when:
- Multiple heterogeneous sources need to be integrated
- Source schemas change frequently
- Full auditability of every source record is required

For contexts where the source is stable and the target model is well-defined (like CabotTrail), Kimball's approach is more efficient. Understanding both helps practitioners choose the right methodology for a given project.

Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann.

Formal Data Quality Frameworks

The data quality dimensions described in section 4.1 (completeness, accuracy, consistency, etc.) are drawn from a formal body of literature on data quality management. The most widely referenced framework is the **DAMA-DMBOK** (Data Management Body of Knowledge), which defines data quality as one of eleven data management disciplines:

DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications.

The **ISO 8000** standard provides international specifications for data quality, particularly for master data and transaction data in supply chain contexts. While ISO 8000 is primarily relevant to large enterprises, understanding its existence positions practitioners in the broader context of data governance.

For practitioners building production ETL systems, the **TDWI (Transforming Data with Intelligence)** organization provides research, training, and best practices specifically for data warehousing and BI: TDWI — Data Quality Resources

S2T Mapping Tools

While spreadsheet-based S2T mappings (Excel or Google Sheets) are the most common format in practice, several tools exist for managing mappings at scale:

- **WhereScape** — metadata-driven ETL automation platform that generates SSIS packages from S2T mapping metadata
- **Manta** — automated data lineage and impact analysis from SQL code
- **Talend Data Catalog** — metadata management with S2T documentation support
- **dbt (Data Build Tool)** — for ELT architectures, dbt's documentation layer provides S2T mapping functionality built into the transformation code

The principle remains the same across all tools: the mapping is the specification, and the specification must be maintained alongside the implementation.

Industry Perspectives

Kimball on Source System Analysis

Kimball dedicates significant attention in *The Data Warehouse Toolkit* to the importance of understanding source data before designing the dimensional model. His guidance is characteristically direct:

> *"The single biggest mistake a data warehouse team can make is to not fully understand the source data before designing the data warehouse. You will be blindsided by data quality issues, structural anomalies, and missing data — and you will discover them in production, not in development."*

The source data analysis work described in this chapter is not optional pre-work — it is a core design activity. Many ETL project failures can be traced to teams that began implementation before source analysis was complete and discovered late-breaking data quality issues that required redesigning already-built packages.

Microsoft on Data Quality in SSIS

Microsoft's documentation for SSIS error handling covers both Conditional Split and Error Output in depth:
- SSIS — Error Handling in Data
- SSIS — Conditional Split Transformation

The SSIS Data Profiling Task (available in the Control Flow) provides automated column profiling:
SSIS — Data Profiling Task

References and Further Reading

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley. — Chapter 16 covers ETL design patterns and source data analysis in detail.
2. Kimball, R., Reeves, L., Ross, M., & Thornthwaite, W. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — The companion to the Toolkit, focused entirely on ETL design and implementation. Chapter 3 covers source data analysis; Chapter 4 covers data quality.
3. DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications. — Chapter 13 defines the data quality dimensions framework.
4. Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann. — Alternative approach to source-to-target design; useful comparative context.
5. Microsoft. (2024). *Error Handling in Data — SSIS*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/error-handling-in-data
6. Microsoft. (2024). *Data Profiling Task — SSIS*. https://learn.microsoft.com/en-us/sql/integration-services/control-flow/data-profiling-task
7. Microsoft. (2024). *SQL Server Data Quality Services*. https://learn.microsoft.com/en-us/sql/data-quality-services/data-quality-services

8. Microsoft. (2024). *Microsoft Purview data governance documentation*. https://learn.microsoft.com/en-us/purview/

9. TDWI. (n.d.). *Data Quality Resources*. https://tdwi.org/home.aspx

10. Redman, T. C. (2001). *Data Quality: The Field Guide*. Digital Press. — A practitioner-focused treatment of data quality management principles that complements the technical ETL approach.

Chapter 4: ETL Implementation — Dimensions, Lookups, and Derivations

Chapter Overview

Chapters 1 through 3 covered the *what* and *why* of ETL — the architecture, the dimensional model, and the design specifications that precede implementation. This chapter crosses the line into the *how*: building actual ETL pipelines in SQL Server Integration Services.

The S2T mapping from Chapter 3 is the specification. This chapter is the implementation guide. Every SSIS component introduced here maps directly to a transformation rule type from the mapping. By the end of this chapter, you will be able to translate a completed S2T mapping into a working, tested SSIS solution.

By the end of this chapter you will be able to:

- Describe the SSIS Control Flow and Data Flow and explain the purpose of each
- Build a dimension load package using OLE DB Source, Lookup, Derived Column, and OLE DB Destination
- Implement surrogate key resolution using the SSIS Lookup transformation
- Apply the Conditional Split transformation to implement data quality error handling
- Build a fact table load package that implements all lookup types identified in the Chapter 3 S2T mapping
- Write and execute reconciliation queries that verify ETL correctness
- Explain the load dependency order for a dimensional model and implement it in a master package

Table of Contents

1. SSIS Architecture: Control Flow and Data Flow

An SSIS package has two distinct execution environments: the **Control Flow** and the **Data Flow**. Understanding the separation between them is the foundation for designing packages that are correct, maintainable, and performant.

1.1 The Control Flow

The Control Flow is the **orchestration layer**. It determines what runs, in what order, under what conditions, and what to do when something fails. It contains tasks and containers connected by precedence constraints.

Think of the Control Flow as the conductor of an orchestra — it does not play any instruments itself, but it directs everything that does.

```
Control Flow of a typical dimension load package:

[Truncate DimCustomer]         ← Execute SQL Task
        ↓ Success
[Load DimCustomer]             ← Data Flow Task
        ↓ Success
[Run Reconciliation Tests]     ← Execute SQL Task
        ↓ Failure
[Log Error to ETL.TestResults] ← Execute SQL Task
```

Key Control Flow tasks:

Task	Purpose	Common use cases
Execute SQL Task	Run T-SQL against a connection	Truncate tables, call stored procedures, log results, create temp tables
Data Flow Task	Execute a data movement pipeline	Any extract-transform-load operation
Execute Package Task	Call a child package	Master package orchestration
Sequence Container	Group tasks with shared error handling	Group all dimension loads; group all fact loads
For Loop Container	Repeat a fixed number of times	Rarely used in ETL; more common in administrative scripts
Foreach Loop Container	Iterate over a collection	Process multiple files; iterate over a result set
Script Task	Execute C# or VB.NET code	Complex logic that cannot be expressed in other tasks

Precedence constraints define when a connected task runs:

Constraint	Condition	Visual
Success	Run next task only if this task succeeded	Green arrow
Failure	Run next task only if this task failed	Red arrow
Completion	Run next task regardless of outcome	Blue arrow
Expression	Run next task if a custom expression evaluates to TRUE	Dashed arrow

The most important constraint to understand is **Failure**. In production ETL, failure paths are as important as success paths. A package that silently swallows errors and reports success is dangerous. Every critical task should have a failure path that logs the error and alerts operations.

1.2 The Data Flow

The Data Flow is the **transformation engine**. It operates as a streaming pipeline — rows flow from a source through a series of transformations to a destination. Unlike the Control Flow, which runs tasks sequentially, the Data Flow processes rows in parallel — as rows leave the source, they flow into the first transformation without waiting for all rows to be read.

This streaming architecture is fundamental to SSIS performance. A Data Flow pipeline with one million rows does not wait for all one million to be read before starting transformations — rows flow through the entire pipeline continuously, processed in memory buffers.

```
Data Flow pipeline for a dimension load:

[OLE DB Source]              ← Read from OLTP
        ↓ (all rows)
[Derived Column]             ← Compute SalesTerritory, date keys, flags
        ↓
[Lookup: CustomerKey]        ← Resolve surrogate key (only for fact loads)
        ├── Match output ↓
        └── No Match → [Error OLE DB Destination]
[OLE DB Destination]         ← Write to DW dimension table
```

Key Data Flow components:

Component	Type	Purpose
OLE DB Source	Source	Read rows from SQL Server
Flat File Source	Source	Read rows from CSV or fixed-width files
OLE DB Destination	Destination	Write rows to SQL Server
Lookup	Transformation	Join to a reference table; resolve keys
Derived Column	Transformation	Add or replace columns using expressions
Conditional Split	Transformation	Route rows to different outputs based on a condition
Data Conversion	Transformation	Change a column's data type
Aggregate	Transformation	Group and summarize rows
Union All	Transformation	Combine rows from multiple inputs
Multicast	Transformation	Send the same rows to multiple outputs
Sort	Transformation	Order rows (use sparingly — breaks streaming)
Row Count	Transformation	Count rows passing through; store in a variable

1.3 The Buffer Architecture

The Data Flow's streaming performance comes from its **buffer architecture**. SSIS allocates memory buffers of fixed size (default 10 MB, configurable). Rows from the source fill a buffer; once the buffer is full, it is passed downstream to the next transformation while the source continues filling a new buffer.

This means that at any point during execution, multiple buffers may be in flight simultaneously — one being filled by the source, another being transformed, another being written to the destination. The pipeline processes continuously rather than in discrete steps.

The practical implication: the Data Flow is optimized for throughput (moving many rows efficiently) rather than latency (processing one row as fast as possible). For high-volume dimension and fact loads, the buffer architecture makes SSIS exceptionally fast.

---

2. The OLE DB Source

The **OLE DB Source** is the entry point for most ETL Data Flows. It reads rows from a SQL Server table, view, or query result and passes them into the pipeline.

2.1 Configuration

The OLE DB Source has three data access modes:

Mode	Description	When to use
Table or view	Read all rows from a named table or view	Simple lookups; complete table loads
Table or view — fast load	Same, with performance optimizations	Large tables where all rows are needed
SQL command	Execute a T-SQL query	Any transformation-at-source (joins, filters, derivations)
SQL command from variable	Execute a T-SQL query stored in an SSIS variable	Dynamic SQL; parameterized queries

For ETL loading dimensional models, **SQL command** is almost always the correct choice. The extract query from the S2T mapping (Chapter 3) is entered directly here.

2.2 Column Output

After configuring the source query, SSIS automatically detects the output columns and their data types. Review these carefully:

- Verify that date columns are mapped to `DT_DBDATE` or `DT_DBTIMESTAMP`, not `DT_WSTR` (string)
- Verify that decimal columns have the correct precision and scale
- Verify that integer columns are the appropriate size (`DT_I4` for INT, `DT_I2` for SMALLINT, `DT_I1` for TINYINT)

Type mismatches at the source cause failures deep in the pipeline — a type detected incorrectly at the source will fail at the destination, after all transformation work has already been done.

2.3 The Extract Query in the OLE DB Source

The extract query is the SQL developed in Chapter 3 — the complete SELECT statement that implements all structural gaps and joins from the S2T mapping. For `DimCustomer`:

```
-- Entered in OLE DB Source > SQL Command
SELECT
    c.CustomerID,
    c.CustomerName,
    c.CustomerGroupName                         AS CustomerCategoryName,
    c.CreditLimit,
    c.AccountOpenedDate,
    0                                           AS IsOnCreditHold,
    ci.CityName,
    sp.StateProvinceName                        AS ProvinceName,
    sp.StateProvinceCode                        AS ProvinceCode,
    co.CountryName,
    NULL                                        AS PostalCode,
    CASE sp.StateProvinceCode
        WHEN 'NS' THEN 'Atlantic — Nova Scotia'
        WHEN 'NB' THEN 'Atlantic — New Brunswick'
        WHEN 'PE' THEN 'Atlantic — PEI'
        WHEN 'NL' THEN 'Atlantic — Newfoundland'
        WHEN 'QC' THEN 'Quebec'
        WHEN 'ON' THEN 'Ontario'
        WHEN 'MB' THEN 'Prairies'
        WHEN 'SK' THEN 'Prairies'
        WHEN 'AB' THEN 'Alberta'
        WHEN 'BC' THEN 'British Columbia'
        ELSE 'Other'
    END                                         AS SalesTerritory,
    CAST('2022-01-01' AS DATE)                  AS ValidFrom,
    CAST('9999-12-31' AS DATE)                  AS ValidTo,
    1                                           AS IsCurrent
FROM    Sales.Customers c
INNER JOIN Application.Cities ci
    ON ci.CityID = c.DeliveryCityID
INNER JOIN Application.StateProvinces sp
    ON sp.StateProvinceID = ci.StateProvinceID
INNER JOIN Application.Countries co
    ON co.CountryID = sp.CountryID
WHERE   c.CustomerID <> 0
```

Notice that the `SalesTerritory` derivation from the S2T mapping is implemented entirely in the source SQL — not in a Derived Column transformation. This is a deliberate design choice: **transformations that can be expressed cleanly in SQL belong in SQL**. Moving complex CASE expressions into SSIS Derived Column expressions makes them harder to read, harder to debug, and harder to change.

The general principle: use the extract query for structural transformations (joins, renames, CASE expressions, NULL handling). Use SSIS transformations for operations that genuinely require in-pipeline processing (surrogate key lookups, conditional routing, data type conversions that SQL cannot handle cleanly).

---

3. The Lookup Transformation

The **Lookup** transformation is the most important transformation in dimensional ETL. It resolves natural keys from source data to surrogate keys in dimension tables — the core operation that links fact rows to their dimensions.

3.1 How the Lookup Works

A Lookup transformation is conceptually equivalent to a SQL LEFT JOIN — it takes each row flowing through the pipeline, searches a reference dataset for a matching row, and adds columns from the matching row to the pipeline row.

```
Pipeline row:  CustomerID = 42, LineTotal = 249.99, ...
Lookup dataset: SELECT CustomerID, CustomerKey FROM DimCustomer
                42 → CustomerKey = 15

Output row: CustomerKey = 15, LineTotal = 249.99, ...
```

The natural key (`CustomerID`) is the join column. The surrogate key (`CustomerKey`) is the output column added to the pipeline. After the lookup, the pipeline carries both the natural key (no longer needed for loading but useful for debugging) and the surrogate key (required as the FK in the fact table).

3.2 Lookup Configuration

Step 1 — Connection: Set the lookup connection to the DW database.

Step 2 — Query: Define the reference dataset — the query that produces the lookup table. Keep it minimal — only the columns needed for the join and the output:

```
-- Lookup reference query: DimCustomer
SELECT  CustomerID, CustomerKey
FROM    Dimension.DimCustomer
WHERE   CustomerID <> 0;

-- Lookup reference query: DimProduct
SELECT  ProductID, ProductKey
FROM    Dimension.DimProduct
WHERE   ProductID <> 0;

-- Lookup reference query: DimDate
SELECT  DateKey
FROM    Dimension.DimDate
WHERE   DateKey <> 0;
-- (No output column needed — just validates the key exists)
```

Step 3 — Join column: Map the pipeline column to the lookup column.
- Pipeline column: `CustomerID` → Lookup column: `CustomerID`

Step 4 — Output column: Select the column(s) to add to the pipeline from the lookup result.
- Add `CustomerKey` to the pipeline output

Step 5 — No match behaviour: This is the most important configuration decision.

No match behaviour	Effect	When to use
Fail component	Any unmatched row fails the entire Data Flow	Never use in production
Ignore failure	Unmatched rows pass through with NULL for lookup columns	Dangerous — silently loads NULL FKs
Redirect rows to no match output	Unmatched rows are sent to a separate error output	**Always use this in production**

The **redirect to no match output** option is the correct approach for all production ETL. Unmatched rows — rows where the natural key has no corresponding dimension record — are redirected to an error destination for investigation. The main load continues with clean rows.

3.3 Lookup Caching

By default, SSIS loads the entire lookup reference dataset into memory once and caches it for the duration of the Data Flow. This is **Full Cache mode** — the fastest option for lookups against small to medium reference tables (dimension tables typically qualify).

For very large reference tables (millions of rows), **Partial Cache mode** loads only the rows actually referenced, querying the database for each miss. This uses less memory but generates many small database round-trips, which can be slow.

For extremely large reference tables or when the reference data changes during the ETL run, **No Cache mode** queries the database for every single row — maximum flexibility but lowest performance.

Recommendation for CabotTrail: All lookups use Full Cache. The largest CabotTrail dimension table (`DimCustomer`) has 100 rows — trivially small for full cache loading.

3.4 Multiple Lookups in a Single Data Flow

A fact table load typically requires multiple Lookup transformations — one for each foreign key. They are chained sequentially in the Data Flow:

```
OLE DB Source (with CustomerID, ProductID, EmployeeID, CityID, DeliveryMethodID)
        ↓
Lookup: CustomerID → CustomerKey
        ↓ Match
Lookup: ProductID → ProductKey
        ↓ Match
Lookup: EmployeeID → EmployeeKey
        ↓ Match
Lookup: CityID → GeographyKey
        ↓ Match
Lookup: DeliveryMethodID → DeliveryMethodKey
        ↓ Match
OLE DB Destination (FactSales)
```

Each lookup adds its surrogate key to the pipeline. No-match outputs from each lookup are redirected to error tables for investigation. By the time rows reach the destination, all five surrogate keys are present and all rows have passed five validation checks.

3.5 Date Key Lookups

Date keys in the CabotTrail DW are YYYYMMDD integers computed in the extract query. They do not require a Lookup transformation — the integer key is computed from the source date and is by definition correct if the source date is valid. However, a **validation lookup** against `DimDate` is good practice to confirm that the computed key actually exists in the calendar:

```
-- Validation lookup reference: just confirm the key exists
SELECT DateKey FROM Dimension.DimDate WHERE DateKey <> 0;
```

This catches dates outside the DW's calendar range (e.g., a future order date that predates a calendar extension) that would violate the FK constraint at load time.

---

4. The Derived Column Transformation

The **Derived Column** transformation adds new columns to the pipeline or replaces existing column values using SSIS expressions. It is used when transformation logic cannot be expressed in the source SQL — typically when it depends on values computed by earlier transformations in the same pipeline, or when type conversions require SSIS-specific syntax.

4.1 SSIS Expression Language

SSIS uses its own expression language — syntactically similar to C# but distinct from T-SQL. Key differences:

Feature	T-SQL syntax	SSIS expression syntax
String literals	`'single quotes'`	`"double quotes"`
Conditional	`CASE WHEN x THEN y ELSE z END`	`x ? y : z` (ternary)
NULL check	`IS NULL`	`ISNULL(column)`
NULL substitute	`ISNULL(col, default)`	`ISNULL(col) ? default : col`
String concat	`col1 + col2`	`col1 + col2` (same)
Type cast	`CAST(x AS INT)`	`(DT_I4)x`
Date part	`YEAR(date)`	`YEAR(date)` (same)

Common SSIS data type codes:

Type code	SQL Server equivalent
`DT_I1`	TINYINT
`DT_I2`	SMALLINT
`DT_I4`	INT
`DT_I8`	BIGINT
`DT_R4`	REAL
`DT_R8`	FLOAT
`DT_DECIMAL`	DECIMAL/NUMERIC
`DT_WSTR`	NVARCHAR
`DT_STR`	VARCHAR
`DT_DBDATE`	DATE
`DT_DBTIMESTAMP`	DATETIME
`DT_BOOL`	BIT

4.2 Common Derived Column Expressions

Computing an integer date key from a DATE column:

```
(DT_I4)((DT_WSTR,8)(DT_DBDATE)[OrderDate])
```

This casts the date to a string in YYYYMMDD format then converts to INT. Equivalent to `CAST(FORMAT(OrderDate, 'yyyyMMdd') AS INT)` in T-SQL.

A simpler approach using YEAR, MONTH, DAY:

```
YEAR([OrderDate]) * 10000 + MONTH([OrderDate]) * 100 + DAY([OrderDate])
```

Both produce the same integer result; the second is more readable.

CustomerTier derivation from CreditLimit:

```
[CreditLimit] > 10000 ? "Enterprise" : ([CreditLimit] >= 1000 ? "Standard" : "New")
```

NULL substitution:

```
ISNULL([ProductCode]) ? "No Code" : [ProductCode]
```

MarginTier from CategoryName:

```
[CategoryName] == "Accessories" || [CategoryName] == "Hydration" ||
[CategoryName] == "Navigation"  || [CategoryName] == "Safety and First Aid"
? "High"
: ([CategoryName] == "Climbing and Rappelling" || [CategoryName] == "Cooking and Food" ||
   [CategoryName] == "Apparel - Outerwear"     || [CategoryName] == "Apparel - Tops"   ||
   [CategoryName] == "Backpacks and Bags"
   ? "Medium"
   : "Low")
```

IsWeekend flag from DayOfWeekNumber:

```
[DayOfWeekNumber] == 1 || [DayOfWeekNumber] == 7 ? (DT_BOOL)1 : (DT_BOOL)0
```

4.3 When to Use Derived Column vs Source SQL

The choice between implementing a derivation in the source SQL query or in a SSIS Derived Column transformation follows a practical rule:

Use source SQL when:
- The derivation depends only on source table columns
- The logic is a CASE expression, calculation, or JOIN that SQL handles naturally
- The result is needed by downstream SSIS transformations (it can be referenced in Lookup join conditions)

Use SSIS Derived Column when:
- The derivation depends on a column produced by an earlier SSIS transformation (e.g., a surrogate key added by a Lookup)

- The derivation requires SSIS-specific type casting
- A flag or indicator needs to be set based on a Lookup match/no-match outcome

For CabotTrail ETL, most derivations are implemented in source SQL. The SSIS Derived Column transformation is used primarily for:
1. Setting `IsInvoice`, `IsPayment`, `IsCreditNote` flags based on `TransactionTypeName` (after the TransactionType lookup)
2. Type conversions that SSIS requires explicitly

---

5. The Conditional Split Transformation

The **Conditional Split** transformation routes rows to different outputs based on conditions. It is the primary tool for implementing data quality error handling in the Data Flow.

5.1 How Conditional Split Works

A Conditional Split evaluates conditions in order and routes each row to the first matching output. Rows that match no condition go to the **default output**.

```
Rows entering Conditional Split:

Condition 1: UnitPrice > 0 AND Quantity > 0 AND CustomerID IS NOT NULL
  → Output: ValidRows → continues to Lookup transformations

Condition 2: UnitPrice <= 0
  → Output: InvalidPrice → Error destination

Condition 3: Quantity <= 0
  → Output: InvalidQuantity → Error destination

Default: everything else (shouldn't happen if conditions are complete)
  → Output: UnexpectedRows → Error destination
```

5.2 SSIS Expression Syntax for Conditions

```
-- Valid row condition
[UnitPrice] > 0 && [Quantity] > 0 && ![ISNULL([CustomerID])]

-- Invalid price
[UnitPrice] <= 0

-- NULL CustomerID
ISNULL([CustomerID])
```

Note the SSIS logical operators: `&&` (AND), `||` (OR), `!` (NOT).

5.3 The Error Destination

The error output from a Conditional Split (or from a Lookup no-match output) should be directed to an **error logging table** in the DW:

```
-- Error log table for ETL load failures
USE CabotTrailOutdoorDW;
GO

CREATE TABLE ETL.LoadErrors
(
    ErrorID         INT             NOT NULL IDENTITY(1,1),
    PackageName     NVARCHAR(200)   NOT NULL,
    ComponentName   NVARCHAR(200)   NOT NULL,
    ErrorCode       INT             NULL,
    ErrorColumn     INT             NULL,
    ErrorDescription NVARCHAR(MAX)  NULL,
    RejectedRow     NVARCHAR(MAX)   NULL,   -- JSON or concatenated key values
    LoadDate        DATETIME2       NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT PK_ETL_LoadErrors PRIMARY KEY (ErrorID)
);
GO
```

In SSIS, the OLE DB Destination pointed at `ETL.LoadErrors` receives rejected rows. Package-level variables (`@PackageName`, `@ComponentName`) populated by SSIS system variables provide context for each error record.

5.4 Implementing the DQ Strategy in SSIS

Recall the three error handling strategies from Chapter 3. Here is how each maps to SSIS components:

Fail fast: Use a `Fail component` setting on the Lookup or add a Conditional Split that routes invalid rows to a **Script Task** that throws an exception — terminating the entire package.

Reject and continue: Use Conditional Split or Lookup no-match output → **OLE DB Destination** pointing at `ETL.LoadErrors`. The main flow continues with valid rows.

Substitute and continue: Use a **Derived Column** transformation to replace the invalid value with a default before writing to the destination. Add a `Flag_Substituted` column set to TRUE for rows where substitution occurred, and log these to an audit table.

---

6. The OLE DB Destination

The **OLE DB Destination** writes rows from the pipeline into a SQL Server table. It is the final component in most Data Flow pipelines.

6.1 Configuration

Connection: The DW connection manager.

Table: The target dimension or fact table. Always select the table explicitly — never use the auto-create option in production, as it generates tables without constraints, correct data types, or indexes.

Column mapping: Map each pipeline column to its target table column. SSIS attempts automatic mapping by name — always verify this manually, especially when source and target column names differ.

6.2 Fast Load vs Standard Load

OLE DB Destination offers two write modes:

Table or view — fast load (recommended): Uses SQL Server's bulk insert mechanism. Significantly faster than row-by-row insertion for large datasets. Supports options:
- `Keep identity` — do not generate surrogate keys; use values from the pipeline (use when loading from a DW to a data mart where surrogate keys are carried over)
- `Keep nulls` — preserve explicit NULLs rather than substituting column defaults
- `Table lock` — acquire a table-level lock for the duration of the load (faster but blocks concurrent access)
- `Check constraints` — enforce CHECK constraints during load (slight performance cost but catches violations)
- `Rows per batch` — number of rows per bulk insert batch (default 0 = all rows in one batch)
- `Maximum insert commit size` — number of rows per commit (smaller = more frequent commits = better recoverability)

Table or view — standard load: Row-by-row insertion. Much slower. Use only when fast load is not appropriate (e.g., when inserting into tables with triggers that must fire per row).

6.3 Keep Identity: Loading Data Marts from the DW

When loading a data mart (e.g., `CabotTrailOutdoorsSales.dim.Customer`) from the DW (`CabotTrailOutdoorDW.Dimension.DimCustomer`), the surrogate keys from the DW must be preserved exactly. The data mart FK relationships depend on the same surrogate key values existing in both databases.

In the OLE DB Destination for data mart loads:
- Enable `Keep identity` — this tells SQL Server to use the `CustomerKey` value from the pipeline rather

than generating a new IDENTITY value
- The target table's IDENTITY column must have `IDENTITY_INSERT ON` active during the load

In T-SQL, this is done explicitly:

```
SET IDENTITY_INSERT dim.Customer ON;

INSERT INTO dim.Customer (CustomerKey, CustomerID, CustomerName, ...)
SELECT CustomerKey, CustomerID, CustomerName, ...
FROM   CabotTrailOutdoorDW.Dimension.DimCustomer;

SET IDENTITY_INSERT dim.Customer OFF;
```

In SSIS, enabling `Keep identity` on the OLE DB Destination handles this automatically.

---

7. Building a Dimension Load Package

With each individual component understood, we can build a complete dimension load package. This section walks through `Load_DimProduct.dtsx` — the most complex CabotTrail dimension because it requires two lookup transformations.

7.1 Package Design

```
Control Flow:
┌──────────────────────────────────────────────────────────┐
│  [Execute SQL: Truncate DimProduct]                      │
│          ↓ Success                                       │
│  [Data Flow: Load DimProduct]                            │
│          ↓ Success                                       │
│  [Execute SQL: Record reconciliation test]               │
│          ↓ Failure                                       │
│  [Execute SQL: Log error to ETL.LoadErrors]              │
└──────────────────────────────────────────────────────────┘

Data Flow:
┌──────────────────────────────────────────────────────────┐
│  [OLE DB Source: Products extract query]                 │
│          ↓                                               │
│  [Lookup: ProductCategoryID → ProductCategoryKey]        │
│          ├── Match → continues                           │
│          └── No Match → [Error OLE DB Destination]       │
│  [Lookup: SupplierID → SupplierKey]                      │
│          ├── Match → continues                           │
│          └── No Match → [Error OLE DB Destination]       │
│  [OLE DB Destination: DimProduct]                        │
└──────────────────────────────────────────────────────────┘
```

7.2 Execute SQL Task: Truncate

```
-- Truncate statement in Execute SQL Task
TRUNCATE TABLE Dimension.DimProduct;
```

> ***Why truncate instead of delete?*** `TRUNCATE TABLE` *is a minimally logged operation — it deallocates all data pages without logging individual row deletions. For a table with 142 rows, the difference is negligible. For fact tables with millions of rows, truncate is orders of magnitude faster than DELETE.*
>
> ***Constraint note:*** `TRUNCATE TABLE` *is blocked when there are active FK references from other tables. This is why fact tables must be truncated before their dimensions in a full reload. The reverse (truncating a dimension while its fact table still has rows) will fail with a constraint violation.*

7.3 OLE DB Source: Products Extract Query

```
-- Products extract query (in OLE DB Source > SQL Command)
SELECT
    p.ProductID,
    p.ProductCode,
    p.ProductName,
    pca.ProductCategoryID,      -- Used in Lookup 1 below
    c.ColorName,
    pt.PackageTypeName,
    p.UnitPrice,
    p.RecommendedRetailPrice,
    p.TypicalWeightPerUnit,
    p.IsDiscontinued,
    p.SupplierID                -- Used in Lookup 2 below
FROM    Inventory.Products p
LEFT  JOIN Inventory.Colors c
    ON c.ColorID = p.ColorID
INNER JOIN Inventory.PackageTypes pt
    ON pt.PackageTypeID = p.PackageTypeID
INNER JOIN (
    -- Resolve M:M to primary category
    SELECT  ProductID,
            MIN(ProductCategoryID) AS ProductCategoryID
    FROM    Inventory.ProductCategoryAssignments
    GROUP BY ProductID
) pca ON pca.ProductID = p.ProductID
WHERE   p.ProductID <> 0;
```

7.4 Lookup 1: ProductCategoryID → ProductCategoryKey

Reference query:

```
SELECT  ProductCategoryID, ProductCategoryKey
FROM    CabotTrailOutdoorDW.Dimension.DimProductCategory
WHERE   ProductCategoryID <> 0;
```

Join column: `ProductCategoryID` (pipeline) → `ProductCategoryID` (lookup)

Output column: `ProductCategoryKey` → add to pipeline

No match behaviour: Redirect rows to no match output → Error OLE DB Destination

> ***Prerequisite:*** *`DimProductCategory` must be loaded before `DimProduct`. If this lookup runs against an empty `DimProductCategory`, every row will be a no-match and the entire load will produce 0 rows in `DimProduct`. This is the load order dependency in action.*

7.5 Lookup 2: SupplierID → SupplierKey

Reference query:

```
SELECT  SupplierID, SupplierKey
FROM    CabotTrailOutdoorDW.Dimension.DimSupplier
WHERE   SupplierID <> 0;
```

Join column: `SupplierID` (pipeline) → `SupplierID` (lookup)

Output column: `SupplierKey` → add to pipeline

No match behaviour: Redirect rows to no match output → Error OLE DB Destination

7.6 OLE DB Destination: DimProduct

Connection: CabotTrailOutdoorDW

Table: `[Dimension].[DimProduct]`

Keep identity: OFF — `ProductKey` is an IDENTITY column generated by the DW

Column mapping (key mappings to verify):

Pipeline column	Target column
`ProductCategoryKey`	`ProductCategoryKey`
`SupplierKey`	`SupplierKey`
`ProductID`	`ProductID`
`ProductName`	`ProductName`
`ColorName`	`ColorName`
`PackageTypeName`	`PackageTypeName`

7.7 Execute SQL Task: Reconciliation

After the Data Flow completes, the Execute SQL Task runs a reconciliation check and logs the result:

```
-- Reconciliation test for DimProduct
INSERT INTO ETL.TestResults
    (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed)
SELECT
    'DimProduct Row Count vs Source'    AS TestName,
    'Reconciliation'                    AS TestCategory,
    'Dimension.DimProduct'              AS TargetTable,
    CAST(src.SourceCount AS NVARCHAR)   AS ExpectedValue,
    CAST(dw.DWCount AS NVARCHAR)        AS ActualValue,
    CASE WHEN src.SourceCount = dw.DWCount THEN 1 ELSE 0 END AS Passed
FROM
    (SELECT COUNT(*) AS SourceCount
     FROM CabotTrailOutdoor.Inventory.Products
     WHERE ProductID <> 0) src,
    (SELECT COUNT(*) AS DWCount
     FROM Dimension.DimProduct
     WHERE ProductID <> 0) dw;
```

8. Building a Fact Table Load Package

Fact table load packages are structured identically to dimension packages — the complexity lies in the number of lookup transformations required and the precision of the measure derivations.

8.1 Package Design: Load_FactSales.dtsx

```
Control Flow:
[Execute SQL: Truncate Fact.FactSales]
        ↓ Success
[Data Flow: Load FactSales]
        ↓ Success
[Execute SQL: Record reconciliation tests (row count + revenue)]
        ↓ Failure
[Execute SQL: Log error]

Data Flow:
[OLE DB Source: FactSales extract query]
        ↓
[Lookup: CustomerID → CustomerKey]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: ProductID → ProductKey]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: EmployeeID → EmployeeKey]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: CityID → GeographyKey]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: DeliveryMethodID → DeliveryMethodKey]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: OrderDateKey → validate in DimDate]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: InvoiceDateKey → validate in DimDate]
        ├── Match ↓
        └── No Match → [Error Destination]
[Lookup: DueDateKey → validate in DimDate]
        ├── Match ↓
        └── No Match → [Error Destination]
[OLE DB Destination: Fact.FactSales]
```

8.2 Extract Query

The complete extract query developed in Chapter 3 is entered in the OLE DB Source. All measure derivations (UnitCost, GrossProfit, GrossProfitMarginPct) are computed in the source SQL. The output columns include natural keys (`CustomerID`, `ProductID`, etc.) that will be replaced by surrogate keys in the subsequent Lookup transformations.

8.3 Five Surrogate Key Lookups

Each of the five dimension FKs requires a Lookup transformation. The pattern is identical for all five:

Lookup	Reference query	Join col	Output col
Customer	`SELECT CustomerID, CustomerKey FROM DimCustomer WHERE CustomerID <> 0`	`CustomerID`	`CustomerKey`
Product	`SELECT ProductID, ProductKey FROM DimProduct WHERE ProductID <> 0`	`ProductID`	`ProductKey`
Employee	`SELECT EmployeeID, EmployeeKey FROM DimEmployee WHERE EmployeeID <> 0`	`EmployeeID`	`EmployeeKey`
Geography	`SELECT CityID, GeographyKey FROM DimGeography WHERE CityID <> 0`	`CityID`	`GeographyKey`
DeliveryMethod	`SELECT DeliveryMethodID, DeliveryMethodKey FROM DimDeliveryMethod WHERE DeliveryMethodID <> 0`	`DeliveryMethodID`	`DeliveryMethodKey`

8.4 Three Date Key Validations

The three date keys (`OrderDateKey` , `InvoiceDateKey` , `DueDateKey`) were computed in the source SQL as YYYYMMDD integers. Validation lookups confirm they exist in `DimDate` :

```
-- Reference query for all three date validation lookups
SELECT DateKey FROM Dimension.DimDate WHERE DateKey <> 0;
```

No output column is added — the only purpose is to confirm the key exists. A no-match means a date outside the calendar range, which would violate the FK constraint at load time.

8.5 OLE DB Destination: Fact.FactSales

At the destination, after all eight lookups, the pipeline contains:
- The three YYYYMMDD date keys (computed at source)
- The five dimension surrogate keys (added by lookups)
- All degenerate dimensions (InvoiceID, OrderID, OrderLineID)

- All measures (OrderedQuantity, PickedQuantity, UnitPrice, TaxRate, LineTotal, TaxAmount, LineTotalIncludingTax, UnitCost, GrossProfit, GrossProfitMarginPct)

The destination maps each pipeline column to its target column. `SalesKey` is the IDENTITY PK — it is not in the pipeline and not mapped. SQL Server generates it automatically on insert.

8.6 Reconciliation for FactSales

Two reconciliation tests run after the fact load:

```
-- Test 1: Row count
INSERT INTO ETL.TestResults
    (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed)
SELECT
    'FactSales Row Count vs Source',
    'Reconciliation', 'Fact.FactSales',
    CAST(src.c AS NVARCHAR),
    CAST(dw.c  AS NVARCHAR),
    CASE WHEN src.c = dw.c THEN 1 ELSE 0 END
FROM
    (SELECT COUNT(*) AS c
     FROM CabotTrailOutdoor.Sales.InvoiceLines il
     INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
     INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
     WHERE YEAR(o.OrderDate) < 2026) src,
    (SELECT COUNT(*) AS c FROM Fact.FactSales) dw;

-- Test 2: Revenue reconciliation
INSERT INTO ETL.TestResults
    (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed)
SELECT
    'FactSales Revenue vs Source',
    'Reconciliation', 'Fact.FactSales',
    CAST(src.rev AS NVARCHAR),
    CAST(dw.rev  AS NVARCHAR),
    CASE WHEN ABS(src.rev - dw.rev) < 0.01 THEN 1 ELSE 0 END
FROM
    (SELECT ROUND(SUM(il.LineTotal), 2) AS rev
     FROM CabotTrailOutdoor.Sales.InvoiceLines il
     INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
     INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
     WHERE YEAR(o.OrderDate) < 2026) src,
    (SELECT ROUND(SUM(LineTotal), 2) AS rev FROM Fact.FactSales) dw;
```

> ***The 0.01 tolerance*** *in the revenue test accounts for floating-point rounding differences between the source and DW computations. Both are DECIMAL columns, so the tolerance should be zero — but 0.01 is a conservative guard against minor platform-level precision differences.*

9. Reconciliation and Verification

Reconciliation is not optional. It is the mechanism that distinguishes a trusted ETL system from one that might be loading incorrect data silently.

9.1 The Reconciliation Framework

A complete reconciliation framework checks three things after each load:

Row count reconciliation. The number of rows in the target must match the number of rows in the source (after applying the same filters). A discrepancy indicates either dropped rows (lookup failures, filter errors) or duplicated rows (JOIN fan-out).

```
-- Template: row count reconciliation
SELECT
    'Target rows'   AS Metric,
    COUNT(*)        AS Value
FROM    [target_table]
UNION ALL
SELECT
    'Source rows',
    COUNT(*)
FROM    [source_query_with_same_filters];
```

Aggregate reconciliation. At least one key measure must be summed in both source and target and compared. The most meaningful measure for `FactSales` is total revenue (`LineTotal`).

```
-- Template: aggregate reconciliation
SELECT
    'Target revenue'    AS Metric,
    SUM(LineTotal)      AS Value
FROM    Fact.FactSales
UNION ALL
SELECT
    'Source revenue',
    SUM(il.LineTotal)
FROM    Sales.InvoiceLines il
INNER JOIN Sales.Invoices i ON i.InvoiceID = il.InvoiceID
INNER JOIN Sales.Orders o   ON o.OrderID   = i.OrderID
WHERE   YEAR(o.OrderDate) < 2026;
```

Orphan check. After every fact load, verify that no fact row has a FK value that does not exist in the referenced dimension. A passing orphan check confirms referential integrity:

```
-- Orphan check: FactSales
SELECT  COUNT(*) AS OrphanRows
FROM    Fact.FactSales fs
WHERE   NOT EXISTS (SELECT 1 FROM Dimension.DimDate d
                    WHERE d.DateKey = fs.OrderDateKey)
OR      NOT EXISTS (SELECT 1 FROM Dimension.DimCustomer c
                    WHERE c.CustomerKey = fs.CustomerKey)
OR      NOT EXISTS (SELECT 1 FROM Dimension.DimProduct p
                    WHERE p.ProductKey = fs.ProductKey)
OR      NOT EXISTS (SELECT 1 FROM Dimension.DimEmployee e
                    WHERE e.EmployeeKey = fs.EmployeeKey)
OR      NOT EXISTS (SELECT 1 FROM Dimension.DimGeography g
                    WHERE g.GeographyKey = fs.GeographyKey)
OR      NOT EXISTS (SELECT 1 FROM Dimension.DimDeliveryMethod dm
                    WHERE dm.DeliveryMethodKey = fs.DeliveryMethodKey);
-- Expected: 0
```

9.2 Diagnosing Reconciliation Failures

When row count reconciliation fails — the DW has fewer rows than the source — the first diagnostic step is to check the error tables:

```
-- Check for lookup failures during the load
SELECT  PackageName, ComponentName, ErrorDescription, COUNT(*) AS RejectedRows
FROM    ETL.LoadErrors
WHERE   CAST(LoadDate AS DATE) = CAST(GETDATE() AS DATE)
GROUP BY PackageName, ComponentName, ErrorDescription
ORDER BY RejectedRows DESC;
```

The most common cause of row count discrepancies is a surrogate key lookup failure — a source row referenced a natural key that had no matching surrogate in the dimension. The error table will identify exactly which component rejected the rows and how many.

When revenue reconciliation fails but row counts match, the issue is a transformation defect — the derived `LineTotal` or measure computation is producing different values than the source. Compare individual rows between source and target to isolate the discrepancy:

```
-- Find revenue discrepancies at the invoice line level
SELECT
    dw.InvoiceID,
    dw.OrderLineID,
    dw.LineTotal        AS DW_LineTotal,
    src.LineTotal       AS Src_LineTotal,
    dw.LineTotal - src.LineTotal AS Difference
FROM    Fact.FactSales dw
INNER JOIN CabotTrailOutdoor.Sales.InvoiceLines src
    ON src.InvoiceID    = dw.InvoiceID
    AND src.OrderLineID = dw.OrderLineID
WHERE   ABS(dw.LineTotal - src.LineTotal) > 0.01
ORDER BY ABS(dw.LineTotal - src.LineTotal) DESC;
```

---

10. The Master Package and Load Order

Individual packages for each dimension and fact table are the building blocks. The **master package** orchestrates all of them in the correct dependency order.

10.1 The Execute Package Task

The Control Flow task for calling a child package is the **Execute Package Task**. It can reference packages in:

- **The file system** — path to a `.dtsx` file (development only)
- **The SSIS Catalog** — a deployed package in `SSISDB` (production)

For production deployment, always use the SSIS Catalog reference. This decouples the master package from file system paths and supports environment-specific configuration.

10.2 Load Order: Dependency Chain

The complete load order for CabotTrail DW implements the five-tier dependency chain from Chapter 3:

```
Tier 1 — No dependencies (can run in parallel):
    Load_DimDate.dtsx
    Load_DimDeliveryMethod.dtsx
    Load_DimTransactionType.dtsx
    Load_DimPaymentMethod.dtsx
    Load_DimProductCategory.dtsx

Tier 2 — Depends on no other DW dims:
    Load_DimGeography.dtsx

Tier 3 — Depends on DimGeography:
    Load_DimCustomer.dtsx
    Load_DimSupplier.dtsx
    Load_DimEmployee.dtsx

Tier 4 — Depends on DimProductCategory + DimSupplier:
    Load_DimProduct.dtsx

Tier 5 — Depends on all dimensions:
    Load_FactSales.dtsx
    Load_FactPurchasing.dtsx
    Load_FactReturns.dtsx
    Load_FactInventory.dtsx
    Load_FactCustomerTransactions.dtsx
    Load_FactSupplierTransactions.dtsx
```

10.3 Master Package Control Flow

```
Sequence Container: Tier 1 Dimensions
    [Load_DimDate]  [Load_DimDeliveryMethod]  [Load_DimTransactionType]
    [Load_DimPaymentMethod]  [Load_DimProductCategory]
    (all run in parallel within the container)
        ↓ Container Success
Sequence Container: Tier 2 Dimensions
    [Load_DimGeography]
        ↓ Success
Sequence Container: Tier 3 Dimensions
    [Load_DimCustomer]  [Load_DimSupplier]  [Load_DimEmployee]
    (all run in parallel)
        ↓ Container Success
[Load_DimProduct]
        ↓ Success
Sequence Container: Fact Tables
    [Load_FactSales]  [Load_FactPurchasing]  [Load_FactReturns]
    [Load_FactInventory]  [Load_FactCustomerTransactions]
[Load_FactSupplierTransactions]
    (all run in parallel)
        ↓ Container Success / Failure
[Execute SQL: Final reconciliation summary]
```

10.4 Parallelism in the Control Flow

Tasks within a Sequence Container with no precedence constraints between them run in parallel. SSIS manages thread allocation automatically. Parallel execution of independent dimension loads (Tier 1 and Tier 3) significantly reduces total load time.

Important: Tasks in parallel must be truly independent. Parallelizing dimension loads that depend on each other (e.g., trying to load `DimProduct` in parallel with `DimProductCategory`) will produce intermittent failures depending on which loads faster in any given run.

10.5 SSIS Variables for Package Configuration

Rather than hardcoding connection strings and table names in each package, use **SSIS variables** — named values defined at the package or project level that can be changed without modifying package logic.

Project-level variables defined once apply to all packages in the project:

Variable	Type	Value
`@SourceServer`	String	`localhost` (or server name)
`@SourceDatabase`	String	`CabotTrailOutdoor`
`@TargetDatabase`	String	`CabotTrailOutdoorDW`
`@LoadDate`	DateTime	Set to GETDATE() at runtime
`@PackageName`	String	Set from system variable `@[System::PackageName]`

Connection Managers can reference variables using expressions, making each package automatically adapt to the configured server and database names.

---

11. Chapter Summary

- SSIS separates orchestration (**Control Flow**) from data movement (**Data Flow**). The Control Flow determines what runs and in what order. The Data Flow processes rows through a streaming transformation pipeline.
- The **OLE DB Source** reads rows using a SQL query. The extract query from the S2T mapping is entered here directly. Transformations that can be expressed in SQL should be implemented in the source query rather than in SSIS transformations.

- The **Lookup transformation** resolves natural keys to surrogate keys. It is the central operation of dimensional ETL. Every fact FK requires a corresponding Lookup. No-match rows must be redirected to error outputs — never silently dropped or loaded as NULL FKs.
- The **Derived Column transformation** adds computed columns to the pipeline using SSIS expressions. Use it for derivations that depend on pipeline-produced values or that require SSIS-specific type handling.
- The **Conditional Split transformation** routes rows to different outputs based on conditions. It is the primary tool for data quality error handling — valid rows continue to the destination; invalid rows go to error tables.
- The **OLE DB Destination** writes rows to SQL Server. Fast load mode uses bulk insert for performance. `Keep identity` must be enabled when preserving surrogate keys across database boundaries (DW to data mart loads).
- **Reconciliation** — row count, aggregate, and orphan checks — must run after every load. Discrepancies indicate defects that must be diagnosed and resolved before the ETL is considered correct.
- The **master package** orchestrates all dimension and fact loads in dependency order using Execute Package Tasks. Dimensions at the same tier can be parallelized within Sequence Containers.

---

12. Review Questions

1. Explain why a Lookup transformation with "Fail component" behaviour is never appropriate for a production ETL package. What should be used instead, and what happens to the rejected rows?
2. The `DimProduct` load package runs successfully but produces 0 rows in the target. The OLE DB Source returned 142 rows. What is the most likely cause, and how would you diagnose it?
3. You need to load `DimSupplier`. Its extract query JOINs `Purchasing.Suppliers` to `Application.Cities`, `Application.StateProvinces`, and `Application.Countries`. In the SSIS Data Flow, should these JOINs be implemented in the OLE DB Source SQL query or in SSIS Lookup transformations? Justify your answer.
4. After loading `Fact.FactSales`, the row count reconciliation passes (16,359 rows in both source and DW) but the revenue reconciliation fails — the DW total is $42 lower than the source total. List three possible causes and describe the diagnostic query you would run for each.
5. A new column `PromotionCode` is being added to `Fact.FactSales`. It comes from a new OLTP table `Marketing.PromoCodes` which has `OrderLineID` and `PromotionCode`. About 30% of order lines

have a promo code; the other 70% do not. Describe the complete implementation: the join type in the source query, the NULL handling, and any SSIS components needed beyond the source and destination.

6. The Tier 3 dimensions `DimCustomer`, `DimSupplier`, and `DimEmployee` are currently loaded sequentially. A colleague suggests loading them in parallel to reduce load time. Is this safe? Under what conditions could parallel loading of these three dimensions cause problems?
7. In the master package, what happens if `Load_DimProduct` fails? Describe the behaviour of the downstream fact loads and explain why this is the correct behaviour.
8. Explain the difference between `TRUNCATE TABLE` and `DELETE FROM` in the context of an ETL load. When would you use DELETE instead of TRUNCATE, and why?

Deeper Dive

Going Further with ETL Implementation

SSIS Buffer Tuning for Performance

The SSIS Data Flow buffer architecture described in section 1.3 is configurable. For high-volume loads, tuning buffer parameters can significantly improve throughput:

`DefaultBufferMaxRows` — maximum number of rows in a single buffer (default: 10,000). Increasing this for wide rows (many columns) reduces the number of buffer handoffs in the pipeline.

`DefaultBufferSize` — maximum size of a single buffer in bytes (default: 10 MB). The actual buffer size is the minimum of `DefaultBufferMaxRows × row_size` and `DefaultBufferSize`. For narrow rows, the row limit is binding; for wide rows, the byte limit is binding.

`EngineThreads` — number of threads available to the Data Flow engine (default: 10). Increasing this allows more parallel processing within a single Data Flow.

These parameters are set at the Data Flow Task level (not the package level). Microsoft's documentation provides detailed guidance:
SSIS — Data Flow Performance Features

The SSIS Catalog: Deployment and Monitoring

The **SSIS Catalog** (`SSISDB`) is the production deployment and monitoring platform for SSIS packages. It provides:

- **Package deployment** — packages are deployed as projects to the catalog, versioned, and executed from there
- **Environment configuration** — environment variables (server names, passwords, paths) are stored separately from package logic, allowing the same package to run in DEV, Test, and Production without modification
- **Execution logging** — every package execution is logged with start/end times, parameter values, and messages
- **Operation messages** — error messages, warnings, and informational messages are stored per execution, accessible via SSMS or T-SQL views

Key catalog views for monitoring:

```
-- Recent executions and their status
SELECT  execution_id, package_name, project_name,
        start_time, end_time, status,
        DATEDIFF(SECOND, start_time, end_time) AS DurationSeconds
FROM    SSISDB.catalog.executions
ORDER BY start_time DESC;

-- Error messages from a failed execution
SELECT  message_time, message_type, message
FROM    SSISDB.catalog.operation_messages
WHERE   operation_id = [execution_id_from_above]
AND     message_type IN (120, 130)  -- 120=Error, 130=Warning
ORDER BY message_time;

-- Package performance: rows per second for each data flow component
SELECT  execution_id, package_name, task_name,
        rows_read, rows_sent
FROM    SSISDB.catalog.execution_data_statistics
WHERE   execution_id = [execution_id_from_above]
ORDER BY task_name;
```

Microsoft documentation for the SSIS Catalog:

SSIS Catalog (SSISDB)

The Script Component: Extending the Data Flow

When built-in SSIS transformations cannot meet a requirement, the **Script Component** allows custom C# or VB.NET code to execute within the Data Flow pipeline. Use cases include:

- Parsing complex or non-standard file formats
- Calling web services to enrich pipeline rows
- Implementing custom fuzzy matching logic for deduplication
- Complex string manipulation that SSIS expressions cannot handle

The Script Component can act as a Source, Transformation, or Destination. It has full access to ADO.NET and .NET Framework libraries, making it enormously powerful — but also complex to maintain.

> ***Design principle:*** *Use the Script Component as a last resort. Every Script Component requires a developer who understands both SSIS and C#/.NET to maintain it. If the same transformation can be achieved in a source SQL query or a Derived Column expression, prefer those.*

Microsoft documentation:
Script Component — SSIS

T-SQL as an ETL Alternative

SSIS is not the only way to implement ETL against SQL Server. T-SQL stored procedures implementing `TRUNCATE TABLE` + `INSERT INTO ... SELECT` (the pattern from DBAS 5010) are a legitimate alternative, particularly for:

- Simple dimension loads with no complex transformations
- Environments where SSIS licensing or tooling is not available
- ETL processes that run within a single SQL Server instance (no cross-server movement)
- Rapid prototyping and development

The trade-offs compared to SSIS:

Factor	SSIS	T-SQL
Performance	Streaming buffers; parallel within pipeline	Bulk insert; set-based operations
Error handling	Row-level redirect to error output	Entire statement fails; no row-level routing without TRY/CATCH
Monitoring	SSIS Catalog with full execution history	SQL Agent job history; manual logging required
Cross-source	Any OLEDB source; files, APIs	Linked servers required for cross-instance
Maintainability	Visual representation of pipeline	Code-readable, version-controllable
Deployment	Requires SSIS infrastructure	Standard SQL; runs anywhere SQL Server runs

For the CabotTrail environment, both approaches are valid. The T-SQL ETL scripts developed in DBAS 5010 and DBAS 2010 are functionally equivalent to the SSIS packages in this chapter. Production environments typically use SSIS for its monitoring, error handling, and scheduling integration capabilities.

Change Data Capture Integration with SSIS

For incremental ETL (loading only rows changed since the last load), SSIS has a dedicated **CDC Source** component that reads directly from SQL Server's Change Data Capture tables:

```
-- CDC tables created by SQL Server when CDC is enabled
-- Accessible via Change Data Capture functions
SELECT  *
FROM    cdc.fn_cdc_get_all_changes_Sales_Orders(
            @from_lsn,      -- Log sequence number from last run
            @to_lsn,        -- Current log sequence number
            'all'           -- Include inserts, updates, deletes
        );
```

The SSIS CDC Source component wraps this function and provides:
- Automatic LSN (Log Sequence Number) management
- Row classification: `INSERT`, `UPDATE BEFORE`, `UPDATE AFTER`, `DELETE`
- Integration with the CDC Control Task for managing the extraction window

For the CabotTrail full-reload pattern, CDC is not needed. For incremental ETL in production environments with high data volumes, CDC integration with SSIS is the standard approach.

Microsoft documentation:
CDC Source — SSIS
Using CDC with SSIS

Industry Perspectives

Kimball on the ETL System

Kimball's *The Data Warehouse ETL Toolkit* (co-authored with Joe Caserta) dedicates an entire chapter to what they call the "ETL System Architecture." Their key insight is that ETL is not a collection of scripts — it is a system with distinct subsystems:

- **Extract subsystem** — manages connection to sources, handles extraction timing and volume
- **Cleanse and conform subsystem** — implements data quality rules and conforming logic
- **Deliver subsystem** — manages the loading of dimensions and facts in the correct order
- **Manage subsystem** — handles scheduling, monitoring, error recovery, and metadata

Thinking of ETL as a system rather than a collection of packages produces better-organized, more maintainable solutions. The SSIS master package pattern in this chapter is a practical implementation of Kimball's "deliver subsystem."

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley.

Microsoft on SSIS Best Practices

Microsoft's SSIS documentation includes a best practices guide that covers package design, performance, and maintainability:
Integration Services Best Practices

Key recommendations aligned with this chapter:
- Use parameterized queries where possible to enable plan reuse
- Set `ValidateExternalMetadata = False` in development to avoid validation errors on disconnected machines
- Use project-level connection managers (not package-level) to share connections across packages
- Name all components descriptively — default names like "OLE DB Source" and "Lookup" make debugging impossible

References and Further Reading

1. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — The definitive reference for ETL system design. Chapters 4–7 cover transformation patterns, surrogate key management, and fact table loading in detail.

2. Microsoft. (2024). *Integration Services Data Flow*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/integration-services-data-flow

3. Microsoft. (2024). *Lookup Transformation*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/transformations/lookup-transformation

4. Microsoft. (2024). *Derived Column Transformation*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/transformations/derived-column-transformation

5. Microsoft. (2024). *Conditional Split Transformation*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/transformations/conditional-split-transformation

6. Microsoft. (2024). *OLE DB Destination*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/ole-db-destination

7. Microsoft. (2024). *Data Flow Performance Features*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/data-flow-performance-features

8. Microsoft. (2024). *SSIS Catalog*. https://learn.microsoft.com/en-us/sql/integration-services/catalog/ssis-catalog

9. Microsoft. (2024). *Integration Services Best Practices*. https://learn.microsoft.com/en-us/sql/integration-services/integration-services-best-practices

10. Microsoft. (2024). *CDC Source — SSIS*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/cdc-source

11. Knight, B., Veerman, E., Davis, J., Dickinson, B., & Moss, J. (2012). *Professional Microsoft SQL Server 2012 Integration Services*. Wrox. — Comprehensive practical reference for SSIS development; covers all components and patterns described in this chapter with extensive worked examples.

Chapter 5: Testing, Data Quality, and Governance

Chapter Overview

Chapters 3 and 4 covered the design and implementation of ETL pipelines. A pipeline that runs without errors is not the same as a pipeline that produces *correct* results. This chapter addresses the discipline that bridges that gap: testing and data quality governance.

An ETL system without a formal test suite is an assertion — "I believe this is correct." An ETL system with a formal test suite is a verified claim — "this is correct, and here is the evidence." In a business intelligence context, the difference matters enormously. Analysts, managers, and executives make decisions based on data from the warehouse. If that data is wrong — even subtly, even occasionally — the decisions built on it are compromised.

Data governance provides the organizational framework that makes testing sustainable: the policies, standards, and accountability structures that ensure data quality is maintained not just at initial load but continuously, through system changes, source data changes, and business rule changes.

By the end of this chapter you will be able to:

- Describe the five levels of ETL testing and explain when each is applied
- Build a formal ETL test suite using the `ETL.TestResults` table pattern
- Implement automated reconciliation tests that run after every load
- Distinguish between data governance and data quality, and explain the relationship between them
- Describe the key elements of a data governance framework for a BI environment
- Implement SSIS logging and connect it to a monitoring strategy
- Explain the role of metadata in ETL governance

Table of Contents

---

1. Why ETL Testing Is Different

Testing an ETL system presents challenges that do not arise in application software testing. Understanding these differences shapes the testing strategy.

1.1 The Output Is Data, Not Behaviour

Application software testing verifies that a function returns the correct value or that a UI element behaves correctly. ETL testing verifies that millions of data rows have been transformed correctly according to business rules. There is no return value to assert against — there are rows in a table that must collectively satisfy a set of conditions.

This means ETL tests are fundamentally **set-based**: they operate on aggregations, distributions, and relationships across the entire dataset, not on individual records.

1.2 The Source Data Changes

An application can be tested against a fixed, controlled input. ETL source data changes with every load — new orders are placed, new customers are created, prices are updated. Tests that passed yesterday must pass again today against new data. The test suite must be designed to accommodate evolving data without requiring constant maintenance.

The solution is to write tests that verify **structural properties** (the relationship between source and target counts, the referential integrity of FK relationships, the mathematical correctness of derivations) rather than specific values. A test that says "revenue must equal $6,350,582.50" breaks every time a new sale is loaded. A test that says "DW revenue must equal source revenue" is valid for the life of the system.

1.3 Failures Can Be Silent

An application that crashes announces its failure. An ETL pipeline that silently drops 3% of rows — because a lookup found no match and the no-match rows were not redirected to an error output — delivers a subtly wrong data warehouse with no error messages. Analysts will eventually notice that numbers are slightly off, but tracing the issue back to a silent lookup failure may take days.

The test suite is the mechanism that makes silent failures audible. If every load is followed by a reconciliation test that compares source and target counts, a 3% row loss is detected immediately.

1.4 The "Correct Answer" Is a Business Agreement

In application testing, correct behaviour is defined by a specification. In ETL testing, correct data is defined by a business agreement — the S2T mapping from Chapter 3. This agreement must be documented and maintained. If the business changes its mind about how `SalesTerritory` is derived, the test that validates `SalesTerritory` values must be updated along with the ETL code.

This coupling between business rules, ETL implementation, and tests is why the S2T mapping is a living document: it is the single source of truth that keeps all three in alignment.

---

2. The Five Levels of ETL Testing

ETL testing is structured in levels that correspond to the stages of development and deployment. Each level catches different categories of defects.

2.1 Unit Testing

Unit testing validates a single transformation in isolation. For ETL, a unit test verifies that one specific transformation rule produces the correct output for a known input.

Example: Testing the `SalesTerritory` derivation rule for `DimCustomer`.

```
-- Unit test: SalesTerritory derivation
-- Input: ProvinceCode values from source
-- Expected output: corresponding SalesTerritory values

-- Build a controlled test dataset
WITH TestCases AS (
    SELECT 'NS' AS ProvinceCode, 'Atlantic — Nova Scotia'     AS ExpectedTerritory UNION
ALL
    SELECT 'NB',                 'Atlantic — New Brunswick'                          UNIO
N ALL
    SELECT 'ON',                 'Ontario'                                           UNIO
N ALL
    SELECT 'AB',                 'Alberta'                                           UNIO
N ALL
    SELECT 'XX',                 'Other'   -- Unknown province
),
ActualResults AS (
    SELECT
        ProvinceCode,
        CASE ProvinceCode
            WHEN 'NS' THEN 'Atlantic — Nova Scotia'
            WHEN 'NB' THEN 'Atlantic — New Brunswick'
            WHEN 'PE' THEN 'Atlantic — PEI'
            WHEN 'NL' THEN 'Atlantic — Newfoundland'
            WHEN 'QC' THEN 'Quebec'
            WHEN 'ON' THEN 'Ontario'
            WHEN 'MB' THEN 'Prairies'
            WHEN 'SK' THEN 'Prairies'
            WHEN 'AB' THEN 'Alberta'
            WHEN 'BC' THEN 'British Columbia'
            ELSE 'Other'
        END AS ActualTerritory
    FROM TestCases
)
SELECT
    t.ProvinceCode,
    t.ExpectedTerritory,
    a.ActualTerritory,
    CASE WHEN t.ExpectedTerritory = a.ActualTerritory THEN 'PASS' ELSE 'FAIL' END AS Resu
lt
FROM TestCases t
INNER JOIN ActualResults a ON a.ProvinceCode = t.ProvinceCode;
```

Unit tests are written by the ETL developer, run frequently during development, and do not require a loaded target table — they test the logic of individual rules.

2.2 Integration Testing

Integration testing validates a complete package end-to-end: source → all transformations → destination. It verifies that the components work together correctly and that the package produces the expected output in the target table.

Integration tests are run after each package is built and after any modification. They use the live source data against a test copy of the DW.

Key integration test checks:
- Package runs to completion without errors (no red tasks in Control Flow)
- Target table has the expected row count
- A sample of rows in the target have correct values for key columns
- No rows appear in error tables (no lookup failures, no DQ rejections)

2.3 System Testing

System testing validates the complete ETL solution — all packages running together in the correct sequence, against the full dataset. It is the first test that exercises the master package and the inter-package dependencies.

System testing catches:
- Load order violations (a fact loads before its dimension is populated)
- Parallelism issues (two packages that should be independent but actually share a resource)
- Cumulative performance problems (the full load takes longer than the available window)
- Cascading failures (one package failure propagates incorrectly to downstream packages)

2.4 Reconciliation Testing

Reconciliation testing verifies that the data in the target matches the data in the source — numerically and structurally. This is the most important ongoing test category and the one that runs in production after every load.

Reconciliation tests are the subject of section 4.

2.5 Regression Testing

Regression testing verifies that changes to an existing ETL system have not broken previously correct behaviour. It is run whenever a package is modified — a new column added, a transformation rule changed, a join updated.

A regression test suite is a collection of tests that represent "known correct" behaviour at a point in time. Every test that passed before the change must still pass after. New tests are added for new behaviour.

```
-- Regression test: verify DimCustomer row count has not decreased
-- (guards against accidentally adding a WHERE clause that filters too aggressively)
DECLARE @ExpectedMinRows INT = 100;  -- Known count from last verified load

SELECT
    CASE WHEN COUNT(*) >= @ExpectedMinRows
         THEN 'PASS — Row count maintained'
         ELSE 'FAIL — Row count below expected minimum (' +
              CAST(COUNT(*) AS VARCHAR) + ' vs ' +
              CAST(@ExpectedMinRows AS VARCHAR) + ')'
    END AS RegressionResult
FROM Dimension.DimCustomer
WHERE CustomerID <> 0;
```

---

3. Building a Formal Test Suite

A formal test suite is not an ad-hoc collection of verification queries — it is a structured, versioned, and executed set of tests with recorded results. The `ETL.TestResults` table introduced in Chapter 4 is the foundation.

3.1 The Test Results Table

```
-- Full definition of the test infrastructure tables
USE CabotTrailOutdoorDW;
GO

CREATE SCHEMA ETL;
GO

-- Test results: one row per test execution
CREATE TABLE ETL.TestResults
(
    TestID              INT             NOT NULL IDENTITY(1,1),
    TestName            NVARCHAR(200)   NOT NULL,
    TestCategory        NVARCHAR(50)    NOT NULL,
    TargetTable         NVARCHAR(100)   NOT NULL,
    ExpectedValue       NVARCHAR(200)   NULL,
    ActualValue         NVARCHAR(200)   NULL,
    Passed              BIT             NOT NULL,
    RunDate             DATETIME2       NOT NULL DEFAULT SYSDATETIME(),
    PackageName         NVARCHAR(200)   NULL,
    LoadDurationSeconds INT             NULL,
    Notes               NVARCHAR(500)   NULL,
    CONSTRAINT PK_ETL_TestResults PRIMARY KEY (TestID)
);
GO

-- Test run summary: one row per complete ETL execution
CREATE TABLE ETL.LoadRuns
(
    RunID               INT             NOT NULL IDENTITY(1,1),
    RunStartTime        DATETIME2       NOT NULL,
    RunEndTime          DATETIME2       NULL,
    RunStatus           NVARCHAR(20)    NOT NULL DEFAULT 'Running',
    TotalTests          INT             NOT NULL DEFAULT 0,
    TestsPassed         INT             NOT NULL DEFAULT 0,
    TestsFailed         INT             NOT NULL DEFAULT 0,
    ErrorSummary        NVARCHAR(MAX)   NULL,
    CONSTRAINT PK_ETL_LoadRuns      PRIMARY KEY (RunID),
    CONSTRAINT CK_ETL_LoadRuns_Status
        CHECK (RunStatus IN ('Running', 'Passed', 'Failed', 'Partial'))
);
GO

-- Load errors: rows rejected during ETL
CREATE TABLE ETL.LoadErrors
(
    ErrorID             INT             NOT NULL IDENTITY(1,1),
    RunID               INT             NULL,
    PackageName         NVARCHAR(200)   NOT NULL,
    ComponentName       NVARCHAR(200)   NOT NULL,
    NaturalKeyValue     NVARCHAR(200)   NULL,
    ErrorCode           INT             NULL,
    ErrorDescription    NVARCHAR(MAX)   NULL,
    LoadDate            DATETIME2       NOT NULL DEFAULT SYSDATETIME(),
```

```
    CONSTRAINT PK_ETL_LoadErrors    PRIMARY KEY (ErrorID),
    CONSTRAINT FK_ETL_LoadErrors_Run
        FOREIGN KEY (RunID) REFERENCES ETL.LoadRuns (RunID)
);
GO
```

3.2 Test Registration and Execution

Tests are not just ad-hoc queries — they are registered entries that are executed consistently on every run. A stored procedure wraps the test logic and records the result:

```
-- Stored procedure: execute and record a row count test
CREATE PROCEDURE ETL.usp_TestRowCount
    @TestName       NVARCHAR(200),
    @TargetTable    NVARCHAR(100),
    @ExpectedCount  INT,
    @PackageName    NVARCHAR(200) = NULL
AS
BEGIN
    DECLARE @ActualCount    INT;
    DECLARE @SQL            NVARCHAR(500);
    DECLARE @Passed         BIT;

    -- Dynamic count against target table
    SET @SQL = N'SELECT @cnt = COUNT(*) FROM ' + @TargetTable;
    EXEC sp_executesql @SQL, N'@cnt INT OUTPUT', @cnt = @ActualCount OUTPUT;

    SET @Passed = CASE WHEN @ActualCount = @ExpectedCount THEN 1 ELSE 0 END;

    INSERT INTO ETL.TestResults
        (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed, Package
Name)
    VALUES
        (@TestName, 'RowCount', @TargetTable,
         CAST(@ExpectedCount AS NVARCHAR),
         CAST(@ActualCount   AS NVARCHAR),
         @Passed,
         @PackageName);

    -- Return pass/fail for SSIS package to evaluate
    RETURN CASE WHEN @Passed = 1 THEN 0 ELSE 1 END;  -- 0 = success, 1 = failure
END;
GO
```

```
-- Example: running the test from an Execute SQL Task in SSIS
EXEC ETL.usp_TestRowCount
    @TestName       = 'DimCustomer Row Count',
    @TargetTable    = 'Dimension.DimCustomer',
    @ExpectedCount  = 100,
    @PackageName    = 'Load_DimCustomer';
```

3.3 The Complete Test Suite for CabotTrail

A production-ready test suite covers every dimension and fact table with at minimum two tests each:

```
-- Execute all reconciliation tests (run from master package after all loads)
USE CabotTrailOutdoorDW;

-- ── Dimension tests ──────────────────────────────────────────
EXEC ETL.usp_TestRowCount 'DimDeliveryMethod Row Count',
    'Dimension.DimDeliveryMethod', 12, 'Load_DimDeliveryMethod';

EXEC ETL.usp_TestRowCount 'DimEmployee Row Count',
    'Dimension.DimEmployee', 50, 'Load_DimEmployee';

EXEC ETL.usp_TestRowCount 'DimCustomer Row Count',
    'Dimension.DimCustomer', 100, 'Load_DimCustomer';

EXEC ETL.usp_TestRowCount 'DimProduct Row Count',
    'Dimension.DimProduct', 142, 'Load_DimProduct';

EXEC ETL.usp_TestRowCount 'DimProductCategory Row Count',
    'Dimension.DimProductCategory', 13, 'Load_DimProductCategory';

-- ── Fact table tests ─────────────────────────────────────────

-- FactSales: row count
EXEC ETL.usp_TestRowCount 'FactSales Row Count',
    'Fact.FactSales', 16359, 'Load_FactSales';

-- FactSales: revenue reconciliation (aggregate test)
INSERT INTO ETL.TestResults
    (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed,
PackageName)
SELECT
    'FactSales Revenue Reconciliation',
    'Aggregate',
    'Fact.FactSales',
    CAST(ROUND(src.Revenue, 2) AS NVARCHAR),
    CAST(ROUND(dw.Revenue,  2) AS NVARCHAR),
    CASE WHEN ABS(src.Revenue - dw.Revenue) < 0.01 THEN 1 ELSE 0 END,
    'Load_FactSales'
FROM
    (SELECT SUM(il.LineTotal) AS Revenue
     FROM CabotTrailOutdoor.Sales.InvoiceLines il
     INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
     INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
     WHERE YEAR(o.OrderDate) < 2026) src,
    (SELECT SUM(LineTotal) AS Revenue FROM Fact.FactSales) dw;

-- FactSales: referential integrity (orphan check)
INSERT INTO ETL.TestResults
    (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed,
PackageName)
SELECT
    'FactSales Orphan Check',
    'ReferentialIntegrity',
    'Fact.FactSales',
```

```
    '0',
    CAST(OrphanCount AS NVARCHAR),
    CASE WHEN OrphanCount = 0 THEN 1 ELSE 0 END,
    'Load_FactSales'
FROM (
    SELECT COUNT(*) AS OrphanCount
    FROM Fact.FactSales fs
    WHERE NOT EXISTS (SELECT 1 FROM Dimension.DimCustomer c
                      WHERE c.CustomerKey = fs.CustomerKey)
    OR    NOT EXISTS (SELECT 1 FROM Dimension.DimProduct p
                      WHERE p.ProductKey  = fs.ProductKey)
    OR    NOT EXISTS (SELECT 1 FROM Dimension.DimEmployee e
                      WHERE e.EmployeeKey = fs.EmployeeKey)
    OR    NOT EXISTS (SELECT 1 FROM Dimension.DimGeography g
                      WHERE g.GeographyKey = fs.GeographyKey)
    OR    NOT EXISTS (SELECT 1 FROM Dimension.DimDeliveryMethod dm
                      WHERE dm.DeliveryMethodKey = fs.DeliveryMethodKey)
) orph;

-- FactPurchasing: row count and aggregate
EXEC ETL.usp_TestRowCount 'FactPurchasing Row Count',
    'Fact.FactPurchasing', 905, 'Load_FactPurchasing';

-- FactReturns: row count and aggregate
EXEC ETL.usp_TestRowCount 'FactReturns Row Count',
    'Fact.FactReturns', 500, 'Load_FactReturns';

-- FactInventory: row count
EXEC ETL.usp_TestRowCount 'FactInventory Row Count',
    'Fact.FactInventory', 142, 'Load_FactInventory';

-- FactCustomerTransactions: row count
EXEC ETL.usp_TestRowCount 'FactCustomerTransactions Row Count',
    'Fact.FactCustomerTransactions', 9346, 'Load_FactCustomerTransactions';

-- FactSupplierTransactions: row count
EXEC ETL.usp_TestRowCount 'FactSupplierTransactions Row Count',
    'Fact.FactSupplierTransactions', 240, 'Load_FactSupplierTransactions';
```

3.4 Test Suite Summary Query

After all tests run, a summary query shows the overall pass/fail status:

```
-- Test suite summary: current run
SELECT
    TestCategory,
    COUNT(*)                                          AS TotalTests,
    SUM(CASE WHEN Passed = 1 THEN 1 ELSE 0 END)       AS Passed,
    SUM(CASE WHEN Passed = 0 THEN 1 ELSE 0 END)       AS Failed,
    MAX(RunDate)                                      AS LastRun
FROM    ETL.TestResults
WHERE   CAST(RunDate AS DATE) = CAST(GETDATE() AS DATE)
GROUP BY TestCategory
ORDER BY Failed DESC, TestCategory;
```

```
-- Failed tests: detail for investigation
SELECT
    TestName,
    TestCategory,
    TargetTable,
    ExpectedValue,
    ActualValue,
    RunDate,
    PackageName
FROM    ETL.TestResults
WHERE   Passed = 0
AND     CAST(RunDate AS DATE) = CAST(GETDATE() AS DATE)
ORDER BY RunDate DESC;
```

---

4. Reconciliation Testing in Depth

Reconciliation testing is the most critical ongoing test category. It runs in production after every load and provides the evidence that the ETL system is functioning correctly.

4.1 The Three Layers of Reconciliation

Chapter 4 introduced the three-part reconciliation framework. This section extends it with the diagnostic queries needed when each layer fails.

Layer 1: Row Count Reconciliation

What it checks: The number of rows in the target matches the number of rows expected from the source.

Why it fails: Lookup failures (rows dropped at a no-match output), incorrect WHERE clause filters (rows excluded that should not be), JOIN fan-out (rows multiplied by a Cartesian JOIN).

```
-- Diagnostic: find the row count difference
WITH SourceCount AS (
    SELECT COUNT(*) AS Rows
    FROM CabotTrailOutdoor.Sales.InvoiceLines il
    INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
    INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
    WHERE YEAR(o.OrderDate) < 2026
),
TargetCount AS (
    SELECT COUNT(*) AS Rows FROM CabotTrailOutdoorDW.Fact.FactSales
),
ErrorCount AS (
    SELECT COUNT(*) AS Rows FROM CabotTrailOutdoorDW.ETL.LoadErrors
    WHERE CAST(LoadDate AS DATE) = CAST(GETDATE() AS DATE)
    AND   PackageName = 'Load_FactSales'
)
SELECT
    s.Rows          AS SourceRows,
    t.Rows          AS TargetRows,
    e.Rows          AS ErrorRows,
    s.Rows - t.Rows AS DiscrepancyRows,
    -- If discrepancy ≈ error count, lookup failure is the cause
    CASE WHEN s.Rows - t.Rows = e.Rows THEN 'Lookup failures explain discrepancy'
         WHEN s.Rows - t.Rows > e.Rows THEN 'Additional rows lost — check JOINs'
         WHEN t.Rows > s.Rows          THEN
'More rows in target than source — check for duplicates'
    END AS Diagnosis
FROM SourceCount s, TargetCount t, ErrorCount e;
```

Finding fan-out (duplicate rows): A Cartesian JOIN in the source query multiplies rows. This is a common error when joining to a table without a proper join condition:

```
-- Detect fan-out: any InvoiceID appearing more times in DW than in source
SELECT  dw.InvoiceID,
        COUNT(*) AS DW_Count,
        src.Src_Count,
        COUNT(*) - src.Src_Count AS ExtraRows
FROM    CabotTrailOutdoorDW.Fact.FactSales dw
INNER JOIN (
    SELECT InvoiceID, COUNT(*) AS Src_Count
    FROM CabotTrailOutdoor.Sales.InvoiceLines
    GROUP BY InvoiceID
) src ON src.InvoiceID = dw.InvoiceID
GROUP BY dw.InvoiceID, src.Src_Count
HAVING COUNT(*) > src.Src_Count
ORDER BY ExtraRows DESC;
```

Layer 2: Aggregate Reconciliation

What it checks: Key measures sum to the same value in source and target.

Why it fails: Transformation defects (wrong formula for a derived measure), data type precision differences (DECIMAL vs FLOAT in intermediate computations), incorrect filter (some rows included in target but not source aggregate, or vice versa).

```
-- Diagnostic: find rows where DW LineTotal differs from source LineTotal
SELECT
    dw.InvoiceID,
    dw.OrderLineID,
    dw.LineTotal        AS DW_LineTotal,
    src.LineTotal       AS Src_LineTotal,
    ABS(dw.LineTotal - src.LineTotal) AS AbsDifference
FROM CabotTrailOutdoorDW.Fact.FactSales dw
INNER JOIN CabotTrailOutdoor.Sales.InvoiceLines src
    ON  src.InvoiceID   = dw.InvoiceID
    AND src.OrderLineID = dw.OrderLineID
WHERE ABS(dw.LineTotal - src.LineTotal) > 0.01
ORDER BY AbsDifference DESC;
```

Layer 3: Derived Measure Verification

What it checks: Computed measures (`GrossProfit`, `GrossProfitMarginPct`, `UnitCost`) match the documented derivation formula.

This layer verifies that the ETL's transformation logic implements the S2T mapping correctly. The test queries the DW and recomputes the measure independently, comparing stored vs computed values:

```
-- Verify GrossProfit derivation: stored value vs recomputed value
SELECT
    fs.SalesKey,
    fs.GrossProfit                                      AS StoredGrossProfit,
    ROUND(fs.LineTotal - fs.UnitCost, 2)                AS ComputedGrossProfit,
    ABS(fs.GrossProfit - ROUND(fs.LineTotal - fs.UnitCost, 2)) AS Discrepancy
FROM CabotTrailOutdoorDW.Fact.FactSales fs
WHERE ABS(fs.GrossProfit - ROUND(fs.LineTotal - fs.UnitCost, 2)) > 0.01
ORDER BY Discrepancy DESC;
-- Expected: 0 rows

-- Verify GrossProfitMarginPct: must match GrossProfit / LineTotal * 100
SELECT
    fs.SalesKey,
    fs.GrossProfitMarginPct                                     AS StoredPct,
    CASE WHEN fs.LineTotal = 0 THEN 0
         ELSE ROUND(fs.GrossProfit / fs.LineTotal * 100, 2)
    END                                                         AS ComputedPct,
    ABS(fs.GrossProfitMarginPct -
        CASE WHEN fs.LineTotal = 0 THEN 0
             ELSE ROUND(fs.GrossProfit / fs.LineTotal * 100, 2)
        END)                                                    AS Discrepancy
FROM CabotTrailOutdoorDW.Fact.FactSales fs
WHERE ABS(fs.GrossProfitMarginPct -
          CASE WHEN fs.LineTotal = 0 THEN 0
               ELSE ROUND(fs.GrossProfit / fs.LineTotal * 100, 2)
          END) > 0.01
ORDER BY Discrepancy DESC;
-- Expected: 0 rows
```

4.2 Cross-Database Reconciliation

When data flows through multiple layers (OLTP → DW → data mart), reconciliation must be verified at each boundary:

```
-- Cross-database reconciliation: OLTP → DW → datamart
-- All three revenue figures must agree

SELECT 'OLTP Source'       AS Layer, ROUND(SUM(il.LineTotal), 2) AS Revenue
FROM CabotTrailOutdoor.Sales.InvoiceLines il
INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
WHERE YEAR(o.OrderDate) < 2026
UNION ALL
SELECT 'DW Fact.FactSales', ROUND(SUM(LineTotal), 2)
FROM CabotTrailOutdoorDW.Fact.FactSales
UNION ALL
SELECT 'Sales Datamart',    ROUND(SUM(LineTotal), 2)
FROM CabotTrailOutdoorsSales.fact.Sales;
```

All three figures must match. A discrepancy between OLTP and DW indicates a DW ETL defect. A discrepancy between DW and datamart indicates a datamart ETL defect.

---

5. Data Quality in Production

Chapter 3 introduced data quality assessment as a design-time activity — profiling source data before ETL is built. This section addresses data quality as a **production concern** — managing and monitoring quality continuously after the ETL system is deployed.

5.1 The Data Quality Lifecycle

Data quality is not a one-time assessment. It has a lifecycle that spans the entire operational life of the ETL system:

```
Design time:
  Source profiling → DQ rule definition → Severity assignment → ETL error handling design

Load time:
  DQ rule evaluation → Reject/substitute/continue → Error logging → Notification

Post-load:
  Error table review → Root cause analysis → Source system feedback → Rule adjustment

Ongoing:
  Trend monitoring → Rule maintenance → Periodic source re-profiling
```

The critical point: **data quality degrades over time** if not actively managed. Sources change — new applications are deployed, data entry processes evolve, migrations introduce inconsistencies. A DQ rule that passed 100% on day one may start failing months later as source data patterns change.

5.2 Data Quality Trend Monitoring

Recording DQ check results over time enables trend detection:

```
-- DQ trend: track NULL rate for key columns over time
-- Run this on a schedule and store results

INSERT INTO ETL.TestResults
    (TestName, TestCategory, TargetTable, ExpectedValue, ActualValue, Passed)
SELECT
    'DimCustomer: NULL SalesTerritory rate',
    'DataQuality',
    'Dimension.DimCustomer',
    '0',
    CAST(SUM(CASE WHEN SalesTerritory IS NULL THEN 1 ELSE 0 END) AS NVARCHAR),
    CASE WHEN SUM(CASE WHEN SalesTerritory IS NULL THEN 1 ELSE 0 END) = 0 THEN 1 ELSE 0 E
ND
FROM Dimension.DimCustomer
WHERE CustomerID <> 0;
```

```
-- DQ trend report: how has the NULL rate changed over time?
SELECT
    CAST(RunDate AS DATE)              AS TestDate,
    TestName,
    ActualValue                        AS NullCount,
    Passed
FROM    ETL.TestResults
WHERE   TestName LIKE '%NULL%'
AND     TestCategory = 'DataQuality'
ORDER BY TestName, TestDate;
```

5.3 The Unknown Member as a Quality Indicator

In a dimensional model, the unknown member (surrogate key = 0) serves as the default FK for fact rows whose dimension value cannot be resolved. The count of fact rows pointing to the unknown member is a direct measure of data quality:

```
-- How many fact rows point to the unknown member for each dimension?
SELECT
    'CustomerKey = 0' AS Dimension,
    COUNT(*) AS UnknownMemberRows,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Fact.FactSales), 2) AS PctOfTotal
FROM Fact.FactSales WHERE CustomerKey = 0
UNION ALL
SELECT 'ProductKey = 0',
    COUNT(*),
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Fact.FactSales), 2)
FROM Fact.FactSales WHERE ProductKey = 0
UNION ALL
SELECT 'EmployeeKey = 0',
    COUNT(*),
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Fact.FactSales), 2)
FROM Fact.FactSales WHERE EmployeeKey = 0;
```

In a healthy ETL system this query returns 0 for all dimensions. Any non-zero value indicates that lookup failures were handled by routing to the unknown member (acceptable if designed that way) or that dimension records were deleted after fact rows were loaded (a referential integrity concern).

5.4 Source Data Change Detection

When source data changes in unexpected ways between loads, the ETL system may produce subtly different results without any errors being raised. Change detection queries compare the current load to the previous load:

```
-- Detect unexpected changes in source data between loads
-- Compare today's OLTP row counts to yesterday's DW values

WITH CurrentSourceCounts AS (
    SELECT 'Sales.Customers'    AS TableName, COUNT(*) AS RowCount
    FROM CabotTrailOutdoor.Sales.Customers
    UNION ALL
    SELECT 'Sales.InvoiceLines', COUNT(*)
    FROM CabotTrailOutdoor.Sales.InvoiceLines
    UNION ALL
    SELECT 'Inventory.Products', COUNT(*)
    FROM CabotTrailOutdoor.Inventory.Products WHERE ProductID <> 0
),
PreviousLoadCounts AS (
    -- Most recent passed RowCount test for each table
    SELECT TargetTable AS TableName,
           CAST(ActualValue AS INT) AS RowCount
    FROM (
        SELECT TargetTable, ActualValue,
               ROW_NUMBER() OVER (PARTITION BY TargetTable ORDER BY RunDate DESC) AS rn
        FROM ETL.TestResults
        WHERE TestCategory = 'RowCount' AND Passed = 1
    ) ranked
    WHERE rn = 1
)
SELECT
    c.TableName,
    c.RowCount          AS CurrentRows,
    p.RowCount          AS PreviousRows,
    c.RowCount - ISNULL(p.RowCount, 0) AS RowDelta,
    CASE
        WHEN c.RowCount < ISNULL(p.RowCount, 0)
        THEN '   Row count decreased — investigate'
        WHEN c.RowCount > ISNULL(p.RowCount, 0) + 1000
        THEN '   Unusually large increase — verify'
        ELSE '✓ Normal'
    END AS Status
FROM CurrentSourceCounts c
LEFT JOIN PreviousLoadCounts p ON p.TableName = c.TableName;
```

6. SSIS Logging and Monitoring

An ETL system that runs without monitoring is operating blind. SSIS provides built-in logging infrastructure that, when properly configured, gives complete visibility into every execution.

6.1 The SSIS Catalog as a Monitoring Platform

When packages are deployed to the SSIS Catalog (`SSISDB`), every execution is automatically logged. No additional logging configuration is required — the catalog captures:

- Start and end time for every package and every task
- Parameters passed to the execution
- Row counts for every Data Flow component
- All error, warning, and informational messages
- Performance statistics (rows processed, buffer usage)

```
-- Recent executions with duration and status
SELECT
    e.execution_id,
    e.package_name,
    e.project_name,
    e.start_time,
    e.end_time,
    DATEDIFF(SECOND, e.start_time, e.end_time)  AS DurationSeconds,
    CASE e.status
        WHEN 1 THEN 'Created'
        WHEN 2 THEN 'Running'
        WHEN 3 THEN 'Cancelled'
        WHEN 4 THEN 'Failed'
        WHEN 5 THEN 'Pending'
        WHEN 6 THEN 'Ended unexpectedly'
        WHEN 7 THEN 'Succeeded'
        WHEN 8 THEN 'Stopping'
        WHEN 9 THEN 'Completed'
    END AS StatusDescription
FROM    SSISDB.catalog.executions e
ORDER BY e.start_time DESC;
```

```
-- Error messages from a specific failed execution
SELECT
    om.message_time,
    om.package_name,
    om.task_name,
    om.message
FROM    SSISDB.catalog.operation_messages om
WHERE   om.operation_id = [execution_id]
AND     om.message_type = 120   -- 120 = Error
ORDER BY om.message_time;
```

```
-- Row counts per Data Flow component — performance analysis
SELECT
    eds.package_name,
    eds.task_name,
    eds.dataflow_path_id_string  AS Component,
    eds.rows_sent
FROM    SSISDB.catalog.execution_data_statistics eds
WHERE   eds.execution_id = [execution_id]
ORDER BY eds.package_name, eds.task_name;
```

6.2 Custom Logging with Execute SQL Tasks

The SSIS Catalog provides operational logs — what happened during execution. Custom logging records business-meaningful metrics — what was loaded, how much, and whether it was correct. Both are needed.

The `ETL.LoadRuns` table introduced in section 3.1 implements custom run-level logging:

```
-- Start of master package: create a load run record
-- (Execute SQL Task at the beginning of Master_DW_Load.dtsx)
DECLARE @RunID INT;

INSERT INTO ETL.LoadRuns (RunStartTime, RunStatus)
VALUES (SYSDATETIME(), 'Running');

SET @RunID = SCOPE_IDENTITY();

-- Store RunID in an SSIS package variable for use by child packages
-- (Return value from Execute SQL Task → SSIS variable @RunID)
SELECT @RunID AS RunID;
```

```
-- End of master package: update the run record
-- (Execute SQL Task at the end, on both Success and Failure paths)
UPDATE ETL.LoadRuns
SET
    RunEndTime  = SYSDATETIME(),
    RunStatus   = CASE WHEN FailedTests.c = 0 THEN 'Passed' ELSE 'Failed' END,
    TotalTests  = TotalTests.c,
    TestsPassed = PassedTests.c,
    TestsFailed = FailedTests.c
FROM ETL.LoadRuns lr
CROSS JOIN (SELECT COUNT(*) AS c FROM ETL.TestResults
            WHERE CAST(RunDate AS DATE) = CAST(GETDATE() AS DATE)) TotalTests
CROSS JOIN (SELECT COUNT(*) AS c FROM ETL.TestResults
            WHERE CAST(RunDate AS DATE) = CAST(GETDATE() AS DATE) AND Passed = 1) PassedT
ests
CROSS JOIN (SELECT COUNT(*) AS c FROM ETL.TestResults
            WHERE CAST(RunDate AS DATE) = CAST(GETDATE() AS DATE) AND Passed = 0) FailedT
ests
WHERE lr.RunID = ?;  -- ? = @RunID SSIS variable
```

6.3 Load Performance Monitoring

Tracking load duration over time detects performance degradation before it becomes a production problem:

```
-- Load duration trend: is the ETL getting slower over time?
SELECT
    CAST(RunStartTime AS DATE)          AS LoadDate,
    DATEDIFF(SECOND, RunStartTime, RunEndTime) AS DurationSeconds,
    TotalTests,
    TestsPassed,
    TestsFailed,
    RunStatus
FROM    ETL.LoadRuns
WHERE   RunStatus IN ('Passed', 'Failed')
ORDER BY RunStartTime DESC;
```

A sudden increase in load duration typically indicates one of:

- Source data volume grew significantly (expected, but worth confirming)
- A new JOIN or subquery was added without a supporting index
- Blocking or locking from a concurrent process
- A hardware or infrastructure change

6.4 Alerting on Failure

In production, ETL failures must trigger immediate notification. SQL Server Agent provides this through **Notifications** — email alerts sent when a job fails. This requires Database Mail to be configured:

```
-- Enable Database Mail (run once by DBA)
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'Database Mail XPs', 1;
RECONFIGURE;

-- Send a test email
EXEC msdb.dbo.sp_send_dbmail
    @profile_name   = 'ETL Alerts',
    @recipients     = 'Patrick.Dolinger@nscc.ca',
    @subject        = 'CabotTrail ETL — Load Failure',
    @body           = 'The nightly ETL load has failed. Please check ETL.LoadRuns for
details.';
```

```
-- Stored procedure: send alert on ETL failure
CREATE PROCEDURE ETL.usp_SendLoadAlert
    @RunID  INT
AS
BEGIN
    DECLARE @Subject    NVARCHAR(200);
    DECLARE @Body       NVARCHAR(MAX);
    DECLARE @Status     NVARCHAR(20);
    DECLARE @Failed     INT;

    SELECT  @Status = RunStatus,
            @Failed = TestsFailed
    FROM    ETL.LoadRuns
    WHERE   RunID = @RunID;

    IF @Status = 'Failed'
    BEGIN
        SET @Subject = 'CabotTrail ETL FAILED — ' +
                       CAST(@Failed AS NVARCHAR) + ' test(s) failed';
        SET @Body    = 'Load RunID ' + CAST(@RunID AS NVARCHAR) +
                       ' completed with failures. ' + CHAR(13) +
                       'Review ETL.TestResults and ETL.LoadErrors for details.';

        EXEC msdb.dbo.sp_send_dbmail
            @profile_name   = 'ETL Alerts',
            @recipients     = 'Patrick.Dolinger@nscc.ca',
            @subject        = @Subject,
            @body           = @Body;
    END
END;
GO
```

7. Introduction to Data Governance

Testing and monitoring ensure that the ETL system produces correct data. **Data governance** is the broader organizational framework that ensures data *remains* correct, *is understood* by its users, *is trusted* by decision-makers, and *is managed* responsibly throughout its lifecycle.

7.1 What Data Governance Is

Data governance is not a technology — it is a set of **policies, standards, processes, roles, and accountabilities** that define how data is managed within an organization. Technology (including ETL tools, data catalogs, and data quality platforms) supports governance, but governance itself is an organizational capability.

A useful working definition:

> ***Data governance*** *is the exercise of decision-making authority over data assets — defining who can do what with which data, under what conditions, in service of what organizational objectives.*

For a BI and ETL context, data governance addresses questions like:
- Who owns each data domain (Sales, Finance, HR) and is accountable for its quality?
- What does "Revenue" mean, precisely, and who approved that definition?
- Who can modify the ETL pipeline, and what approval process governs changes?
- How long is data retained, and when is it archived or deleted?
- Who can access sensitive data (customer PII, financial details), and how is that enforced?

7.2 The Four Governance Pillars for BI

In a business intelligence environment, governance clusters around four pillars:

Pillar 1: Data Definitions and Standards

Every key business metric must have a single, agreed, documented definition. In the absence of governance, different teams calculate the same metric differently:

- The sales team reports "revenue" as the value on the invoice
- Finance reports "revenue" as the value after returns and adjustments
- Operations reports "revenue" as orders placed, before cancellations

Each team is correct by their own definition. The organization has three different "revenue" numbers and cannot agree on performance. This is the **definition problem** — the most common and most damaging data governance failure.

The ETL developer's role: implement the agreed definition in the S2T mapping and the ETL transformation logic. If no agreed definition exists, **stop and get one before building**. An ETL that implements an unratified definition is encoding a political decision as technical fact.

Pillar 2: Data Stewardship

A **data steward** is a business-side person who is accountable for the quality and appropriate use of data within a specific domain. Data stewards:
- Define and maintain business definitions for their domain
- Approve changes to transformation rules that affect their domain
- Review and act on data quality reports
- Arbitrate disputes about data interpretations

In the CabotTrail context:
- The Sales Director is the data steward for customer, product, and revenue data
- The Finance Manager is the data steward for transaction, invoice, and payment data
- The Operations Manager is the data steward for inventory and purchasing data

The ETL developer does not decide what "Customer Category" means — the data steward does. The ETL developer implements the steward's decision.

Pillar 3: Data Quality Management

Data quality management is the operational process of monitoring, measuring, and improving data quality continuously. It includes:

- **Quality measurement:** The DQ rules and test suite from this chapter
- **Quality reporting:** Regular reports to data stewards showing quality metrics and trends
- **Issue management:** A process for reporting, tracking, and resolving data quality issues
- **Root cause analysis:** Finding and fixing the underlying source of quality problems, not just the symptoms

Quality problems found in the DW should be traced back to the source system and fixed there — not patched in the ETL transformation. Patching in the ETL creates hidden business rules that the source system continues to violate:

```
Wrong approach: OLTP has inconsistent province codes →
    ETL maps 'N.S.' and 'Nova Scotia' to 'NS' →
    The OLTP continues producing inconsistent codes →
    ETL silently "corrects" them forever

Right approach: OLTP has inconsistent province codes →
    Report to data steward →
    Data steward escalates to application team →
    Application team fixes the data entry validation →
    ETL no longer needs the mapping correction
```

Pillar 4: Access Control and Data Security

Not all users should have access to all data. Data governance defines access control policies — who can see what, under what conditions.

For BI systems, access control typically operates at two levels:

Database level: SQL Server security — GRANT/DENY/REVOKE on schemas, tables, and views. Analysts get read access to dimension and fact tables; ETL developers get read-write access to their target schema; DBAs have full access.

```
-- Example: grant read access to BI analysts on the data mart
CREATE ROLE DataMartReader;

GRANT SELECT ON SCHEMA::fact TO DataMartReader;
GRANT SELECT ON SCHEMA::dim  TO DataMartReader;

-- Add analysts to the role
ALTER ROLE DataMartReader ADD MEMBER [NSCC\john.analyst];
ALTER ROLE DataMartReader ADD MEMBER [NSCC\jane.analyst];
```

Row and column level: Some data requires finer-grained access control — for example, hiding salary data from users without HR clearance, or limiting regional managers to their own territory's data. SQL Server supports this through:
- **Row-Level Security (RLS)** — filter rows based on the current user
- **Column-level permissions** — DENY SELECT on specific sensitive columns
- **Dynamic Data Masking** — show masked versions of sensitive values to unauthorized users

7.3 The Data Governance Council

In organizations with mature governance programs, a **Data Governance Council** (or Data Stewardship Committee) provides oversight:
- Composed of data stewards from each business domain, IT leadership, and legal/compliance

- Meets regularly to review data quality reports, approve definition changes, and address escalations
- Has authority to allocate resources for data quality remediation

For smaller organizations (and for CabotTrail as a teaching context), governance is lighter — but the principles are the same. Even a single ETL developer and a single business analyst practicing governance (agreeing on definitions, documenting transformations, reviewing DQ reports) is more valuable than a larger team that builds without it.

---

8. Metadata Management

Metadata is data about data. In an ETL context, metadata describes the structure, lineage, meaning, and quality of the data in the warehouse. Managing metadata is the operational expression of data governance.

8.1 Types of Metadata

Type	Description	Examples
Technical metadata	Physical structure of data	Table names, column names, data types, indexes, constraints
Business metadata	Business meaning of data	Column descriptions, metric definitions, business rules
Operational metadata	ETL execution history	Load run times, row counts, error counts, last successful load
Lineage metadata	Where data came from	Source table, transformation applied, load date
Quality metadata	Quality measurements	NULL rates, DQ rule pass/fail history, anomaly counts

8.2 Technical Metadata: The System Catalog

SQL Server's system catalog provides complete technical metadata programmatically. The data dictionary query introduced in Chapter 5 of the lab book generates technical metadata from the catalog:

```
-- Technical metadata: complete column inventory for the DW
SELECT
    t.TABLE_SCHEMA                  AS [Schema],
    t.TABLE_NAME                    AS [Table],
    c.COLUMN_NAME                   AS [Column],
    c.DATA_TYPE +
    CASE
        WHEN c.CHARACTER_MAXIMUM_LENGTH IS NOT NULL
        THEN '(' + CAST(c.CHARACTER_MAXIMUM_LENGTH AS VARCHAR) + ')'
        WHEN c.NUMERIC_PRECISION IS NOT NULL
         AND c.DATA_TYPE IN ('decimal', 'numeric')
        THEN '(' + CAST(c.NUMERIC_PRECISION AS VARCHAR) + ','
                 + CAST(c.NUMERIC_SCALE AS VARCHAR) + ')'
        ELSE ''
    END                             AS DataType,
    c.IS_NULLABLE                   AS Nullable,
    c.COLUMN_DEFAULT                AS DefaultValue,
    CASE WHEN pk.COLUMN_NAME IS NOT NULL THEN 'PK' ELSE '' END AS KeyType,
    CASE WHEN fk.COLUMN_NAME IS NOT NULL THEN 'FK' ELSE '' END AS FKIndicator
FROM    INFORMATION_SCHEMA.TABLES t
INNER JOIN INFORMATION_SCHEMA.COLUMNS c
    ON c.TABLE_SCHEMA = t.TABLE_SCHEMA AND c.TABLE_NAME = t.TABLE_NAME
LEFT JOIN (
    SELECT ku.TABLE_SCHEMA, ku.TABLE_NAME, ku.COLUMN_NAME
    FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE ku
    INNER JOIN INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
        ON tc.CONSTRAINT_NAME = ku.CONSTRAINT_NAME
        AND tc.CONSTRAINT_TYPE = 'PRIMARY KEY'
) pk ON pk.TABLE_SCHEMA = t.TABLE_SCHEMA
     AND pk.TABLE_NAME  = t.TABLE_NAME
     AND pk.COLUMN_NAME = c.COLUMN_NAME
LEFT JOIN (
    SELECT ku.TABLE_SCHEMA, ku.TABLE_NAME, ku.COLUMN_NAME
    FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE ku
    INNER JOIN INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
        ON tc.CONSTRAINT_NAME = ku.CONSTRAINT_NAME
        AND tc.CONSTRAINT_TYPE = 'FOREIGN KEY'
) fk ON fk.TABLE_SCHEMA = t.TABLE_SCHEMA
     AND fk.TABLE_NAME  = t.TABLE_NAME
     AND fk.COLUMN_NAME = c.COLUMN_NAME
WHERE   t.TABLE_TYPE = 'BASE TABLE'
AND     t.TABLE_SCHEMA IN ('Dimension', 'Fact', 'ETL')
ORDER BY t.TABLE_SCHEMA, t.TABLE_NAME, c.ORDINAL_POSITION;
```

8.3 Operational Metadata: The ETL Audit Trail

The `ETL.LoadRuns` and `ETL.TestResults` tables constitute the **operational metadata repository** — a record of every ETL execution with its outcomes.

This operational metadata enables:

Load history analysis:

```
-- When was each table last successfully loaded?
SELECT
    TargetTable,
    MAX(RunDate)    AS LastSuccessfulLoad,
    COUNT(*)        AS TotalRuns,
    SUM(CASE WHEN Passed = 1 THEN 1 ELSE 0 END) AS PassedRuns
FROM    ETL.TestResults
WHERE   TestCategory = 'RowCount'
AND     Passed = 1
GROUP BY TargetTable
ORDER BY LastSuccessfulLoad;
```

Freshness monitoring:

```
-- Is the DW data current? Alert if last load was > 25 hours ago
SELECT
    TargetTable,
    MAX(RunDate)                AS LastLoad,
    DATEDIFF(HOUR, MAX(RunDate), GETDATE()) AS HoursSinceLoad,
    CASE WHEN DATEDIFF(HOUR, MAX(RunDate), GETDATE()) > 25
         THEN '   DATA MAY BE STALE'
         ELSE '✓ Current'
    END                         AS FreshnessStatus
FROM ETL.TestResults
WHERE TestCategory = 'RowCount' AND Passed = 1
GROUP BY TargetTable
ORDER BY HoursSinceLoad DESC;
```

8.4 Business Metadata: The Data Dictionary

The S2T mapping from Chapter 3 serves as the business metadata repository for the ETL system. A formal data dictionary extends it with business-side descriptions:

Column	Technical definition (from catalog)	Business description
`GrossProfit`	`DECIMAL(18,2) NOT NULL`	Revenue minus cost of goods sold for this invoice line. Cost is computed as UnitPrice × StandardCostPct × PickedQuantity. StandardCostPct is set per product category. Do not sum GrossProfitMarginPct — compute from GrossProfit/LineTotal × 100.
`DaysUntilInvoice`	`AS DATEDIFF(DAY, OrderDate, InvoiceDate) PERSISTED`	Number of days elapsed between the customer placing the order and the invoice being issued. Used as a proxy for order fulfillment speed.
`SalesTerritory`	`NVARCHAR(60) NULL`	Regional grouping derived from the customer's delivery province code during ETL. Not stored in the OLTP — derived during DW load.

The business description column is the most valuable — and the one that requires business stakeholder input to write correctly. A data dictionary without business descriptions is incomplete metadata.

---

9. Chapter Summary

- **ETL testing** is fundamentally different from application testing: it is set-based, must accommodate changing source data, must catch silent failures, and depends on business-agreed specifications.
- The **five testing levels** are: unit testing (individual transformation logic), integration testing (complete package end-to-end), system testing (all packages together), reconciliation testing (source vs target data agreement), and regression testing (changes have not broken existing behaviour).
- A **formal test suite** records test results in a persistent table (`ETL.TestResults`) after every load. Tests cover row counts, aggregates, referential integrity, and derived measure verification. A summary query provides immediate pass/fail visibility.
- **Reconciliation testing** has three layers: row count (was every row loaded?), aggregate (do key measures sum correctly?), and derived measure verification (are computed columns correct?). Diagnostic queries accompany each layer to isolate defects.
- **Data quality in production** is a continuous activity — monitoring trends, tracking the unknown member rate, detecting source data changes, and feeding quality issues back to source system owners.

- **SSIS logging** through the Catalog provides operational execution history. Custom logging to `ETL.LoadRuns` and `ETL.TestResults` provides business-meaningful audit trails. Alerting via Database Mail ensures failures receive immediate attention.
- **Data governance** is the organizational framework — policies, stewardship, quality management, and access control — that makes data trustworthy over time. The ETL developer's governance responsibilities include implementing agreed definitions, documenting transformations, and not patching source data quality problems silently.
- **Metadata management** covers technical (system catalog), operational (load history), and business (data dictionary) metadata. All three types are necessary for a fully governed BI environment.

---

10. Review Questions

1. Explain why ETL reconciliation tests should verify *structural relationships* (e.g., "DW revenue equals source revenue") rather than *specific values* (e.g., "total revenue equals $6,350,582.50"). Under what circumstances would a specific value test be appropriate?
2. The row count reconciliation test for `Fact.FactSales` shows 16,200 rows in the DW versus 16,359 rows expected. The error table shows 159 records logged with `ErrorDescription = 'No match found in lookup Dimension.DimCustomer'`. What does this tell you about the ETL failure, and what two follow-up investigations would you conduct?
3. Describe the difference between a unit test and an integration test in the context of ETL. Give a specific example of each for the `SalesTerritory` derivation in `DimCustomer`.
4. A data steward reports that "revenue" in the BI dashboard does not match the revenue figure in the finance system's monthly close report. Before changing the ETL, what governance process should be followed? Who needs to be involved?
5. The `GrossProfitMarginPct` column has a DQ trend that shows it has been 45.0% for all records for the past three months, then suddenly dropped to an average of 38.5% after a recent load. Describe the investigation you would conduct and what business context might explain this change.
6. Explain the concept of the unknown member in a dimension table. Write a query that reports the unknown member usage rate across all five dimension FKs in `Fact.FactSales`. What would a 5% unknown member rate in `CustomerKey` tell you?
7. A new junior developer has written an ETL package that "corrects" province codes in the OLTP data — mapping `'N.S.'` and `'Nova Scotia'` to `'NS'` within the ETL transformation. Explain why this approach is problematic from a governance perspective, and what the correct approach should be.

8. What is the difference between technical metadata, business metadata, and operational metadata? For the column `DaysToReturn` in `Fact.FactReturns`, write an example of each type.

Deeper Dive

Going Further with Testing, Quality, and Governance

Test-Driven ETL Development

Test-Driven Development (TDD) is a software development practice where tests are written before code. The developer writes a failing test, writes the minimum code to make it pass, then refactors. Applied to ETL, this is called **Test-Driven ETL Development**.

The process:
1. Write the S2T mapping (the specification)
2. Write the test suite from the mapping (all tests fail — the ETL does not exist yet)
3. Build the ETL until all tests pass
4. Refactor for performance and maintainability (tests continue to pass)

This discipline ensures that the test suite is complete before the ETL is built — not retrofitted afterward, when the temptation is to write tests that match what the ETL actually does rather than what it should do.

For SQL-based ETL, tSQLt is a popular unit testing framework for SQL Server:
tSQLt — T-SQL Unit Testing Framework

Data Observability

Data observability is an emerging discipline that extends traditional data quality monitoring with automated anomaly detection. Rather than checking predefined rules ("UnitPrice must be > 0"), data observability tools learn the normal patterns of data and alert when anomalies are detected:

- Unusual changes in row counts (column value distributions shifting)
- Schema changes (a column disappearing or changing type)
- Freshness violations (data not updated within the expected window)
- Volume anomalies (an order-of-magnitude change in row counts)

Commercial data observability platforms include Monte Carlo, Acceldata, and Bigeye. dbt (Data Build Tool) has built-in testing features that implement many data observability concepts:
dbt Testing Documentation

Microsoft's own **Azure Monitor** and **Azure Data Factory monitoring** provide similar capabilities in the Azure ecosystem:
Monitor Azure Data Factory

The DAMA-DMBOK Data Quality Framework

The **DAMA-DMBOK** (Data Management Body of Knowledge) provides the most comprehensive academic and practitioner treatment of data quality as an organizational discipline. Its framework defines:

- Six data quality dimensions (accuracy, completeness, consistency, timeliness, uniqueness, validity) — consistent with what this chapter covers
- A data quality management lifecycle (define, measure, analyse, improve, monitor)
- Roles and responsibilities for data quality management
- Integration of data quality with other data management disciplines (governance, metadata, security)

DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications.

Row-Level Security in SQL Server

Row-Level Security (RLS) is a SQL Server feature that transparently filters rows based on the identity of the executing user. For BI systems where different users should see different subsets of data (regional managers see only their region's sales, HR personnel see only their division's records), RLS implements the filter at the database layer rather than in the application or BI tool.

```
-- RLS example: restrict dim.Customer rows by SalesTerritory
-- based on the current user

CREATE SCHEMA Security;
GO

-- Security predicate: function that returns 1 for allowed rows
CREATE FUNCTION Security.fn_TerritoryFilter(@SalesTerritory NVARCHAR(60))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS IsAllowed
    WHERE   @SalesTerritory = (
                SELECT UserTerritory
                FROM   Security.UserTerritoryMapping
                WHERE  UserName = USER_NAME()
            )
    OR USER_NAME() IN ('ETL_Service', 'DBA_Admin');  -- Full access for service accounts
GO

-- Apply the security policy
CREATE SECURITY POLICY Security.TerritoryPolicy
    ADD FILTER PREDICATE Security.fn_TerritoryFilter(SalesTerritory)
    ON dim.Customer
WITH (STATE = ON);
GO
```

Microsoft documentation:

Row-Level Security — SQL Server

The Data Contract Pattern

An emerging pattern in data engineering is the **data contract** — a formal, versioned agreement between a data producer (source system) and a data consumer (ETL / data warehouse) that specifies:

- The schema of the data being produced
- The quality guarantees (completeness, freshness, constraints)
- The change notification process (how consumers are notified of schema changes)
- The SLA for data availability

Data contracts formalize what this chapter describes informally as the S2T mapping and DQ rule table. They shift the data quality responsibility upstream — the source system agrees to produce data that meets the contract's standards, rather than the ETL developer silently correcting whatever arrives.

Tools implementing data contracts include Soda Core and Great Expectations:
Soda Core Documentation
Great Expectations Documentation

Industry Perspectives

Kimball on ETL Testing

Kimball's *The Data Warehouse ETL Toolkit* dedicates a full chapter to ETL testing and quality assurance. His core principle aligns with this chapter:

> *"The ETL system must be tested at every level — unit, integration, system, and acceptance. Testing is not a phase that happens at the end of development; it is an activity that is woven throughout development. An ETL system that has not been thoroughly tested is not ready for production, regardless of how well it was designed."*

Kimball's acceptance testing concept — where business users sign off on data correctness before go-live — is the data governance connection: the business, not just the ETL developer, verifies that the data is correct.

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapter 11 covers testing and quality assurance.

Microsoft on SQL Server Security

Microsoft provides comprehensive documentation on all SQL Server security features relevant to data governance:
- Row-Level Security
- Dynamic Data Masking
- SQL Server Audit
- Always Encrypted

References and Further Reading

1. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapter 11 covers ETL testing; Chapter 12 covers quality assurance systems.

2. DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications. — The authoritative reference for data governance and data quality management disciplines.

3. Redman, T. C. (2008). *Data Driven: Profiting from Your Most Important Business Asset*. Harvard Business Press. — Practitioner-focused treatment of data quality as a business asset.

4. Loshin, D. (2010). *The Practitioner's Guide to Data Quality Improvement*. Morgan Kaufmann. — Covers the data quality lifecycle, measurement frameworks, and improvement processes.

5. Microsoft. (2024). *SSIS Catalog — Monitoring*. https://learn.microsoft.com/en-us/sql/integration-services/catalog/ssis-catalog

6. Microsoft. (2024). *Row-Level Security*. https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security

7. Microsoft. (2024). *Dynamic Data Masking*. https://learn.microsoft.com/en-us/sql/relational-databases/security/dynamic-data-masking

8. tSQLt. (n.d.). *Database Unit Testing for SQL Server*. https://tsqlt.org/

9. dbt Labs. (2024). *Data Tests — dbt Documentation*. https://docs.getdbt.com/docs/build/data-tests

10. Soda. (2024). *Soda Core Documentation*. https://docs.soda.io/

11. Eckerson, W. W. (2010). *Performance Dashboards: Measuring, Monitoring, and Managing Your Business* (2nd ed.). Wiley. — Covers the governance and stewardship structures that underpin trusted BI.

Chapter 6: ETL Documentation — Data Dictionaries and Process Flows

Chapter Overview

The previous chapters have built a complete, tested ETL pipeline. This chapter addresses the work that makes that pipeline maintainable, auditable, and transferable: documentation.

Documentation is the most consistently undervalued activity in ETL development. It is easy to skip when deadlines press, easy to defer when the immediate problem is getting data to load, and easy to rationalize away when only one person knows the system. It is also the activity whose absence causes the most long-term damage — to system maintainability, to data trust, and to the organization's ability to onboard new team members or adapt to change.

This chapter approaches documentation not as a burden but as a professional discipline that pays compounding returns. A well-documented ETL system is faster to debug, safer to change, easier to hand off, and more likely to be trusted by the business users who depend on it.

By the end of this chapter you will be able to:

- Explain the purpose and components of a formal ETL design document
- Build a complete data dictionary for a dimensional model
- Draw process flow diagrams for ETL pipelines using standard notation
- Describe the components of a complete source-to-target mapping document
- Apply PMI-standard documentation practices to ETL design deliverables
- Use SQL queries to generate documentation artifacts directly from the system catalog
- Explain what makes documentation maintainable over the life of a system

Table of Contents

---

1. The Case for ETL Documentation

Before examining what good documentation looks like, it is worth understanding precisely what its absence costs.

1.1 The Undocumented ETL Tax

An ETL system without documentation imposes what practitioners call the **undocumented ETL tax** — a growing overhead of time and risk on every activity that involves the system:

Debugging: When a reconciliation test fails and the developer must find the cause, undocumented systems require reading every SQL expression, every SSIS component, every stored procedure to understand what the system was supposed to do. With documentation, the expected behaviour is stated; the actual behaviour can be compared immediately.

Change management: A business analyst requests that the `SalesTerritory` mapping be updated — Manitoba should move from 'Prairies' to 'Central Canada'. In a documented system this takes twenty minutes: find the mapping in the S2T document, update the transformation rule, update the SSIS expression, run the tests. In an undocumented system, the developer must first spend an hour finding where the CASE expression lives, then determine whether it appears in multiple places.

Knowledge transfer: A new developer joins the team. In a documented system, the ETL design document, data dictionary, and S2T mapping provide a complete mental model of the system within a day of reading. In

an undocumented system, the new developer spends weeks reverse-engineering the system from package structures — and even then, the *intent* of design decisions may never be recoverable.

Audit and compliance: A regulator or auditor asks: "Show me exactly how revenue is calculated and where that data comes from." A documented system answers this question immediately from the S2T mapping and data dictionary. An undocumented system requires a multi-week forensic exercise.

Trust: Business users who cannot get a clear answer to "how is this number calculated?" will eventually stop trusting the BI system — regardless of whether the number is actually correct. Documentation makes correctness visible and verifiable.

1.2 Documentation as a Design Activity

The most effective documentation is written *during* design and *before* implementation, not as an afterthought. This is not merely a matter of efficiency — it is a matter of quality.

Writing the data dictionary before loading data forces precision about column meanings. Writing the S2T mapping before building the SSIS package surfaces gaps and ambiguities that would otherwise only be discovered during testing. Writing the process flow before coding reveals missing error paths and dependency violations.

The S2T mapping from Chapter 3 is already an example of this principle — it was developed before any ETL code was written, and every ETL decision in Chapters 3 and 4 was derived from it. This chapter formalizes that approach into a complete documentation framework.

---

2. The ETL Design Document

The **ETL Design Document** is the master reference for an ETL project. It collects all design decisions, specifications, and rationale in a single, structured deliverable. It is the document that a developer should be able to read and then build the ETL system — and the document that a new developer should be able to read and then maintain it.

2.1 Document Structure

A complete ETL Design Document has eight sections:

Section	Content	Primary audience
1. Executive Summary	Scope, objectives, key design decisions	Management, stakeholders
2. Source System Analysis	Source schemas, row counts, date ranges, DQ findings	ETL developers, DBAs
3. Target System Overview	DW/mart schema, all dims and facts with row estimates	ETL developers, BI developers
4. Gap Analysis	All identified gaps with type, description, resolution	ETL developers, data stewards
5. Source-to-Target Mapping	Complete column-level specification for all tables	ETL developers, QA analysts
6. Data Quality Rules	Rule catalogue with SQL, severity, and results	ETL developers, data stewards
7. Process Flow Diagrams	Visual ETL workflow for each fact table load	ETL developers, operations
8. Testing Strategy	Test types, reconciliation metrics, pass/fail criteria	ETL developers, QA analysts

2.2 Section 1: Executive Summary

The executive summary is written for readers who will not read the rest of the document. It states:

- **Scope:** What source systems are included, what target system is being loaded, what date range the data covers
- **Objectives:** What business questions the DW/mart is designed to answer
- **Architecture:** A one-paragraph description of the pipeline (OLTP → DW → data marts)
- **Key design decisions:** The three to five most important design choices and their rationale
- **Known limitations:** Gaps that were accepted (e.g., PostalCode not available), SCD Type choices, any data that is excluded and why

CabotTrail Executive Summary example:

> *This document specifies the ETL design for `CabotTrailOutdoorDW`, a dimensional data warehouse loaded nightly from `CabotTrailOutdoor` (the operational OLTP system). The DW covers sales, purchasing, returns, inventory, and financial transaction data for the period 2022–2025, representing four years of business activity for CabotTrail Outdoor, a Nova Scotia-based outdoor gear retailer.*
>
> *The DW uses a Kimball-style star schema with 10 dimension tables and 6 fact tables. Five subject-specific data marts are derived from the DW for consumption by BI tools.*

> ***Key design decisions:***
> *1. Product cost is derived from `StandardCostPct` per product category (not from purchase order cost), producing realistic margin variation across 13 categories*
> *2. All dimensions are SCD Type 1 (overwrite) — historical customer/product changes are not tracked in this version*
> *3. The primary product category is resolved using MIN(ProductCategoryID) for products assigned to multiple categories*
> *4. Open orders (2026, no invoices) are excluded from all fact tables*
> *5. `SalesTerritory` is derived during ETL from province code — it does not exist in the OLTP*

2.3 Section 2: Source System Analysis

The source system analysis summarizes the profiling work from Chapter 3. It should include:

Schema inventory table:

Schema	Tables	Total rows	Date range	Key entities
Sales	8	~35,000	2022–2026	Customers, Orders, Invoices, Returns
Purchasing	4	~2,000	2022–2026	Suppliers, PurchaseOrders
Inventory	4	~500	Current snapshot	Products, Categories, Holdings
Application	8	~300	Reference data	Employees, Geography, Lookups

Data quality summary: A condensed version of the DQ rule results, organized by severity:

Severity	Rules checked	Passed	Failed
High	8	8	0
Medium	4	3	1
Low	2	2	0

For any failed rules, describe the issue and how it was resolved.

2.4 Section 3: Target System Overview

The target system overview describes the DW schema that will be loaded. For each dimension and fact table, document:

- Table name and schema

- Grain (for facts) or entity description (for dimensions)
- Row count estimate
- Primary source tables
- Special design notes

```
Dimension.DimCustomer
  Description: One row per customer, sourced from Sales.Customers
               with geography flattened from Application.Cities/StateProvinces/Countries
  Rows: 100 (plus unknown member row)
  Source tables: Sales.Customers, Application.Cities, Application.StateProvinces,
                 Application.Countries
  Notes: SCD Type 1. SalesTerritory derived from ProvinceCode (not in OLTP).
         PostalCode not available in source — NULL.

Fact.FactSales
  Grain: One row per product on each customer invoice
  Rows: ~16,359
  Source tables: Sales.InvoiceLines, Sales.Invoices, Sales.Orders,
                 Sales.OrderLines, Inventory.ProductCategories
  Dimensions: DimDate (×3 roles), DimCustomer, DimProduct, DimEmployee,
              DimGeography, DimDeliveryMethod
  Notes: UnitCost derived from StandardCostPct × UnitPrice × PickedQuantity.
         GrossProfitMarginPct is non-additive — do not SUM.
         Excludes 2026 open orders (YEAR(OrderDate) < 2026).
```

---

3. The Data Dictionary

The **data dictionary** is the most referenced document in a BI environment. Analysts, developers, and business users consult it to understand what columns mean, where they come from, and how to use them correctly. A good data dictionary is not just a list of column names and types — it is a vocabulary for the business.

3.1 What a Complete Data Dictionary Contains

For every table and column in the DW and data marts, the data dictionary records:

Field	Description	Example
Schema	Database schema	`Fact`
Table	Table name	`FactSales`
Column	Column name	`GrossProfitMarginPct`
Data type	SQL Server data type with precision	`DECIMAL(8,2)`
Nullable	Whether NULL is permitted	`NOT NULL`
Default	Default value if any	None
Key type	PK, FK, or blank	FK
Source	Origin (table.column or derivation)	Computed from GrossProfit / LineTotal
Business description	Plain-language meaning	Gross profit as a percentage of revenue for this invoice line. Computed as (LineTotal − UnitCost) / LineTotal × 100. **Non-additive — never sum this column.** Use SUM(GrossProfit) / SUM(LineTotal) × 100 for aggregate margin.
Usage notes	How to use correctly; common mistakes	Always compute aggregate margin from GrossProfit and LineTotal, not by averaging this column
DQ rules	Which rules apply	DQ-009: GrossProfit ≥ 0

The **business description** and **usage notes** are the columns that require business stakeholder input. They cannot be generated from the system catalog — they must be written by a human who understands what the data means.

3.2 The Data Dictionary as a SQL Query

The technical portion of the data dictionary can be generated directly from the system catalog. This ensures it is always accurate and never becomes stale due to schema changes:

```
-- Generate the technical data dictionary for the DW
USE CabotTrailOutdoorDW;

SELECT
    s.name                              AS [Schema],
    t.name                              AS [Table],
    c.name                              AS [Column],
    -- Full data type with precision/scale
    tp.name +
    CASE
        WHEN tp.name IN ('nvarchar','varchar','char','nchar')
            AND c.max_length > 0
        THEN '(' + CAST(
                CASE WHEN tp.name IN ('nvarchar','nchar')
                     THEN c.max_length / 2
                     ELSE c.max_length END
             AS VARCHAR) + ')'
        WHEN tp.name IN ('nvarchar','varchar')
            AND c.max_length = -1
        THEN '(MAX)'
        WHEN tp.name IN ('decimal','numeric')
        THEN '(' + CAST(c.precision AS VARCHAR) + ',' + CAST(c.scale AS VARCHAR) + ')'
        ELSE ''
    END                                 AS [DataType],
    CASE c.is_nullable WHEN 1 THEN 'NULL' ELSE 'NOT NULL' END AS [Nullable],
    -- Key type
    CASE
        WHEN pk.column_id IS NOT NULL THEN 'PK'
        WHEN fk.parent_column_id IS NOT NULL THEN 'FK'
        ELSE ''
    END                                 AS [KeyType],
    -- Referenced table for FK columns
    CASE WHEN fk.parent_column_id IS NOT NULL
         THEN ref_s.name + '.' + ref_t.name + '.' + ref_c.name
         ELSE ''
    END                                 AS [References],
    -- Computed column flag
    CASE c.is_computed WHEN 1 THEN 'Computed' ELSE '' END AS [Computed],
    -- Column ordinal position for ordering
    c.column_id                         AS [Position]
FROM    sys.tables t
INNER JOIN sys.schemas s        ON s.schema_id   = t.schema_id
INNER JOIN sys.columns c        ON c.object_id   = t.object_id
INNER JOIN sys.types tp         ON tp.user_type_id = c.user_type_id
-- PK detection
LEFT JOIN (
    SELECT  ic.object_id, ic.column_id
    FROM    sys.index_columns ic
    INNER JOIN sys.indexes i ON i.object_id = ic.object_id
                            AND i.index_id = ic.index_id
                            AND i.is_primary_key = 1
) pk ON pk.object_id = t.object_id AND pk.column_id = c.column_id
-- FK detection
```

```
LEFT JOIN sys.foreign_key_columns fk
    ON fk.parent_object_id = t.object_id AND fk.parent_column_id = c.column_id
LEFT JOIN sys.tables ref_t
    ON ref_t.object_id = fk.referenced_object_id
LEFT JOIN sys.schemas ref_s
    ON ref_s.schema_id = ref_t.schema_id
LEFT JOIN sys.columns ref_c
    ON ref_c.object_id = fk.referenced_object_id
    AND ref_c.column_id = fk.referenced_column_id
WHERE   s.name IN ('Dimension', 'Fact', 'ETL')
ORDER BY s.name, t.name, c.column_id;
```

This query produces the technical skeleton of the data dictionary. Export it to Excel or a documentation tool and add the business description and usage notes columns manually.

3.3 Extended Properties: Storing Descriptions in the Database

SQL Server supports **extended properties** — metadata stored directly in the database, attached to any database object (table, column, index, schema). They provide a way to store business descriptions alongside the schema definition, making them accessible to documentation tools:

```
-- Add a business description to a column using extended properties
EXEC sys.sp_addextendedproperty
    @name       = N'MS_Description',
    @value      = N'Gross profit as a percentage of revenue for this invoice line.
                    Computed as (LineTotal - UnitCost) / LineTotal * 100.
                    NON-ADDITIVE: never sum this column.
                    Always compute aggregate margin as SUM(GrossProfit) / SUM(LineTotal)
* 100.',
    @level0type = N'SCHEMA',  @level0name = N'Fact',
    @level1type = N'TABLE',   @level1name = N'FactSales',
    @level2type = N'COLUMN',  @level2name = N'GrossProfitMarginPct';

-- Update an existing extended property
EXEC sys.sp_updateextendedproperty
    @name       = N'MS_Description',
    @value      = N'[Updated description]',
    @level0type = N'SCHEMA',  @level0name = N'Fact',
    @level1type = N'TABLE',   @level1name = N'FactSales',
    @level2type = N'COLUMN',  @level2name = N'GrossProfitMarginPct';

-- Query all extended properties: generate data dictionary with descriptions
SELECT
    s.name          AS [Schema],
    t.name          AS [Table],
    c.name          AS [Column],
    ep.value        AS [Description]
FROM    sys.extended_properties ep
INNER JOIN sys.columns c  ON c.object_id  = ep.major_id
                         AND c.column_id  = ep.minor_id
INNER JOIN sys.tables t   ON t.object_id  = c.object_id
INNER JOIN sys.schemas s  ON s.schema_id  = t.schema_id
WHERE   ep.name = 'MS_Description'
AND     ep.class = 1    -- Column-level properties
ORDER BY s.name, t.name, c.column_id;
```

Extended properties are preserved through database backups and restores. Tools like SSMS, Azure Data Studio, and many third-party documentation generators read `MS_Description` extended properties automatically.

3.4 Data Dictionary Entries for Key CabotTrail Columns

The following entries illustrate the depth expected for a production data dictionary. Each entry combines technical precision with business-meaningful description:

`Fact.FactSales.GrossProfitMarginPct`

- **Data type:** `DECIMAL(8,2) NOT NULL`

- **Source:** Computed — `(LineTotal − UnitCost) / LineTotal × 100`

- **Business description:** Gross profit expressed as a percentage of revenue for this individual invoice line. Computed during ETL using `StandardCostPct` per product category. For example, a Sleeping Bag line with LineTotal = $249.99 and UnitCost = $167.49 has GrossProfitMarginPct = 33.0%.
- **Usage notes: This measure is non-additive — do not use SUM(GrossProfitMarginPct).** To compute aggregate margin, always use `SUM(GrossProfit) / SUM(LineTotal) × 100`. Averaging this column also produces incorrect results because it weights all lines equally regardless of value.
- **DQ rules:** Must satisfy `LineTotal >= UnitCost` (if violated, margin is negative — investigate source data)

`Dimension.DimCustomer.SalesTerritory`

- **Data type:** `NVARCHAR(60) NULL`
- **Source:** Derived — CASE expression on `Application.StateProvinces.StateProvinceCode` during ETL load
- **Business description:** Regional grouping assigned to each customer based on their delivery province. Values: 'Atlantic — Nova Scotia', 'Atlantic — New Brunswick', 'Atlantic — PEI', 'Atlantic — Newfoundland', 'Quebec', 'Ontario', 'Prairies', 'Alberta', 'British Columbia'. Customers with unrecognized province codes receive 'Other'.
- **Usage notes:** This attribute does not exist in the OLTP system — it is derived during ETL. If the territory mapping needs to change, the S2T mapping document must be updated and the ETL redeployed.
- **DQ rules:** NULL indicates a customer whose DeliveryCityID could not be resolved to a province — investigate the geography hierarchy

---

`Fact.FactInventory.QuantityOnHand`

- **Data type:** `INT NOT NULL`
- **Source:** `Inventory.ProductHoldings.QuantityOnHand`
- **Business description:** Number of units of this product in stock as of the snapshot date. Sourced from the most recent physical stocktake recorded in `ProductHoldings.LastStocktakeDate`.
- **Usage notes: Semi-additive** — can be summed across products (total units in stock) but NOT across snapshot dates (which would double-count inventory). If this table is loaded with multiple snapshots, use `MAX(SnapshotDate)` to filter to the current snapshot before summing. Currently loaded as a single snapshot per product.
- **DQ rules:** Must be ≥ 0

---

4. Process Flow Diagrams

A **process flow diagram** is a visual representation of an ETL pipeline — what happens, in what order, under what conditions, and what happens when things go wrong. It is the visual complement to the S2T mapping's column-level specification.

4.1 Standard Process Flow Notation

Process flows use a standardized symbol set. BPMN (Business Process Model and Notation) is the most widely used formal standard, but ETL process flows typically use a simplified subset:

Symbol	Shape	Represents
Start / End	Rounded rectangle (oval)	The beginning or end of the process
Process step	Rectangle	A single ETL operation
Decision	Diamond	A conditional branch (Yes/No, Pass/Fail)
Data store	Cylinder	A database table (source or target)
Document	Rectangle with wavy bottom	A file (CSV, Excel)
Manual operation	Trapezoid	A step requiring human intervention
Flow arrow	Directed arrow	Direction of execution
Annotation	Dashed bracket with text	Notes on a step

> ***Rule:*** *Every decision diamond must have exactly two exit paths, each labelled. A decision with only one exit path is a logic error — what happens in the other case?*

4.2 Process Flow for Load_DimProduct.dtsx

```
( START )
    |
    v
[ Truncate Dimension.DimProduct ]
    |
    v
[ Extract: JOIN Products + Colors + PackageTypes + PrimaryCategoryAssignment ]
    |
    v
< DQ Check: ProductID <> 0 AND UnitPrice > 0 >
    |Yes                    |No
    v                       v
[ Lookup:                [ Write to ETL.LoadErrors ]
  ProductCategoryID         |
  → ProductCategoryKey ]    v
    |Match    |No Match  ( FAIL PATH — log and continue )
    v         v
    |    [ Write to ETL.LoadErrors: 'No match in DimProductCategory' ]
    v
[ Lookup:
  SupplierID
  → SupplierKey ]
    |Match    |No Match
    v         v
    |    [ Write to ETL.LoadErrors: 'No match in DimSupplier' ]
    v
[ Write to Dimension.DimProduct ]
    |
    v
[ Run Reconciliation Test:
  DimProduct row count vs source ]
    |
    v
< Test Passed? >
    |Yes                 |No
    v                    v
[ Record PASS        [ Record FAIL in ETL.TestResults ]
  in ETL.TestResults ]     |
    |                    [ Send alert: ETL.usp_SendLoadAlert ]
    v                        |
( END — Success )       ( END — Failure )
```

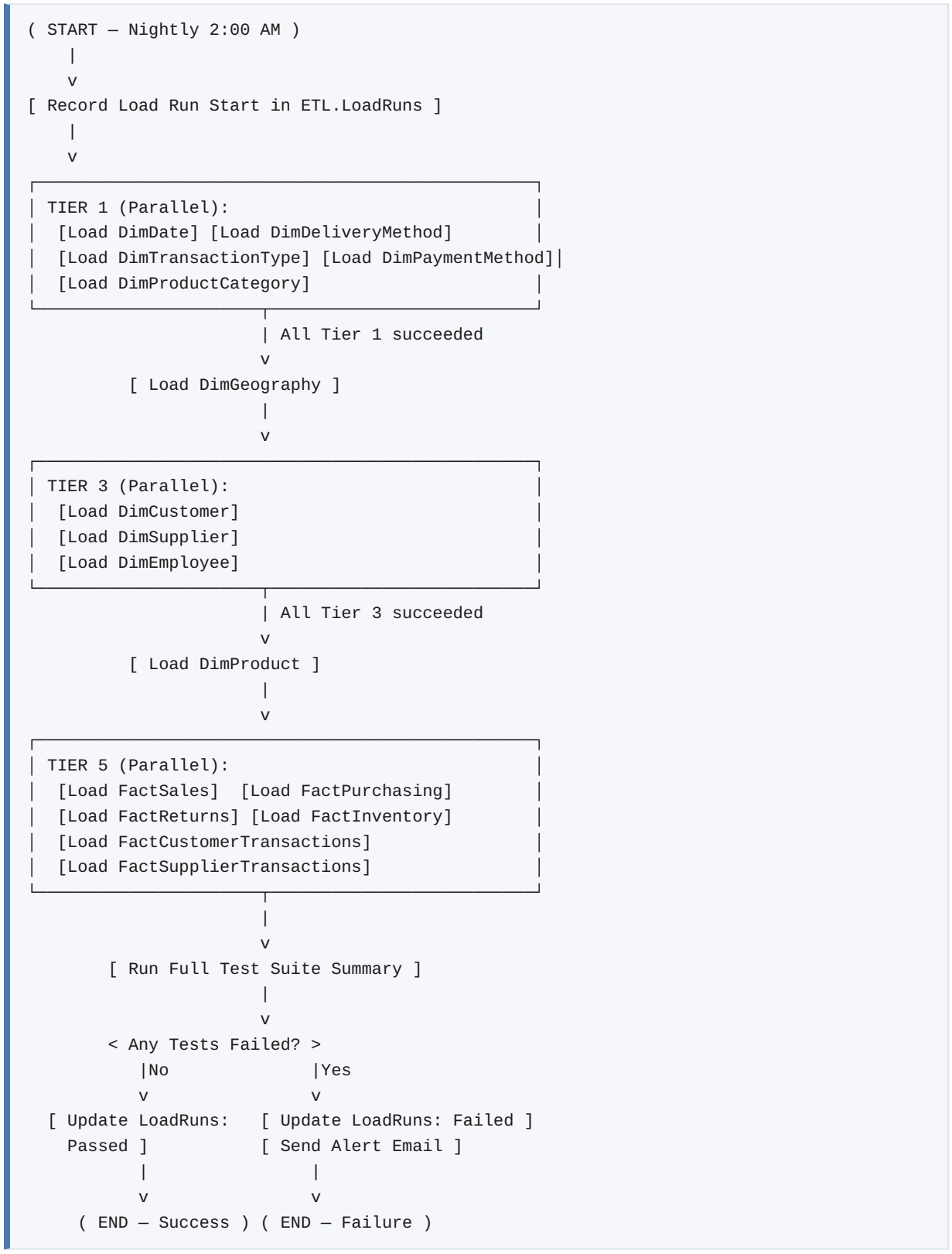
4.3 Process Flow for Load_FactSales.dtsx

The fact table load is more complex — it has eight lookup transformations, each with an error path:

```
( START )
    |
    v
[ Truncate Fact.FactSales ]
    |
    v
[ Extract: JOIN InvoiceLines + Invoices + Orders + OrderLines +
           Customers + PrimaryCategoryAssignment + ProductCategories ]
    |
    v
< DQ Checks: UnitPrice > 0 AND Quantity > 0 AND InvoiceID NOT NULL >
    |Pass                      |Fail
    v                          v
[ Lookup: CustomerID        [ Write to ETL.LoadErrors ]
  → CustomerKey ]
    |Match    |No Match
    v         v
    |    [ Write to ETL.LoadErrors ]
    v
[ Lookup: ProductID → ProductKey ] → (error path as above)
    |
[ Lookup: EmployeeID → EmployeeKey ] → (error path as above)
    |
[ Lookup: CityID → GeographyKey ] → (error path as above)
    |
[ Lookup: DeliveryMethodID → DeliveryMethodKey ] → (error path as above)
    |
[ Lookup: OrderDateKey → validate in DimDate ] → (error path as above)
    |
[ Lookup: InvoiceDateKey → validate in DimDate ] → (error path as above)
    |
[ Lookup: DueDateKey → validate in DimDate ] → (error path as above)
    |
    v
[ Write to Fact.FactSales ]
    |
    v
[ Run Reconciliation Test: Row count ]
[ Run Reconciliation Test: Revenue sum ]
[ Run Reconciliation Test: Orphan check ]
    |
    v
< All Tests Passed? >
    |Yes                 |No
    v                    v
( END — Success )    [ Send alert ] → ( END — Failure )
```

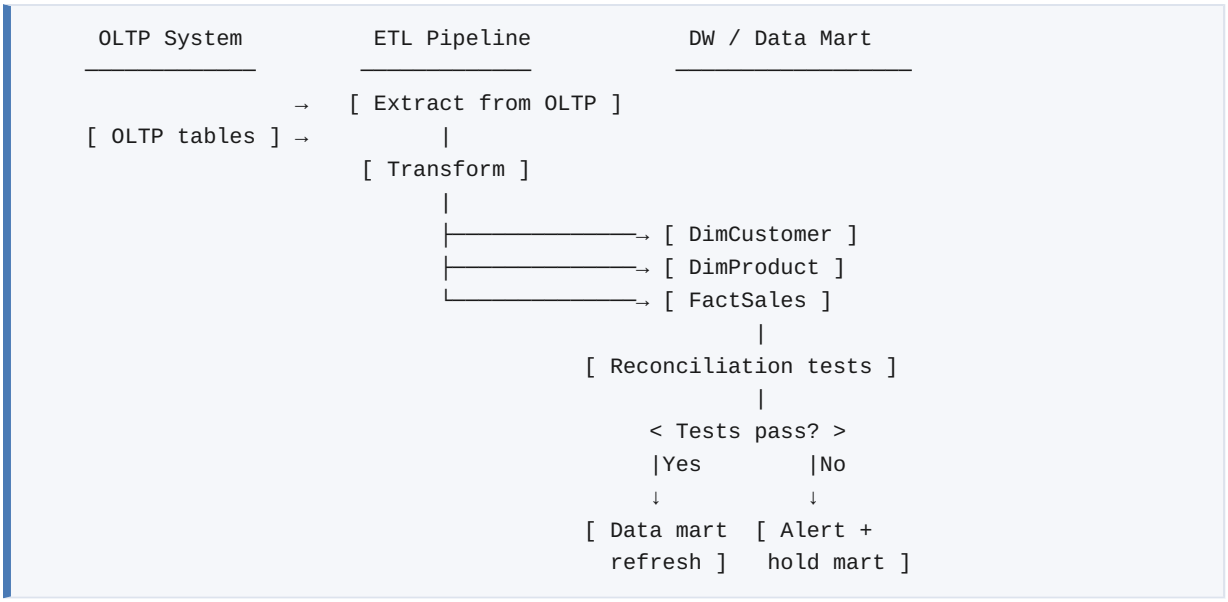
4.4 Process Flow for Master_DW_Load.dtsx

The master package process flow shows the dependency structure across all packages:


(START — Nightly 2:00 AM)
[Record Load Run Start in ETL.LoadRuns]
TIER 1 (Parallel):
[Load DimDate] [Load DimDeliveryMethod]
[Load DimTransactionType] [Load DimPaymentMethod]
[Load DimProductCategory]
All Tier 1 succeeded
[Load DimGeography]
TIER 3 (Parallel):
[Load DimCustomer]
[Load DimSupplier]
[Load DimEmployee]
All Tier 3 succeeded
[Load DimProduct]
TIER 5 (Parallel):
[Load FactSales] [Load FactPurchasing]
[Load FactReturns] [Load FactInventory]
[Load FactCustomerTransactions]
[Load FactSupplierTransactions]
[Run Full Test Suite Summary]
< Any Tests Failed? >
No
Yes
[Update LoadRuns: Passed]
[Update LoadRuns: Failed]
[Send Alert Email]
(END — Success)
(END — Failure)

4.5 Swimlane Diagrams

For processes that involve multiple systems or teams, a **swimlane diagram** adds a vertical or horizontal partitioning layer showing which actor or system is responsible for each step:



Swimlane diagrams are particularly useful for communicating ETL architecture to non-technical stakeholders — they make the system boundary between OLTP and DW visible.

---

5. The Complete Source-to-Target Mapping

The S2T mapping was introduced in Chapter 3 as a design tool. This section describes what makes a mapping document *complete* — ready to serve as a specification, a test oracle, and an audit trail.

5.1 Completeness Criteria

An S2T mapping is complete when:

1. **Every target column has a row** — no column is left undocumented
2. **Every transformation rule is unambiguous** — a developer who has never seen the system can implement it from the rule alone, without asking questions
3. **All gaps are documented** — missing data, data quality issues, and structural gaps are explicitly noted with their agreed resolution
4. **Load order dependencies are shown** — the document makes clear which tables must be loaded before which

5. **Non-additive measures are flagged** — every measure whose aggregation behaviour is not straightforward must be explicitly labelled
6. **SCD type is stated** — for every dimension, the SCD handling strategy is documented
7. **Test references are included** — each column references the DQ rules and reconciliation tests that verify it

5.2 Transformation Rule Precision

The transformation rule column is where most S2T mappings fall short. Vague rules produce inconsistent implementations. Every rule must be precise enough that two developers, working independently, produce identical SSIS expressions.

Imprecise rule (insufficient):

```
SalesTerritory: Derived from province
```

Precise rule (sufficient):

```
SalesTerritory: CASE StateProvinceCode
    WHEN 'NS' THEN 'Atlantic — Nova Scotia'
    WHEN 'NB' THEN 'Atlantic — New Brunswick'
    WHEN 'PE' THEN 'Atlantic — PEI'
    WHEN 'NL' THEN 'Atlantic — Newfoundland'
    WHEN 'QC' THEN 'Quebec'
    WHEN 'ON' THEN 'Ontario'
    WHEN 'MB' THEN 'Prairies'
    WHEN 'SK' THEN 'Prairies'
    WHEN 'AB' THEN 'Alberta'
    WHEN 'BC' THEN 'British Columbia'
    ELSE 'Other'
END
Source of StateProvinceCode: Application.StateProvinces.StateProvinceCode,
joined via Application.Cities.StateProvinceID →
Application.StateProvinces.StateProvinceID
and Customers.DeliveryCityID → Application.Cities.CityID
```

Imprecise rule (insufficient):

```
UnitCost: Computed from category cost rate
```

Precise rule (sufficient):

```
UnitCost: ROUND(il.UnitPrice × pcat.StandardCostPct × ol.PickedQuantity, 2)
Where:
  il.UnitPrice = Sales.InvoiceLines.UnitPrice
  pcat.StandardCostPct = Inventory.ProductCategories.StandardCostPct
  ol.PickedQuantity = Sales.OrderLines.PickedQuantity
  pcat is joined via primary category: MIN(ProductCategoryID) per ProductID
  from Inventory.ProductCategoryAssignments
```

5.3 The S2T Mapping as a Spreadsheet

The S2T mapping is typically maintained as an Excel spreadsheet with one worksheet per target table. A complete workbook for CabotTrail would have:

Worksheet	Content
`00_Index`	Table of contents with links to each sheet; load order; SCD summary
`01_DimDate`	S2T mapping for DimDate
`02_DimGeography`	S2T mapping for DimGeography
`03_DimCustomer`	S2T mapping for DimCustomer
`04_DimProduct`	S2T mapping for DimProduct
`05_DimSupplier`	S2T mapping for DimSupplier
`06_DimEmployee`	S2T mapping for DimEmployee
`07_DimDeliveryMethod`	S2T mapping for DimDeliveryMethod
`08_DimTransactionType`	S2T mapping for DimTransactionType
`09_DimPaymentMethod`	S2T mapping for DimPaymentMethod
`10_DimProductCategory`	S2T mapping for DimProductCategory
`11_FactSales`	S2T mapping for FactSales
`12_FactPurchasing`	S2T mapping for FactPurchasing
`13_FactReturns`	S2T mapping for FactReturns
`14_FactInventory`	S2T mapping for FactInventory
`15_FactCustomerTransactions`	S2T mapping for FactCustomerTransactions
`16_FactSupplierTransactions`	S2T mapping for FactSupplierTransactions
`Gaps`	All identified gaps with type and resolution
`DQ_Rules`	Complete DQ rule catalogue

5.4 Load Order Index (Worksheet 00_Index)

The index worksheet documents the dependency chain that governs load order in the master package:

Load order	Package	Depends on	SCD type	Est. rows
1	Load_DimDate	None	Static	4,017
2	Load_DimDeliveryMethod	None	Static	12
3	Load_DimTransactionType	None	Static	varies
4	Load_DimPaymentMethod	None	Static	varies
5	Load_DimProductCategory	None	Static	13
6	Load_DimGeography	None	Type 1	varies
7	Load_DimCustomer	DimGeography	Type 1	100
8	Load_DimSupplier	DimGeography	Type 1	varies
9	Load_DimEmployee	None	Type 1	50
10	Load_DimProduct	DimProductCategory, DimSupplier	Type 1	142
11	Load_FactSales	All dims	N/A	16,359
12	Load_FactPurchasing	All dims	N/A	905
13	Load_FactReturns	All dims	N/A	500
14	Load_FactInventory	DimDate, DimProduct, DimSupplier, DimProductCategory	N/A	142
15	Load_FactCustomerTransactions	DimDate, DimCustomer, DimTransactionType, DimEmployee	N/A	9,346
16	Load_FactSupplierTransactions	DimDate, DimSupplier, DimTransactionType, DimEmployee	N/A	240

6. Documentation Standards and Formatting

Professional ETL documentation follows standards that make it consistent, readable, and maintainable. The PMI (Project Management Institute) standard is widely used in enterprise environments.

6.1 PMI Document Standards

Document header: Every document begins with a header block containing:

- Document title
- Project name
- Author(s)
- Version number
- Date created / last revised
- Status (Draft / Under Review / Approved)
- Approved by (when applicable)

Version history table: Below the header, a table recording every significant revision:

Version	Date	Author	Description of changes
0.1	2027-01-06	P. Dolinger	Initial draft — source analysis and gap analysis complete
0.2	2027-01-15	P. Dolinger	S2T mapping complete for all dimensions
0.3	2027-01-22	P. Dolinger	S2T mapping complete for all facts; DQ rules added
1.0	2027-02-12	P. Dolinger	Final version — all sections complete; approved by J. Smith

Table of contents: Auto-generated from headers in Word or Google Docs. Required for documents over 5 pages.

Consistent heading hierarchy: Use heading levels consistently — H1 for major sections, H2 for subsections, H3 for sub-subsections. Never skip a level.

Page numbers and section references: All pages numbered; section references in cross-references (e.g., "see Section 5.3").

6.2 Naming Conventions

Consistent naming makes a documentation set navigable:

Document files:

```
CabotTrail_ETL_DesignDocument_v1.0.pdf
CabotTrail_S2T_Mapping_v1.0.xlsx
CabotTrail_DataDictionary_v1.0.xlsx
CabotTrail_DW_ERD_v1.0.pdf
```

Section names in the S2T mapping: Match the database object names exactly — no abbreviations, no alternate spellings. `Fact.FactSales` in the database means the worksheet is titled `Fact.FactSales`, not "Sales Fact" or "FactSales".

Column names in documentation: Quote column names in backticks or monospace font to distinguish them from prose text: "`GrossProfitMarginPct` is a non-additive measure."

6.3 Writing for Multiple Audiences

An ETL Design Document is read by at least three distinct audiences who have different needs:

Business stakeholders (executives, data stewards): Read Section 1 (Executive Summary) and the business descriptions in the data dictionary. Write for them: avoid jargon, lead with business impact, explain decisions in business terms.

ETL developers: Read Sections 2–8 in full; use the S2T mapping as a daily reference during implementation. Write for them: be technically precise, include complete SQL expressions, document all edge cases.

BI developers and analysts: Read the data dictionary and the target system overview. Write for them: explain how to use each measure correctly, identify common misuse patterns, provide example queries.

The same fact — that `GrossProfitMarginPct` is non-additive — needs to be communicated differently to each audience:

For business stakeholders: "Total company margin must be calculated as total profit divided by total revenue — adding up individual line margins would produce an incorrect result."

For ETL developers: "`GrossProfitMarginPct DECIMAL(8,2)` — non-additive. Computed as `ROUND((LineTotal - UnitCost) / NULLIF(LineTotal, 0) * 100, 2)`. Stored for convenience; not to be aggregated. Document in data dictionary with usage warning."

For BI developers: "Do not use `SUM(GrossProfitMarginPct)` or `AVG(GrossProfitMarginPct)`. To compute company-wide margin: `SUM(GrossProfit) / SUM(LineTotal) * 100`. To compute margin by product category: `SUM(GrossProfit) / SUM(LineTotal) * 100 GROUP BY CategoryName`."

---

7. Generating Documentation from the System Catalog

The most reliable documentation is documentation that is generated from the system itself — it cannot become out of sync with reality because it is derived from reality. SQL Server's system catalog provides the raw material for several documentation artifacts.

7.1 Entity-Relationship Diagram Data

The FK relationships in the system catalog can be extracted to provide the data for an ERD. While SQL Server cannot draw the ERD itself, the relationship data drives any diagramming tool:

```
-- Extract all relationships for ERD generation
SELECT
    s_parent.name       AS ParentSchema,
    t_parent.name       AS ParentTable,
    c_parent.name       AS ParentColumn,
    s_child.name        AS ChildSchema,
    t_child.name        AS ChildTable,
    c_child.name        AS ChildColumn,
    fk.name             AS RelationshipName,
    -- Cardinality is always 1 (PK side) → Many (FK side) in normalized design
    '1'                 AS ParentCardinality,
    'M'                 AS ChildCardinality
FROM    sys.foreign_keys fk
INNER JOIN sys.foreign_key_columns fkc
    ON fkc.constraint_object_id = fk.object_id
-- Child table (FK side)
INNER JOIN sys.tables t_child   ON t_child.object_id  = fkc.parent_object_id
INNER JOIN sys.schemas s_child  ON s_child.schema_id  = t_child.schema_id
INNER JOIN sys.columns c_child  ON c_child.object_id  = fkc.parent_object_id
                               AND c_child.column_id  = fkc.parent_column_id
-- Parent table (PK side)
INNER JOIN sys.tables t_parent  ON t_parent.object_id = fkc.referenced_object_id
INNER JOIN sys.schemas s_parent ON s_parent.schema_id = t_parent.schema_id
INNER JOIN sys.columns c_parent ON c_parent.object_id = fkc.referenced_object_id
                               AND c_parent.column_id = fkc.referenced_column_id
ORDER BY s_parent.name, t_parent.name, s_child.name, t_child.name;
```

7.2 Index Inventory

Documenting existing indexes helps future developers understand the performance design decisions and avoids duplicate or conflicting index creation:

```
-- Index inventory: what indexes exist and what columns they cover
SELECT
    s.name                          AS [Schema],
    t.name                          AS [Table],
    i.name                          AS [IndexName],
    i.type_desc                     AS [IndexType],
    CASE i.is_primary_key   WHEN 1 THEN 'Yes' ELSE 'No' END AS [IsPK],
    CASE i.is_unique        WHEN 1 THEN 'Yes' ELSE 'No' END AS [IsUnique],
    STRING_AGG(c.name, ', ')
        WITHIN GROUP (ORDER BY ic.key_ordinal)  AS [IndexColumns],
    -- Include columns (non-key columns in a covering index)
    STRING_AGG(
        CASE WHEN ic.is_included_column = 1 THEN c.name ELSE NULL END, ', ')
        WITHIN GROUP (ORDER BY ic.index_column_id) AS [IncludedColumns]
FROM    sys.indexes i
INNER JOIN sys.tables t     ON t.object_id  = i.object_id
INNER JOIN sys.schemas s    ON s.schema_id  = t.schema_id
INNER JOIN sys.index_columns ic ON ic.object_id = i.object_id
                               AND ic.index_id  = i.index_id
INNER JOIN sys.columns c    ON c.object_id  = ic.object_id
                           AND c.column_id  = ic.column_id
WHERE   i.type > 0          -- Exclude heaps (tables with no clustered index)
AND     s.name IN ('Dimension', 'Fact', 'ETL')
GROUP BY s.name, t.name, i.name, i.type_desc, i.is_primary_key, i.is_unique
ORDER BY s.name, t.name, i.name;
```

7.3 Row Count and Freshness Report

A living documentation artifact — generated on demand to show the current state of every DW table:

```
-- DW state snapshot: row counts and last load time
SELECT
    s.name                  AS [Schema],
    t.name                  AS [Table],
    p.rows                  AS [RowCount],
    -- Last update from ETL.TestResults
    ISNULL(
        CAST(MAX(tr.RunDate) AS NVARCHAR),
        'Never loaded'
    )                       AS [LastLoadTime],
    ISNULL(
        CAST(DATEDIFF(HOUR, MAX(tr.RunDate), GETDATE()) AS NVARCHAR) + ' hours ago',
        'N/A'
    )                       AS [DataAge]
FROM    sys.tables t
INNER JOIN sys.schemas s    ON s.schema_id = t.schema_id
INNER JOIN sys.partitions p ON p.object_id = t.object_id
                           AND p.index_id IN (0, 1)
LEFT JOIN CabotTrailOutdoorDW.ETL.TestResults tr
    ON tr.TargetTable = s.name + '.' + t.name
    AND tr.TestCategory = 'RowCount'
    AND tr.Passed = 1
WHERE   s.name IN ('Dimension', 'Fact')
GROUP BY s.name, t.name, p.rows
ORDER BY s.name, t.name;
```

---

8. Keeping Documentation Current

Documentation that is accurate on the day it is written but never updated becomes more dangerous than no documentation at all — a developer who trusts stale documentation makes decisions based on incorrect information.

8.1 The Documentation Change Process

Every change to the ETL system must be accompanied by a corresponding change to the documentation. This is a process requirement, not a good-intention aspiration. The process:

1. **Change request received** (new column, changed business rule, new source table)
2. **S2T mapping updated first** — the specification changes before the code
3. **Code change implemented** from the updated specification
4. **Tests updated** to reflect the new expected behaviour
5. **Data dictionary updated** with new column descriptions or changed descriptions
6. **Process flow diagrams updated** if the pipeline structure changed

7. **Version number incremented** in the document header
8. **Version history updated** with a description of the change

Steps 1–5 must happen for every change. Steps 6–7 are required for structural changes (new packages, new dependencies). Steps 8 is always required.

8.2 Documentation Reviews

Schedule periodic documentation reviews — at minimum once per quarter — to verify that documentation matches reality:

```
-- Generate a comparison: documented row counts vs actual row counts
-- (Documentation claims vs system reality)
-- This query assumes documented expected counts are stored in ETL.TestResults

SELECT
    tr.TargetTable,
    CAST(tr.ExpectedValue AS INT)                  AS DocumentedCount,
    p.rows                                         AS ActualCount,
    CAST(tr.ExpectedValue AS INT) - p.rows         AS Discrepancy
FROM (
    SELECT TargetTable, ExpectedValue,
           ROW_NUMBER() OVER (PARTITION BY TargetTable ORDER BY RunDate DESC) AS rn
    FROM   CabotTrailOutdoorDW.ETL.TestResults
    WHERE  TestCategory = 'RowCount' AND Passed = 1
) tr
INNER JOIN sys.tables t     ON t.name = PARSENAME(tr.TargetTable, 1)
INNER JOIN sys.schemas s    ON s.schema_id = t.schema_id
                           AND s.name = PARSENAME(tr.TargetTable, 2)
INNER JOIN sys.partitions p ON p.object_id = t.object_id
                           AND p.index_id IN (0, 1)
WHERE tr.rn = 1
ORDER BY ABS(CAST(tr.ExpectedValue AS INT) - p.rows) DESC;
```

8.3 Documentation as Code

The most maintainable documentation is documentation that lives alongside the code, in the same version control system, and changes are tracked together. **Docs-as-code** approaches maintain documentation in Markdown or similar text formats in a Git repository:

```
etl-cabot-trail/
├── docs/
│   ├── design-document.md          ← ETL Design Document in Markdown
│   ├── data-dictionary.md          ← Data Dictionary
│   ├── s2t-mapping/
│   │   ├── dim-customer.md         ← One file per mapping
│   │   ├── dim-product.md
│   │   └── fact-sales.md
│   └── process-flows/
│       ├── master-package-flow.md  ← Process flows in Markdown/Mermaid
│       └── fact-sales-flow.md
├── packages/
│   ├── Load_DimCustomer.dtsx
│   └── Load_FactSales.dtsx
└── sql/
    ├── ddl/
    └── reconciliation/
```

GitHub and GitLab render Markdown natively, making this approach particularly effective when the repository is hosted there. Process flow diagrams can be maintained as **Mermaid diagrams** — a text-based diagramming syntax that renders in GitHub Markdown:

````
```mermaid
graph TD
    A([Start]) --> B[Truncate DimProduct]
    B --> C[Extract Products]
    C --> D{DQ Check}
    D -- Pass --> E[Lookup: CategoryKey]
    D -- Fail --> F[Write to LoadErrors]
    E -- Match --> G[Lookup: SupplierKey]
    E -- No Match --> F
    G -- Match --> H[Write to DimProduct]
    G -- No Match --> F
    H --> I[Reconciliation Test]
    I --> J{Passed?}
    J -- Yes --> K([End — Success])
    J -- No --> L[Send Alert]
    L --> M([End — Failure])
```
````

```
This kind of diagram renders directly in GitHub as a visual flowchart, making the
repository itself a navigable documentation portal.

---

## 9. Chapter Summary

- **ETL documentation** is a professional discipline with compounding returns — it makes
systems faster to debug, safer to change, easier to hand off, and more likely to be
trusted. Its absence imposes the **undocumented ETL tax** on every subsequent activity.

- The **ETL Design Document** has eight sections covering the executive summary, source
analysis, target overview, gap analysis, S2T mapping, data quality rules, process flows,
and testing strategy. It is written before implementation begins and maintained
throughout the system's life.

- The **data dictionary** records every column in the DW and data marts with technical
definition, source, business description, usage notes, and DQ rule references. Business
descriptions cannot be auto-generated — they require business stakeholder input. Extended
properties store descriptions inside the database.

- **Process flow diagrams** use standard notation (oval=start/end, rectangle=process,
diamond=decision, cylinder=data store) to visualize ETL pipelines. Every decision must
have two labeled exit paths. Error paths are as important as success paths.

- The **S2T mapping** is complete only when every target column is documented with an
unambiguous transformation rule. Vague rules produce inconsistent implementations.

- **PMI documentation standards** — version history, document headers, table of contents,
consistent heading hierarchy — make documents professional, navigable, and trustworthy.

- The **system catalog** provides auto-generated technical metadata (column types, FK
relationships, indexes) that can be extracted with SQL queries. This portion of the data
dictionary cannot become stale. Business descriptions require human input and must be
maintained manually.

- **Keeping documentation current** requires a formal change process: specification
changes before code, followed by code, tests, and documentation updates in sequence.
Docs-as-code in a Git repository is the most maintainable approach.

---

## 10. Review Questions

1. Explain the "undocumented ETL tax" using a specific scenario from the CabotTrail
environment. Choose one of the four cost categories (debugging, change management,
knowledge transfer, audit/compliance) and describe concretely what would be required
without documentation.

2. A colleague argues that "the code is the documentation" — the SSIS packages are self-
documenting because you can read the component configuration to understand what they do.
Identify two specific situations where this argument fails, using examples from the
CabotTrail ETL.
```

3. Write a complete data dictionary entry for `Fact.FactSales.DaysUntilInvoice` — the computed column that measures days between order date and invoice date. Include all fields from the data dictionary template (data type, source, business description, usage notes, DQ rules).

4. Draw a process flow diagram for `Load_DimCustomer.dtsx` using the standard notation described in this chapter. Include: the extract from three source tables, a DQ check for NULL CustomerID, the write to DimCustomer, and a reconciliation test with success and failure paths.

5. The S2T mapping for `FactSales.UnitCost` has the transformation rule: "Derived from category cost." A developer implements this as `UnitPrice * 0.55`. Why is the mapping's rule insufficient? Write a precise rule that would prevent this incorrect implementation.

6. Explain why it is important that the S2T mapping be updated *before* the SSIS package is modified when a business rule changes. What risk arises when code is changed first and documentation is updated afterward?

7. The data dictionary lists `Fact.FactInventory.QuantityOnHand` as semi-additive. A Power BI developer creates a report that shows total `QuantityOnHand` summed over a date range (January through March 2024). Write the explanation you would include in the data dictionary usage notes to prevent this mistake, and describe the correct approach.

8. Describe the "docs-as-code" approach to ETL documentation. What specific advantages does storing documentation in a Git repository alongside the SSIS packages provide, compared to storing the documentation in a shared network folder?

Deeper Dive

Going Further with ETL Documentation

The TOGAF Architecture Documentation Framework

TOGAF (The Open Group Architecture Framework) is an enterprise architecture methodology that includes formal standards for documenting architecture components, including data architecture. TOGAF's Architecture Description concepts — viewpoints, views, and stakeholders — provide a framework for producing documentation that addresses multiple audiences systematically.

For ETL and data warehouse projects, TOGAF's **Data Architecture** domain is directly relevant. It defines:
- The **Data Entity / Data Component Catalog** (equivalent to the data dictionary)
- The **Data Architecture Diagram** (equivalent to the ERD and process flow)
- The **Data Lifecycle Diagram** (documents how data moves through the pipeline)
- The **Data Lineage Diagram** (traces data from source to final consumption)

While full TOGAF adoption is primarily relevant to large enterprise environments, the documentation structures it defines are valuable patterns for any BI project:
[The Open Group — TOGAF Standard](https://www.opengroup.org/togaf)

```
#### Mermaid Diagrams for Process Flows in GitHub

**Mermaid** is a text-based diagramming language that renders directly in GitHub Markdown, making it ideal for docs-as-code documentation of ETL pipelines. GitHub has supported Mermaid natively since 2022.

The full syntax reference covers flowcharts, sequence diagrams, entity-relationship diagrams, Gantt charts, and more:
[Mermaid Documentation](https://mermaid.js.org/intro/)

For ETL process flows, the `flowchart` and `graph` diagram types are most useful. For data model documentation, the `erDiagram` type renders entity-relationship diagrams directly in Markdown:
```

```
erDiagram
    DimCustomer {
        int CustomerKey PK
        int CustomerID
        nvarchar CustomerName
        nvarchar SalesTerritory
    }
    FactSales {
        int SalesKey PK
        int CustomerKey FK
        int ProductKey FK
        decimal LineTotal
    }
    DimProduct {
        int ProductKey PK
        int ProductID
        nvarchar ProductName
        nvarchar CategoryName
    }
    FactSales ||--o{ DimCustomer : "CustomerKey"
    FactSales ||--o{ DimProduct  : "ProductKey"
```

````
This produces a rendered ERD directly in the GitHub README, making the repository its own documentation portal.

#### The DAMA International Data Dictionary Standard

DAMA International's *DMBOK* defines standards for data dictionaries at enterprise scale, including:
- Metadata repository architecture (centralized vs federated)
- Metadata exchange standards (the Common Warehouse Metamodel, OMG XMI)
- Governance of the metadata repository (who can add, update, archive entries)
- Integration with data catalog tools

For practitioners moving beyond spreadsheet-based data dictionaries, DAMA's guidance on metadata repository design provides a structured path:
DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications. — Chapter 12 covers metadata management.

#### Modern Data Catalog Tools

Enterprise-scale data catalogues automate much of what this chapter documents manually. They crawl connected data sources, automatically extract technical metadata, track lineage, and provide a searchable interface for business users and developers:

**Microsoft Purview:** Microsoft's unified data governance platform. Scans Azure and SQL Server sources, extracts metadata automatically, tracks lineage across SSIS packages and Azure Data Factory pipelines, and provides a business-facing data catalog:
[Microsoft Purview — Data Catalog](https://learn.microsoft.com/en-us/purview/purview)

**Apache Atlas:** Open-source metadata management and governance platform. Widely used in Hadoop/Spark environments:
[Apache Atlas](https://atlas.apache.org/)

**Alation / Collibra:** Commercial data catalog platforms widely used in enterprise environments. Provide automated metadata harvesting, business glossary management, and data quality integration.

The principles in this chapter — data dictionary, lineage documentation, business descriptions — are exactly what these tools automate at scale. Understanding the manual process makes the automated tools more valuable, not redundant.

#### SQL Server Extended Properties in Depth

Extended properties in SQL Server support a richer metadata model than just column descriptions. They can be used to store:
- Column sensitivity classifications (for data privacy compliance)
- Display format hints for BI tools
- Custom tags for documentation generators
- Business rule references

```sql
-- Store multiple extended properties on a column
-- Sensitivity classification
EXEC sys.sp_addextendedproperty
````
````

```
    @name = N'DataSensitivity', @value = N'PII — Customer Name',
    @level0type = N'SCHEMA', @level0name = N'Dimension',
    @level1type = N'TABLE',  @level1name = N'DimCustomer',
    @level2type = N'COLUMN', @level2name = N'CustomerName';

-- BI tool display format hint
EXEC sys.sp_addextendedproperty
    @name = N'DisplayFormat', @value = N'$#,##0.00',
    @level0type = N'SCHEMA', @level0name = N'Fact',
    @level1type = N'TABLE',  @level1name = N'FactSales',
    @level2type = N'COLUMN', @level2name = N'LineTotal';

-- Additive behaviour flag (custom property)
EXEC sys.sp_addextendedproperty
    @name = N'Additivity', @value = N'NON-ADDITIVE — compute from GrossProfit/LineTotal',
    @level0type = N'SCHEMA', @level0name = N'Fact',
    @level1type = N'TABLE',  @level1name = N'FactSales',
    @level2type = N'COLUMN', @level2name = N'GrossProfitMarginPct';
```

Microsoft documentation:
Extended Properties — SQL Server

Industry Perspectives

Kimball on ETL Documentation

Kimball's *The Data Warehouse ETL Toolkit* addresses documentation pragmatically — acknowledging that documentation is often skipped in practice while making the case that it is professionally non-negotiable:

> *"Every ETL system should be fully documented. This is not optional. An undocumented ETL system is a liability — it cannot be maintained, audited, or extended reliably. The documentation set consists of the source-to-target mapping, the data dictionary, and the process flow diagrams. These three documents are the minimum for a professionally delivered system."*

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapter 13 covers ETL documentation standards.

References and Further Reading

1. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapter 13 covers ETL documentation standards and deliverables.

2. DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications. — Chapter 12 on metadata management provides the enterprise-scale context for data dictionary practices.
3. Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (7th ed.). PMI. — Documentation standards and deliverable management practices.
4. The Open Group. (2018). *TOGAF Standard, Version 9.2*. The Open Group. — Enterprise architecture documentation framework including data architecture standards. https://www.opengroup.org/togaf
5. Microsoft. (2024). *sp_addextendedproperty — SQL Server*. https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-addextendedproperty-transact-sql
6. Microsoft. (2024). *Microsoft Purview — Data Catalog*. https://learn.microsoft.com/en-us/purview/purview
7. Mermaid. (2024). *Mermaid Diagramming and Charting Tool*. https://mermaid.js.org/intro/
8. Apache Software Foundation. (2024). *Apache Atlas*. https://atlas.apache.org/
9. Loshin, D. (2010). *The Practitioner's Guide to Data Quality Improvement*. Morgan Kaufmann. — Chapter 8 covers data quality documentation and metadata management.
10. Brackett, M. H. (2012). *Data Resource Design: Reality Beyond Illusion*. Technics Publications. — A deep treatment of data resource documentation and its organizational value.

Chapter 7: Advanced ETL — MERGE, Slowly Changing Dimensions, and Multiple Sources

> ***ETL for Business Intelligence***
>
> *A practical guide to data provisioning, dimensional modelling, and pipeline design*
>
>

Chapter Overview

Chapters 3 through 6 built a complete, documented ETL pipeline using the full-reload pattern: truncate the target and reload from scratch on every run. This approach is simple, reliable, and entirely correct for the CabotTrail environment where data volumes are modest and the load window is sufficient.

Most production ETL systems eventually outgrow the full-reload pattern — either because data volumes make full reloads too slow, because source data changes frequently and incremental processing is more efficient, or because the business requires historical dimension tracking that full reloads cannot provide.

This chapter introduces the advanced techniques that address these needs: the T-SQL `MERGE` statement for upsert operations, Slowly Changing Dimension (SCD) implementations in depth, and patterns for integrating data from multiple heterogeneous sources. These are the techniques that distinguish a junior ETL developer from a senior one.

By the end of this chapter you will be able to:

- Write correct, production-quality T-SQL `MERGE` statements for ETL upsert operations
- Explain and implement SCD Types 1, 2, 3, and the Type 6 hybrid
- Choose the appropriate SCD type for a given business requirement
- Design and implement an incremental ETL strategy using watermark timestamps
- Handle data from multiple source systems with conflicting schemas and overlapping keys
- Use the SSIS `MERGE` JOIN and `UNION ALL` transformations for multi-source ETL
- Understand the SSIS SCD Wizard and its limitations

Table of Contents

1. Beyond Full Reload: When Incremental ETL Is Needed

The full-reload pattern — truncate and reload — has a clean simplicity that makes it the right default for many ETL systems. But it has two fundamental limitations that emerge as systems grow.

1.1 The Scalability Limit of Full Reload

Full reload costs are directly proportional to data volume. For CabotTrail with 16,359 fact rows, a full reload completes in seconds. For an enterprise system with 500 million fact rows, a full reload may take 8–12 hours — longer than the available nightly batch window.

The scalability break-even point varies by hardware and query complexity, but a practical rule of thumb: **full reload is appropriate when the load completes within 20% of the available batch window.** This leaves capacity for re-runs on failure and for the window to grow as data accumulates over time.

1.2 The History Preservation Limit

Full reload has a more fundamental limitation that has nothing to do with volume: it cannot preserve dimensional history. When a customer moves from Nova Scotia to Ontario, a full reload of `DimCustomer`

reflects the new province — and all historical sales rows now appear to be from an Ontario customer, regardless of when they were placed.

SCD Type 2 requires that the ETL detect changes and handle them differently from inserts — which requires incremental processing, not full reload.

1.3 Choosing Between Full Reload and Incremental

The choice is not absolute. Many production systems use full reload for small, stable dimensions (lookup tables like `DimDeliveryMethod`) and incremental processing for large, frequently changing tables (transaction facts, SCD Type 2 dimensions).

Scenario	Recommended approach
Small dimension, changes infrequent	Full reload — simplicity outweighs overhead
Large fact table, new rows only (append-only)	Incremental insert — filter by `LastModifiedDate > watermark`
Dimension with tracked history	Incremental upsert — MERGE with SCD Type 2 logic
Large fact table with corrections	Incremental upsert — MERGE on natural key
Multiple sources with conflicting records	Staging → MERGE with conflict resolution

---

2. The T-SQL MERGE Statement

The `MERGE` statement (introduced in SQL Server 2008) combines `INSERT`, `UPDATE`, and `DELETE` into a single atomic operation. It compares a **source dataset** to a **target table** row by row, applying different actions depending on whether a match is found.

2.1 MERGE Syntax

```
MERGE INTO [target_table] AS tgt
USING [source_query_or_table] AS src
ON ([join_condition])

WHEN MATCHED [AND additional_condition] THEN
    [UPDATE SET ... | DELETE]

WHEN NOT MATCHED [BY TARGET] THEN
    INSERT ([columns]) VALUES ([values])

WHEN NOT MATCHED BY SOURCE [AND additional_condition] THEN
    [UPDATE SET ... | DELETE]

[OUTPUT $action, inserted.*, deleted.*];
```

Key clauses:

Clause	Meaning	Typical ETL use
`WHEN MATCHED`	A source row matches a target row	Update changed attributes
`WHEN MATCHED AND (condition)`	Match found AND condition is TRUE	Update only when specific columns changed
`WHEN NOT MATCHED BY TARGET`	Source row has no match in target	Insert new row
`WHEN NOT MATCHED BY SOURCE`	Target row has no match in source	Handle deleted source records (use with caution in DW)
`OUTPUT`	Returns information about each row affected	Audit logging; capturing generated identity values

2.2 A Minimal MERGE Example

Before applying MERGE to dimensional ETL, a simple example illustrates the mechanics:

```
-- Source: current data from the OLTP
-- Target: DimEmployee in the DW
-- Action: Update job titles if changed; insert new employees

MERGE INTO CabotTrailOutdoorDW.Dimension.DimEmployee AS tgt
USING
(
    SELECT
        e.EmployeeID,
        p.FullName,
        p.PreferredName,
        e.JobTitle,
        e.Department,
        p.EmailAddress,
        e.HireDate,
        e.IsActive
    FROM CabotTrailOutdoor.Application.Employees e
    INNER JOIN CabotTrailOutdoor.Application.People p
        ON p.PersonID = e.PersonID
    WHERE e.EmployeeID <> 0
) AS src
ON (tgt.EmployeeID = src.EmployeeID)

-- Update only when something has actually changed
WHEN MATCHED AND (
    tgt.FullName    <> src.FullName     OR
    tgt.JobTitle    <> src.JobTitle     OR
    tgt.Department  <> src.Department   OR
    tgt.IsActive    <> src.IsActive
) THEN
    UPDATE SET
        tgt.FullName    = src.FullName,
        tgt.PreferredName = src.PreferredName,
        tgt.JobTitle    = src.JobTitle,
        tgt.Department  = src.Department,
        tgt.EmailAddress = src.EmailAddress,
        tgt.IsActive    = src.IsActive

-- Insert employees that don't exist yet
WHEN NOT MATCHED BY TARGET THEN
    INSERT (EmployeeID, FullName, PreferredName, JobTitle,
            Department, EmailAddress, HireDate, IsActive)
    VALUES (src.EmployeeID, src.FullName, src.PreferredName, src.JobTitle,
            src.Department, src.EmailAddress, src.HireDate, src.IsActive);
```

2.3 The Change Detection Condition

The `WHEN MATCHED AND (...)` condition is the most important part of an ETL MERGE. Without it, every matched row is updated on every run — correct data, but:

- Unnecessary writes degrade performance

- `LastModifiedDate` or audit timestamp columns are updated without meaningful changes
- In SCD Type 2 implementations, spurious "changes" trigger new dimension rows

The change detection condition uses OR across all tracked columns — if *any* tracked column has changed, the row is updated:

```
-- Change detection: update if ANY tracked column differs
WHEN MATCHED AND (
    tgt.CustomerName    <> src.CustomerName     OR
    tgt.CreditLimit     <> src.CreditLimit      OR
    tgt.ProvinceCode    <> src.ProvinceCode
) THEN UPDATE SET ...
```

NULL handling in change detection: Standard comparison operators (`<>`) return NULL (not TRUE or FALSE) when either side is NULL. A column that was NULL and is now populated would not be detected as changed. Use `ISNULL()` or `COALESCE()` to handle NULLable columns:

```
-- NULL-safe change detection
WHEN MATCHED AND (
    tgt.CustomerName        <> src.CustomerName                         OR
    tgt.CreditLimit         <> src.CreditLimit                          OR
    -- NULL-safe comparison for nullable column
    ISNULL(tgt.AccountOpenedDate, '1900-01-01')
        <> ISNULL(src.AccountOpenedDate, '1900-01-01')
) THEN UPDATE SET ...
```

2.4 The OUTPUT Clause

The `OUTPUT` clause captures information about every row affected by the MERGE, returning it as a result set. In ETL, it is used to:

- Log which rows were inserted, updated, or deleted
- Capture generated surrogate keys (`inserted.CustomerKey`) for use in subsequent operations
- Produce audit records without a separate query

```
-- MERGE with OUTPUT for audit logging
DECLARE @AuditLog TABLE (
    Action          NVARCHAR(10),
    EmployeeID      INT,
    OldJobTitle     NVARCHAR(100),
    NewJobTitle     NVARCHAR(100),
    ChangeTime      DATETIME2 DEFAULT SYSDATETIME()
);

MERGE INTO Dimension.DimEmployee AS tgt
USING (...) AS src
ON (tgt.EmployeeID = src.EmployeeID)
WHEN MATCHED AND tgt.JobTitle <> src.JobTitle THEN
    UPDATE SET tgt.JobTitle = src.JobTitle
WHEN NOT MATCHED BY TARGET THEN
    INSERT (EmployeeID, FullName, JobTitle, ...)
    VALUES (src.EmployeeID, src.FullName, src.JobTitle, ...)
OUTPUT
    $action                 AS Action,          -- 'INSERT' or 'UPDATE'
    COALESCE(deleted.EmployeeID, inserted.EmployeeID) AS EmployeeID,
    deleted.JobTitle        AS OldJobTitle,
    inserted.JobTitle       AS NewJobTitle
INTO @AuditLog (Action, EmployeeID, OldJobTitle, NewJobTitle);

-- Review the audit log
SELECT * FROM @AuditLog;
```

3. MERGE Patterns for ETL

Different ETL scenarios require different MERGE configurations. This section catalogs the most common patterns.

3.1 Pattern 1: Upsert (Insert or Update)

The most common ETL MERGE pattern: insert new rows, update changed rows, leave deleted rows untouched.

```
-- Pattern: Upsert DimCustomer
-- No WHEN NOT MATCHED BY SOURCE clause — do not delete dimension rows
-- even if source customers are no longer active

MERGE INTO Dimension.DimCustomer AS tgt
USING
(
    SELECT
        c.CustomerID,
        c.CustomerName,
        c.CustomerGroupName         AS CustomerCategoryName,
        c.CreditLimit,
        c.AccountOpenedDate,
        ci.CityName,
        sp.StateProvinceName        AS ProvinceName,
        sp.StateProvinceCode        AS ProvinceCode,
        co.CountryName
    FROM CabotTrailOutdoor.Sales.Customers c
    INNER JOIN CabotTrailOutdoor.Application.Cities ci
        ON ci.CityID = c.DeliveryCityID
    INNER JOIN CabotTrailOutdoor.Application.StateProvinces sp
        ON sp.StateProvinceID = ci.StateProvinceID
    INNER JOIN CabotTrailOutdoor.Application.Countries co
        ON co.CountryID = sp.CountryID
    WHERE c.CustomerID <> 0
) AS src
ON (tgt.CustomerID = src.CustomerID)

WHEN MATCHED AND (
    tgt.CustomerName        <> src.CustomerName         OR
    ISNULL(tgt.CreditLimit, 0) <> ISNULL(src.CreditLimit, 0)
) THEN
    UPDATE SET
        tgt.CustomerName        = src.CustomerName,
        tgt.CustomerCategoryName = src.CustomerCategoryName,
        tgt.CreditLimit         = src.CreditLimit

WHEN NOT MATCHED BY TARGET THEN
    INSERT (CustomerID, CustomerName, CustomerCategoryName,
            CreditLimit, AccountOpenedDate,
            CityName, ProvinceName, ProvinceCode, CountryName)
    VALUES (src.CustomerID, src.CustomerName, src.CustomerCategoryName,
            src.CreditLimit, src.AccountOpenedDate,
            src.CityName, src.ProvinceName, src.ProvinceCode, src.CountryName);
```

3.2 Pattern 2: Insert-Only (Append)

For fact tables that only ever grow (new transactions are added; existing transactions are never modified), the MERGE simplifies to an insert-only pattern. A LEFT JOIN detects new rows more efficiently than a full MERGE for large datasets:

```
-- Pattern: Insert-only for FactSales (new invoice lines since last load)
-- Assumes watermark-based filtering (covered in section 7)

INSERT INTO Fact.FactSales
(
    OrderDateKey, InvoiceDateKey, DueDateKey,
    CustomerKey, ProductKey, EmployeeKey, GeographyKey, DeliveryMethodKey,
    InvoiceID, OrderID, OrderLineID,
    OrderedQuantity, PickedQuantity, UnitPrice, TaxRate,
    LineTotal, TaxAmount, LineTotalIncludingTax,
    UnitCost, GrossProfit, GrossProfitMarginPct
)
SELECT
    [... extract query ...]
FROM [source tables]
-- Only insert rows not already in the fact table
WHERE NOT EXISTS (
    SELECT 1
    FROM Fact.FactSales existing
    WHERE existing.InvoiceID   = [source].InvoiceID
    AND   existing.OrderLineID = [source].OrderLineID
);
```

3.3 Pattern 3: Full Replace for Small Dimensions

For small, stable lookup dimensions (DeliveryMethod, TransactionType), the overhead of MERGE change detection is not worth the added complexity. Full reload (TRUNCATE + INSERT) remains appropriate:

```
-- Pattern: Full replace for small static dimensions
TRUNCATE TABLE Dimension.DimDeliveryMethod;

INSERT INTO Dimension.DimDeliveryMethod (DeliveryMethodID, DeliveryMethodName)
SELECT DeliveryMethodID, DeliveryMethodName
FROM CabotTrailOutdoor.Application.DeliveryMethods
WHERE DeliveryMethodID <> 0;
```

When to use full replace vs MERGE:

- Full replace: table has < 10,000 rows AND changes are rare AND no downstream dependencies require stable surrogate keys
- MERGE: any table where you need to detect and respond to individual row changes

3.4 Pattern 4: Merge with Soft Delete

When source records are logically deleted (marked inactive rather than physically removed), the `WHEN NOT MATCHED BY SOURCE` clause can update the DW dimension to reflect the inactive state:

```
-- Pattern: Soft delete — mark inactive in DW when not in active source
MERGE INTO Dimension.DimEmployee AS tgt
USING
(
    -- Source: only active employees
    SELECT e.EmployeeID, p.FullName, e.JobTitle, e.IsActive
    FROM Application.Employees e
    INNER JOIN Application.People p ON p.PersonID = e.PersonID
    WHERE e.EmployeeID <> 0
) AS src
ON (tgt.EmployeeID = src.EmployeeID)

WHEN MATCHED AND tgt.IsActive <> src.IsActive THEN
    UPDATE SET tgt.IsActive = src.IsActive

WHEN NOT MATCHED BY TARGET THEN
    INSERT (EmployeeID, FullName, JobTitle, IsActive)
    VALUES (src.EmployeeID, src.FullName, src.JobTitle, src.IsActive)

-- Employee exists in DW but not in active source: mark inactive
WHEN NOT MATCHED BY SOURCE AND tgt.EmployeeID <> 0 THEN
    UPDATE SET tgt.IsActive = 0;
```

> ***Warning on `NOT MATCHED BY SOURCE`:*** *This clause updates (or deletes) DW rows that have no corresponding source row. In dimensional modelling, physically deleting dimension rows that have historical fact row references would break referential integrity. Use `NOT MATCHED BY SOURCE` only for soft updates (marking `IsActive = 0`) — never for hard deletes on dimensions that have existing fact references.*

---

4. Slowly Changing Dimensions in Depth

Chapter 2 introduced SCD types conceptually. This chapter implements them. The decision of which SCD type to apply to each dimension attribute is one of the most consequential ETL design decisions — it determines what historical questions the DW can and cannot answer.

4.1 The Business Question Test

The correct SCD type for an attribute is determined by the business question it enables. Ask: *"Does a historical report need to show the value at the time of the event, or the value as it is today?"*

Attribute: Customer Province

- Historical question: "What was total sales revenue by province in 2022?" — needs the province at the time

of each 2022 sale
- Answer requires: SCD Type 2 — preserve historical province values

Attribute: Customer Credit Limit
- Historical question: "What was the credit limit for this customer when they placed this order?" — useful for credit analysis
- But: the business may accept "show current credit limit" as sufficient for all reports
- Answer requires: business decision. If historical accuracy matters: Type 2. If current state is sufficient: Type 1.

Attribute: Employee Job Title
- Historical question: "Which job title was the sales rep when they made this sale?" — relevant for commission calculations
- Answer requires: SCD Type 2

Attribute: Product Name
- Historical question: "Did the product name ever change?"
- Usually: a name correction is a Type 1 fix. A rebrand might warrant Type 2.
- Answer requires: business decision based on frequency and significance of changes.

4.2 SCD Type 0: Fixed (Retain Original)

SCD Type 0 is the opposite of Type 1 — the dimension value never changes once loaded. It is appropriate for attributes that represent the state at the time of first occurrence and should never be overwritten:

- `AccountOpenedDate` — the date a customer account was first created; this never changes
- `HireDate` — an employee's original hire date; even if re-hired, the original date is preserved separately

In practice, SCD Type 0 requires no special ETL logic — simply exclude the column from the `WHEN MATCHED THEN UPDATE SET` list and it will never be overwritten.

4.3 SCD Type 1: Overwrite

Type 1 is the simplest — update in place, no history preserved. The `WHEN MATCHED AND (change_detected) THEN UPDATE` pattern from section 3.1 implements it.

CabotTrail dimensions with Type 1 attributes:
- All dimensions in the current CabotTrail DW are Type 1. This is a deliberate design decision — the business has not yet identified a need for historical dimension tracking. When that need arises, specific attributes can be upgraded to Type 2.

When Type 1 is acceptable:

- Corrections (fixing a typo in a customer name — the wrong name was never "correct")
- Attributes whose historical value adds no analytical value
- Attributes that change so rarely that the lack of history is not a practical concern

4.4 SCD Type 2: Add New Row (Full History)

Type 2 is the workhorse of dimensional history tracking. Every change to a tracked attribute produces a new dimension row, preserving the old value for historical fact analysis.

The Type 2 row signature:

```
CustomerKey | CustomerID | ProvinceCode | ValidFrom   | ValidTo     | IsCurrent
──────────────────────────────────────────────────────────────────────────────
15          | 42         | NS           | 2022-01-01  | 2024-06-30  | 0
87          | 42         | ON           | 2024-07-01  | 9999-12-31  | 1
```

- `CustomerKey = 15` is the surrogate key for the historical NS row — fact rows from 2022–2024 reference this key
- `CustomerKey = 87` is the surrogate key for the current ON row — future fact rows reference this key
- `ValidFrom` / `ValidTo` define the date range during which each row was "current"
- `IsCurrent = 1` identifies the active row for each CustomerID

Querying Type 2 dimensions:

```
-- Current customer province (IsCurrent filter)
SELECT CustomerID, CustomerName, ProvinceCode
FROM Dimension.DimCustomer
WHERE IsCurrent = 1
AND   CustomerID <> 0
ORDER BY CustomerName;

-- Historical province: what province was this customer in at the time of each sale?
-- (No IsCurrent filter needed — fact rows already reference the correct version)
SELECT
    fs.InvoiceID,
    fs.OrderDate,
    dc.CustomerName,
    dc.ProvinceCode     AS ProvinceAtTimeOfSale,
    fs.LineTotal
FROM Fact.FactSales fs
INNER JOIN Dimension.DimCustomer dc ON dc.CustomerKey = fs.CustomerKey
WHERE dc.CustomerID = 42
ORDER BY fs.OrderDate;
-- Each fact row joins to the dimension version that was current when it was loaded
-- — historical ProvinceCode is preserved correctly
```

4.5 SCD Type 3: Add Column (Limited History)

Type 3 adds a `PreviousValue` column to store the immediately prior value alongside the current value:

```
CustomerKey | CustomerID | ProvinceCode | PreviousProvinceCode | ChangedDate
──────────────────────────────────────────────────────────────────────────────
15          | 42         | ON           | NS                   | 2024-07-01
```

Limitations of Type 3:

- Only one prior value is preserved — a second change overwrites `PreviousProvinceCode`
- Fact rows cannot be correctly attributed to historical values — there is only one dimension row per entity, so joining a fact always returns the current (or previous) value, not the value at the time of the event

When Type 3 is useful:

- When only the "before and after the most recent change" is needed
- For slowly changing attributes where a single transition is the maximum expected
- As a lightweight alternative when Type 2's additional rows are not justified

---

5. Implementing SCD Type 2

SCD Type 2 requires the most sophisticated ETL logic of all SCD types. The implementation involves detecting changes, expiring old rows, and inserting new ones — all in a single, atomic operation.

5.1 The Type 2 Implementation Steps

For each source row arriving in the ETL pipeline:

1. **Look up the current DW row** for this natural key (where `IsCurrent = 1`)
2. **Compare tracked attributes**: has any Type 2 column changed?
3. **If changed:**
   a. Expire the old row: set `ValidTo = today - 1 day` and `IsCurrent = 0`
   b. Insert new row: all current attributes, `ValidFrom = today`, `ValidTo = '9999-12-31'`, `IsCurrent = 1`, new surrogate key generated by IDENTITY
4. **If not changed but row exists:** optionally update Type 1 attributes (non-tracked changes that overwrite in place)
5. **If no current row exists:** insert as a new Type 2 row (first appearance)

5.2 T-SQL Implementation: Type 2 for DimCustomer

The cleanest T-SQL approach for Type 2 uses two separate operations: a MERGE that handles Type 1 updates and detects Type 2 changes, followed by an INSERT for the new Type 2 rows.

```
-- Step 1: Expire current rows where Type 2 attributes have changed
-- Sets IsCurrent = 0 and ValidTo = today for rows about to be superseded

UPDATE tgt
SET
    tgt.ValidTo     = CAST(GETDATE() AS DATE),
    tgt.IsCurrent   = 0
FROM Dimension.DimCustomer tgt
INNER JOIN
(
    -- Source: current OLTP customer data
    SELECT
        c.CustomerID,
        sp.StateProvinceCode    AS ProvinceCode,
        ci.CityName,
        sp.StateProvinceName    AS ProvinceName
    FROM CabotTrailOutdoor.Sales.Customers c
    INNER JOIN CabotTrailOutdoor.Application.Cities ci
        ON ci.CityID = c.DeliveryCityID
    INNER JOIN CabotTrailOutdoor.Application.StateProvinces sp
        ON sp.StateProvinceID = ci.StateProvinceID
    WHERE c.CustomerID <> 0
) src ON src.CustomerID = tgt.CustomerID
-- Only target current rows where Type 2 attributes have changed
WHERE   tgt.IsCurrent = 1
AND (
    tgt.ProvinceCode    <> src.ProvinceCode     OR
    tgt.CityName        <> src.CityName
);
-- Returns: number of rows expired
```

```
-- Step 2: Insert new current rows for changed and new customers

INSERT INTO Dimension.DimCustomer
(
    CustomerID, CustomerName, CustomerCategoryName,
    CreditLimit, AccountOpenedDate, IsOnCreditHold,
    CityName, ProvinceName, ProvinceCode, CountryName,
    PostalCode, SalesTerritory,
    ValidFrom, ValidTo, IsCurrent
)
SELECT
    src.CustomerID,
    src.CustomerName,
    src.CustomerGroupName,
    src.CreditLimit,
    src.AccountOpenedDate,
    0,
    ci.CityName,
    sp.StateProvinceName,
    sp.StateProvinceCode,
    co.CountryName,
    NULL,
    CASE sp.StateProvinceCode
        WHEN 'NS' THEN 'Atlantic — Nova Scotia'
        WHEN 'NB' THEN 'Atlantic — New Brunswick'
        WHEN 'PE' THEN 'Atlantic — PEI'
        WHEN 'NL' THEN 'Atlantic — Newfoundland'
        WHEN 'QC' THEN 'Quebec'
        WHEN 'ON' THEN 'Ontario'
        WHEN 'MB' THEN 'Prairies'
        WHEN 'SK' THEN 'Prairies'
        WHEN 'AB' THEN 'Alberta'
        WHEN 'BC' THEN 'British Columbia'
        ELSE 'Other'
    END,
    CAST(GETDATE() AS DATE),    -- ValidFrom: today
    '9999-12-31',               -- ValidTo: open-ended
    1                           -- IsCurrent: this is the current row
FROM CabotTrailOutdoor.Sales.Customers src
INNER JOIN CabotTrailOutdoor.Application.Cities ci
    ON ci.CityID = src.DeliveryCityID
INNER JOIN CabotTrailOutdoor.Application.StateProvinces sp
    ON sp.StateProvinceID = ci.StateProvinceID
INNER JOIN CabotTrailOutdoor.Application.Countries co
    ON co.CountryID = sp.CountryID
WHERE src.CustomerID <> 0
-- Insert new rows where:
-- (a) No current row exists yet (new customer), OR
-- (b) The current row was just expired in Step 1 (changed customer)
AND NOT EXISTS (
    SELECT 1
    FROM Dimension.DimCustomer existing
    WHERE existing.CustomerID = src.CustomerID
```

```
    AND   existing.IsCurrent  = 1
);
```

5.3 Fact Table Loading with Type 2 Dimensions

When a dimension uses SCD Type 2, the fact table load must resolve the natural key to the surrogate key that was **current at the time of the event** — not the current surrogate key.

For CabotTrail, sales orders are loaded with their actual order date. The Lookup transformation in SSIS must find the dimension row where:

- `CustomerID` matches, AND
- `ValidFrom <= OrderDate AND ValidTo >= OrderDate`

```
-- Surrogate key lookup for Type 2 dimension: find the row current at order date
-- (Used in SSIS Lookup reference query or T-SQL join)
SELECT
    CustomerID,
    CustomerKey,
    ValidFrom,
    ValidTo
FROM Dimension.DimCustomer
WHERE CustomerID <> 0;

-- In the fact ETL join:
INNER JOIN Dimension.DimCustomer dc
    ON dc.CustomerID  = il_src.CustomerID
    AND dc.ValidFrom  <= CAST(o.OrderDate AS DATE)
    AND dc.ValidTo    >= CAST(o.OrderDate AS DATE)
```

> ***Important:*** *The SSIS Lookup transformation does not natively support range-based join conditions (`ValidFrom <= OrderDate AND ValidTo >= OrderDate`). For Type 2 dimensions, the lookup must either be implemented as a T-SQL JOIN in the source query, or using the SSIS Lookup with a filtered reference query that pre-resolves to the correct version.*

---

6. Implementing SCD Type 6

SCD Type 6 (the hybrid of Types 1, 2, and 3) adds current-value columns to the Type 2 structure. Every row carries both the historical attribute value (Type 2, preserved at the time of insertion) and the current attribute value (Type 1, updated on every load to reflect today's state).

6.1 The Type 6 Row Structure

```
-- Type 6 DimCustomer: both historical and current province
ALTER TABLE Dimension.DimCustomer
ADD CurrentProvinceCode NCHAR(2)        NULL,
    CurrentSalesTerritory NVARCHAR(60)  NULL;
```

After adding these columns, the dimension rows look like:

```
CustomerKey | CustomerID | ProvinceCode | CurrentProvinceCode | ValidFrom   | ValidTo
| IsCurrent
_____________________________________________________________________________________
_________
15          | 42         | NS           | ON                  | 2022-01-01  | 2024-06-30
| 0
87          | 42         | ON           | ON                  | 2024-07-01  | 9999-12-31
| 1
```

Both rows now carry `CurrentProvinceCode = 'ON'`. Historical fact rows can join to `CustomerKey = 15` to get `ProvinceCode = 'NS'` (the province at time of sale), or to `CurrentProvinceCode = 'ON'` (the customer's current province) — from the same join.

6.2 Updating Type 6 Current-Value Columns

On each ETL run, after inserting new Type 2 rows, update the `CurrentProvinceCode` on ALL rows for each CustomerID (both current and expired rows):

```
-- Update CurrentProvinceCode on ALL rows for each CustomerID
-- (both IsCurrent = 1 and IsCurrent = 0)
UPDATE tgt
SET
    tgt.CurrentProvinceCode     = src.CurrentProvince,
    tgt.CurrentSalesTerritory   = src.CurrentTerritory
FROM Dimension.DimCustomer tgt
INNER JOIN
(
    -- Get the current province for each customer from the IsCurrent = 1 row
    SELECT
        CustomerID,
        ProvinceCode        AS CurrentProvince,
        SalesTerritory      AS CurrentTerritory
    FROM Dimension.DimCustomer
    WHERE IsCurrent = 1
    AND   CustomerID <> 0
) src ON src.CustomerID = tgt.CustomerID
WHERE tgt.CustomerID <> 0;
```

After this update, every row in `DimCustomer` — historical and current — reflects the customer's *current* province in `CurrentProvinceCode`, while `ProvinceCode` continues to reflect the province at the time that row was created.

---

7. Watermark-Based Incremental Extraction

A **watermark** is a stored value — typically a timestamp or sequence number — that marks the point up to which source data has already been extracted. On each ETL run, only records modified after the watermark are extracted.

7.1 The Watermark Pattern

```
-- Watermark table: stores the last extraction point per source table
USE CabotTrailOutdoorDW;

CREATE TABLE ETL.Watermarks
(
    WatermarkID         INT             NOT NULL IDENTITY(1,1),
    SourceSchema        NVARCHAR(60)    NOT NULL,
    SourceTable         NVARCHAR(100)   NOT NULL,
    WatermarkColumn     NVARCHAR(100)   NOT NULL,
    LastExtractedValue  DATETIME2       NOT NULL,
    LastUpdated         DATETIME2       NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT PK_ETL_Watermarks PRIMARY KEY (WatermarkID),
    CONSTRAINT UQ_ETL_Watermarks UNIQUE (SourceSchema, SourceTable)
);

-- Initialize watermarks for CabotTrail source tables
INSERT INTO ETL.Watermarks (SourceSchema, SourceTable, WatermarkColumn, LastExtractedValu
e)
VALUES
    ('Sales',       'Orders',       'LastEditedWhen', '2022-01-01 00:00:00'),
    ('Sales',       'Customers',    'ValidTo',        '2022-01-01 00:00:00'),
    ('Inventory',   'Products',     'ValidTo',        '2022-01-01 00:00:00');
```

7.2 Incremental Extraction Using a Watermark

```
-- Read the current watermark
DECLARE @LastExtract DATETIME2;

SELECT @LastExtract = LastExtractedValue
FROM ETL.Watermarks
WHERE SourceSchema = 'Sales' AND SourceTable = 'Orders';

-- Extract only rows modified since last extract
SELECT
    o.OrderID,
    o.CustomerID,
    o.OrderDate,
    o.LastEditedWhen
FROM Sales.Orders o
WHERE o.LastEditedWhen > @LastExtract
ORDER BY o.LastEditedWhen;

-- After successful extract and load, update the watermark
-- (Use the MAX LastEditedWhen from the extracted rows, not GETDATE(),
-- to avoid missing rows edited during the extract window)
UPDATE ETL.Watermarks
SET
    LastExtractedValue  = (SELECT MAX(LastEditedWhen) FROM Sales.Orders
                           WHERE LastEditedWhen > @LastExtract),
    LastUpdated         = SYSDATETIME()
WHERE SourceSchema = 'Sales' AND SourceTable = 'Orders';
```

7.3 Watermark Requirements in the Source

Watermark-based extraction requires that source tables have a column that reliably reflects when a row was last modified. In practice:

Column type	Examples	Reliability
Server-maintained timestamp	`ROWVERSION`, `TIMESTAMP`, database-maintained `LastModifiedAt`	High — cannot be bypassed by application
Application-maintained timestamp	`LastEditedWhen` (application writes this)	Medium — depends on application being well-behaved
Sequence number	Auto-incrementing `ChangeSequence`	High for inserts; does not capture updates
No timestamp	No modification tracking	Not viable for watermark extraction; use Change Data Capture instead

The CabotTrail OLTP includes `LastEditedWhen` columns on several tables — a common pattern in applications built for ETL compatibility.

7.4 Watermark Pitfalls

Late-arriving data: Records with a `LastEditedWhen` earlier than the watermark will be missed if they arrive after the extract ran. This is the core limitation of watermark extraction — it assumes that records are modified *before* they arrive in the extraction window.

Time zone issues: If the source system and the ETL server are in different time zones, `GETDATE()` on the source and `GETDATE()` on the ETL server may disagree. Always use UTC (`GETUTCDATE()` or `SYSUTCDATETIME()`) for watermark values.

Batch window overlap: If the ETL runs while the source system is also being updated, a record modified at exactly the watermark boundary may be missed. Add a small overlap — subtract 5 minutes from the watermark — to ensure records on the boundary are not missed.

---

8. Multiple Source Integration

Most enterprise BI environments have more than one source system. The ETL must integrate data from multiple sources into a coherent, consistent dimensional model.

8.1 The Integration Challenges

Overlapping natural keys. Source System A has `CustomerID = 42`. Source System B also has `CustomerID = 42`. These are different customers. The DW cannot use either natural key directly — it must create a combined key or use a separate mapping table.

```
-- Solution: composite natural key with source system identifier
-- DimCustomer gains a SourceSystem column
ALTER TABLE Dimension.DimCustomer
ADD SourceSystem NVARCHAR(20) NOT NULL DEFAULT 'CabotTrailOLTP';

-- The unique constraint becomes the combination
ALTER TABLE Dimension.DimCustomer
ADD CONSTRAINT UQ_DimCustomer_Source UNIQUE (CustomerID, SourceSystem);

-- Lookup in fact ETL uses both columns
INNER JOIN Dimension.DimCustomer dc
    ON dc.CustomerID   = src.CustomerID
    AND dc.SourceSystem = 'CabotTrailOLTP'
    AND dc.IsCurrent    = 1
```

Conflicting definitions. Source A defines "Revenue" as invoice amount including tax. Source B defines it as invoice amount excluding tax. The ETL must normalize both to a single agreed definition before loading.

Different granularity. Source A records sales at the invoice-line level. Source B records sales as daily summaries. They cannot both feed the same fact table at line-item grain — one must be transformed to match the other's grain.

Schema differences. Source A has `CustomerFirstName` and `CustomerLastName` as separate columns. Source B has `CustomerFullName` as a single column. The DW needs `FullName` — the ETL must concatenate from Source A and use directly from Source B.

8.2 The Staging Area as an Integration Layer

For multi-source ETL, the **staging area** (introduced in Chapter 1) becomes essential. Each source extracts to its own staging tables, then a conforming layer integrates the staged data before loading the DW dimensions and facts:

```
Source A → Staging.CustomerA  ┐
                               ├── [Conform + deduplicate] → Dimension.DimCustomer
Source B → Staging.CustomerB  ┘

Source A → Staging.SalesA     ┐
                               ├── [Normalize to line-item grain] → Fact.FactSales
Source B → Staging.SalesDailyB┘
```

```
-- Staging tables: raw landed data, no transformation
CREATE TABLE Staging.CustomerA
(
    CustomerID      INT             NOT NULL,
    CustomerName    NVARCHAR(100)   NOT NULL,
    ProvinceCode    NCHAR(2)        NULL,
    LoadDate        DATETIME2       NOT NULL DEFAULT SYSDATETIME()
);

CREATE TABLE Staging.CustomerB
(
    ClientCode      NVARCHAR(20)    NOT NULL,   -- Different key format
    ClientFullName  NVARCHAR(200)   NOT NULL,   -- Different column name
    Region          NVARCHAR(50)    NULL,        -- Different attribute
    LoadDate        DATETIME2       NOT NULL DEFAULT SYSDATETIME()
);
```

8.3 Deduplication and Master Data

When the same real-world entity (a customer, a product, a supplier) exists in multiple source systems, the ETL must decide whether they represent the same entity or different ones. This is the **master data problem** — one of the most complex challenges in enterprise data integration.

Deterministic matching: Match records that share an exact key value (email address, tax ID, ISIN):

```
-- Deterministic deduplication: same email = same customer
SELECT
    COALESCE(a.CustomerID, -1 * ROW_NUMBER() OVER (ORDER BY b.ClientCode))
                               AS MasterCustomerID,
    COALESCE(a.CustomerName, b.ClientFullName) AS CustomerName,
    COALESCE(a.ProvinceCode,
             -- Map Source B region codes to province codes
             CASE b.Region
                 WHEN 'Atlantic' THEN 'NS'
                 WHEN 'Central'  THEN 'ON'
                 ELSE NULL
             END)              AS ProvinceCode,
    CASE WHEN a.CustomerID IS NOT NULL THEN 'SourceA'
         ELSE 'SourceB'
    END                        AS MasterSource
FROM Staging.CustomerA a
FULL OUTER JOIN Staging.CustomerB b
    ON b.ClientCode = CAST(a.CustomerID AS NVARCHAR)  -- Key mapping
ORDER BY MasterCustomerID;
```

Probabilistic matching: Match records based on similarity scores across multiple attributes (name similarity, address similarity). This requires more sophisticated tooling — covered in the Deeper Dive.

8.4 Conformed Dimensions Across Sources

A **conformed dimension** used by multiple sources must use consistent keys so that cross-source analysis is possible. When `DimCustomer` is loaded from both Source A and Source B, both sources' fact tables must reference the same `CustomerKey` values.

The critical discipline: **load conformed dimensions before any fact tables from any source.** The dimension load integrates all sources into a single coherent dimension set. Fact loads from all sources then reference those shared dimension keys.

---

9. SSIS Patterns for Advanced ETL

The T-SQL techniques above can be implemented directly using Execute SQL Tasks in SSIS. But SSIS also provides dedicated Data Flow transformations for some advanced scenarios.

9.1 The SSIS SCD Transformation Wizard

SSIS includes a built-in **SCD Wizard** that generates a Data Flow for SCD Type 1, Type 2, and Type 3 handling. While convenient for simple cases, the wizard has significant limitations:

Wizard approach:

1. Drag an SCD transformation onto the Data Flow canvas
2. Configure: choose the dimension table, identify the natural key column, classify each attribute as Fixed/Type 1/Type 2/Type 3
3. SSIS generates the downstream transformations and OLE DB Commands to expire old rows and insert new ones

Limitations of the SCD Wizard:

- Generates `OLE DB Command` transformations for row-by-row updates — extremely slow for large dimensions (one UPDATE statement per changed row, not a set-based operation)
- Does not support composite natural keys well
- Generated packages are difficult to read and maintain
- Cannot be easily extended for Type 6 or custom SCD logic

Recommendation: Use the SCD Wizard for learning and prototyping. In production, implement SCD logic as T-SQL stored procedures called from Execute SQL Tasks — set-based operations that are orders of magnitude faster:

```
-- Production approach: SCD in a stored procedure called from Execute SQL Task
CREATE PROCEDURE ETL.usp_LoadDimCustomerSCD2
AS
BEGIN
    BEGIN TRANSACTION;

    -- Step 1: Expire changed rows
    UPDATE tgt
    SET ValidTo = CAST(GETDATE() AS DATE), IsCurrent = 0
    FROM Dimension.DimCustomer tgt
    INNER JOIN [... source query ...] src
        ON src.CustomerID = tgt.CustomerID
    WHERE tgt.IsCurrent = 1
    AND (tgt.ProvinceCode <> src.ProvinceCode OR tgt.CityName <> src.CityName);

    -- Step 2: Insert new rows
    INSERT INTO Dimension.DimCustomer (...)
    SELECT ... FROM [... source query ...]
    WHERE NOT EXISTS (SELECT 1 FROM Dimension.DimCustomer WHERE CustomerID = src.Customer
ID AND IsCurrent = 1);

    COMMIT TRANSACTION;
END;
GO
```

9.2 The Merge Join Transformation

The SSIS **Merge Join** transformation performs a JOIN between two sorted inputs within the Data Flow. Unlike the Lookup transformation (which caches a reference table and performs key resolution), Merge Join performs a full relational join between two data streams.

Use cases:

- Joining two large source datasets that are too big to cache in a Lookup
- Detecting changes between two sorted datasets (the "New/Unchanged/Changed/Deleted" change detection pattern)
- Combining rows from two similarly structured sources

Requirements:

- Both inputs must be sorted on the join key
- Sorting within the Data Flow uses the Sort transformation (which breaks the streaming architecture and buffers all rows in memory) — expensive
- Prefer pre-sorting in the OLE DB Source SQL query (`ORDER BY`) for best performance

```
Source A (sorted by CustomerID)  ──┐
                                   ├── [Merge Join: Full Outer JOIN on CustomerID]
Source B (sorted by CustomerID)  ──┘
    |
    v
[Conditional Split: detect New/Changed/Deleted]
    ├── New (in A only)      → Insert to target
    ├── Changed (in both)    → Update target
    └── Deleted (in B only)  → Soft delete target
```

9.3 The Union All Transformation

The **Union All** transformation combines rows from multiple inputs into a single output stream. It is the SSIS equivalent of the SQL `UNION ALL` operator — no deduplication, no sorting, simply merge all rows.

Use cases:

- Loading a dimension from multiple source tables that have the same structure
- Combining similar data from Source A and Source B after schema normalization

```
[OLE DB Source: Customers from System A] ──┐
                                           ├── [Union All] → [OLE DB Destination:
DimCustomer]
[OLE DB Source: Clients from System B]  ──┘
```

After the Union All, a Derived Column transformation can add `SourceSystem = 'A'` or `SourceSystem = 'B'` to distinguish rows by origin.

---

10. Late Arriving Facts and Dimensions

In a full-reload ETL, late arriving facts are not a concern — every load re-processes all source data. In incremental ETL, they are a real design challenge.

10.1 Late Arriving Facts

A **late arriving fact** is a transaction that is loaded into the DW after the period it belongs to has already been processed. Examples:

- An invoice created in December 2024 is entered into the system in January 2025 (data entry backlog)
- A correction to a 2023 order is made in 2025 (retroactive adjustment)
- Data from a partner system arrives 48 hours late (batch timing)

Detection:

```
-- Find rows where the fact's date is earlier than the last watermark
-- (These are late arrivals that the incremental ETL may have missed)
SELECT
    fs.InvoiceID,
    fs.OrderDate,
    CAST(fs.OrderDate AS DATE) AS FactDate,
    w.LastExtractedValue       AS LastWatermark
FROM CabotTrailOutdoor.Sales.Invoices fs_src
CROSS JOIN (
    SELECT LastExtractedValue
    FROM ETL.Watermarks
    WHERE SourceSchema = 'Sales' AND SourceTable = 'Orders'
) w
WHERE CAST(fs_src.InvoiceDate AS DATE) < CAST(w.LastExtractedValue AS DATE)
AND   fs_src.InvoiceID NOT IN (SELECT InvoiceID FROM Fact.FactSales);
```

Handling strategies:

Strategy	Approach	Trade-off
Re-process full period	Re-run ETL for the affected period (e.g., full December reload)	Accurate but expensive; only practical for recent periods
Insert and flag	Insert the late fact with correct date keys; add `IsLateArrival` flag	Simple; downstream reports must filter if needed
Hold in staging	Queue late facts until the next scheduled full re-run	Clean; adds latency
Accept the gap	Document that late arrivals beyond N days will not be captured	Practical for infrequent late arrivals; requires business agreement

10.2 Late Arriving Dimensions

A **late arriving dimension** is more serious: a fact row references a dimension entity (a customer, a product) that does not yet exist in the DW dimension table.

Example: A sales order is loaded before the new customer who placed it has been loaded into `DimCustomer`. The Lookup transformation fails with a no-match.

Handling strategies:

Use the unknown member: Route the fact row's CustomerKey to the unknown member (CustomerKey = 0). The fact is loaded; the customer is unknown. When the customer loads later, the historical fact rows cannot be retroactively corrected (they reference CustomerKey = 0, not the actual customer's key).

Hold the fact in staging: Do not load the fact until its dimension is available. On each run, re-attempt loading from the staging table for any rows previously rejected as late-arriving.

Create a placeholder dimension row: Insert a minimal dimension row with known values and a flag `IsPlaceholder = 1`. When the real dimension row arrives, update the placeholder with correct values and clear the flag. Historical fact rows now reference a complete (corrected) dimension row.

```
-- Create placeholder for a late-arriving customer
INSERT INTO Dimension.DimCustomer
    (CustomerID, CustomerName, CustomerCategoryName,
     CreditLimit, ProvinceCode, SalesTerritory,
     ValidFrom, ValidTo, IsCurrent, IsPlaceholder)
VALUES
    (@CustomerID, 'Unknown Customer ' + CAST(@CustomerID AS NVARCHAR),
     'Unknown', 0, 'XX', 'Other',
     CAST(GETDATE() AS DATE), '9999-12-31', 1, 1);

-- Later: update with real values when the customer loads
UPDATE Dimension.DimCustomer
SET
    CustomerName        = @RealName,
    CustomerCategoryName = @RealCategory,
    CreditLimit         = @RealCreditLimit,
    ProvinceCode        = @RealProvinceCode,
    IsPlaceholder       = 0
WHERE CustomerID = @CustomerID AND IsCurrent = 1;
```

11. Chapter Summary

- The **full-reload pattern** is appropriate for small tables and short load windows. As data volumes grow or historical tracking becomes required, **incremental ETL** with MERGE becomes necessary.
- The **T-SQL MERGE statement** combines INSERT, UPDATE, and DELETE in a single atomic operation. The change detection condition (`WHEN MATCHED AND (...)`) is critical — without it, every matched row is unnecessarily rewritten. NULL-safe comparisons are required for nullable columns.
- **MERGE patterns** include upsert (the most common), insert-only append, full replace for small dimensions, and soft delete with `NOT MATCHED BY SOURCE`. The `NOT MATCHED BY SOURCE` clause should never physically delete dimension rows with historical fact references.
- **SCD types** are chosen based on the business question: does a historical report need the attribute value at the time of the event (Type 2) or the current value (Type 1)?
- **SCD Type 2** requires two steps: expire changed rows (UPDATE `ValidTo` and `IsCurrent = 0`) then insert new rows. Fact table loading must join to the dimension version current at the time of the event.

- **SCD Type 6** adds current-value columns to all rows, enabling both historical (`ProvinceCode`) and current-state (`CurrentProvinceCode`) queries from a single join.
- **Watermark-based incremental extraction** stores the last-extracted timestamp per source table and extracts only rows modified since the watermark. It requires source tables to have reliable modification timestamps.
- **Multiple source integration** requires a staging area, key normalization (composite keys or source system identifiers), definition conforming, and granularity alignment. Conformed dimensions must be loaded before any source's fact tables.
- **Late arriving facts** are handled by re-processing, inserting with flags, or staging. **Late arriving dimensions** require placeholder dimension rows or staging until the dimension is available.

---

12. Review Questions

1. A production ETL system loads `Fact.FactSales` nightly using full reload. The table currently has 16 million rows and the load takes 4.5 hours. The available batch window is 6 hours. Explain why this system is approaching a scalability problem and describe the incremental ETL strategy you would implement.
2. Write a MERGE statement that loads `Dimension.DimProductCategory` from `Inventory.ProductCategories` . The merge should insert new categories and update `CategoryName` if it has changed. Include appropriate NULL-safe change detection and exclude the unknown member (CategoryID = 0).
3. The `WHEN NOT MATCHED BY SOURCE THEN DELETE` clause of a MERGE would delete dimension rows that no longer exist in the source. Explain specifically why this is dangerous in a dimensional model and what the correct alternative is.
4. A customer moves from Halifax to Toronto. The S2T mapping for `DimCustomer` designates `CityName` and `ProvinceCode` as SCD Type 2 attributes, and `CreditLimit` as SCD Type 1. Describe step by step what happens in the DW when this change is detected during the nightly ETL run. What does the DimCustomer table look like before and after?
5. Explain the difference between `ProvinceCode` and `CurrentProvinceCode` in an SCD Type 6 dimension. Write two queries — one that uses each column — and explain what business question each answers.
6. The source system has a `LastEditedWhen` column on `Sales.Orders` . After implementing watermark-based incremental extraction, you discover that some orders from last week are missing from the DW.

Investigation reveals that a batch update process modified these orders without updating `LastEditedWhen`. Identify the root cause and describe two possible solutions.

7. Your ETL is loading facts from both Source A (CabotTrailOutdoor) and a new Source B (a partner retailer's system). Source B uses `PartnerCustomerCode` (a NVARCHAR) as its customer identifier, while Source A uses `CustomerID` (INT). They may or may not represent the same customers. Describe the staging and conforming strategy you would implement to load both into a shared `DimCustomer`.
8. During incremental ETL, a new invoice line arrives for `CustomerID = 999`. Your Lookup transformation against `DimCustomer` finds no matching row for CustomerID 999. Describe the three strategies for handling this late-arriving dimension scenario and explain when each is appropriate.

Deeper Dive

Going Further with Advanced ETL

MERGE Performance and Deadlocks

The `MERGE` statement in SQL Server has a documented edge case with deadlocks: when multiple sessions execute MERGE against the same target table simultaneously, they can deadlock. This is because MERGE acquires locks in a non-deterministic order depending on which rows it encounters first.

Mitigation strategies:

1. **Serialize MERGE execution:** Ensure only one session executes MERGE against a given target at a time (enforced by the master package's sequential execution model)
2. **Use explicit table hints:** `WITH (HOLDLOCK)` forces MERGE to acquire a range lock upfront, preventing phantom inserts from other sessions:

```
MERGE INTO Dimension.DimCustomer WITH (HOLDLOCK) AS tgt
USING (...) AS src
ON (...)
...
```

1. **Use a staging table:** Stage all source data first, then MERGE from the stage to the target in a single session — no concurrency concern

Microsoft Knowledge Base article on MERGE deadlocks:
MERGE Statement May Cause Deadlocks

Temporal Tables: Database-Managed SCD Type 2

SQL Server 2016 introduced **system-versioned temporal tables** — a database engine feature that automatically tracks row history without requiring custom SCD logic in the ETL.

A temporal table has a parallel history table maintained by SQL Server:

```
-- Create a temporal (system-versioned) table
CREATE TABLE Dimension.DimCustomer_Temporal
(
    CustomerKey     INT             NOT NULL IDENTITY(1,1) PRIMARY KEY,
    CustomerID      INT             NOT NULL,
    CustomerName    NVARCHAR(100)   NOT NULL,
    ProvinceCode    NCHAR(2)        NOT NULL,
    -- System-managed period columns
    ValidFrom       DATETIME2       GENERATED ALWAYS AS ROW START NOT NULL,
    ValidTo         DATETIME2       GENERATED ALWAYS AS ROW END   NOT NULL,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (
    HISTORY_TABLE = Dimension.DimCustomer_Temporal_History
));

-- When you UPDATE a row, SQL Server automatically moves the old version
-- to the history table with correct ValidFrom/ValidTo values
UPDATE Dimension.DimCustomer_Temporal
SET ProvinceCode = 'ON'
WHERE CustomerID = 42;

-- Query history: what did this row look like on a given date?
SELECT * FROM Dimension.DimCustomer_Temporal
FOR SYSTEM_TIME AS OF '2023-06-01'
WHERE CustomerID = 42;
```

Temporal tables implement SCD Type 2 semantics at the database engine level — no ETL code required to manage `ValidFrom` / `ValidTo` / `IsCurrent`. The trade-off: temporal tables are designed for operational auditing rather than dimensional modelling, and the history table structure does not match the Kimball SCD Type 2 pattern exactly (no surrogate key per version, no `IsCurrent` flag). They are best suited for OLTP auditing rather than DW dimension tracking.

Microsoft documentation:
Temporal Tables — SQL Server

Probabilistic Matching for Multi-Source Deduplication

When source systems do not share common keys, deterministic matching (same email address = same customer) may not be sufficient. **Probabilistic matching** (also called **fuzzy matching** or **entity resolution**) scores potential matches across multiple attributes and accepts matches above a confidence threshold.

SQL Server Integration Services includes a **Fuzzy Lookup** transformation and a **Fuzzy Grouping** transformation for probabilistic matching within the Data Flow:

```
[Source A Customers]  ──┐
                        ├── [Fuzzy Lookup: match on CustomerName + ProvinceCode]
[Reference: DimCustomer]┘
    |
    v
[Conditional Split: Confidence > 0.85 → matched; else → unmatched]
    ├── Matched → UPDATE existing DimCustomer row
    └── Unmatched → INSERT new DimCustomer row
```

The Fuzzy Lookup uses token-based similarity matching, similar to the Jaro-Winkler or Levenshtein distance algorithms used in natural language processing. It is effective for human names and addresses where exact matches are unreliable.

Microsoft documentation:
Fuzzy Lookup Transformation — SSIS

Change Data Capture as an Alternative to Watermarks

Change Data Capture (CDC) reads SQL Server's transaction log to detect changed rows, avoiding the need for `LastEditedWhen` columns in the source schema. It captures INSERT, UPDATE, and DELETE events with the exact before and after values for each changed column.

CDC is more reliable than watermark-based extraction because:
- It captures deletes (watermarks cannot detect deleted rows)
- It detects all changes even when the application does not update a `LastEditedWhen` column
- It provides before-and-after values for each changed column, enabling precise SCD Type 2 detection

```
-- CDC: get all changes to Sales.Orders since last extraction
DECLARE @from_lsn BINARY(10) = sys.fn_cdc_get_min_lsn('Sales_Orders');
DECLARE @to_lsn   BINARY(10) = sys.fn_cdc_get_max_lsn();

SELECT
    __$operation,   -- 1=Delete, 2=Insert, 3=Before Update, 4=After Update
    OrderID,
    CustomerID,
    OrderDate,
    __$start_lsn    -- Log sequence number for ordering
FROM cdc.fn_cdc_get_all_changes_Sales_Orders(
    @from_lsn, @to_lsn, 'all with merge'
)
ORDER BY __$start_lsn;
```

CDC requires setup by a DBA and adds some overhead to the source SQL Server (log reading). It is the gold standard for incremental ETL when source schemas do not provide reliable modification timestamps.

Microsoft documentation:
About Change Data Capture — SQL Server

The Data Vault Approach to Multi-Source Integration

The **Data Vault 2.0** methodology (discussed in earlier chapters) was specifically designed for multi-source integration scenarios. Its three table types directly address the challenges of section 8:

- **Hubs** — store the unique natural keys from each source system, one row per business entity
- **Links** — store relationships between hub entities, including cross-source relationships
- **Satellites** — store descriptive attributes from each source separately, with full history

The Hub pattern elegantly handles overlapping natural keys:

```
-- Hub: one row per unique customer, regardless of source
CREATE TABLE DV.HubCustomer
(
    CustomerHashKey     BINARY(16)      NOT NULL PRIMARY KEY,  -- MD5 hash of source key
+ source
    LoadDate            DATETIME2       NOT NULL,
    RecordSource        NVARCHAR(100)   NOT NULL,
    CustomerID          NVARCHAR(50)    NOT NULL,              -- Natural key from source
    SourceSystem        NVARCHAR(20)    NOT NULL
);

-- Satellite: history of customer attributes from Source A
CREATE TABLE DV.SatCustomerA
(
    CustomerHashKey     BINARY(16)      NOT NULL,
    LoadDate            DATETIME2       NOT NULL,
    LoadEndDate         DATETIME2       NULL,
    HashDiff            BINARY(16)      NOT NULL,  -- Hash of all attribute values
(change detection)
    CustomerName        NVARCHAR(100),
    ProvinceCode        NCHAR(2),
    PRIMARY KEY (CustomerHashKey, LoadDate)
);
```

Data Vault handles SCD automatically — every change produces a new Satellite row with a new `LoadDate`. The SCD logic is in the pattern, not in custom ETL code.

Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann.

Industry Perspectives

Kimball on Incremental ETL

Kimball's position on incremental ETL is practical: use full reload wherever it works, and introduce incremental processing only where full reload genuinely fails. From *The Data Warehouse ETL Toolkit*:

> *"Incremental ETL is more complex, more fragile, and harder to maintain than full reload. Every incremental ETL system trades simplicity for efficiency. Make that trade only when the efficiency gain is necessary — not as a matter of principle or architectural preference."*

This is important corrective guidance against the tendency to over-engineer ETL with incremental processing before it is needed.

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapters 5–6 cover incremental extraction strategies and SCD implementation patterns in detail.

References and Further Reading

1. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapters 5–6 cover SCD patterns and incremental ETL; Chapter 7 covers multi-source integration.
2. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley. — Chapters 5–6 cover SCD Types 1–7 in detail with design guidance.
3. Kimball Group. (n.d.). *Slowly Changing Dimension Techniques*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/050-slowly-changing-dimension-types-1-2-3-4-5-6/
4. Microsoft. (2024). *MERGE (T-SQL)*. https://learn.microsoft.com/en-us/sql/t-sql/statements/merge-transact-sql
5. Microsoft. (2024). *Temporal Tables*. https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables
6. Microsoft. (2024). *About Change Data Capture*. https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/about-change-data-capture-sql-server
7. Microsoft. (2024). *Fuzzy Lookup Transformation — SSIS*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/transformations/fuzzy-lookup-transformation
8. Microsoft. (2024). *MERGE Statement May Cause Deadlocks*. https://learn.microsoft.com/en-us/troubleshoot/sql/analysis-services/merge-statement-may-cause-deadlocks

9. Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann. — Chapters 4–6 cover Hub, Link, and Satellite patterns for multi-source integration.
10. Olson, J. E. (2003). *Data Quality: The Accuracy Dimension*. Morgan Kaufmann. — Covers fuzzy matching and entity resolution techniques for deduplication.
11. Rozenshtein, D. (2012). *The Art of SQL*. O'Reilly Media. — Advanced T-SQL patterns including complex MERGE scenarios and performance optimization.

Chapter 8: ETL Administration — Scheduling, Hierarchies, Aggregates, and Migrations

Chapter Overview

The previous chapters built a complete, documented, and tested ETL pipeline with advanced transformation capabilities. This chapter addresses the operational side of ETL: how a pipeline is scheduled and monitored in production, how hierarchies and aggregate tables extend the analytical value of the warehouse, and how ETL systems are promoted safely through development, test, and production environments.

These are the disciplines that separate a working ETL system from a *production ETL system* — one that runs reliably, night after night, with monitoring, alerting, and governance supporting it.

By the end of this chapter you will be able to:

- Configure and manage SQL Server Agent jobs to schedule SSIS packages
- Monitor ETL execution using the SSIS Catalog and Agent job history
- Design and implement aggregate tables for analytical performance
- Describe balanced and ragged hierarchies and implement them in a dimensional model
- Explain the DEV → Test → QA → Production promotion process
- Use SSIS environment variables to manage connection strings across environments
- Implement a rollback strategy for failed ETL deployments
- Describe best practices for ETL performance tuning

Table of Contents

1. ETL in Production: The Operational Mindset

An ETL system in production is not a project — it is a service. A service has users who depend on it, SLAs (service level agreements) that define acceptable performance and availability, and operational procedures that govern how it is managed, changed, and recovered.

This shift from project mindset to service mindset changes how ETL developers think about their work:

Project mindset	Service mindset
"It runs without errors"	"It runs reliably, within the SLA, every night"
"I know how to fix it if it breaks"	"The on-call engineer can diagnose and fix it without my involvement"
"I can change it quickly when needed"	"Changes are tested, documented, and approved before deployment"
"The data looks right"	"The data is verified by automated tests after every load"
"I'll document it later"	"Documentation is current and reviewed regularly"

Every technique in this chapter — scheduling, monitoring, environment promotion, rollback — exists in service of this operational mindset.

1.1 The Production ETL Contract

A production ETL system carries an implicit contract with its users:

- **Freshness:** Data will be available in the DW within a defined window after source data changes (e.g., "DW reflects previous day's transactions by 6:00 AM")
- **Completeness:** All source records within scope will be loaded — no silent drops
- **Accuracy:** All transformations and derivations are correct per the agreed S2T mapping
- **Availability:** The DW is queryable throughout the business day (ETL runs during off-peak hours)
- **Notification:** Failures are reported immediately — business users are not the first to discover a problem

Each element of the contract drives a specific operational practice. Freshness drives the scheduling strategy. Completeness drives the reconciliation test suite. Accuracy drives the S2T mapping and derived measure verification. Availability drives the batch window management. Notification drives the alerting configuration.

---

2. SQL Server Agent: Scheduling ETL

SQL Server Agent is the built-in job scheduler for SQL Server. It runs as a Windows service and executes jobs on defined schedules — nightly, hourly, or on demand. For ETL, it is the mechanism that runs the master SSIS package automatically without human intervention.

2.1 Job Architecture

A SQL Server Agent job consists of:

- **Job:** The top-level unit — named, described, and owned by a login
- **Steps:** One or more ordered steps within the job. Each step executes a specific action (run an SSIS package, execute T-SQL, run a PowerShell script)
- **Schedule:** One or more schedules that define when the job runs automatically
- **Notifications:** Email or event log entries triggered on job success, failure, or completion
- **History:** A log of every job execution — start time, end time, duration, success/failure, and step-level messages

2.2 Creating the Nightly ETL Job

The following steps create a SQL Server Agent job that executes the CabotTrail master ETL package nightly at 2:00 AM.

Step 1 — Create the job:

```
USE msdb;
GO

EXEC sp_add_job
    @job_name           = N'CabotTrail_DW_Nightly_Load',
    @description        = N'Loads CabotTrailOutdoorDW from CabotTrailOutdoor OLTP.
                           Full reload of all dimensions and facts.
                           Runs nightly at 02:00. Alerts on failure.',
    @owner_login_name   = N'sa',   -- or a dedicated ETL service account
    @notify_level_email = 2,       -- 2 = On failure
    @notify_email_operator_name = N'ETL Operations';
GO
```

Step 2 — Add the SSIS package execution step:

```
EXEC sp_add_jobstep
    @job_name       = N'CabotTrail_DW_Nightly_Load',
    @step_name      = N'Execute Master ETL Package',
    @step_id        = 1,
    @subsystem      = N'SSIS',
    @command        = N'/ISSERVER
"\"\SSISDB\CabotTrailETL\Packages\Master_DW_Load.dtsx\""
                      /SERVER localhost
                      /ENVREFERENCE 1',    -- Environment reference ID
    @on_success_action = 1,  -- 1 = Quit reporting success
    @on_fail_action    = 2;  -- 2 = Quit reporting failure
GO
```

Step 3 — Add the schedule:

```
EXEC sp_add_schedule
    @schedule_name      = N'Daily_2AM',
    @freq_type          = 4,        -- 4 = Daily
    @freq_interval      = 1,        -- Every 1 day
    @active_start_time  = 020000,   -- 02:00:00
    @active_end_time    = 235959;   -- Run until end of day if needed
GO

EXEC sp_attach_schedule
    @job_name       = N'CabotTrail_DW_Nightly_Load',
    @schedule_name  = N'Daily_2AM';
GO

EXEC sp_add_jobserver
    @job_name       = N'CabotTrail_DW_Nightly_Load',
    @server_name    = N'(local)';
GO
```

Step 4 — Set up the email operator:

```
-- Create the ETL Operations notification operator
EXEC sp_add_operator
    @name                   = N'ETL Operations',
    @enabled                = 1,
    @email_address          = N'Patrick.Dolinger@nscc.ca',
    @pager_days             = 62;   -- Mon-Fri
GO
```

2.3 The ETL Job Schedule Design

The schedule must balance two competing concerns:

The load window must be long enough to complete the ETL plus all reconciliation tests, with capacity for at least one retry on failure.

The load must complete before business users arrive. In most organizations, the DW must be current by 6:00–8:00 AM. If the load starts at 2:00 AM and typically completes in 45 minutes, there is a 3–4 hour window for retries and recovery.

For CabotTrail, the full load completes in under 5 minutes (16,359 fact rows is modest). A 2:00 AM start provides ample recovery time before a 6:00 AM business start.

For large-scale systems, the schedule design becomes more complex:

```
-- Multi-step job: dimensions load first, then facts
-- Allows dimension failures to be caught before fact loads begin

EXEC sp_add_jobstep
    @job_name           = N'CabotTrail_DW_Nightly_Load',
    @step_name          = N'Step 1: Load Dimensions',
    @step_id            = 1,
    @subsystem          = N'SSIS',
    @command            = N'... Master_DimLoad.dtsx ...',
    @on_success_action  = 3,    -- 3 = Go to next step
    @on_fail_action     = 2;    -- 2 = Quit reporting failure

EXEC sp_add_jobstep
    @job_name           = N'CabotTrail_DW_Nightly_Load',
    @step_name          = N'Step 2: Load Facts',
    @step_id            = 2,
    @subsystem          = N'SSIS',
    @command            = N'... Master_FactLoad.dtsx ...',
    @on_success_action  = 1,    -- 1 = Quit reporting success
    @on_fail_action     = 2;    -- 2 = Quit reporting failure
GO
```

2.4 On-Demand Execution

In addition to scheduled runs, ETL must sometimes be executed on demand — after a data correction, a failed overnight run, or a one-time historical reload. SQL Server Agent supports manual execution:

```
-- Execute the job immediately (on demand)
EXEC msdb.dbo.sp_start_job
    @job_name = N'CabotTrail_DW_Nightly_Load';
```

For cases where only a specific package needs reloading (e.g., only `DimCustomer` needs refreshing), a stored procedure wrapper provides controlled on-demand execution:

```
-- On-demand dimension refresh procedure
CREATE PROCEDURE dbo.usp_RefreshDimension
    @DimensionName      NVARCHAR(100),
    @EnvironmentRef     INT = 1         -- Default to Production environment
AS
BEGIN
    DECLARE @ExecutionID BIGINT;

    -- Create an execution in the SSIS Catalog
    EXEC SSISDB.catalog.create_execution
        @package_name       = @DimensionName,
        @execution_id       = @ExecutionID OUTPUT,
        @folder_name        = N'CabotTrailETL',
        @project_name       = N'CabotTrailETL',
        @use32bitruntime    = FALSE,
        @reference_id       = @EnvironmentRef;

    -- Start the execution
    EXEC SSISDB.catalog.start_execution
        @execution_id = @ExecutionID;

    -- Return the execution ID for monitoring
    SELECT @ExecutionID AS ExecutionID;
END;
GO

-- Usage
EXEC dbo.usp_RefreshDimension @DimensionName = 'Load_DimCustomer.dtsx';
```

---

3. Monitoring and Alerting

A production ETL system must be actively monitored. "No news is good news" is not an acceptable monitoring strategy — a failed load that is not reported is a load that business users will discover when their dashboards show yesterday's data.

3.1 Three Layers of Monitoring

Layer 1: Job-level monitoring (SQL Server Agent)

Did the job start? Did it finish? Did it report success or failure?

Layer 2: Package-level monitoring (SSIS Catalog)

Which packages ran? How long did each take? Did any package log errors?

Layer 3: Business-level monitoring (ETL.TestResults)

Did the data load correctly? Do row counts match? Does revenue reconcile?

All three layers are necessary. A job that reports success is not necessarily a job that loaded correct data — a package can succeed while silently dropping rows into error tables.

3.2 Job History Queries

```
-- Job execution history: recent runs with status and duration
USE msdb;

SELECT
    j.name                                  AS JobName,
    jh.run_date,
    -- Format run_time as HH:MM:SS
    STUFF(STUFF(RIGHT('000000' + CAST(jh.run_time AS VARCHAR), 6), 5, 0, ':'), 3, 0, ':')
                                            AS RunTime,
    -- Format run_duration as HH:MM:SS
    STUFF(STUFF(RIGHT('000000' + CAST(jh.run_duration AS VARCHAR), 6), 5, 0, ':'), 3, 0,
':')
                                            AS Duration,
    CASE jh.run_status
        WHEN 0 THEN '   Failed'
        WHEN 1 THEN '   Succeeded'
        WHEN 2 THEN '   Retry'
        WHEN 3 THEN '    Cancelled'
        WHEN 4 THEN '   Running'
    END                                     AS Status,
    jh.message
FROM    sysjobs j
INNER JOIN sysjobhistory jh
    ON jh.job_id = j.job_id
WHERE   j.name = N'CabotTrail_DW_Nightly_Load'
AND     jh.step_id = 0      -- 0 = job-level outcome (not individual steps)
ORDER BY jh.run_date DESC, jh.run_time DESC;
```

```
-- Detect jobs that did not run when expected
-- (Identifies missed schedules — a job that should have run at 2 AM but did not)
SELECT
    j.name                      AS JobName,
    CAST(s.next_run_date AS VARCHAR) + ' ' +
        STUFF(STUFF(RIGHT('000000' + CAST(s.next_run_time AS VARCHAR), 6), 5, 0, ':'),
3, 0, ':')
                                AS NextScheduledRun,
    ISNULL(
        CAST(last_run.run_date AS VARCHAR) + ' ' +
        STUFF(STUFF(RIGHT('000000' + CAST(last_run.run_time AS VARCHAR), 6), 5, 0, ':'),
3, 0, ':'),
        'Never'
    )                           AS LastActualRun,
    CASE last_run.run_status WHEN 1 THEN 'Success' ELSE 'Failed/Unknown' END AS LastStatu
s
FROM    sysjobs j
INNER JOIN sysjobschedules js   ON js.job_id = j.job_id
INNER JOIN sysschedules s       ON s.schedule_id = js.schedule_id
LEFT JOIN (
    SELECT job_id, run_date, run_time, run_status,
           ROW_NUMBER() OVER (PARTITION BY job_id ORDER BY run_date DESC, run_time DESC)
AS rn
    FROM sysjobhistory WHERE step_id = 0
) last_run ON last_run.job_id = j.job_id AND last_run.rn = 1
WHERE   j.name LIKE N'CabotTrail%';
```

3.3 SSIS Catalog Monitoring Queries

```
-- Package execution summary: last 7 days
USE SSISDB;

SELECT
    e.package_name,
    e.start_time,
    e.end_time,
    DATEDIFF(SECOND, e.start_time, e.end_time)  AS DurationSeconds,
    CASE e.status
        WHEN 7 THEN '   Succeeded'
        WHEN 4 THEN '   Failed'
        WHEN 2 THEN '   Running'
        WHEN 3 THEN '    Cancelled'
        ELSE        '    Other (' + CAST(e.status AS VARCHAR) + ')'
    END                                         AS StatusLabel
FROM    catalog.executions e
WHERE   e.start_time >= DATEADD(DAY, -7, GETDATE())
ORDER BY e.start_time DESC;
```

```
-- Error messages from the most recent failed execution
SELECT  TOP 1 execution_id INTO #LastFailed
FROM    SSISDB.catalog.executions
WHERE   status = 4      -- Failed
ORDER BY start_time DESC;

SELECT
    om.message_time,
    om.package_name,
    om.task_name,
    om.message
FROM    SSISDB.catalog.operation_messages om
WHERE   om.operation_id = (SELECT execution_id FROM #LastFailed)
AND     om.message_type IN (120, 130)   -- 120=Error, 130=Warning
ORDER BY om.message_time;

DROP TABLE #LastFailed;
```

```
-- Performance trend: daily average load duration
SELECT
    CAST(start_time AS DATE)                        AS LoadDate,
    COUNT(*)                                        AS ExecutionCount,
    AVG(DATEDIFF(SECOND, start_time, end_time))     AS AvgDurationSeconds,
    MAX(DATEDIFF(SECOND, start_time, end_time))     AS MaxDurationSeconds,
    SUM(CASE WHEN status = 7 THEN 1 ELSE 0 END)     AS Successes,
    SUM(CASE WHEN status = 4 THEN 1 ELSE 0 END)     AS Failures
FROM    SSISDB.catalog.executions
WHERE   package_name = N'Master_DW_Load.dtsx'
AND     start_time >= DATEADD(DAY, -30, GETDATE())
GROUP BY CAST(start_time AS DATE)
ORDER BY LoadDate DESC;
```

3.4 The Operational Dashboard Query

A single query that gives the on-call engineer an immediate picture of last night's load:

```
-- ETL Operational Dashboard: last load status
USE CabotTrailOutdoorDW;

SELECT
    '── LAST LOAD RUN ──────────────────────────────' AS Section,
    NULL AS Detail
UNION ALL
SELECT 'Run Status',
    RunStatus + ' | Tests: ' +
    CAST(TestsPassed AS VARCHAR) + ' passed, ' +
    CAST(TestsFailed AS VARCHAR) + ' failed | ' +
    'Duration: ' +
    CAST(DATEDIFF(SECOND, RunStartTime, RunEndTime) AS VARCHAR) + 's'
FROM ETL.LoadRuns
WHERE RunID = (SELECT MAX(RunID) FROM ETL.LoadRuns)
UNION ALL
SELECT '── FAILED TESTS (if any) ──────────────────────', NULL
UNION ALL
SELECT TestName + ' [' + TargetTable + ']',
    'Expected: ' + ExpectedValue + ' | Actual: ' + ActualValue
FROM ETL.TestResults
WHERE Passed = 0
AND   CAST(RunDate AS DATE) = CAST((SELECT MAX(RunStartTime) FROM ETL.LoadRuns) AS DATE)
UNION ALL
SELECT '── TABLE FRESHNESS ────────────────────────────', NULL
UNION ALL
SELECT s.name + '.' + t.name,
    CAST(p.rows AS VARCHAR) + ' rows | Last loaded: ' +
    ISNULL(CAST(MAX(tr.RunDate) AS VARCHAR), 'Unknown')
FROM sys.tables t
INNER JOIN sys.schemas s ON s.schema_id = t.schema_id
INNER JOIN sys.partitions p ON p.object_id = t.object_id AND p.index_id IN (0,1)
LEFT  JOIN ETL.TestResults tr
    ON tr.TargetTable = s.name + '.' + t.name
    AND tr.TestCategory = 'RowCount' AND tr.Passed = 1
WHERE s.name IN ('Dimension','Fact')
GROUP BY s.name, t.name, p.rows
ORDER BY Section, Detail;
```

3.5 Database Mail for Alerting

Database Mail must be configured before SQL Server Agent notifications work. Once configured, it sends emails automatically on job failure:

```
-- Configure Database Mail (requires sysadmin permissions)
EXEC msdb.dbo.sysmail_add_account_sp
    @account_name           = N'ETL Alert Account',
    @description            = N'Account for ETL failure notifications',
    @email_address          = N'etl-alerts@nscc.ca',
    @display_name           = N'CabotTrail ETL Alerts',
    @mailserver_name        = N'smtp.nscc.ca',
    @port                   = 587,
    @enable_ssl             = 1,
    @username               = N'etl-service@nscc.ca',
    @password               = N'[service_account_password]';

EXEC msdb.dbo.sysmail_add_profile_sp
    @profile_name = N'ETL Alerts',
    @description  = N'Profile for ETL failure notifications';

EXEC msdb.dbo.sysmail_add_profileaccount_sp
    @profile_name   = N'ETL Alerts',
    @account_name   = N'ETL Alert Account',
    @sequence_number = 1;

-- Test the configuration
EXEC msdb.dbo.sp_send_dbmail
    @profile_name   = N'ETL Alerts',
    @recipients     = N'Patrick.Dolinger@nscc.ca',
    @subject        = N'Database Mail Test',
    @body           = N'If you receive this, Database Mail is configured correctly.';
```

---

4. Hierarchies in Dimensional Models

A **hierarchy** is a structured set of levels in a dimension that defines drill-down and roll-up paths for analysis. Hierarchies are not just organizational conveniences — they are the mechanism by which BI tools enable users to move from summary to detail.

4.1 Balanced Hierarchies

A **balanced hierarchy** has the same number of levels in every branch. Every leaf node is at the same depth. These are the most common and the easiest to implement.

CabotTrail Calendar hierarchy (balanced):

```
Level 1: FiscalYear      (4 values: 2022/23, 2023/24, 2024/25, 2025/26)
Level 2: FiscalQuarter   (16 values: FQ1 2022/23, FQ2 2022/23, ...)
Level 3: MonthName       (48 values: one per month across all years)
Level 4: FullDate        (4,017 values: one per day)
```

In the CabotTrail DW, this hierarchy is implicit in the `DimDate` column structure. BI tools detect it by naming convention or explicit definition:

```
-- Calendar hierarchy columns in DimDate
SELECT DISTINCT
    FiscalYearNumber                AS FiscalYear,
    FiscalYearNumber * 10 + FiscalQuarterNumber AS FiscalYearQuarter,
    FiscalYearNumber * 100 + FiscalPeriodNumber AS FiscalYearMonth,
    FullDate                        AS Day
FROM Dimension.DimDate
ORDER BY FiscalYear, FiscalYearQuarter, FiscalYearMonth, Day;
```

CabotTrail Geography hierarchy (balanced):

```
Level 1: CountryName     (typically 1: Canada)
Level 2: ProvinceName    (10 values: NS, NB, ON, ...)
Level 3: CityName        (many values: Halifax, Truro, ...)
```

In `dim.Customer` (datamart), all three levels exist as columns, enabling BI tools to navigate the geography hierarchy:

```
-- Geography hierarchy drill-down example
SELECT
    cust.CountryName,
    cust.ProvinceName,
    cust.CityName,
    COUNT(DISTINCT fs.InvoiceID)    AS Invoices,
    SUM(fs.LineTotal)               AS Revenue
FROM CabotTrailOutdoorsSales.fact.Sales fs
INNER JOIN CabotTrailOutdoorsSales.dim.Customer cust
    ON cust.CustomerKey = fs.CustomerKey
GROUP BY
    GROUPING SETS (
        (cust.CountryName),
        (cust.CountryName, cust.ProvinceName),
        (cust.CountryName, cust.ProvinceName, cust.CityName)
    )
ORDER BY cust.CountryName, cust.ProvinceName, cust.CityName;
```

The `GROUPING SETS` clause produces all three levels of the geography hierarchy in a single query — country total, province subtotals, and city detail — which is exactly what a BI tool needs to render a drill-down table.

4.2 Ragged (Non-Balanced) Hierarchies

A **ragged hierarchy** has branches of different depths. Some nodes have children; others do not. The employee organizational chart is the classic example — a CEO may have direct reports who are also VPs with managers below them, while other direct reports are individual contributors with no team.

Handling ragged hierarchies in dimensional models:

Option 1 — **Bridge table:** A separate table stores the parent-child relationships, allowing recursive navigation. Used when the hierarchy is deep and the depth varies significantly:

```
-- Bridge table for a ragged employee hierarchy
CREATE TABLE Dimension.DimEmployeeHierarchy
(
    EmployeeKey         INT     NOT NULL,   -- Child employee
    AncestorKey         INT     NOT NULL,   -- All ancestors (including self)
    DepthDifference     INT     NOT NULL,   -- 0=self, 1=direct parent, 2=grandparent,
etc.
    IsDirectParent      BIT     NOT NULL,
    CONSTRAINT PK_DimEmployeeHierarchy
        PRIMARY KEY (EmployeeKey, AncestorKey)
);
```

Option 2 — **Fixed-depth with NULL padding:** If the maximum hierarchy depth is known and small (e.g., 5 levels), store all levels as columns and use NULL for missing levels:

```
-- Fixed-depth geography with NULL for missing levels
SELECT
    CountryName,
    ProvinceName,
    SalesTerritory,             -- Territory level (not in all geographies)
    CityName
FROM dim.Customer
-- CityName is always populated; SalesTerritory may be NULL
-- for cities that don't fall into a named territory
```

4.3 Product Category Hierarchy in CabotTrail

The CabotTrail product dimension has a two-level hierarchy: Category → Product. The `DimProductCategory` and `DimProduct` relationship represents this:

```
-- Product hierarchy: category roll-up
SELECT
    prod.CategoryName,
    prod.ProductName,
    COUNT(*)            AS SalesLines,
    SUM(fs.LineTotal)   AS Revenue
FROM Fact.FactSales fs
INNER JOIN Dimension.DimProduct dp      ON dp.ProductKey       = fs.ProductKey
INNER JOIN Dimension.DimProductCategory pc ON pc.ProductCategoryKey = dp.ProductCategoryK
ey
GROUP BY
    GROUPING SETS (
        (prod.CategoryName),
        (prod.CategoryName, prod.ProductName)
    )
ORDER BY prod.CategoryName, prod.ProductName;
```

---

5. Aggregate Tables and Performance

As fact tables grow, queries that scan millions of rows for summary-level analysis become increasingly slow. **Aggregate tables** pre-compute summaries at higher grains and store them as separate tables, enabling queries to scan thousands of rows instead of millions.

5.1 When to Build Aggregate Tables

Aggregate tables are justified when:

1. **Queries are slow:** Key analytical queries take longer than acceptable (typically > 10 seconds for interactive BI)
2. **Query patterns are predictable:** The same aggregation levels are consistently queried (monthly revenue by product category is a common aggregate candidate)
3. **The source fact table is large:** Pre-aggregating a 16-million-row fact table to monthly grain may produce a 50,000-row aggregate — a 320x reduction in scan size
4. **The aggregation is expensive:** Complex measures (rolling averages, percentiles) that are costly to compute at query time benefit from pre-computation

For CabotTrail at 16,359 rows, aggregate tables are not needed for performance. They are built here as a design pattern demonstration — the same pattern applies when the fact table grows to millions of rows.

5.2 Building the Monthly Sales Aggregate

```
-- Create the aggregate table
USE CabotTrailOutdoorDW;
GO

CREATE TABLE Fact.FactSalesMonthly
(
    -- Grain: one row per calendar month per customer per product
    MonthKey            INT             NOT NULL,   -- YYYYMM (e.g., 202403)
    CustomerKey         INT             NOT NULL,
    ProductKey          INT             NOT NULL,

    -- Pre-aggregated measures
    SalesLineCount      INT             NOT NULL DEFAULT 0,
    InvoiceCount        INT             NOT NULL DEFAULT 0,
    TotalRevenue        DECIMAL(18,2)   NOT NULL DEFAULT 0,
    TotalGrossProfit    DECIMAL(18,2)   NOT NULL DEFAULT 0,
    TotalTaxAmount      DECIMAL(18,2)   NOT NULL DEFAULT 0,
    TotalUnitCost       DECIMAL(18,2)   NOT NULL DEFAULT 0,
    TotalOrderedQty     INT             NOT NULL DEFAULT 0,
    TotalPickedQty      INT             NOT NULL DEFAULT 0,

    -- Derived measure (computed at aggregate level — correct because additive)
    AvgMarginPct        AS CASE
                            WHEN TotalRevenue = 0 THEN 0
                            ELSE ROUND(TotalGrossProfit / TotalRevenue * 100, 2)
                           END PERSISTED,

    CONSTRAINT PK_FactSalesMonthly
        PRIMARY KEY (MonthKey, CustomerKey, ProductKey),
    CONSTRAINT FK_FactSalesMonthly_Customer
        FOREIGN KEY (CustomerKey) REFERENCES Dimension.DimCustomer (CustomerKey),
    CONSTRAINT FK_FactSalesMonthly_Product
        FOREIGN KEY (ProductKey)  REFERENCES Dimension.DimProduct (ProductKey)
);
GO

-- Indexes on common filter columns
CREATE INDEX IX_FactSalesMonthly_MonthKey    ON Fact.FactSalesMonthly (MonthKey);
CREATE INDEX IX_FactSalesMonthly_CustomerKey ON Fact.FactSalesMonthly (CustomerKey);
CREATE INDEX IX_FactSalesMonthly_ProductKey  ON Fact.FactSalesMonthly (ProductKey);
GO
```

5.3 Populating the Aggregate

The aggregate is populated from the line-item fact table. This population runs as the final step of the FactSales load — after FactSales is loaded and reconciled:

```
-- Populate FactSalesMonthly from FactSales
-- Run after Load_FactSales.dtsx completes

TRUNCATE TABLE Fact.FactSalesMonthly;

INSERT INTO Fact.FactSalesMonthly
(
    MonthKey, CustomerKey, ProductKey,
    SalesLineCount, InvoiceCount,
    TotalRevenue, TotalGrossProfit, TotalTaxAmount, TotalUnitCost,
    TotalOrderedQty, TotalPickedQty
)
SELECT
    d.YearNumber * 100 + d.MonthNumber  AS MonthKey,
    fs.CustomerKey,
    fs.ProductKey,
    COUNT(*)                            AS SalesLineCount,
    COUNT(DISTINCT fs.InvoiceID)        AS InvoiceCount,
    SUM(fs.LineTotal)                   AS TotalRevenue,
    SUM(fs.GrossProfit)                 AS TotalGrossProfit,
    SUM(fs.TaxAmount)                   AS TotalTaxAmount,
    SUM(fs.UnitCost)                    AS TotalUnitCost,
    SUM(fs.OrderedQuantity)             AS TotalOrderedQty,
    SUM(fs.PickedQuantity)              AS TotalPickedQty
FROM    Fact.FactSales fs
INNER JOIN Dimension.DimDate d ON d.DateKey = fs.OrderDateKey
GROUP BY
    d.YearNumber * 100 + d.MonthNumber,
    fs.CustomerKey,
    fs.ProductKey;
GO
```

5.4 Query Comparison: Line-Item vs Aggregate

The performance benefit of the aggregate is visible even at CabotTrail's modest scale. At millions of rows the difference is dramatic:

```
-- Query using line-item fact (scans 16,359 rows)
SELECT
    d.YearMonthName,
    SUM(fs.LineTotal)   AS Revenue
FROM Fact.FactSales fs
INNER JOIN Dimension.DimDate d ON d.DateKey = fs.OrderDateKey
GROUP BY d.YearMonthName, d.YearNumber, d.MonthNumber
ORDER BY d.YearNumber, d.MonthNumber;

-- Same query using monthly aggregate
-- (scans far fewer rows at scale because one row covers all products per customer per
month)
SELECT
    -- Reconstruct YearMonthName from MonthKey integer
    CAST(MonthKey / 100 AS VARCHAR) + '-' +
        RIGHT('0' + CAST(MonthKey % 100 AS VARCHAR), 2)     AS YearMonth,
    SUM(TotalRevenue)   AS Revenue
FROM Fact.FactSalesMonthly
GROUP BY MonthKey
ORDER BY MonthKey;
```

5.5 Aggregate Table Maintenance

Aggregate tables are **derived data** — they must be repopulated whenever their source fact table changes. In the master package, the aggregate load runs immediately after the fact load and its reconciliation:

```
[Load_FactSales.dtsx] → (success) → [Load_FactSalesMonthly: TRUNCATE + INSERT] →
(success) → [Reconcile Aggregate]
```

Reconciliation for the aggregate:

```
-- Aggregate reconciliation: total revenue must match between line-item and aggregate
SELECT
    'Line-item total'   AS Source,
    SUM(LineTotal)      AS TotalRevenue
FROM Fact.FactSales
UNION ALL
SELECT
    'Aggregate total',
    SUM(TotalRevenue)
FROM Fact.FactSalesMonthly;
-- Both values must be identical
```

6. The SSIS Catalog: Deployment and Configuration

The SSIS Catalog (`SSISDB`) is the production deployment platform for SSIS packages. It provides deployment management, environment-based configuration, and integrated monitoring.

6.1 Deploying to the SSIS Catalog

From Visual Studio (SSDT):

1. Right-click the SSIS project in Solution Explorer → Deploy
2. Select destination: `SQL Server` → enter server name → select the `SSISDB` catalog
3. Select or create a folder: `CabotTrailETL`
4. Review the deployment summary → Deploy

From T-SQL (for automated deployment pipelines):

```
-- Deploy an ISPAC (Integration Services Project Archive) file
USE SSISDB;

DECLARE @ProjectBinary VARBINARY(MAX);
DECLARE @Operation     INT;

-- Read the ISPAC file (requires xp_cmdshell or pre-staged binary)
-- Typically automated via PowerShell in CI/CD pipelines

EXEC catalog.deploy_project
    @folder_name    = N'CabotTrailETL',
    @project_name   = N'CabotTrailETL',
    @project_stream = @ProjectBinary,
    @operation_id   = @Operation OUTPUT;
```

6.2 SSIS Environments for Configuration

The most powerful feature of the SSIS Catalog is **environment-based configuration**. An SSIS environment is a named collection of variables — essentially a configuration profile that can be applied to a package execution without modifying the package itself.

Create environments for each deployment tier:

```
-- Create the Production environment
EXEC catalog.create_environment
    @folder_name        = N'CabotTrailETL',
    @environment_name   = N'Production',
    @environment_description = N'Production connection strings and settings';

-- Add variables to the environment
EXEC catalog.create_environment_variable
    @folder_name        = N'CabotTrailETL',
    @environment_name   = N'Production',
    @variable_name      = N'SourceConnectionString',
    @sensitive          = TRUE,         -- Encrypted at rest
    @value              = N'Data Source=PROD-SQL01;Initial
Catalog=CabotTrailOutdoor;Integrated Security=SSPI;',
    @data_type          = N'String';

EXEC catalog.create_environment_variable
    @folder_name        = N'CabotTrailETL',
    @environment_name   = N'Production',
    @variable_name      = N'TargetConnectionString',
    @sensitive          = TRUE,
    @value              = N'Data Source=PROD-SQL01;Initial
Catalog=CabotTrailOutdoorDW;Integrated Security=SSPI;',
    @data_type          = N'String';

EXEC catalog.create_environment_variable
    @folder_name        = N'CabotTrailETL',
    @environment_name   = N'Production',
    @variable_name      = N'AlertEmail',
    @sensitive          = FALSE,
    @value              = N'Patrick.Dolinger@nscc.ca',
    @data_type          = N'String';
```

```
-- Reference the environment from the project
EXEC catalog.create_environment_reference
    @folder_name        = N'CabotTrailETL',
    @project_name       = N'CabotTrailETL',
    @environment_name   = N'Production',
    @environment_folder_name = N'CabotTrailETL',
    @reference_type     = N'A';     -- 'A' = Absolute reference (same folder)
```

Create a parallel environment for Test:

```
EXEC catalog.create_environment
    @folder_name        = N'CabotTrailETL',
    @environment_name   = N'Test',
    @environment_description = N'Test environment — points to test databases';

EXEC catalog.create_environment_variable
    @folder_name        = N'CabotTrailETL',
    @environment_name   = N'Test',
    @variable_name      = N'SourceConnectionString',
    @sensitive          = TRUE,
    @value              = N'Data Source=TEST-SQL01;Initial
Catalog=CabotTrailOutdoor_Test;Integrated Security=SSPI;',
    @data_type          = N'String';
-- (repeat for all variables with test values)
```

Now the same deployed packages can execute against Production or Test simply by selecting a different environment reference — no package modification required.

---

7. Environment Promotion: DEV → Test → QA → Production

Professional ETL development follows a structured promotion path. Code is developed in one environment and promoted through increasingly production-like environments, with testing and approval at each stage.

7.1 The Four-Environment Model

Environment	Purpose	Data	Who uses it	Promotion requires
DEV	Build and debug packages	Subset / sample data	ETL developer	Developer self-review
Test	Verify packages against complete data	Full copy of production data	ETL developer + QA analyst	All reconciliation tests pass
QA	Business validation against production-like data	Production data copy (recent)	QA analyst + business users	Business sign-off
Production	Live system	Live OLTP source	Automated (Agent job)	Change management approval

7.2 What Changes Between Environments

The SSIS packages themselves do not change between environments — they are the same `.dtsx` files. What changes are the environment variables in the SSIS Catalog:

Variable	DEV	Test	QA	Production
`SourceConnectionString`	Local DEV SQL	TEST-SQL01	QA-SQL01	PROD-SQL01
`TargetConnectionString`	Local DEV DW	TEST-DW	QA-DW	PROD-DW
`AlertEmail`	developer@nscc.ca	dev-team@nscc.ca	qa-team@nscc.ca	ops@nscc.ca
`LoadBatchSize`	1,000	10,000	100,000	Unlimited

7.3 The Promotion Checklist

Each promotion between environments requires completing a checklist:

```
PROMOTION CHECKLIST: Test → QA
Project: CabotTrailETL v1.3
Date: [Date]
Promoted by: [Name]
Approved by: [Name]

Pre-promotion:
[ ] All reconciliation tests pass in Test environment
[ ] ETL.TestResults shows no failures for latest Test run
[ ] Data dictionary updated to reflect any schema changes in this version
[ ] S2T mapping updated and version-incremented
[ ] Process flow diagrams updated if pipeline structure changed
[ ] Change log entry added to ETL Design Document

Deployment:
[ ] SSIS Catalog environment variables updated for QA
[ ] ISPAC deployed to QA server
[ ] QA Agent job updated to reference new environment
[ ] Initial manual test execution run — packages complete without errors

Post-deployment validation:
[ ] All reconciliation tests pass in QA environment
[ ] Row counts match Test environment results
[ ] Revenue reconciliation passes
[ ] Orphan check returns 0
[ ] Business user spot-check of sample data completed
[ ] Sign-off obtained from: [Business stakeholder name]

Rollback plan:
[ ] Previous ISPAC version retained and labelled v1.2_backup
[ ] Rollback procedure documented: redeploy v1.2_backup, revert environment variables
```

7.4 Automating Promotion with PowerShell

Production-grade ETL shops automate environment promotion using PowerShell or CI/CD pipelines (Azure DevOps, GitHub Actions):

```
# PowerShell: Deploy SSIS project to a specified environment
param(
    [string]$ServerName,
    [string]$FolderName    = "CabotTrailETL",
    [string]$ProjectName   = "CabotTrailETL",
    [string]$IspacPath     = ".\bin\Development\CabotTrailETL.ispac",
    [string]$Environment   = "Production"
)

# Load the SSIS assembly
[System.Reflection.Assembly]::LoadWithPartialName("Microsoft.SqlServer.Management.Integra
tionServices") | Out-Null

$sqlConnection = New-Object System.Data.SqlClient.SqlConnection
$sqlConnection.ConnectionString = "Data Source=$ServerName;Initial
Catalog=SSISDB;Integrated Security=SSPI;"
$sqlConnection.Open()

$integrationServices = New-Object Microsoft.SqlServer.Management.IntegrationServices.Inte
grationServices($sqlConnection)
$catalog = $integrationServices.Catalogs["SSISDB"]
$folder  = $catalog.Folders[$FolderName]

# Read the ISPAC file
$projectBytes = [System.IO.File]::ReadAllBytes($IspacPath)

# Deploy
Write-Host "Deploying $ProjectName to $ServerName/$FolderName (Environment:
$Environment)..."
$folder.DeployProject($ProjectName, $projectBytes)

Write-Host "Deployment complete. Validating..."
$project = $folder.Projects[$ProjectName]
$project.Validate()
Write-Host "Validation: $($project.ValidationStatus)"
```

---

8. ETL Performance Tuning

Performance problems in ETL manifest in two ways: the load takes too long (window violation) or individual transforms are slow (bottleneck). Diagnosing and resolving each requires different techniques.

8.1 Identifying the Bottleneck

The SSIS Catalog execution statistics reveal where time is spent:

```
-- Time spent in each Data Flow component for the most recent execution
SELECT
    eds.package_name,
    eds.task_name,
    eds.dataflow_path_id_string     AS Component,
    eds.rows_sent,
    -- Rows per second (approximate throughput)
    CAST(eds.rows_sent AS DECIMAL)
        / NULLIF(DATEDIFF(SECOND, e.start_time, e.end_time), 0) AS RowsPerSecond
FROM    SSISDB.catalog.execution_data_statistics eds
INNER JOIN SSISDB.catalog.executions e ON e.execution_id = eds.execution_id
WHERE   e.execution_id = (SELECT MAX(execution_id) FROM SSISDB.catalog.executions)
ORDER BY eds.rows_sent DESC;
```

Low rows-per-second for a specific component identifies the bottleneck. Common bottleneck sources:

Symptom	Likely cause	Resolution
OLE DB Source slow	Missing index on source WHERE clause	Add index on source table
Lookup transformation slow	Repeated database calls (no-cache mode)	Switch to Full Cache mode
OLE DB Destination slow	Row-by-row insert mode	Switch to Fast Load mode
Sort transformation slow	Sorting all rows in memory	Pre-sort in source SQL ORDER BY
Script component slow	Complex C# per-row logic	Rewrite as set-based SQL or Derived Column

8.2 Source Query Optimization

The extract query is the first opportunity for performance improvement. Every join, filter, and computation in the source query runs against the OLTP — an operational system that must remain responsive for business users.

```
-- Index the OLTP source for ETL extraction performance
-- (Run during maintenance window on the OLTP server)

-- Index for incremental extraction by LastEditedWhen
CREATE NONCLUSTERED INDEX IX_InvoiceLines_LastEdited
    ON CabotTrailOutdoor.Sales.InvoiceLines (LastEditedWhen)
    INCLUDE (InvoiceID, ProductID, Quantity, UnitPrice, LineTotal, TaxAmount);

-- Index for the M:M category join (primary category lookup)
CREATE NONCLUSTERED INDEX IX_ProductCategoryAssignments_ProductID
    ON CabotTrailOutdoor.Inventory.ProductCategoryAssignments (ProductID)
    INCLUDE (ProductCategoryID);
```

8.3 Destination Write Optimization

For large fact table loads, destination write performance is often the primary bottleneck.

Minimize index maintenance during load:

```
-- Option 1: Disable non-clustered indexes before load, rebuild after
-- (Effective for full-reload patterns; not for incremental)

-- Before load:
ALTER INDEX IX_FactSales_CustomerKey   ON Fact.FactSales DISABLE;
ALTER INDEX IX_FactSales_ProductKey    ON Fact.FactSales DISABLE;
ALTER INDEX IX_FactSales_OrderDateKey  ON Fact.FactSales DISABLE;

-- [Run SSIS load]

-- After load:
ALTER INDEX IX_FactSales_CustomerKey   ON Fact.FactSales REBUILD;
ALTER INDEX IX_FactSales_ProductKey    ON Fact.FactSales REBUILD;
ALTER INDEX IX_FactSales_OrderDateKey  ON Fact.FactSales REBUILD;
```

Minimize logging with minimal logging:

```
-- Use minimal logging for bulk inserts (reduces transaction log growth)
-- Requires: target table uses SIMPLE or BULK-LOGGED recovery model
--           and table has no non-clustered indexes (or they are disabled)
ALTER DATABASE CabotTrailOutdoorDW SET RECOVERY BULK_LOGGED;

-- [Run SSIS load with TABLOCK hint on destination]

ALTER DATABASE CabotTrailOutdoorDW SET RECOVERY SIMPLE;
```

8.4 Parallelism in SSIS

Multiple Data Flow Tasks can execute in parallel within the same package or across packages in the master package. Configuring the correct degree of parallelism improves throughput without overloading the server.

```
Master Package Control Flow:
┌──────────────────────────────────────────────────────────────────┐
│  Sequence Container: Tier 1 Dimensions                           │
│  MaxConcurrentExecutables = 5  (all five run simultaneously)     │
│                                                                  │
│  [DimDate]  [DimDeliveryMethod]  [DimTransactionType]            │
│  [DimPaymentMethod]  [DimProductCategory]                        │
└──────────────────────────────────────────────────────────────────┘
```

SSIS package-level parallelism is configured via the `MaxConcurrentExecutables` property on the package or container. Setting it to -1 (default) uses the number of logical processors; setting it to 1 forces sequential execution.

Important: Parallelism increases both throughput and resource contention. Monitor SQL Server CPU, memory, and I/O during parallel loads to ensure the server is not overloaded. For CabotTrail's small data volumes, sequential execution is fine and simpler to debug.

8.5 Connection Pooling

Opening and closing database connections is expensive. SSIS reuses connections within a package through its connection manager pooling. Best practices:

- Use **project-level connection managers** (shared across all packages in the project) rather than package-level ones (recreated for each package)
- Set `RetainSameConnection = True` on connection managers where possible (maintains the connection across multiple tasks in the same package)
- Do not create connection managers inside Foreach Loop containers (creates and destroys connections on each iteration)

---

9. ETL Versioning and Rollback

Every production ETL system needs a strategy for versioning packages and rolling back changes that cause problems.

9.1 Version Control for SSIS Packages

SSIS packages (`.dtsx` files) are XML files. They can be stored in any source control system — Git, Azure DevOps, SVN. The standard practice:

```
etl-cabot-trail/                    ← Git repository root
├── CabotTrailETL.dtproj            ← SSIS project file
├── Master_DW_Load.dtsx             ← Packages
├── Load_DimCustomer.dtsx
├── Load_DimProduct.dtsx
├── Load_FactSales.dtsx
├── [... all packages ...]
└── docs/
    ├── ETL_DesignDocument_v1.3.pdf
    └── S2T_Mapping_v1.3.xlsx
```

Git commit discipline for ETL:

- One commit per logical change (adding a column, changing a derivation rule)
- Commit message references the business change: "Add PromotionCode to FactSales per CR-047"
- Tag release versions: `git tag v1.3-production-2027-02-12`

9.2 The SSIS Catalog's Built-In Versioning

The SSIS Catalog retains previous versions of deployed projects. When a new ISPAC is deployed, the previous version is retained (up to the configured retention limit):

```
-- View deployment history for the project
SELECT
    pv.project_version_lsn,
    pv.name             AS ProjectName,
    pv.description,
    pv.deployed_by_name,
    pv.last_deployed_time
FROM    SSISDB.catalog.projects p
INNER JOIN SSISDB.catalog.project_versions pv
    ON pv.project_id = p.project_id
WHERE   p.name = N'CabotTrailETL'
ORDER BY pv.last_deployed_time DESC;
```

```
-- Restore a previous version (rollback)
EXEC SSISDB.catalog.restore_project
    @folder_name    = N'CabotTrailETL',
    @project_name   = N'CabotTrailETL',
    @project_version_lsn = [version_lsn_from_above];
```

9.3 The Rollback Decision

A rollback is warranted when a deployment causes:

- Reconciliation test failures that cannot be quickly resolved
- Business users reporting incorrect data
- Performance degradation beyond the load window

The rollback decision should be made within a defined time window — typically 2 hours after deployment. If the issue is not resolved within the window, roll back and fix forward.

Rollback procedure:

```
1. Stop the current Agent job if running:
   EXEC msdb.dbo.sp_stop_job @job_name = 'CabotTrail_DW_Nightly_Load';

2. Restore previous SSIS package version from Catalog:
   EXEC SSISDB.catalog.restore_project
       @folder_name = 'CabotTrailETL',
       @project_name = 'CabotTrailETL',
       @project_version_lsn = [previous_version_lsn];

3. If schema changes were made (new columns, tables):
   -- Revert the schema change
   ALTER TABLE Fact.FactSales DROP COLUMN PromotionCode;
   -- or restore from backup

4. Re-run the load:
   EXEC msdb.dbo.sp_start_job @job_name = 'CabotTrail_DW_Nightly_Load';

5. Verify: all reconciliation tests pass with previous version's results.
```

9.4 Schema Change Management

When ETL changes require schema changes (new columns, new tables), the schema change and the package change must be deployed together — and rolled back together:

```
-- Pre-deployment schema migration script
-- Run BEFORE deploying the new SSIS packages

-- Step 1: Add new column (nullable first for safe deployment)
ALTER TABLE Fact.FactSales
ADD PromotionCode NVARCHAR(20) NULL;

-- [Deploy new SSIS packages]
-- [Run reconciliation tests]

-- Step 2: If tests pass, enforce NOT NULL constraint after data is loaded
-- (Cannot add NOT NULL constraint on existing rows until they have values)
ALTER TABLE Fact.FactSales
ALTER COLUMN PromotionCode NVARCHAR(20) NOT NULL;

-- Rollback schema migration (if new packages fail):
ALTER TABLE Fact.FactSales DROP COLUMN PromotionCode;
```

The pattern of adding columns as nullable first, then constraining after the load, is standard practice for zero-downtime schema changes in production environments.

---

10. Chapter Summary

- **Production ETL** is a service with a contract covering freshness, completeness, accuracy, availability, and notification. Every operational practice in this chapter exists to fulfil one or more elements of that contract.
- **SQL Server Agent** schedules SSIS packages through jobs with defined schedules, multi-step execution, and email notifications on failure. On-demand execution is available through stored procedure wrappers.
- **Three layers of monitoring** are necessary: job-level (Agent history), package-level (SSIS Catalog), and business-level (ETL.TestResults). An operational dashboard query combines all three into a single status view.
- **Balanced hierarchies** (calendar, geography) are implemented as multiple columns in dimension tables. **Ragged hierarchies** require bridge tables or fixed-depth NULL-padded columns. `GROUPING SETS` enables multi-level aggregation in a single query.
- **Aggregate tables** pre-compute summaries at higher grains to accelerate common queries. They must be repopulated whenever the source fact changes and reconciled against the source fact.
- The **SSIS Catalog** provides deployment management, environment-based configuration, and execution monitoring. SSIS environments store connection strings and parameters separately from packages, enabling the same packages to run in DEV, Test, QA, and Production without modification.

- **Environment promotion** follows a four-stage model (DEV → Test → QA → Production) with a formal checklist at each stage, including reconciliation test validation and business sign-off before Production.
- **Performance tuning** starts with identifying the bottleneck using Catalog execution statistics. Common interventions include source query indexing, Lookup cache mode, destination fast load, index disabling during bulk loads, and parallelism configuration.
- **Versioning and rollback** require source control for packages, SSIS Catalog version retention, and a defined rollback procedure. Schema changes must be deployed and rolled back alongside their corresponding package changes.

---

11. Review Questions

1. Explain the three layers of ETL monitoring and describe a scenario where all three are needed together. Specifically: what would a successful job with a failed business-level test look like, and how would you detect it?
2. Write a SQL Server Agent job definition (using `sp_add_job`, `sp_add_jobstep`, and `sp_add_schedule`) for a job that runs `Load_DimCustomer.dtsx` every Sunday at midnight. Configure it to email `Patrick.Dolinger@nscc.ca` on failure.
3. The CabotTrail nightly load currently runs the five Tier 1 dimension loads sequentially. A colleague proposes running them in parallel within a Sequence Container. What are the performance benefits, and what conditions must be true for this parallelism to be safe?
4. `Fact.FactSalesMonthly` is an aggregate of `Fact.FactSales` at monthly grain. A new sale is added to `Fact.FactSales` during today's nightly load. Describe what must happen to keep `Fact.FactSalesMonthly` consistent with `Fact.FactSales`. Write the reconciliation query that verifies consistency.
5. Explain why SSIS environment variables are essential for the DEV → Production promotion model. What specific problem would arise if connection strings were hardcoded in the packages instead?
6. The production nightly load fails at 2:47 AM. Your monitoring query shows that `Load_FactSales.dtsx` succeeded (16,359 rows loaded) but the revenue reconciliation test failed — the DW shows $3,200 less revenue than the source. Describe your diagnostic process step by step.
7. A new `PromotionCode` column needs to be added to `Fact.FactSales`. Describe the complete deployment sequence: schema migration, package deployment, testing, and the rollback procedure if tests fail.

8. The `FactSales` load takes 45 minutes and the batch window is 2 hours. A performance analysis shows that the OLE DB Destination is the bottleneck, writing at 200 rows/second. The table has five non-clustered indexes. Describe two specific interventions that could improve write performance and explain why each works.

Deeper Dive

Going Further with ETL Administration

SQL Server Agent Under the Hood

SQL Server Agent runs as a Windows service (`SQLSERVERAGENT`) and communicates with the SQL Server engine through the `msdb` system database. Understanding the `msdb` schema enables sophisticated monitoring and automation:

Key `msdb` tables:
- `dbo.sysjobs` — job definitions
- `dbo.sysjobsteps` — individual step definitions
- `dbo.sysjobschedules` — schedule-to-job mappings
- `dbo.sysjobhistory` — execution history (purged automatically based on retention settings)
- `dbo.sysalerts` — alert definitions (CPU, disk space, SQL error numbers)
- `dbo.sysoperators` — notification recipients

The history retention setting is important for monitoring — by default Agent retains only the last 1,000 history rows per job. For long-running production systems, increase the retention or export history to a custom audit table before it is purged:

```
-- Configure Agent history retention (requires sysadmin)
EXEC msdb.dbo.sp_set_sqlagent_properties
    @jobhistory_max_rows            = 10000,    -- Per job
    @jobhistory_max_rows_per_job    = 1000;
```

Microsoft documentation:

SQL Server Agent

GROUPING SETS, ROLLUP, and CUBE

The `GROUPING SETS` clause used in section 4.1 is one of three SQL extension operators for multi-level aggregation:

`GROUPING SETS`: Explicit list of grouping combinations. Most flexible — you specify exactly which combinations you want.

`ROLLUP`: Automatically generates a hierarchy of aggregations from right to left in the column list. `GROUP BY ROLLUP(Country, Province, City)` produces City totals, Province totals, Country totals, and a grand total — same as a balanced hierarchy drill-up.

`CUBE`: Generates all possible combinations of the columns. `GROUP BY CUBE(Country, Province, City)` produces every combination including City-only, Province-only, Country-only subtotals — useful for cross-tabulation but can produce many combinations for large column lists.

```
-- ROLLUP: hierarchy roll-up (Country → Province → City)
SELECT
    ISNULL(CountryName, '(All Countries)')   AS Country,
    ISNULL(ProvinceName, '(All Provinces)')  AS Province,
    ISNULL(CityName, '(All Cities)')         AS City,
    SUM(LineTotal)                           AS Revenue
FROM CabotTrailOutdoorsSales.fact.Sales fs
INNER JOIN CabotTrailOutdoorsSales.dim.Customer cust
    ON cust.CustomerKey = fs.CustomerKey
GROUP BY ROLLUP (cust.CountryName, cust.ProvinceName, cust.CityName)
ORDER BY Country, Province, City;
```

Microsoft documentation:
GROUP BY ROLLUP, CUBE, GROUPING SETS

Columnstore Indexes for Large Fact Tables

For fact tables with tens of millions of rows, the performance difference between row-store and column-store indexing is dramatic. **Columnstore indexes** store data column by column rather than row by row, enabling extremely efficient compression and vectorized processing for analytical aggregations.

```
-- Create a clustered columnstore index on a large fact table
-- (Replaces the clustered row-store index)
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales
    ON Fact.FactSales
    WITH (DROP_EXISTING = OFF, ONLINE = OFF);

-- Or add a non-clustered columnstore as an additional index
-- (Leaves the existing clustered row-store index in place)
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_FactSales_Analytics
    ON Fact.FactSales
    (OrderDateKey, CustomerKey, ProductKey, LineTotal, GrossProfit);
```

For CabotTrail at 16,359 rows, columnstore indexing provides no benefit — the overhead of index maintenance exceeds any query acceleration. At 10+ million rows, a columnstore index on the fact table typically reduces analytical query time by 10–100×.

Microsoft documentation:
Columnstore Indexes — SQL Server

CI/CD for ETL: Automating the Promotion Pipeline

Modern DevOps practices can be applied to ETL development through **Continuous Integration / Continuous Deployment (CI/CD)** pipelines. Tools like Azure DevOps, GitHub Actions, and Jenkins can automate the promotion process described in section 7.4:

A typical CI/CD pipeline for SSIS:

```
Developer pushes code → Git
    ↓
CI Pipeline (Azure DevOps):
  1. Build the SSIS project → generate ISPAC
  2. Run unit tests (SQL scripts against DEV database)
  3. Deploy ISPAC to Test environment
  4. Run integration tests (full reconciliation suite against Test)
  5. If all tests pass → create release candidate

CD Pipeline (manual approval gate):
  6. Approve promotion to QA
  7. Deploy to QA
  8. Run QA validation tests
  9. Business user sign-off
 10. Approve promotion to Production
 11. Deploy to Production
 12. Run production reconciliation tests
 13. Notify on success/failure
```

This pipeline eliminates manual deployment steps and enforces the promotion checklist automatically. Any failed test at any stage blocks the promotion.

Microsoft documentation:
Build and deploy SSIS packages — Azure DevOps

Partition Switching for Large Fact Tables

For very large fact tables (hundreds of millions of rows), **table partitioning** with **partition switching** enables near-instantaneous load operations by staging data in a separate table and then switching the partition into the main table in a metadata-only operation.

```
-- Create a partitioned fact table (partition by year)
CREATE PARTITION FUNCTION pf_FactSales_Year (INT)
AS RANGE RIGHT FOR VALUES (20220101, 20230101, 20240101, 20250101);

CREATE PARTITION SCHEME ps_FactSales_Year
AS PARTITION pf_FactSales_Year
TO ([PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY]);

CREATE TABLE Fact.FactSales_Partitioned
(
    [... same columns as FactSales ...]
    OrderDateKey INT NOT NULL
) ON ps_FactSales_Year(OrderDateKey);

-- Load a staging table for the new period, then switch it in
-- (O(1) metadata operation — instant regardless of row count)
ALTER TABLE Fact.FactSales_Staging
    SWITCH TO Fact.FactSales_Partitioned PARTITION 5;  -- 2025 partition
```

Partition switching is the gold standard for large-scale fact table loads — it enables daily or even hourly incremental loads of billions of rows with minimal impact on query performance.

Microsoft documentation:
Partitioned Tables and Indexes

Industry Perspectives

Kimball on ETL Operations

Kimball dedicates significant attention in *The Data Warehouse ETL Toolkit* to the operational aspects of ETL that this chapter covers. His perspective on monitoring is particularly relevant:

> *"The ETL system must be fully monitored. Operations staff should never be in a position where they are unsure whether last night's load ran successfully. A complete monitoring infrastructure — scheduler, error notification, reconciliation reporting — is not optional. It is part of the ETL system."*

Kimball treats monitoring not as an add-on but as a core deliverable of the ETL system — the same status as the dimension and fact loads themselves.

Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapter 14 covers ETL operations, scheduling, and monitoring.

References and Further Reading

1. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — Chapter 14 covers ETL operations and scheduling.
2. Microsoft. (2024). *SQL Server Agent*. https://learn.microsoft.com/en-us/sql/ssms/agent/sql-server-agent
3. Microsoft. (2024). *SSIS Catalog*. https://learn.microsoft.com/en-us/sql/integration-services/catalog/ssis-catalog
4. Microsoft. (2024). *GROUP BY ROLLUP, CUBE, GROUPING SETS*. https://learn.microsoft.com/en-us/sql/t-sql/queries/select-group-by-transact-sql
5. Microsoft. (2024). *Columnstore Indexes Overview*. https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview
6. Microsoft. (2024). *Partitioned Tables and Indexes*. https://learn.microsoft.com/en-us/sql/relational-databases/partitions/partitioned-tables-and-indexes
7. Microsoft. (2024). *Deploy SSIS Projects and Packages*. https://learn.microsoft.com/en-us/sql/integration-services/packages/deploy-integration-services-ssis-projects-and-packages
8. Microsoft. (2024). *Database Mail*. https://learn.microsoft.com/en-us/sql/relational-databases/database-mail/database-mail
9. Microsoft. (2024). *Data Flow Performance Features*. https://learn.microsoft.com/en-us/sql/integration-services/data-flow/data-flow-performance-features
10. Itzik Ben-Gan. (2015). *T-SQL Querying*. Microsoft Press. — Chapters 7–8 cover GROUPING SETS, ROLLUP, and CUBE in depth with performance guidance.
11. Fritchey, G. (2018). *SQL Server Query Performance Tuning* (5th ed.). Apress. — Comprehensive guide to SQL Server performance analysis and tuning, including index strategies for analytical workloads.

Chapter 9: Putting It Together — The Complete ETL Pipeline

Chapter Overview

This final chapter steps back from individual techniques and views the complete ETL system as a whole. Each of the preceding chapters introduced a piece: the foundational concepts, the dimensional model, the source analysis, the SSIS implementation, the test suite, the documentation, the advanced techniques, and the operational practices. Here they converge into an integrated picture.

The chapter does four things. First, it traces the complete data journey for a single CabotTrail sale — from the moment it enters the OLTP system to the moment it is queryable in the BI data mart — illustrating how every concept from every chapter contributes to that journey. Second, it reviews the complete pipeline architecture as a coherent design with specific design decisions documented. Third, it maps the ETL knowledge in this book to the broader landscape of the data engineering profession so that readers know where to go next. Fourth, it ends with the practitioner principles that distinguish experienced ETL developers from novices — not rules, but ways of thinking about the work.

By the end of this chapter you will be able to:

- Trace the end-to-end journey of a single transaction through the complete CabotTrail ETL pipeline
- Describe the complete architecture of the CabotTrail ETL system and justify its design decisions
- Map the techniques in this book to the broader data engineering landscape
- Apply practitioner principles to new ETL design and implementation challenges
- Identify the next learning steps appropriate to your career direction

Table of Contents

---

1. The Complete Data Journey: One Sale, Five Systems

The most grounding way to understand a complete ETL pipeline is to follow a single transaction through every layer. This section traces what happens from the moment a CabotTrail customer places an order to the moment an analyst queries it in a Power BI report.

1.1 The Event: A Sale Is Placed

On the morning of March 15, 2024, **Fundy Bay Outfitters** (CustomerID = 42, located in Halifax, Nova Scotia) places an order with sales representative Alice MacLean (EmployeeID = 7) for:

- 2 units of **Cape Breton 3-Season Sleeping Bag** (ProductID = 83, CategoryName = 'Sleeping', UnitPrice = $249.99)
- Delivery method: **Road Freight** (DeliveryMethodID = 5)

The order is fulfilled the same day. An invoice is issued with:
- InvoiceDate: March 15, 2024
- DueDate: April 14, 2024 (30-day terms)
- LineTotal: $499.98 (2 × $249.99)
- TaxAmount: $74.997 (~$75.00 at 15% HST)
- LineTotalIncludingTax: $574.98

1.2 Layer 1 — The OLTP: CabotTrailOutdoor

The order creates rows in three related tables:

```
-- Sales.Orders (the order header)
INSERT INTO Sales.Orders
    (CustomerID, SalesRepID, OrderDate, CityID, DeliveryMethodID, ...)
VALUES (42, 7, '2024-03-15', [Halifax CityID], 5, ...);
-- OrderID = 4801 assigned by IDENTITY

-- Sales.OrderLines (one row per product line)
INSERT INTO Sales.OrderLines
    (OrderID, ProductID, Quantity, PickedQuantity, UnitPrice, ...)
VALUES (4801, 83, 2, 2, 249.99, ...);
-- OrderLineID = 14201 assigned by IDENTITY

-- Sales.Invoices (the invoice for this order)
INSERT INTO Sales.Invoices
    (OrderID, CustomerID, InvoiceDate, DueDate, SalesRepID, DeliveryMethodID, ...)
VALUES (4801, 42, '2024-03-15', '2024-04-14', 7, 5, ...);
-- InvoiceID = 10341 assigned by IDENTITY

-- Sales.InvoiceLines (the invoiced line items)
INSERT INTO Sales.InvoiceLines
    (InvoiceID, OrderLineID, ProductID, Quantity, UnitPrice,
     TaxRate, LineTotal, TaxAmount, ...)
VALUES (10341, 14201, 83, 2, 249.99, 15.0, 499.98, 74.997, ...);
-- Note: LineTotalIncludingTax is a computed column: 499.98 + 74.997 = 574.977
```

At this point the sale exists in the OLTP. It is queryable by operational staff but not yet visible in the data warehouse or any data mart.

1.3 Layer 2 — The Nightly ETL: 2:00 AM

At 2:00 AM, SQL Server Agent fires the `CabotTrail_DW_Nightly_Load` job. The master package begins executing.

Tier 1 — Dimension loads (parallel):

`Load_DimDate.dtsx` runs but produces no new rows — `20240315` already exists in `DimDate`.

`Load_DimDeliveryMethod.dtsx` truncates and reloads 12 rows — no change.

`Load_DimProductCategory.dtsx` truncates and reloads 13 rows — no change.

Tier 2–4 — Dimension loads:

`Load_DimGeography.dtsx`, `Load_DimCustomer.dtsx`, `Load_DimEmployee.dtsx`, `Load_DimProduct.dtsx` all run without changes — Halifax/Fundy Bay Outfitters/Alice MacLean/Cape Breton Sleeping Bag are all already in their respective dimensions.

Tier 5 — FactSales load:

`Load_FactSales.dtsx` executes. The extract query reads from `Sales.InvoiceLines` and its related tables. Among the 16,359 rows it processes is the new invoice line:

Extract stage — the OLE DB Source produces this pipeline row:

Column	Value	Source
`OrderDateKey`	`20240315`	`FORMAT(o.OrderDate, 'yyyyMMdd')`
`InvoiceDateKey`	`20240315`	Same date
`DueDateKey`	`20240414`	`FORMAT(i.DueDate, 'yyyyMMdd')`
`CustomerID`	`42`	`Sales.Customers`
`ProductID`	`83`	`Sales.InvoiceLines`
`EmployeeID`	`7`	`Sales.Invoices.SalesRepID`
`CityID`	`[Halifax CityID]`	`Sales.Orders.CityID`
`DeliveryMethodID`	`5`	`Sales.Invoices.DeliveryMethodID`
`InvoiceID`	`10341`	Degenerate dim
`OrderLineID`	`14201`	Degenerate dim
`OrderedQuantity`	`2`	`Sales.OrderLines.Quantity`
`UnitPrice`	`249.99`	`Sales.InvoiceLines`
`LineTotal`	`499.98`	`Sales.InvoiceLines`
`TaxAmount`	`74.997`	`Sales.InvoiceLines`
`UnitCost`	`167.49`	`249.99 × 0.67 × 2 / 2` = `249.99 × 0.67`
`GrossProfit`	`166.49`	`499.98 − 167.49 × 2 = 499.98 − 334.98`
`GrossProfitMarginPct`	`33.00`	`166.49 / 499.98 × 100`

> ***The 0.67 StandardCostPct:*** *The 'Sleeping' category has `StandardCostPct = 0.67` (67% cost, 33% margin), producing the margin seen above.*

Transform stage — five Lookup transformations resolve natural keys to surrogate keys:

Natural key	Surrogate key resolved
`CustomerID = 42`	`CustomerKey = 15`
`ProductID = 83`	`ProductKey = 76`
`EmployeeID = 7`	`EmployeeKey = 7`
`CityID = [Halifax]`	`GeographyKey = 23`
`DeliveryMethodID = 5`	`DeliveryMethodKey = 5`

Three date validation lookups confirm `20240315`, `20240315`, and `20240414` all exist in `DimDate`.

Load stage — the pipeline row is written to `Fact.FactSales`:

```
SalesKey: [new IDENTITY value]
OrderDateKey:       20240315
InvoiceDateKey:     20240315
DueDateKey:         20240414
CustomerKey:        15
ProductKey:         76
EmployeeKey:        7
GeographyKey:       23
DeliveryMethodKey:  5
InvoiceID:          10341
OrderID:            4801
OrderLineID:        14201
OrderedQuantity:    2
PickedQuantity:     2
UnitPrice:          249.99
TaxRate:            15.0
LineTotal:          499.98
TaxAmount:          74.997
LineTotalIncludingTax: 574.977
UnitCost:           334.98
GrossProfit:        165.00
GrossProfitMarginPct: 33.00
```

Reconciliation — both the row count test and revenue test pass. The load completes at 2:04 AM. SQL Server Agent records success.

1.4 Layer 3 — The Data Marts

After `Fact.FactSales` completes, `Load_FactSalesMonthly.dtsx` runs and updates `Fact.FactSalesMonthly` — the March 2024 aggregate row for CustomerKey=15, ProductKey=76 now includes the new line.

At 2:07 AM, the Sales datamart (`CabotTrailOutdoorsSales`) runs its reload. `dim.Customer` , `dim.Product` , and all other dimensions are refreshed, and `fact.Sales` is fully reloaded from the DW. The new invoice line is now in the datamart.

1.5 Layer 4 — The BI Tool Queries the Data

At 9:15 AM, an analyst opens Power BI and refreshes the Sales dashboard. Power BI executes a query against `CabotTrailOutdoorsSales` :

```
-- Power BI generated query (simplified)
SELECT
    prod.CategoryName,
    cal.MonthName,
    cal.CalendarYear,
    SUM(fs.LineTotal)   AS Revenue,
    SUM(fs.GrossProfit) AS GrossProfit
FROM fact.Sales fs
INNER JOIN dim.Product prod ON prod.ProductKey = fs.ProductKey
INNER JOIN dim.Calendar cal ON cal.DateKey     = fs.OrderDateKey
WHERE cal.CalendarYear = 2024
GROUP BY prod.CategoryName, cal.MonthName, cal.MonthNumber, cal.CalendarYear
ORDER BY cal.MonthNumber, prod.CategoryName;
```

In the result, the Sleeping category for March 2024 now includes the $499.98 from Fundy Bay Outfitters' order. The analyst sees it in the chart.

The complete elapsed time from order placement to analyst visibility: approximately 7 hours (order at 9:00 AM, visible by 9:15 AM the following morning, assuming the nightly load ran as scheduled).

This end-to-end journey — OLTP → staging → DW → datamart → BI tool — is the lived experience of every dimension and fact you have built in this course.

---

2. The Complete Pipeline Architecture

2.1 System Inventory

The complete CabotTrail ETL environment consists of:

Component	Type	Purpose
`CabotTrailOutdoor`	SQL Server database (OLTP)	Source of truth — all operational data
`CabotTrailOutdoorDW`	SQL Server database (DW)	Integrated dimensional model
`CabotTrailOutdoorsSales`	SQL Server database (data mart)	Sales-focused star schema for BI
`CabotTrailOutdoorsReturns`	SQL Server database (data mart)	Returns star schema
`CabotTrailOutdoorsPurchasing`	SQL Server database (data mart)	Purchasing star schema
`CabotTrailOutdoorsInventory`	SQL Server database (data mart)	Inventory snapshot schema
`CabotTrailOutdoorsTransactions`	SQL Server database (data mart)	Financial transactions
`CabotTrailExecutive`	SQL Server database (executive mart)	Monthly summary for leadership
`CabotTrailETL`	SSIS project	All ETL packages
`SSISDB`	SSIS Catalog	Deployment, configuration, monitoring
SQL Server Agent	Windows service	Scheduling and notification
`ETL.TestResults`	Table in DW	Automated reconciliation test results
`ETL.LoadRuns`	Table in DW	Load execution audit trail
`ETL.LoadErrors`	Table in DW	Rejected row audit trail
`ETL.Watermarks`	Table in DW	Incremental extraction state

2.2 The Two-Layer Load Architecture

The CabotTrail ETL follows a two-layer architecture:

Layer 1: OLTP → DW

- Source: `CabotTrailOutdoor`
- Target: `CabotTrailOutdoorDW`
- Schedule: Nightly at 2:00 AM
- Pattern: Full reload (truncate + reload all dimensions and facts)
- Package: `Master_DW_Load.dtsx`

Layer 2: DW → Data Marts

- Source: `CabotTrailOutdoorDW`
- Targets: All five data marts + executive mart
- Schedule: Runs immediately after Layer 1 completes (dependent job or sequential package)
- Pattern: Full reload (DELETE + INSERT preserving surrogate keys with IDENTITY_INSERT ON)
- Package: `Master_Mart_Load.dtsx`

This two-layer separation is a deliberate design decision. The DW layer performs all complex transformations (surrogate key generation, measure derivations, gap resolutions, SCD handling). The mart layer performs only simple projections and flattening — structurally straightforward work that is easy to test and maintain.

```
CabotTrailOutdoor (OLTP)
        │
        │  Layer 1 — Master_DW_Load.dtsx
        │  (2:00 AM nightly, ~4 minutes)
        ↓
CabotTrailOutdoorDW (Data Warehouse)
        │
        ├── Layer 2a — Load_SalesMart.dtsx
        ├── Layer 2b — Load_ReturnsMart.dtsx
        ├── Layer 2c — Load_PurchasingMart.dtsx
        ├── Layer 2d — Load_InventoryMart.dtsx
        ├── Layer 2e — Load_TransactionsMart.dtsx
        └── Layer 2f — Load_ExecutiveMart.dtsx
        │
        │  Layer 2 — Master_Mart_Load.dtsx
        │  (2:04 AM, immediately after Layer 1, ~3 minutes)
        ↓
All data marts available by 2:10 AM
```

---

3. Design Decisions Revisited

Every ETL system is a collection of design decisions. Understanding which decisions were made in the CabotTrail system, and why, prepares you to make similar decisions in new contexts.

3.1 Full Reload vs Incremental

Decision: Full reload for all dimensions and facts.

Why: CabotTrail has modest data volumes — 16,359 fact rows load in under 4 minutes. The full-reload pattern is simpler, more reliable, and easier to test. The load window (2:00 AM, 4 hours before business start) is more than sufficient.

Trade-off accepted: No SCD Type 2 history is tracked. Customer province changes overwrite historical values. The business accepted this limitation — CabotTrail does not currently need to answer "what province was this customer in at the time of this sale?"

When to revisit: When data volumes grow past the 20% batch window rule, or when the business requires historical dimension tracking.

3.2 Kimball vs Inmon

Decision: Kimball bottom-up approach — dimensional model at both the DW and mart layers.

Why: The primary consumers are BI tools (Power BI) and analysts using SQL. Dimensional models are well-supported by these tools and produce simple, readable queries. The organization is small enough that a full enterprise Inmon-style normalized DW would add complexity without corresponding benefit.

Trade-off accepted: No fully normalized integration layer. Cross-subject area analysis (e.g., correlating sales and purchasing patterns at a granular level) requires joining across multiple datamarts or going back to the DW.

3.3 StandardCostPct per Category

Decision: Unit cost is derived from `StandardCostPct` per product category, not from actual purchase cost.

Why: The OLTP `CabotTrailOutdoor` has purchase order line costs (`Purchasing.PurchaseOrderLines.UnitCost`) but these represent supplier invoice prices, not the standard cost basis used for margin analysis. A per-category standard cost rate produces analytically meaningful and consistent margin figures.

Trade-off accepted: Margin percentages are uniform within a category (all Sleeping products have 33% margin, for example) rather than varying by individual product. This limitation is documented in the data dictionary.

When to revisit: If the business implements product-level standard costing in the OLTP, `UnitCost` can be sourced directly from there in the ETL.

3.4 Integer Date Keys

Decision: Date keys are stored as YYYYMMDD integers (e.g., `20240315`).

Why: Integer keys are human-readable, enable range filtering without joins, and are universally compatible with dimensional modelling conventions. SSIS computes them from source DATE columns using `CAST(FORMAT(date, 'yyyyMMdd') AS INT)`.

Trade-off accepted: ETL must compute the integer key; it cannot use the date directly. The computation is trivial but must be present in every extract query.

3.5 Single Unknown Member Value

Decision: Unknown member row uses surrogate key = 0 for all dimensions.

Why: Consistent handling across all dimensions makes error detection easier. Any fact row with a key value of 0 is immediately identifiable as an unresolved dimension reference.

Trade-off accepted: SQL queries filtering to exclude unknown members must use `WHERE DimensionKey <> 0`, which developers must remember to apply consistently.

---

4. The Full Package Inventory

A complete ETL solution is documented by its package inventory — the authoritative list of every SSIS package, its purpose, its dependencies, and its typical execution time.

4.1 Layer 1: OLTP → DW Packages

Package	Purpose	Dependencies	Est. rows	Est. time
`Master_DW_Load.dtsx`	Orchestrates all Layer 1 packages	None	N/A	~4 min
`Load_DimDate.dtsx`	Loads DimDate from system generation	None	4,017	<1s
`Load_DimDeliveryMethod.dtsx`	Loads DimDeliveryMethod from OLTP	None	12	<1s
`Load_DimTransactionType.dtsx`	Loads DimTransactionType from OLTP	None	varies	<1s
`Load_DimPaymentMethod.dtsx`	Loads DimPaymentMethod from OLTP	None	varies	<1s
`Load_DimProductCategory.dtsx`	Loads DimProductCategory from OLTP	None	13	<1s
`Load_DimGeography.dtsx`	Loads DimGeography from OLTP	None	varies	<1s
`Load_DimCustomer.dtsx`	Loads DimCustomer from OLTP + geo	DimGeography	100	<1s
`Load_DimSupplier.dtsx`	Loads DimSupplier from OLTP + geo	DimGeography	varies	<1s
`Load_DimEmployee.dtsx`	Loads DimEmployee from OLTP	None	50	<1s
`Load_DimProduct.dtsx`	Loads DimProduct from OLTP	DimProductCategory, DimSupplier	142	<1s
`Load_FactSales.dtsx`	Loads FactSales from OLTP	All dims	16,359	~30s
`Load_FactPurchasing.dtsx`		DimDate, DimProduct,	905	~5s

Package	Purpose	Dependencies	Est. rows	Est. time
	Loads FactPurchasing from OLTP	DimSupplier, DimEmployee		
`Load_FactReturns.dtsx`	Loads FactReturns from OLTP	DimDate, DimCustomer, DimProduct, DimEmployee, DimGeography	500	~3s
`Load_FactInventory.dtsx`	Loads FactInventory from OLTP	DimDate, DimProduct, DimSupplier, DimProductCategory	142	<1s
`Load_FactCustomerTransactions.dtsx`	Loads FactCustomerTxn from OLTP	DimDate, DimCustomer, DimTransactionType, DimEmployee	9,346	~15s
`Load_FactSupplierTransactions.dtsx`	Loads FactSupplierTxn from OLTP	DimDate, DimSupplier, DimTransactionType, DimEmployee	240	~2s
`Load_FactSalesMonthly.dtsx`	Loads aggregate from FactSales	FactSales	varies	<1s

4.2 Layer 2: DW → Data Mart Packages

Package	Target database	Key tables loaded	Est. time
`Master_Mart_Load.dtsx`	Orchestrates all Layer 2 packages	N/A	~3 min
`Load_SalesMart.dtsx`	`CabotTrailOutdoorsSales`	dim.Calendar, dim.Customer, dim.Product, dim.Employee, dim.DeliveryMethod, fact.Sales	~45s
`Load_ReturnsMart.dtsx`	`CabotTrailOutdoorsReturns`	dim.Calendar, dim.Customer, dim.Product, dim.Employee, fact.Returns	~15s
`Load_PurchasingMart.dtsx`	`CabotTrailOutdoorsPurchasing`	dim.Calendar, dim.Supplier, dim.Product, dim.Employee, fact.Purchasing	~10s
`Load_InventoryMart.dtsx`	`CabotTrailOutdoorsInventory`	dim.Calendar, dim.Supplier, dim.Product, fact.Inventory	~5s
`Load_TransactionsMart.dtsx`	`CabotTrailOutdoorsTransactions`	All dims + fact.CustomerTransaction + fact.SupplierTransaction	~30s
`Load_ExecutiveMart.dtsx`	`CabotTrailExecutive`	dim.Period, dim.Territory, dim.Product, fact.ExecutiveSummary	~15s

5. End-to-End Reconciliation

The most important property of a complete ETL pipeline is that data is never lost and never duplicated as it moves through layers. End-to-end reconciliation verifies this property at every boundary.

5.1 The Three-Boundary Reconciliation

```
Boundary 1: OLTP → DW
    Source: CabotTrailOutdoor.Sales.InvoiceLines (filtered to < 2026)
    Target: CabotTrailOutdoorDW.Fact.FactSales
    Metric: Row count + SUM(LineTotal)

Boundary 2: DW → Sales Datamart
    Source: CabotTrailOutdoorDW.Fact.FactSales
    Target: CabotTrailOutdoorsSales.fact.Sales
    Metric: Row count + SUM(LineTotal)

Boundary 3: DW → Executive Mart (aggregated)
    Source: CabotTrailOutdoorDW.Fact.FactSales
    Target: CabotTrailExecutive.fact.ExecutiveSummary
    Metric: SUM(Revenue) only (row counts differ because grain changed)
```

```
-- Complete end-to-end revenue reconciliation across all layers
SELECT 'OLTP Source'               AS Layer,
       COUNT(*)                    AS Rows,
       ROUND(SUM(il.LineTotal), 2)  AS Revenue
FROM CabotTrailOutdoor.Sales.InvoiceLines il
INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
WHERE YEAR(o.OrderDate) < 2026
UNION ALL
SELECT 'DW Fact.FactSales',
       COUNT(*), ROUND(SUM(LineTotal), 2)
FROM CabotTrailOutdoorDW.Fact.FactSales
UNION ALL
SELECT 'Sales Datamart fact.Sales',
       COUNT(*), ROUND(SUM(LineTotal), 2)
FROM CabotTrailOutdoorsSales.fact.Sales
UNION ALL
SELECT 'Executive Mart (Revenue)',
       COUNT(*), ROUND(SUM(Revenue), 2)
FROM CabotTrailExecutive.fact.ExecutiveSummary;
```

Expected result: The first three layers show identical row counts and revenue. The Executive Mart shows different row counts (fewer rows because grain is monthly × territory × product rather than individual invoice lines) but identical total revenue.

5.2 Dimension Consistency Across Marts

Every data mart that shares a conformed dimension (`dim.Customer` , `dim.Product`) should have the same number of rows in that dimension:

```
-- Conformed dimension consistency check across all sales-related marts
SELECT 'DW DimCustomer'             AS Source, COUNT(*) AS CustomerRows
FROM CabotTrailOutdoorDW.Dimension.DimCustomer WHERE CustomerID <> 0
UNION ALL
SELECT 'Sales Mart dim.Customer',   COUNT(*) FROM CabotTrailOutdoorsSales.dim.Customer
UNION ALL
SELECT 'Returns Mart dim.Customer', COUNT(*) FROM CabotTrailOutdoorsReturns.dim.Customer
UNION ALL
SELECT 'Transactions Mart dim.Customer', COUNT(*) FROM
CabotTrailOutdoorsTransactions.dim.Customer;
-- All four should return 100
```

5.3 The Golden Record Test

The ultimate reconciliation: pick a specific invoice line and trace it from source through every layer, verifying that the values are correct at each point:

```
-- Golden record trace: InvoiceID = 10341, OrderLineID = 14201
-- Trace through all layers

-- Layer 1: OLTP source
SELECT 'OLTP' AS Layer, il.InvoiceID, il.OrderLineID,
       il.Quantity, il.UnitPrice, il.LineTotal,
       o.OrderDate, i.InvoiceDate
FROM CabotTrailOutdoor.Sales.InvoiceLines il
INNER JOIN CabotTrailOutdoor.Sales.Invoices i ON i.InvoiceID = il.InvoiceID
INNER JOIN CabotTrailOutdoor.Sales.Orders o   ON o.OrderID   = i.OrderID
WHERE il.InvoiceID = 10341 AND il.OrderLineID = 14201
UNION ALL
-- Layer 2: Data Warehouse
SELECT 'DW', InvoiceID, OrderLineID,
       OrderedQuantity, UnitPrice, LineTotal,
       CAST(OrderDate AS NVARCHAR), CAST(InvoiceDate AS NVARCHAR)
FROM CabotTrailOutdoorDW.Fact.FactSales
WHERE InvoiceID = 10341 AND OrderLineID = 14201
UNION ALL
-- Layer 3: Sales Datamart
SELECT 'Sales Mart', InvoiceID, OrderLineID,
       OrderedQuantity, UnitPrice, LineTotal,
       CAST(OrderDate AS NVARCHAR), CAST(InvoiceDate AS NVARCHAR)
FROM CabotTrailOutdoorsSales.fact.Sales
WHERE InvoiceID = 10341 AND OrderLineID = 14201;
```

If all three layers return identical values, the pipeline has correctly preserved the data through every transformation.

---

6. The ETL System at a Glance

A concise reference summary of the complete CabotTrail ETL system — the single page an on-call engineer needs to understand the environment.

6.1 Quick Reference Card

```
═══════════════════════════════════════════════════════
CABOTTRAIL ETL — QUICK REFERENCE
═══════════════════════════════════════════════════════

SCHEDULE
  Nightly at 2:00 AM
  Agent job: CabotTrail_DW_Nightly_Load
  Typical duration: ~7 minutes (Layer 1: ~4min + Layer 2: ~3min)

DATA WINDOW
  Sales:        2022-01-01 through yesterday
  Returns:      2022 through present
  Purchasing:   2022 through present
  Inventory:    Current snapshot only

EXPECTED ROW COUNTS (verify against ETL.TestResults)
  Fact.FactSales:                  16,359
  Fact.FactPurchasing:                905
  Fact.FactReturns:                   500
  Fact.FactInventory:                 142
  Fact.FactCustomerTransactions:    9,346
  Fact.FactSupplierTransactions:      240

ON FAILURE
  1. Check ETL.LoadRuns for RunStatus and TestsFailed
  2. Check ETL.TestResults WHERE Passed = 0 (today)
  3. Check ETL.LoadErrors for rejected rows
  4. Check SSISDB.catalog.operation_messages for package errors
  5. Re-run: EXEC msdb.dbo.sp_start_job 'CabotTrail_DW_Nightly_Load'

KEY CONTACTS
  ETL Developer:  Patrick Dolinger | Patrick.Dolinger@nscc.ca
  DBA:            [DBA name and contact]
  Alert email:    Patrick.Dolinger@nscc.ca

ROLLBACK
  SSIS Catalog previous version available
  See: Chapter 9 rollback procedure
═══════════════════════════════════════════════════════
```

---

7. ETL in the Broader Data Engineering Landscape

The techniques in this book — dimensional modelling, SSIS ETL, SQL Server data warehousing — represent one corner of a broader and rapidly evolving field. Understanding where this corner sits helps practitioners navigate their career development.

7.1 The Modern Data Stack

The **modern data stack** is a term describing the collection of tools typically used in cloud-native data engineering today:

Layer	Traditional (this book)	Modern cloud equivalent
Source	SQL Server OLTP	Same, or SaaS APIs (Salesforce, HubSpot)
Ingestion/Extract	SSIS OLE DB Source	Fivetran, Airbyte, Azure Data Factory
Storage	SQL Server DW	Snowflake, BigQuery, Azure Synapse, Databricks
Transformation	SSIS Data Flow, T-SQL	dbt (Data Build Tool), Spark SQL
Orchestration	SQL Server Agent	Apache Airflow, Azure Data Factory, Prefect
Presentation	SSRS, SSAS	Power BI, Tableau, Looker, Metabase
Observability	Custom ETL.TestResults	Monte Carlo, Acceldata, dbt tests

The principles remain constant across this mapping: dimensional modelling (Chapter 2), gap analysis (Chapter 3), data quality governance (Chapter 5), documentation (Chapter 6), SCD handling (Chapter 7), monitoring and testing (Chapters 5 and 8). The tools change; the discipline does not.

7.2 ELT vs ETL in the Cloud

Chapter 1 introduced ELT (Extract, Load, Transform) as the cloud-era pattern. The difference:

ETL (this book): Transform data *before* loading into the warehouse. SSIS applies all transformation logic in flight. The warehouse receives already-transformed data.

ELT (modern cloud): Load raw data *first* into the cloud warehouse (e.g., Snowflake). Transform *inside* the warehouse using SQL tools like dbt. The transformation history is queryable; raw data is always available for re-transformation.

The dimensional modelling concepts from Chapter 2 apply equally to both. In an ELT architecture with dbt, a `dim_customer` model is still a flattened, wide dimension built from multiple source tables — the same structure described in Chapter 3's S2T mapping, implemented as a dbt SQL file instead of an SSIS package.

7.3 The Data Engineer vs the BI Developer

The data engineering landscape has two related but distinct roles:

Data Engineer: Builds and maintains data pipelines, data warehouses, and the infrastructure that moves data. Strong in SQL, Python, distributed systems, and data modelling. This book has primarily addressed data engineering skills.

BI Developer / Analyst: Builds reports, dashboards, and analytical models that consume the data warehouse. Strong in BI tools (Power BI, Tableau), data storytelling, and understanding business requirements. Needs dimensional modelling literacy (Chapter 2) but not deep ETL knowledge.

Most practitioners in small-to-medium organizations play both roles. Understanding the distinction helps identify which skills to develop next.

7.4 Certifications and Standards

For practitioners who want to formalize their knowledge:

Certification	Issuer	Relevance
Microsoft Certified: Azure Data Engineer Associate	Microsoft	Azure data platform, ADF, Synapse, Databricks
Microsoft Certified: Data Analyst Associate	Microsoft	Power BI, DAX, data modelling
Snowflake SnowPro Core	Snowflake	Cloud data warehousing
dbt Fundamentals	dbt Labs	ELT transformation patterns
CDMP (Certified Data Management Professional)	DAMA International	Data governance, data quality, metadata
CBIP (Certified Business Intelligence Professional)	TDWI	BI strategy, dimensional modelling, ETL

The CBIP from TDWI is the most directly aligned with the content of this book. The Azure Data Engineer Associate is the most practical for SQL Server/Azure practitioners.

---

8. Practitioner Principles

After nine chapters of technique, it is worth articulating the underlying principles that guide experienced ETL practitioners. These are not rules — every rule has exceptions — but habits of thought that produce better outcomes consistently.

8.1 Design Before You Build

Every ETL defect that is caught in the S2T mapping takes 5 minutes to fix. The same defect caught in testing takes an hour. Caught in production, it may take days and erode data trust that takes months to rebuild.

The S2T mapping, the data dictionary, and the process flow diagrams are not bureaucratic overhead — they are the cheapest form of quality assurance available. A design decision documented in a mapping table takes 2 minutes to change. The same decision encoded in an SSIS expression and loaded into 16 million rows takes a full reload cycle and a re-run of every test.

8.2 The Source Is Always Messier Than You Expect

No source system is as clean as its documentation claims. Budget time for source data analysis (Chapter 3) equivalent to your implementation budget. The surprises will come — NULL values in NOT NULL columns, referential integrity violations, date formats that vary by row, text fields that contain HTML entities. Find them before implementation, not during.

8.3 Reconciliation Is Not Optional

The difference between a trusted BI system and an untrusted one is not the accuracy of the data — it is the *evidence* of accuracy. Reconciliation tests produce that evidence. Without them, users have only faith that the numbers are correct. With them, users have verification.

Run reconciliation after every load. Log every result. Review failures before business users arrive. This is the discipline that earns trust.

8.4 Name Everything Meaningfully

An SSIS package with components named "OLE DB Source 1", "Lookup 2", and "Derived Column 3" is unreadable. A package with components named "Extract InvoiceLines from OLTP", "Lookup: CustomerID → CustomerKey", and "Derive: UnitCost from StandardCostPct" is self-documenting.

The five minutes spent naming components saves hours in debugging. Apply this principle to SQL aliases, constraint names, index names, variable names, and stored procedure names. Code that names things meaningfully communicates intent; code that does not communicates nothing.

8.5 Fix It at the Source

When you find a data quality problem in the source, the instinct is often to fix it in the ETL — add a CASE expression, a REPLACE function, a conditional. This is usually wrong.

Fixing data quality in the ETL creates a hidden business rule that the source system never learns about. The source continues producing bad data; the ETL silently corrects it. Then the ETL changes, the correction is lost, and the problem re-emerges — often in production, months later.

The right path is harder: report the quality problem to the data steward, escalate to the source system owner, get it fixed at the root. This takes longer. It is worth it.

The exception: when the source data is genuinely outside your control (a third-party vendor, a legacy system that cannot be changed), ETL-level corrections are appropriate — but they must be documented as explicitly as if they were business rules, because they are.

8.6 Every Change Is a Risk

A working ETL system is a production asset. Every change — adding a column, modifying a derivation, updating a join — is a risk to that asset. The risk is proportional to the size of the change and the quality of the test coverage.

The discipline of change management (Chapter 8) exists to make risks manageable: test in DEV, verify in Test, validate in QA, deploy to Production with a rollback plan. Skipping steps under schedule pressure is how production incidents happen.

8.7 Document as You Go, Not After You Finish

Documentation written concurrently with development is accurate and takes 20% more time. Documentation written afterward is incomplete, partially inaccurate, and takes 40% more time because the developer must re-discover what they already built.

The S2T mapping is written before development. The data dictionary is extended as each column is implemented. The process flows are drawn as each package is built. After development is complete, the documentation is also complete — and accurate.

8.8 Understand Your Grain Before Anything Else

Kimball's instruction to declare the grain before choosing dimensions or facts is not a design preference — it is a constraint. Every subsequent design decision is bounded by the grain. A dimension that varies at a finer grain than the fact cannot be correctly joined. A measure that is only meaningful at a coarser grain does not belong in this fact. The grain is the foundation.

When in doubt, choose the finest grain the source supports. Aggregates can always be computed upward. Detail cannot be recovered downward.

8.9 The Business Owns the Definitions

ETL developers implement business rules — they do not create them. What "Revenue" means, which customers are in which territory, how cost is allocated across products — these are business decisions, not technical ones.

When a business rule is ambiguous, stop and get a decision from the data steward before implementing anything. Document the decision with the date, the stakeholder, and the rationale. This protects both the developer and the business from disputes about "what the system was supposed to do."

8.10 Simple Is Better

For any given requirement, there are usually three ways to implement it: a complex SSIS solution using six transformations, a moderately complex T-SQL stored procedure, and a simple two-line SELECT with a JOIN. Choose the simplest approach that meets the requirement correctly.

Simple implementations are easier to test, easier to debug, easier to change, and easier to hand off. Complex implementations are impressive in demonstrations and painful in production. The goal is not to demonstrate technical sophistication — it is to reliably move data from source to target according to agreed business rules.

---

9. Where to Go Next

This book covers the foundations of ETL for business intelligence on the SQL Server platform. The following paths extend this knowledge in different directions.

9.1 Deeper into Dimensional Modelling

Kimball's *The Data Warehouse Toolkit* (3rd edition) is the authoritative reference. If you have read only the portions cited in this book, reading it cover to cover reveals patterns and techniques — retail schemas, inventory modelling, financial services dimensions — that apply far beyond CabotTrail's scope. Chapter 20 specifically covers designing for agility and change, which is the next challenge after mastering the fundamentals.

9.2 Cloud Data Engineering

The patterns in this book translate directly to cloud platforms. **Azure Synapse Analytics** is Microsoft's cloud data warehousing platform that integrates Azure Data Factory (cloud ETL), Synapse SQL (T-SQL at

scale), and Apache Spark (distributed processing). The dimensional modelling concepts are identical; the tooling is cloud-native.

The fastest path: take the **Microsoft Azure Data Engineer Associate** certification path, which builds directly on SQL Server knowledge toward cloud-native architecture.

9.3 ELT with dbt

dbt (Data Build Tool) is the dominant ELT transformation tool for cloud data warehouses. It represents SQL models (SELECT queries) as versioned, tested, documented assets — essentially applying software engineering practices to SQL. If you understand the S2T mapping concept from Chapter 3, you understand dbt models.

The **dbt Fundamentals** free course at learn.getdbt.com takes approximately 8 hours and is the best practical introduction.

9.4 Data Governance and Stewardship

DAMA International's **CDMP (Certified Data Management Professional)** certification formalizes the governance concepts introduced in Chapter 5. For practitioners whose work involves data quality, metadata management, or organizational data strategy, the DAMA-DMBOK is the foundational text.

9.5 Python for Data Engineering

Python has become a standard tool in data engineering, primarily for:
- Orchestration (Apache Airflow uses Python for DAG definitions)
- Data processing at scale (PySpark, Pandas)
- API integrations (extracting from REST APIs that SSIS cannot reach)
- Custom data quality testing (Great Expectations, Soda Core)

For SQL Server practitioners, Python is a complement to T-SQL — not a replacement. Learning the fundamentals of Pandas and the requests library opens a large category of data engineering tasks that SQL alone cannot address.

---

10. Chapter Summary

- The **complete data journey** for a single CabotTrail sale spans five systems (OLTP → DW → data mart → BI tool) and seven hours (order placement to analyst visibility). Every chapter in this book contributes a specific piece of that journey.

- The **two-layer architecture** (OLTP → DW → data marts) separates complex transformation from structural projection. The DW performs the expensive integration work; marts serve specific audiences efficiently from the integrated result.
- **Design decisions are trade-offs.** Full reload chose simplicity over scale. Kimball chose query simplicity over normalized integrity. StandardCostPct chose consistent margins over product-level precision. Each decision was appropriate for CabotTrail's context and documented with its rationale.
- **End-to-end reconciliation** verifies that data is neither lost nor duplicated across every pipeline boundary. The golden record test traces a specific transaction through every layer.
- The techniques in this book — dimensional modelling, ETL implementation, testing, documentation, advanced transformation, and administration — are **platform-independent principles** that apply equally in cloud-native, ELT, and alternative tooling environments.
- **Practitioner principles** are the habits of thought that differentiate experienced ETL developers: design before building, expect messy source data, make reconciliation non-optional, name everything meaningfully, fix quality at the source, treat every change as a risk, and always understand the grain before anything else.
- The ETL field is evolving toward cloud platforms, ELT patterns, and Python-based orchestration. The dimensional modelling principles from Chapter 2, the source analysis from Chapter 3, and the governance practices from Chapter 5 are durable across every platform generation.

---

11. Final Review Questions

The following questions are integrative — they draw on concepts from multiple chapters.

1. A new business requirement arrives: the CabotTrail marketing team wants to track which promotional campaign (if any) was active at the time of each sale. Campaigns have a start date, an end date, and apply to specific product categories. Describe the complete design and implementation process: which tables change in the OLTP, what gaps appear in the S2T mapping, what new ETL logic is required, and what tests must be added to the test suite.
2. Three months after deployment, the revenue figure in the Power BI Sales dashboard is $12,400 lower than the finance system's month-end close report for the same period. The ETL reconciliation tests all pass. What are the three most likely explanations for this discrepancy, and what investigation would you conduct for each?
3. The CabotTrail sales team announces that they are opening a new territory — Territories and Nunavut — effective next month. Three customers will be in this territory. Describe every place in the ETL

system that needs to change: the OLTP, the DW ETL, the datamart ETL, the S2T mapping, and the data dictionary.

4. Explain why the golden record test in section 5.3 is more powerful as a reconciliation tool than the row count + revenue aggregate tests alone. In what scenario would the aggregate tests pass while the golden record test would reveal a defect?
5. A new junior ETL developer joins the team and is handed the CabotTrail ETL system to maintain. Using only the documentation artifacts described in this book (ETL Design Document, S2T mapping, data dictionary, process flow diagrams), describe what the developer could understand about the system without reading a single SSIS package. What would they still not know?
6. The `Fact.FactSalesMonthly` aggregate table shows total March 2024 revenue as $191,403. The line-item `Fact.FactSales` shows total March 2024 revenue as $191,403. But the `fact.Sales` table in the Sales datamart shows $193,815. Trace through the pipeline to identify at which boundary the discrepancy occurred and what might have caused it.
7. An analyst reports that for Customer 42 (Fundy Bay Outfitters), the SalesTerritory shows as 'Atlantic — Nova Scotia' in one report and 'Ontario' in another. Both reports connect to `CabotTrailOutdoorsSales`. How is this possible, and what does it tell you about the SCD handling in the system?
8. You are asked to present the CabotTrail ETL system to a non-technical executive who wants to understand why it takes 7 hours from a sale being placed to it appearing in the dashboard. Using the data journey from section 1 as your narrative, explain the delay in business terms — without using the words SSIS, ETL, surrogate key, or fact table.

Deeper Dive

Reflecting on the Full Journey

The Kimball Lifecycle Methodology

Kimball's *The Data Warehouse Lifecycle Toolkit* (not to be confused with the Toolkit) describes the complete organizational and technical lifecycle for delivering a data warehouse project. It covers project planning, business requirements gathering, dimensional modelling, physical design, ETL development, BI application development, deployment, and maintenance — the activities that surround the technical content of this book.

For practitioners who will lead ETL projects rather than just contribute to them, the Lifecycle Toolkit provides the organizational methodology that complements the technical skills described here:

Kimball, R., Ross, M., Thornthwaite, W., Mundy, J., & Becker, B. (2008). *The Data Warehouse Lifecycle Toolkit* (2nd ed.). Wiley.

The Analogy to Software Engineering

ETL development has undergone the same professionalization over the past decade that application software development underwent in the decade before: the adoption of version control, automated testing, continuous integration, documentation standards, and code review practices.

The analogy is not perfect — ETL has unique challenges (the output is data, not behaviour; failures can be silent; correct answers are business agreements) — but the direction is the same. The practitioner principles in section 8 describe the mature end of this professionalization: design-driven, test-verified, documented, governed.

The most current treatment of this convergence is in the emerging **DataOps** discipline, which applies DevOps principles to data engineering:
DataOps Manifesto

Building Your Portfolio

For students completing this course, the CabotTrail ETL system is a concrete, demonstrable portfolio piece. It demonstrates:
- Dimensional modelling design (star schema, conformed dimensions, SCD awareness)
- Complete ETL implementation (14 packages, two-layer architecture)
- Data quality practices (formal test suite, reconciliation framework)
- Documentation standards (S2T mapping, data dictionary, process flows)
- Production operations (scheduling, monitoring, alerting, rollback)

When presenting this work to future employers, emphasize the design decisions (section 3 of this chapter), the testing discipline (Chapter 5), and the governance practices (Chapter 5 and 6). These are the differentiators — many candidates can configure an SSIS Lookup transformation; fewer can explain why they chose Full Cache mode, what happens to the no-match rows, and how they verify that no rows were silently dropped.

Industry Perspectives

Kimball's Enduring Legacy

Ralph Kimball's dimensional modelling methodology was first published in 1996. Nearly thirty years later, it remains the dominant approach to analytical data modelling — used in SQL Server, Snowflake, BigQuery,

Databricks, and virtually every other analytical platform. The specific tools have changed several times; the pattern has not.

The reason for its durability: dimensional modelling solves a human problem (making complex data understandable) as much as a technical one (making queries fast). Star schemas are comprehensible to business users, navigable by BI tools, and producible by ETL developers — a rare alignment of stakeholder needs that no normalization-based alternative has matched.

Kimball officially retired in 2013, but the Kimball Group's techniques library remains online and is the single best reference for dimensional modelling practitioners:
Kimball Group — Techniques Library

References and Further Reading

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd ed.). Wiley. — The primary reference for all dimensional modelling content in this book. Chapter 20 covers agile and iterative dimensional design.
2. Kimball, R., & Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley. — The companion volume covering ETL design and implementation patterns.
3. Kimball, R., Ross, M., Thornthwaite, W., Mundy, J., & Becker, B. (2008). *The Data Warehouse Lifecycle Toolkit* (2nd ed.). Wiley. — The organizational and project management methodology for delivering DW projects.
4. DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications. — The authoritative reference for data governance, metadata, and data quality management.
5. Linstedt, D., & Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann. — The primary alternative to Kimball for enterprise-scale multi-source integration.
6. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media. — Essential reading for practitioners moving into large-scale distributed data systems.
7. Microsoft. (2024). *Azure Synapse Analytics Documentation*. https://learn.microsoft.com/en-us/azure/synapse-analytics/
8. dbt Labs. (2024). *dbt Documentation*. https://docs.getdbt.com/
9. Kimball Group. (n.d.). *Kimball Techniques Library*. https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/
10. DataOps Community. (n.d.). *The DataOps Manifesto*. https://www.dataopsmanifesto.org/

11. TDWI. (n.d.). *CBIP Certification*. https://tdwi.org/pages/education/business-intelligence-certification-program.aspx

12. dbt Labs. (2024). *dbt Fundamentals Course*. https://learn.getdbt.com/courses/dbt-fundamentals